

MST: My Solution of Templates

ACM Template Notes

算法竞赛模板库

HFUT & HDU

Mistry Nihilityn

2026 年 11 月 5 日

目录

第一部分　基础数据结构 **1**

第一章　并查集 **2**
1.1 普通并查集 . . . 2
1.2 可持久化并查集 . . . 3
1.3 区间并查集 . . . 4

第二章　树状数组 **7**

第三章　线段树 **8**
3.1 一般线段树 . . . 8
3.2 动态开点线段树 . . . 10
3.3 标记持久化线段树 . . . 12
3.4 吉司机线段树 . . . 14
3.5 李超线段树 . . . 24
3.6 线段树合并 . . . 28
3.7 可持久化线段树 . . . 33
3.8 可持久化数组 . . . 38

第四章　平衡树 **40**
4.1 Splay 树 . . . 40
4.2 PBDS 平衡树 . . . 43
4.3 笛卡尔树 . . . 49

第五章　分块 **50**
5.1 块状数组 . . . 50
5.2 树分块 . . . 56

第六章　动态树 **60**
6.1 Link-Cut Tree . . . 60

第七章　其他结构 **65**
7.1 KD-Tree . . . 65
7.2 可持久化字典树 . . . 67
7.3 ST 表 . . . 69

第二部分　图论 **70**

第八章　最短路 **71**
8.1 单源最短路 . . . 71
8.2 全源最短路 . . . 75
8.3 差分约束 . . . 77

第九章　树上问题 **79**
9.1 LCA . . . 79

9.2 直径 . . . 79
9.3 偏心距 . . . 84
9.4 树链剖分 . . . 87
9.5 点分治 . . . 95
9.6 动态重心 . . . 97
9.7 虚树 . . . 101
9.8 基环树 . . . 103

第十章 网络流 **107**
10.1 最大流 . . . 107
10.2 费用流 . . . 111
10.3 上下界网络流 . . . 116
10.4 全局最小割（Stoer-Wagner） . . . 117

第十一章 连通性 **120**
11.1 割点 . . . 120
11.2 点双连通分量 . . . 121
11.3 割边 . . . 122
11.4 边双连通分量 . . . 124
11.5 缩点 . . . 125
11.6 2-SAT . . . 127

第十二章 图匹配 **131**
12.1 匈牙利算法 . . . 131
12.2 带花树算法 . . . 132

第十三章 其他图论算法 **135**
13.1 折半搜索 . . . 135
13.2 斯坦纳树 . . . 136
13.3 线段树优化建图 . . . 138

第三部分 数学 **141**

第十四章 数论 **142**
14.1 线性筛 . . . 142
14.2 杜教筛 . . . 142
14.3 迪利克雷前缀和 . . . 155
14.4 迪利克雷卷积 . . . 156
14.5 类欧几里得 . . . 158
14.6 乘法逆元 . . . 160
14.7 大步小步算法（斐波那契同余） . . . 160

第十五章 代数 **163**
15.1 快速幂 . . . 163
15.2 矩阵（min-plus） . . . 163
15.3 线性基 . . . 164
15.4 线性方程组 . . . 169
15.5 拉格朗日插值 . . . 171

第十六章 组合 **172**

16.1 容斥原理 . . . 172
16.2 约瑟夫环问题 . . . 173
16.3 康托展开 . . . 174

第十七章 计算几何 176
17.1 扫描线 . . . 176

第四部分 字符串 178

第十八章 字符串 179
18.1 哈希 . . . 179
18.2 KMP . . . 180
18.3 AC 自动机 . . . 182
18.4 后缀自动机 . . . 183

第五部分 动态规划 186

第十九章 动态规划 187
19.1 线性动规 . . . 187
19.2 区间 DP . . . 193
19.3 树上 DP . . . 199
19.4 背包 . . . 202
19.5 SOSDP . . . 210
19.6 DP 优化 . . . 217

第六部分 杂项与工具 219

第二十章 其他算法 220
20.1 莫队 . . . 220
20.2 环计数 . . . 223
20.3 分数规划 . . . 226
20.4 梯度下降 . . . 227
20.5 高精度 . . . 229
20.6 加工生产调度（Johnson 规则） . . . 232

第二十一章 工具 234
21.1 STL . . . 234
21.2 数据生成器与对拍 . . . 267

附录 A Dev C++ 的使用 269
A.1 快捷键 . . . 269
A.2 调试 . . . 269
A.3 C++ 标准设置 . . . 269
A.4 优化开关 . . . 269

第 I 编

基础数据结构

第 1 章

并查集

1.1 普通并查集

时间复杂度 $O(\alpha(n))$

带按秩合并（size）与路径压缩的并查集。下标从 1 开始，`init(n)` 后每个点自成一个集合。

```
const int DSU_N = 100005;

class DSUNode {
public:
    int fa;
    int r;
};

class DSU {
public:
    DSUNode node[DSU_N];

    /*
     * 初始化编号 1..maxn 的独立集合
     */
    void init(int maxn) {
        for(int i = 0; i ⩽ maxn; i++) {
            node[i] = {i, 1};
        }
    }

    /*
     * 查询 u 所在集合的代表元并压缩路径
     */
    int find(int u) {
        return node[u].fa == u ? u : node[u].fa = find(node[u].fa);
    }

    /*
     * 合并 u、v 所在集合，返回合并后的代表元
     */
    int join(int u, int v) {
        u = find(u);
        v = find(v);
        if(u ≠ v) {
            if(node[v].r == node[u].r) {
                node[u].fa = v;
                node[v].r++;
            } else {
                if(node[v].r < node[u].r) {
                    swap(u, v);
                }
                node[u].fa = v;
            }
        }
        return v;
    }
};
```

注意

下标从 1 开始；`init(n)` 后每个点自成一个集合。

1.2 可持久化并查集

时间复杂度 $O(\log^2 n)$

1.2.1 P3402 【模板】可持久化并查集

```
class Data {
public:
    int fa = 0, dep = 0;
};

class PersistentDSU {
public:
    PersistentArray<Data> arr;
    vector<int> version;
    /*
     * 查询版本 ver 中 x 所在集合的根信息
     */
    Data find(int ver, int x) const {
        Data d = arr.query(version[ver], x);
        while(d.fa ≠ x) {
            x = d.fa;
            d = arr.query(version[ver], x);
        }
        return d;
    }
    /*
     * 建立 n 个独立集合，初始版本为 0
     */
    void init(int n, size_t capacity = 0) {
        vector<Data> a(n + 1);
        for(int i = 1; i ⩽ n; i++) {
            a[i].fa = i;
        }
        arr.build(a, capacity);
        version.assign(1, 0);
    }
    /*
     * 复制版本 ver，返回新版本号
     */
    int copy(int ver) {
        version.push_back(version[ver]);
        return int(version.size()) - 1;
    }
    /*
     * 基于版本 ver 合并 x、y 所在集合，返回新版本号
     */
    int merge(int ver, int x, int y) {
        Data a = find(ver, x), b = find(ver, y);
        int root = version[ver];
        if(a.fa ≠ b.fa) {
            if(a.dep > b.dep) {
                swap(a, b);
            }
            root = arr.modify(root, a.fa, {b.fa, a.dep});
            if(a.dep == b.dep) {
                root = arr.modify(root, b.fa, {b.fa, b.dep + 1});
```

```
            }
        }
        version.push_back(root);
        return int(version.size()) - 1;
    }
    /*
     * 判断版本 ver 中 x、y 是否属于同一集合
     */
    bool same(int ver, int x, int y) const {
        return find(ver, x).fa == find(ver, y).fa;
    }
};
```

注意

按秩合并、不路径压缩；回滚时把根指向旧版本根。

1.3 区间并查集

时间复杂度 $O((n+m)\log n\,\alpha(n))$

```
const int64 N = 100005;
const int64 MODER = 1000000007;
const int64 LOG_N = 20;
int64 n, m;
int64 lg[N];

class Node {
public:
    int64 fa;
    int64 w;
};

Node f[N][LOG_N];

/*
 * 返回以 idx 为起点、长度为 2^log_gap 的区间代表元
 */
int find_root(int64 idx, int64 log_gap) {
    int root = idx;
    while(f[root][log_gap].fa != -1) {
        root = f[root][log_gap].fa;
    }
    while(idx != root) {
        int fa = f[idx][log_gap].fa;
        f[idx][log_gap].fa = root;
        idx = fa;
    }
    return root;
}

/*
 * 合并 u、v 为起点的两个等长区间
 */
void cat(int64 u, int64 v, int64 log_gap) {
    if(u == -1 || v == -1 || u > n || v > n) {
        return;
    }

    u = find_root(u, log_gap);
    v = find_root(v, log_gap);
```

```
    if(u ≠ v) {
        if(f[u][log_gap].w > f[v][log_gap].w) {
            swap(u, v);
        }
        f[u][log_gap].fa = v;
        f[v][log_gap].w += f[u][log_gap].w;
    }
}

void push_down(int idx, int log_gap) {
    if(log_gap > 0 && f[idx][log_gap].fa ≠ -1) {
        cat(idx, f[idx][log_gap].fa, log_gap - 1);
        cat(idx + (1 << (log_gap - 1)), f[idx][log_gap].fa + (1 << (log_gap - 1)), log_gap - 1);
        f[idx][log_gap].fa = -1;
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> m;

    for(int i = 2; i ≤ n; i++) {
        lg[i] = lg[i >> 1] + 1;
    }

    for(int i = 0; i ≤ n; i++) {
        for(int j = 0; j ≤ lg[n]; j++) {
            f[i][j].fa = -1;
            f[i][j].w = 1;
        }
    }

    int64 l, r, L, R, lag, gap;
    while(m--) {
        cin >> l >> r >> L >> R;

        lag = r - l + 1;
        gap = 1 << (lg[n] + 1);
        while(gap) {
            if(gap & lag) {
                cat(l + ((lag - lag % gap) ^ gap), L + ((lag - lag % gap) ^ gap), lg[gap]);
            }
            gap >≥ 1;
        }
    }
    for(int j = lg[n]; j ≥ 0; j--) {
        for(int i = 1; i + (1ll << j) - 1 ≤ n; i++) {
            if(f[i][j].fa ≠ -1) {
                push_down(i, j);
            }
        }
    }
    int cnt = 0;

    for(int i = 1; i ≤ n; i++) {
        if(f[i][0].fa == -1) {
            cnt++;
        }
    }
```

```
102
103     int64 res = 9;
104     cnt--;
105     for(int i = 0; i < cnt; i++) {
106         res = res * 10 % MODER;
107     }
108     cout << (res) << '\n';
109 }
```

注意

倍增并查集：同一层内合并，向下传播。

第 2 章

树状数组

时间复杂度 $O(\log n)$

```
template <class Data = long long>
class BIT {
public:
    vector<Data> arr;
    int size = 0;
    /*
     * 初始化长度为 n 的树状数组
     */
    void init(int n) {
        size = n;
        arr.assign(n + 1, Data());
    }
    /*
     * 在 idx 处加 x，下标从 1 开始
     */
    void add(int idx, Data x) {
        for(int i = idx; i ≤ size; i += lowBit(i)) {
            arr[i] += x;
        }
    }
    /*
     * 查询前缀 [1,r] 的和，r=0 时返回 0
     */
    Data prefix(int r) const {
        Data res{};
        for(int i = r; i; i -= lowBit(i)) {
            res += arr[i];
        }
        return res;
    }
};
```

注意

下标从 1 开始；`prefix(0)` 返回幺元。

第 3 章

线段树

3.1 一般线段树

时间复杂度 $O(\log n)$

```

class Data {
public:
    int64 sum = 0;
    Data operator+(const Data& o) const {
        return {sum + o.sum};
    }
};

class Tag {
public:
    int64 add = 0, mul = 1;
    // a + b: 先执行 a，再执行 b。
    Tag operator+(const Tag& o) const {
        return {add * o.mul + o.add,
                mul * o.mul};
    }
    Data apply(const Data& d, int l, int r) const {
        return {d.sum * mul + add * (r - l + 1)};
    }
};
```

3.1.1 通用实现

下面的类不依赖上述示例；使用时提供自己的 `Data` / `Tag`，实例化 `SegmentTree<Data, Tag>`。

```
template <class Value, class Tag>
class SegmentTree {
public:
    class Node {
    public:
        Value data{};
        Tag tag{};
    };
    int n = 0;
    vector<Node> node;
    void push_up(int u) {
        node[u].data = node[u * 2].data + node[u * 2 + 1].data;
    }
    void apply(int u, int l, int r, const Tag& t) {
        node[u].data = t.apply(node[u].data, l, r);
        node[u].tag = node[u].tag + t;
    }
    void push_down(int u, int l, int r) {
        int mid = l + (r - l) / 2;
        apply(u * 2, l, mid, node[u].tag);
        apply(u * 2 + 1, mid + 1, r, node[u].tag);
        node[u].tag = Tag();
    }
    void build(int u, int l, int r) {
        if(l == r) {
```

```
            node[u].data = Value{};
            return;
        }
        int mid = l + (r - l) / 2;
        build(u * 2, l, mid);
        build(u * 2 + 1, mid + 1, r);
        push_up(u);
    }
    void mod(int u, int l, int r, int ql, int qr, const Tag& t) {
        if(ql <= l && r <= qr) {
            apply(u, l, r, t);
            return;
        }
        int mid = l + (r - l) / 2;
        push_down(u, l, r);
        if(ql <= mid) {
            mod(u * 2, l, mid, ql, qr, t);
        }
        if(qr > mid) {
            mod(u * 2 + 1, mid + 1, r, ql, qr, t);
        }
        push_up(u);
    }

    Value query(int u, int l, int r, int ql, int qr) {
        if(ql <= l && r <= qr) {
            return node[u].data;
        }
        int mid = l + (r - l) / 2;
        push_down(u, l, r);
        Value res{};
        if(ql <= mid) {
            res = res + query(u * 2, l, mid, ql, qr);
        }
        if(qr > mid) {
            res = res + query(u * 2 + 1, mid + 1, r, ql, qr);
        }
        return res;
    }
    /*
     * 建立 [1, n], 所有位置初始化为 Value{}
     */
    void build(int n) {
        this->n = n;
        node.assign(4 * n + 4, Node());
        if(n) {
            build(1, 1, n);
        }
    }
    /*
     * 对闭区间 [l,r] 施加标记 t
     */
    void mod(int l, int r, const Tag& t) {
        mod(1, 1, n, l, r, t);
    }
    /*
     * 合并闭区间 [l,r] 的信息
     */
    Value query(int l, int r) {
        return query(1, 1, n, l, r);
    }
```

```
};
```

注意

Data 的 `operator+` 与 Tag 的 `operator+/apply` 必须满足题目对应的幺元与结合律。

3.2 动态开点线段树

时间复杂度 $O(\log V)$

未开点区间的初值由 `make_value(l, r)` 给出，默认是 `Value{}`；需要与区间相关的初值时（例如让每个点带上自己的下标），继承并重写它。

```
class Data {
public:
    int64 sum = 0;
    Data operator+(const Data& other) const {
        return {sum + other.sum};
    }
};

class Tag {
public:
    int64 add = 0;
    Tag operator+(const Tag& other) const {
        return {add + other.add};
    }
    Data apply(const Data& d, int l, int r) const {
        return {d.sum + add * (r - l + 1)};
    }
};

template <typename Value, typename Tag>
class SegmentTree {
public:
    class Node {
    public:
        int ls = -1, rs = -1;
        Value val{};
        Tag tag{};
    };

    vector<Node> node;
    int ncnt = 0;
    int root;
    int le, re;

    /*
     * 未开点区间 [l, r] 的初值；默认全零
     */
    virtual Value make_value(int, int) {
        return Value{};
    }

    int new_node(int l, int r) {
        int idx = ncnt++;
        node[idx].val = make_value(l, r);
        return idx;
    }

    void apply(int idx, const Tag& t, int l, int r) {
```

```
        node[idx].val = t.apply(node[idx].val, l, r);
        node[idx].tag = node[idx].tag + t;
    }

    void push_down(int l, int r, int u) {
        int mid = l + (r - l) / 2;
        apply(node[u].ls, node[u].tag, l, mid);
        apply(node[u].rs, node[u].tag, mid + 1, r);
        node[u].tag = Tag();
    }

    void mod(int l, int r, int u, const Tag& t, int ql, int qr) {
        if(ql <= l && r <= qr) {
            apply(u, t, l, r);
            return;
        }
        int mid = l + (r - l) / 2;
        if(node[u].ls == -1) {
            node[u].ls = new_node(l, mid);
            node[u].rs = new_node(mid + 1, r);
        }
        push_down(l, r, u);
        if(ql <= mid) {
            mod(l, mid, node[u].ls, t, ql, qr);
        }
        if(qr > mid) {
            mod(mid + 1, r, node[u].rs, t, ql, qr);
        }
        node[u].val = node[node[u].ls].val + node[node[u].rs].val;
    }

    Value query(int l, int r, int u, int ql, int qr) {
        if(u == -1) {
            return make_value(max(l, ql), min(r, qr));
        }
        if(ql <= l && r <= qr) {
            return node[u].val;
        }
        int mid = l + (r - l) / 2;
        Value res{};
        if(ql <= mid) {
            res = res + query(l, mid, node[u].ls, ql, qr);
        }
        if(qr > mid) {
            res = res + query(mid + 1, r, node[u].rs, ql, qr);
        }
        return node[u].tag.apply(res, max(l, ql), min(r, qr));
    }

    /*
     * 初始化闭区间 [l,r]; capacity 按 n * 2 * log V 预留
     */
    void init(int l, int r, int capacity) {
        le = l;
        re = r;
        node.assign(capacity, Node());
        ncnt = 0;
        root = new_node(l, r);
    }

    /*
```

```
     * 对闭区间 [l,r] 施加标记 t，按需开点
     */
    void mod(int l, int r, const Tag& t) {
        mod(le, re, root, t, l, r);
    }

    /*
     * 查询闭区间 [l,r] 的信息，查询不创建节点
     */
    Value query(int l, int r) {
        return query(le, re, root, l, r);
    }
};
```

注意

`init` 的 capacity 按 n * 2 * log V 预留，节点池用 `ncnt` 分配，不 `push_back`；缺失子节点按空信息处理。

3.3 标记持久化线段树

时间复杂度 $O(\log n)$

```
const int INVALID = -2147483648;

template <class Value, class Tag>
class SegmentTree {
public:
    class Node {
    public:
        int ls = INVALID, rs = INVALID;
        Value val{};
        Tag tag{};
    };

    vector<Node> node;
    int root = INVALID;
    int le = 0, re = 0;

    int new_node() {
        int idx = int(node.size());
        node.emplace_back();
        return idx;
    }

    /*
     * 用本节点标记和左右子节点信息重算本节点信息
     */
    void push_up(int u, int l, int r) {
        Value child{};
        if(l != r && node[u].ls != INVALID) {
            child = node[node[u].ls].val + node[node[u].rs].val;
        }
        node[u].val = node[u].tag.apply(child, l, r);
    }

    void mod(int u, int l, int r, int ql, int qr, const Tag& t) {
        if(ql <= l && r <= qr) {
            node[u].tag = node[u].tag + t;
            push_up(u, l, r);
            return;
        }
```

```
        int mid = l + (r - l) / 2;
        if(node[u].ls == INVALID) {
            node[u].ls = new_node();
            node[u].rs = new_node();
        }
        if(ql <= mid) {
            mod(node[u].ls, l, mid, ql, qr, t);
        }
        if(qr > mid) {
            mod(node[u].rs, mid + 1, r, ql, qr, t);
        }
        push_up(u, l, r);
    }

    Value query(int u, int l, int r, int ql, int qr) {
        if(u == INVALID) {
            return Value();
        }
        if(ql <= l && r <= qr) {
            return node[u].val;
        }
        int mid = l + (r - l) / 2;
        Value res{};
        if(ql <= mid) {
            res = res + query(node[u].ls, l, mid, ql, qr);
        }
        if(qr > mid) {
            res = res + query(node[u].rs, mid + 1, r, ql, qr);
        }
        return node[u].tag.apply(res, max(l, ql), min(r, qr));
    }

    /*
     * 初始化闭区间 [l,r]；ncnt 为节点池预留量
     */
    void init(int l, int r, int ncnt = 1) {
        le = l;
        re = r;
        node.clear();
        node.reserve(ncnt);
        root = new_node();
    }

    /*
     * 对闭区间 [l,r] 施加标记 t，标记不下传
     */
    void mod(int l, int r, const Tag& t) {
        mod(root, le, re, l, r, t);
    }

    /*
     * 查询闭区间 [l,r] 的信息
     */
    Value query(int l, int r) {
        return query(root, le, re, l, r);
    }
};
```

注意

标记不下传：push_up 时把本节点累积标记作用在左右儿子合并值上；这是标记持久化，不是懒惰标记。

3.4 吉司机线段树

时间复杂度 $O(\log^2 n)$ 均摊

3.4.1 P6242 【模板】线段树 3（区间最值操作、区间历史最值）

```
const int INVALID = INT_MIN;
const int64 INF = 0x3f3f3f3f3f3f3f3f;
const int N = 500005;
int n, w;

class Tag {
public:
    int64 add, hadd;
    int64 seadd, hseadd;

    Tag operator+(const Tag& other) const {
        Tag res = {
            add + other.add,
            max(hadd, add + other.hadd),
            seadd + other.seadd,
            max(hseadd, seadd + other.hseadd),
        };

        return res;
    }
};

class Data {
public:
    int64 maxi, semax;
    int64 hmaxi;
    int64 span, sum, mcnt;

    Data operator+(const Data& other) const {
        Data res = {
            max(maxi, other.maxi),
            INVALID,
            max(hmaxi, other.hmaxi),
            span + other.span,
            sum + other.sum,
            maxi > other.maxi ? mcnt : other.mcnt,
        };

        if(maxi == other.maxi) {
            res.semax = max(semax, other.semax);
            res.mcnt = mcnt + other.mcnt;
        } else {
            res.semax = max(min(maxi, other.maxi), max(semax, other.semax));
        }

        return res;

    }

    Data operator+(const Tag& tag) const {
        Data res = {
            maxi + tag.add,
            semax + tag.seadd,
            max(hmaxi, maxi + tag.hadd),
            span,
            sum + tag.add * mcnt + tag.seadd * (span - mcnt),
```

```
            mcnt,
        };
        return res;
    }

    void read(int idx) {
        int num;
        cin >> num;
        maxi = hmaxi = sum = num;
        semax = -INF;
        span = mcnt = 1;
    }
};

class Node {
public:
    int ls, rs; // Left Son, Right Son
    Data data;
    Tag tag;
};

class Condition {
public:
};

const Tag TAG_EP = {0, 0, 0, 0};

class SegmentTree {
private:
    static const int MAX_N = N;
    int size, le, re, opl, opr, root;
    Node node[MAX_N * 2];

    void build(int l, int r, int cur) {
        node[cur].tag = TAG_EP;
        if(l == r) {
            node[cur].ls = node[cur].rs = INVALID;
            node[cur].data.read(l);
            return;
        } else {
            int mid = (l + r) >> 1;
            node[cur].ls = cur + 1;
            node[cur].rs = cur + 2 * (mid - l + 1);
            build(l, mid, node[cur].ls);
            build(mid + 1, r, node[cur].rs);
            push_up(cur);
        }
    }

    void update(int cur, Tag tag) {
        node[cur].tag = node[cur].tag + tag;
        node[cur].data = node[cur].data + tag;
    }

    void push_down(int cur) {
        if(node[node[cur].ls].data.maxi == node[node[cur].rs].data.maxi) {
            update(node[cur].ls, node[cur].tag);
            update(node[cur].rs, node[cur].tag);
        } else if(node[node[cur].ls].data.maxi > node[node[cur].rs].data.maxi) {
            Tag se = node[cur].tag;
            se.add = se.seadd;
```

```
            se.hadd = se.hseadd;
            update(node[cur].ls, node[cur].tag);
            update(node[cur].rs, se);
        } else {
            Tag se = node[cur].tag;
            se.add = se.seadd;
            se.hadd = se.hseadd;
            update(node[cur].ls, se);
            update(node[cur].rs, node[cur].tag);
        }
        node[cur].tag = TAG_EP;
    }

    void push_up(int cur) {
        node[cur].data = node[node[cur].ls].data + node[node[cur].rs].data;
    }

    void mod(int l, int r, int cur, const Tag& x) {
        if(opl ⩽ l && r ⩽ opr) {
            update(cur, x);
            return;
        }
        int mid = (l + r) >> 1;
        push_down(cur);
        if(opl ⩽ l && mid ⩽ opr) {
            update(node[cur].ls, x);
        } else if(max(opl, l) ⩽ min(opr, mid)) {
            mod(l, mid, node[cur].ls, x);
        }
        if(opl ⩽ mid + 1 && r ⩽ opr) {
            update(node[cur].rs, x);
        } else if(max(opl, mid + 1) ⩽ min(opr, r)) {
            mod(mid + 1, r, node[cur].rs, x);
        }
        push_up(cur);
    }

    bool cupdate(int cur, int64 mini) {
        if(node[cur].data.maxi ⩽ mini) {
            return true;
        } else if(node[cur].data.semax < mini) {
            int64 g = max(0ll, node[cur].data.maxi - mini);
            node[cur].data.maxi -= g;
            node[cur].data.sum -= g * node[cur].data.mcnt;
            node[cur].tag.add -= g;
            return true;
        } else {
            return false;
        }
    }

    /*
     * 对闭区间 [l,r] 执行取 min 操作
     */
    void cmod(int l, int r, int cur, int64 mini) {
        if(opl ⩽ l && r ⩽ opr && cupdate(cur, mini)) {
            return;
        }
        int mid = (l + r) >> 1;
        push_down(cur);
        if(opl ⩽ l && mid ⩽ opr && cupdate(node[cur].ls, mini)) {
```

```
        } else if(max(opl, l) ⩽ min(opr, mid)) {
            cmod(l, mid, node[cur].ls, mini);
        }
        if(opl ⩽ mid + 1 && r ⩽ opr && cupdate(node[cur].rs, mini)) {
        } else if(max(opl, mid + 1) ⩽ min(opr, r)) {
            cmod(mid + 1, r, node[cur].rs, mini);
        }
        push_up(cur);
    }

    Data query(int l, int r, int cur) {
        if(opl ⩽ l && r ⩽ opr) {
            return node[cur].data;
        }
        int mid = (l + r) >> 1;
        push_down(cur);
        Data resl, resr;
        int state = 3;
        if(opl ⩽ l && mid ⩽ opr) {
            resl = node[node[cur].ls].data;
        } else if(max(opl, l) ⩽ min(opr, mid)) {
            resl = query(l, mid, node[cur].ls);
        } else {
            state ^= 2;
        }
        if(opl ⩽ mid + 1 && r ⩽ opr) {
            resr = node[node[cur].rs].data;
        } else if(max(opl, mid + 1) ⩽ min(opr, r)) {
            resr = query(mid + 1, r, node[cur].rs);
        } else {
            state ^= 1;
        }

        switch(state) {
        case(3):
            return (resl + resr);
        case(2):
            return resl;
        case(1):
            return resr;
        }
        return {};
    }

public:
    /*
     * 建立闭区间 [l,r] 的线段树
     */
    void build(int l, int r) {
        root = 0;
        le = l;
        re = r;
        size = r - l + 1;
        build(l, r, root);
    }

    /*
     * 对闭区间 [l,r] 施加加法标记 x
     */
    void mod(int l, int r, const Tag& x) {
        opl = l;
```

```
        opr = r;
        mod(le, re, root, x);
    }

    /*
     * 对闭区间 [l,r] 执行取 min 操作
     */
    void cmod(int l, int r, int64 mini) {
        opl = l;
        opr = r;
        cmod(le, re, root, mini);
    }

    /*
     * 查询闭区间 [l,r] 的维护信息
     */
    Data query(int l, int r) {
        opl = l;
        opr = r;
        return query(le, re, root);
    }
};

SegmentTree segmentTree;
```

3.4.2 U180387 CTSN loves segment tree

```
const int INVALID = INT_MIN;
const int64 INF = 0x3f3f3f3f3f3f3f3f;
const int N = 300005;
int n, w;
int a[N], b[N];

class Tag {
public:
    int64 Aadd;
    int64 seAadd;
    int64 Badd;
    int64 seBadd;

    Tag operator+(const Tag& other) const {
        Tag res = {
            Aadd + other.Aadd,
            seAadd + other.seAadd,
            Badd + other.Badd,
            seBadd + other.seBadd,
        };

        return res;
    }
};

class CTag {
public:
    int64 miniA;
    int64 miniB;
};

class Data {
public:
    int64 Amaxi, seAmax;
    int64 Bmaxi, seBmax;
```

```
    int64 c[4];
    // 11: Amaxi Bmaxi

    Data operator+(const Data& other) const {
        Data res = {
            max(Amaxi, other.Amaxi),
            Amaxi == other.Amaxi ? max(seAmax, other.seAmax) : max(min(Amaxi, other.Amaxi), max(seAmax,
            ↪  other.seAmax)),
            max(Bmaxi, other.Bmaxi),
            Bmaxi == other.Bmaxi ? max(seBmax, other.seBmax) : max(min(Bmaxi, other.Bmaxi), max(seBmax,
            ↪  other.seBmax)),
            -INF,
            -INF,
            -INF,
            -INF};

        if(Amaxi == res.Amaxi) {
            if(Bmaxi == res.Bmaxi) {
                for(int i = 0; i < 4; i++) {
                    res.c[i] = max(res.c[i], c[i]);
                }
            } else {
                res.c[2] = max(res.c[2], c[3]);
                res.c[2] = max(res.c[2], c[2]);
                res.c[0] = max(res.c[0], c[1]);
                res.c[0] = max(res.c[0], c[0]);
            }
        } else {
            if(Bmaxi == res.Bmaxi) {
                res.c[1] = max(res.c[1], c[3]);
                res.c[1] = max(res.c[1], c[1]);
                res.c[0] = max(res.c[0], c[2]);
                res.c[0] = max(res.c[0], c[0]);
            } else {
                for(int i = 0; i < 4; i++) {
                    res.c[0] = max(res.c[0], c[i]);
                }
            }
        }

        if(other.Amaxi == res.Amaxi) {
            if(other.Bmaxi == res.Bmaxi) {
                for(int i = 0; i < 4; i++) {
                    res.c[i] = max(res.c[i], other.c[i]);
                }
            } else {
                res.c[2] = max(res.c[2], other.c[3]);
                res.c[2] = max(res.c[2], other.c[2]);
                res.c[0] = max(res.c[0], other.c[1]);
                res.c[0] = max(res.c[0], other.c[0]);
            }
        } else {
            if(other.Bmaxi == res.Bmaxi) {
                res.c[1] = max(res.c[1], other.c[3]);
                res.c[1] = max(res.c[1], other.c[1]);
                res.c[0] = max(res.c[0], other.c[2]);
                res.c[0] = max(res.c[0], other.c[0]);
            } else {
                for(int i = 0; i < 4; i++) {
                    res.c[0] = max(res.c[0], other.c[i]);
                }
```

```
            }
        }

        return res;
    }

    Data operator+(const Tag& tag) const {
        Data res = {
            Amaxi + tag.Aadd,
            seAmax + tag.seAadd,
            Bmaxi + tag.Badd,
            seBmax + tag.seBadd,
            c[0] == -INF ? -INF : c[0] + tag.seAadd + tag.seBadd,
            c[1] == -INF ? -INF : c[1] + tag.seAadd + tag.Badd,
            c[2] == -INF ? -INF : c[2] + tag.Aadd + tag.seBadd,
            c[3] == -INF ? -INF : c[3] + tag.Aadd + tag.Badd};
        return res;
    }

    void read(int idx) {
        Amaxi = a[idx];
        Bmaxi = b[idx];
        seAmax = seBmax = -INF;
        c[3] = Amaxi + Bmaxi;
        c[0] = c[1] = c[2] = -INF;
    }
};

class Node {
public:
    int ls, rs; // Left Son, Right Son
    Data data;
    Tag tag;
};

class Condition {
public:
};

const Tag TAG_EP = {0, 0, 0, 0};

class SegmentTree {
private:
    static const int MAX_N = N;
    int size, le, re, opl, opr, root;
    /*
     * le, re: Left Edge, Right Edge
     * opl, opr: Operation l, r
     * root: default 0
     */
    Node node[MAX_N * 2];

    void build(int l, int r, int cur) {
        node[cur].tag = TAG_EP;
        if(l == r) {
            node[cur].ls = node[cur].rs = INVALID;
            node[cur].data.read(l);
            return;
        } else {
            int mid = (l + r) >> 1;
            node[cur].ls = cur + 1;
```

```
            node[cur].rs = cur + 2 * (mid - l + 1);
            build(l, mid, node[cur].ls);
            build(mid + 1, r, node[cur].rs);
            push_up(cur);
        }
    }

    void update(int cur, Tag tag) {
        node[cur].tag = node[cur].tag + tag;
        node[cur].data = node[cur].data + tag;
    }

    void push_down(int cur) {
        if(node[node[cur].ls].data.Amaxi == node[node[cur].rs].data.Amaxi && node[node[cur].ls].data.Bmaxi ==
        ↪  node[node[cur].rs].data.Bmaxi) {
            update(node[cur].ls, node[cur].tag);
            update(node[cur].rs, node[cur].tag);
        } else {
            Tag lt = node[cur].tag, rt = node[cur].tag;
            if(node[node[cur].ls].data.Amaxi ≠ node[node[cur].rs].data.Amaxi) {
                if(node[node[cur].ls].data.Amaxi < node[node[cur].rs].data.Amaxi) {
                    lt.Aadd = lt.seAadd;
                } else {
                    rt.Aadd = rt.seAadd;
                }
            }

            if(node[node[cur].ls].data.Bmaxi ≠ node[node[cur].rs].data.Bmaxi) {
                if(node[node[cur].ls].data.Bmaxi < node[node[cur].rs].data.Bmaxi) {
                    lt.Badd = lt.seBadd;
                } else {
                    rt.Badd = rt.seBadd;
                }
            }

            update(node[cur].ls, lt);
            update(node[cur].rs, rt);
        }
        node[cur].tag = TAG_EP;
    }

    void push_up(int cur) {
        node[cur].data = node[node[cur].ls].data + node[node[cur].rs].data;
    }

    void mod(int l, int r, int cur, const Tag& x) {
        if(opl ≤ l && r ≤ opr) {
            update(cur, x);
            return;
        }
        int mid = (l + r) >> 1;
        push_down(cur);
        if(opl ≤ l && mid ≤ opr) {
            update(node[cur].ls, x);
        } else if(max(opl, l) ≤ min(opr, mid)) {
            mod(l, mid, node[cur].ls, x);
        }
        if(opl ≤ mid + 1 && r ≤ opr) {
            update(node[cur].rs, x);
        } else if(max(opl, mid + 1) ≤ min(opr, r)) {
            mod(mid + 1, r, node[cur].rs, x);
```

```
        }
        push_up(cur);
    }

    bool cupdate(int cur, const CTag& mini) {
        if(node[cur].data.Amaxi ⩽ mini.miniA && node[cur].data.Bmaxi ⩽ mini.miniB) {
            return true;
        } else if(node[cur].data.seAmax < mini.miniA && node[cur].data.seBmax < mini.miniB) {
            int64 gA = max(0ll, node[cur].data.Amaxi - mini.miniA);
            int64 gB = max(0ll, node[cur].data.Bmaxi - mini.miniB);
            node[cur].data.Amaxi -= gA;
            node[cur].tag.Aadd -= gA;
            if(node[cur].data.c[3] ≠ -INF) {
                node[cur].data.c[3] -= gA;
            }
            if(node[cur].data.c[2] ≠ -INF) {
                node[cur].data.c[2] -= gA;
            }

            node[cur].data.Bmaxi -= gB;
            node[cur].tag.Badd -= gB;
            if(node[cur].data.c[3] ≠ -INF) {
                node[cur].data.c[3] -= gB;
            }
            if(node[cur].data.c[1] ≠ -INF) {
                node[cur].data.c[1] -= gB;
            }
            return true;
        } else {
            return false;
        }
    }

    /*
     * 对闭区间 [l,r] 执行取 min 操作
     */
    void cmod(int l, int r, int cur, const CTag& mini) {
        if(opl ⩽ l && r ⩽ opr && cupdate(cur, mini)) {
            return;
        }
        int mid = (l + r) >> 1;
        push_down(cur);
        if(opl ⩽ l && mid ⩽ opr && cupdate(node[cur].ls, mini)) {
        } else if(max(opl, l) ⩽ min(opr, mid)) {
            cmod(l, mid, node[cur].ls, mini);
        }
        if(opl ⩽ mid + 1 && r ⩽ opr && cupdate(node[cur].rs, mini)) {
        } else if(max(opl, mid + 1) ⩽ min(opr, r)) {
            cmod(mid + 1, r, node[cur].rs, mini);
        }
        push_up(cur);
    }

    Data query(int l, int r, int cur) {
        if(opl ⩽ l && r ⩽ opr) {
            return node[cur].data;
        }
        int mid = (l + r) >> 1;
        push_down(cur);
        Data resl, resr;
        int state = 3;
```

```
277         if(opl ⩽ l && mid ⩽ opr) {
278             resl = node[node[cur].ls].data;
279         } else if(max(opl, l) ⩽ min(opr, mid)) {
280             resl = query(l, mid, node[cur].ls);
281         } else {
282             state ^= 2;
283         }
284         if(opl ⩽ mid + 1 && r ⩽ opr) {
285             resr = node[node[cur].rs].data;
286         } else if(max(opl, mid + 1) ⩽ min(opr, r)) {
287             resr = query(mid + 1, r, node[cur].rs);
288         } else {
289             state ^= 1;
290         }
291 
292         switch(state) {
293         case(3):
294             return (resl + resr);
295         case(2):
296             return resl;
297         case(1):
298             return resr;
299         }
300         return {};
301     }
302 
303 public:
304     /*
305      * 建立闭区间 [l,r] 的线段树
306      */
307     void build(int l, int r) {
308         /*
309          * 极端情况下，线段树可能只有一个节点
310          * 在这种情况下上述写法会报错
311          */
312         root = 0;
313         le = l;
314         re = r;
315         size = r - l + 1;
316         build(l, r, root);
317     }
318 
319     /*
320      * 对闭区间 [l,r] 施加加法标记 x
321      */
322     void mod(int l, int r, const Tag& x) {
323         opl = l;
324         opr = r;
325         mod(le, re, root, x);
326     }
327 
328     /*
329      * 对闭区间 [l,r] 执行取 min 操作
330      */
331     void cmod(int l, int r, const CTag& mini) {
332         opl = l;
333         opr = r;
334         cmod(le, re, root, mini);
335     }
336 
337     /*
```

```
     * 查询闭区间 [l,r] 的维护信息
     */
    Data query(int l, int r) {
        opl = l;
        opr = r;
        return query(le, re, root);
    }
};

SegmentTree segmentTree;
```

注意

吉司机线段树：push_down 按左右最大值分配全量/非最大值标记；semax 需要 max_second < x < max_first。

3.5 李超线段树

时间复杂度 $O(\log^2 n)$

3.5.1 P4097 【模板】李超线段树

```
const double FINF = 1e100;
const double EPS = 1e-9;
const int MODER1 = 39989;
const int MODER2 = 1000000000;

int n;
int last_ans = 0;

class Line {
public:
    double k, b;
    int idx;

    inline double operator()(double x) const {
        return k * x + b;
    }
};

class LiChaoSegmentTreeNode {
public:
    int range_l, range_r;
    int l, r;
    Line g;
};

class LiChaoSegmentTree {
private:
    static const int NO_TAG = 0xffffffff;
    static const int NO_SON = -1;
    static const int MAX_N = 40001;
    int size;
    int root = 0;
    int node_used = 1; // 0 for root
    int op_range_l, op_range_r;
    int ans_idx;
    double ans;
    LiChaoSegmentTreeNode node[MAX_N * 2];

    int getNode() {
        return node_used++;
    }
```

```
42
43      void build(int range_l, int range_r, int cur) {
44          node[cur].range_l = range_l;
45          node[cur].range_r = range_r;
46          node[cur].l = NO_SON;
47          node[cur].r = NO_SON;
48          node[cur].g = {FINF, -FINF, -1};
49          if(range_r == range_l) {
50              return;
51          } else {
52              int mid = (range_r + range_l) >> 1;
53              node[cur].l = getNode();
54              node[cur].r = getNode();
55              build(range_l, mid, node[cur].l);
56              build(mid + 1, range_r, node[cur].r);
57          }
58      }
59
60      void update(int cur, Line f) {
61          if(cur == NO_SON) {
62              return;
63          }
64          if(node[cur].g.idx == -1) {
65              node[cur].g = f;
66          } else {
67
68              int mid = (node[cur].range_r + node[cur].range_l) >> 1;
69              double mid_f, mid_g, l_f, l_g, r_f, r_g;
70              mid_f = f(mid);
71              mid_g = node[cur].g(mid);
72
73              if(mid_f - mid_g > EPS || abs(mid_f - mid_g) < EPS && node[cur].g.idx > f.idx) {
74                  swap(node[cur].g, f);
75              }
76
77              if(node[cur].range_l == node[cur].range_r) {
78                  return;
79              }
80              if(f.k == node[cur].g.k && f.b == node[cur].g.b) {
81                  return;
82              }
83              l_f = f(node[cur].range_l);
84              r_f = f(node[cur].range_r);
85              l_g = node[cur].g(node[cur].range_l);
86              r_g = node[cur].g(node[cur].range_r);
87              if(r_f - r_g > EPS || abs(r_f - r_g) < EPS) {
88                  update(node[cur].r, f);
89              }
90              if(l_f - l_g > EPS || abs(l_f - l_g) < EPS) {
91                  update(node[cur].l, f);
92              }
93          }
94      }
95
96      void add_line(int cur, Line f) {
97          if(cur == NO_SON) {
98              return;
99          }
100         if(max(op_range_l, node[cur].range_l) ⩽ min(op_range_r, node[cur].range_r)) {
101             if(op_range_l ⩽ node[cur].range_l && node[cur].range_r ⩽ op_range_r) {
102                 update(cur, f);
```

```
            } else {
                add_line(node[cur].l, f);
                add_line(node[cur].r, f);
            }
        }
    }

    void query(int cur, int x) {
        if(cur == NO_SON) {
            return;
        }
        if(node[cur].range_l <= x && x <= node[cur].range_r) {
            if(node[cur].g.idx != -1) {
                double y_x;
                y_x = node[cur].g(x);
                if(y_x - ans > EPS || abs(y_x - ans) < EPS && ans_idx > node[cur].g.idx) {
                    ans = y_x;
                    ans_idx = node[cur].g.idx;
                }
            }
            query(node[cur].l, x);
            query(node[cur].r, x);
        }
    }

public:
    /*
     * 初始化横坐标闭区间 [range_l,range_r]
     */
    void build(int range_l, int range_r) {
        root = 0;
        node_used = 1;
        build(range_l, range_r, root);
        size = range_r - range_l + 1;
    }

    /*
     * 插入端点为 (x1,y1)、(x2,y2) 的线段，编号为 idx
     */
    void add_line(int x1, int y1, int x2, int y2, int idx) {
        op_range_l = min(x1, x2);
        op_range_r = max(x1, x2);
        Line f;
        if(x1 == x2) {
            f = {0., (double)(max(y2, y1)), idx};
        } else {
            double k = ((double)(y2 - y1)) / (x2 - x1);
            f = {k, y1 - k * (x1), idx};
        }
        add_line(root, f);
    }

    /*
     * 返回横坐标 x 处最高线段的编号，高度相同取编号较小者
     */
    int query(int x) {
        ans = -FINF;
        ans_idx = -1;
        query(root, x);
        if(ans_idx == -1) {
            return 0;
```

```
        }
        return ans_idx;
    }
};

LiChaoSegmentTree segmentTree;
```

3.5.2 离散坐标：区间最小值

用于在线段覆盖的离散坐标上查询区间最小值，与上面的单点最大线段编号版本独立使用。`a[1..n]` 非递减，允许重复坐标；操作参数是下标，满足 `1 <= l <= r <= n`。初始水平线也参与取最小值；`k*x+b` 的结果需在 int64 范围内，中间乘法使用 int128。建树 $O(n)$，区间加线段 $O(\log^2 n)$，区间查询 $O(\log n)$，空间 $O(n)$。

```
class Line {
public:
    int64 k = 0, b = 0;
    int64 operator()(int64 x) const {
        return int64(int128(k) * x + b);
    }
};

class LiChaoSegmentTree {
public:
    class Node {
    public:
        Line g;
        int64 minimum;
    };
    int n = 0;
    vector<int64> x;
    vector<Node> node;

    void push_up(int u, int l, int r) {
        node[u].minimum = min(node[u].g(x[l]), node[u].g(x[r]));
        if(l < r) {
            node[u].minimum = min(node[u].minimum, min(node[u * 2].minimum, node[u * 2 + 1].minimum));
        }
    }
    void update(int u, int l, int r, Line f) {
        int mid = (l + r) / 2;
        if(f(x[mid]) < node[u].g(x[mid])) {
            swap(f, node[u].g);
        }
        if(l < r) {
            if(f(x[l]) < node[u].g(x[l])) {
                update(u * 2, l, mid, f);
            } else if(f(x[r]) < node[u].g(x[r])) {
                update(u * 2 + 1, mid + 1, r, f);
            }
        }
        push_up(u, l, r);
    }
    void add_line(int u, int l, int r, int ql, int qr, Line f) {
        if(ql <= l && r <= qr) {
            update(u, l, r, f);
            return;
        }
        int mid = (l + r) / 2;
        if(ql <= mid) {
            add_line(u * 2, l, mid, ql, qr, f);
        }
```

```
        if(qr > mid) {
            add_line(u * 2 + 1, mid + 1, r, ql, qr, f);
        }
        push_up(u, l, r);
    }
    int64 range_query(int u, int l, int r, int ql, int qr) const {
        if(ql <= l && r <= qr) {
            return node[u].minimum;
        }
        int64 res = min(node[u].g(x[max(l, ql)]), node[u].g(x[min(r, qr)]));
        int mid = (l + r) / 2;
        if(ql <= mid) {
            res = min(res, range_query(u * 2, l, mid, ql, qr));
        }
        if(qr > mid) {
            res = min(res, range_query(u * 2 + 1, mid + 1, r, ql, qr));
        }
        return res;
    }
    /*
     * 由非递减横坐标 a[1..n] 建树，以水平线 value 初始化
     */
    void build(const vector<int64>& a, int64 value) {
        x = a;
        n = int(a.size()) - 1;
        node.assign(4 * n + 4, Node{{0, value}, value});
    }
    /*
     * 在下标闭区间 [l,r] 加入直线 kx+b
     */
    void add_line(int l, int r, int64 k, int64 b) {
        add_line(1, 1, n, l, r, {k, b});
    }
    /*
     * 查询下标闭区间 [l,r] 内的最小值
     */
    int64 range_query(int l, int r) const {
        return range_query(1, 1, n, l, r);
    }
};
```

注意

用 double 比较，EPS 过小时极端坐标可能选错线段；需要精确时改成 int128 交叉相乘。

3.6 线段树合并

时间复杂度 $O(n \log n)$

本例只做**单点加、整树查询、破坏性合并**；mod 的 l、r 必须相等，不能用于区间加。合并后子树节点可能被父树复用，子树答案在 dfs3 返回前保存，之后不要再查询已合并的子树。固定节点池需按 O((n+m) log V) 估算；深链上的 DFS 仍需按赛题栈限制改成迭代。

3.6.1 P4556 [Vani 有约会] 雨天的尾巴 /【模板】线段树合并

```
const int INF = 0x3f3f3f3f;
const int INVALID = -2147483648;
const int N = 100005;
int n, m, moder;

class Tag {
public:
```

```
    int add;
    Tag operator+(const Tag& other) {
        return {add + other.add};
    }
};

class Data {
public:
    int maxi, idx;

    Data operator+(const Data& other) {
        if(maxi ⩾ other.maxi) {
            return *this;
        } else {
            return other;
        }
    }

    void read(int i) {
        idx = i;
        cin >> maxi;
    }
};

inline Data merge(int l, int r, const Data& d, const Tag& tag) {
    return {
        d.maxi + tag.add,
        d.idx};
}

class SegmentNode {
public:
    int ls, rs; // Left Son, Right Son
    Data data;
    Tag tag;
};

const Data DATA_EP = {0, 0};
const Tag TAG_EP = {0};

SegmentNode stNode[200 * N];
int stNode_cnt;

class SegmentTree {
private:
    static const int MAX_N = N;
    int size, le, re, opl, opr, root;

    int getNode(int idx) {
        stNode[stNode_cnt] = {
            INVALID, INVALID, {0, idx}, {0}};
        return stNode_cnt++;
    }

    void push_up(int cur) {
        if(stNode[cur].ls ≠ INVALID && stNode[cur].rs ≠ INVALID) {
            stNode[cur].data = stNode[stNode[cur].ls].data + stNode[stNode[cur].rs].data;
        } else if(stNode[cur].ls ≠ INVALID) {
            stNode[cur].data = stNode[stNode[cur].ls].data;
        } else if(stNode[cur].rs ≠ INVALID) {
            stNode[cur].data = stNode[stNode[cur].rs].data;
```

```
        } else {
            return;
        }
    }

    void mod(int l, int r, int cur, const Tag& x) {
        if(opl ⩽ l && r ⩽ opr) {
            stNode[cur].data = merge(l, r, stNode[cur].data, x);
            stNode[cur].tag = stNode[cur].tag + x;
        } else {
            int mid = (l + r) >> 1;
            if(stNode[cur].ls == INVALID) {
                stNode[cur].ls = getNode(l);
                stNode[cur].rs = getNode(mid + 1);
            }
            if(max(opl, l) ⩽ min(opr, mid)) {
                mod(l, mid, stNode[cur].ls, x);
            }
            if(max(opl, mid + 1) ⩽ min(opr, r)) {
                mod(mid + 1, r, stNode[cur].rs, x);
            }
            push_up(cur);
        }
    }

    Data query(int l, int r, int cur) {
        if(cur == INVALID) {
            return DATA_EP;
        }
        if(opl ⩽ l && r ⩽ opr) {
            return stNode[cur].data;
        } else if(cur ≠ INVALID) {
            int mid = (l + r) >> 1;
            bool dual = max(opl, l) ⩽ min(opr, mid) && max(opl, mid + 1) ⩽ min(opr, r);

            Data res;
            if(dual) {
                res = query(l, mid, stNode[cur].ls) +
                      query(mid + 1, r, stNode[cur].rs);
            } else if(max(opl, l) ⩽ min(opr, mid)) {
                res = query(l, mid, stNode[cur].ls);
            } else {
                res = query(mid + 1, r, stNode[cur].rs);
            }
            return merge(max(opl, l), min(opr, r), res, stNode[cur].tag);
        } else {
            return DATA_EP;
        }
    }

    int dfsMerge(int l, int r, int curS, int curO, const SegmentTree& other) {
        if(curS == INVALID && curO == INVALID) {
            return INVALID;
        } else if(curS ≠ INVALID && curO ≠ INVALID) {
            if(l == r) {
                stNode[curS].data.maxi += stNode[curO].data.maxi;
            } else {
                int mid = (l + r) >> 1;
                stNode[curS].ls = dfsMerge(l, mid, stNode[curS].ls, stNode[curO].ls, other);
                stNode[curS].rs = dfsMerge(mid + 1, r, stNode[curS].rs, stNode[curO].rs, other);
                push_up(curS);
```

```
            }
            return curS;
        } else if(curS != INVALID) {
            return curS;
        } else {
            return curO;
        }
    }

public:
    /*
     * 建立值域闭区间 [l,r] 的空树
     */
    void build(int l, int r) {
        root = getNode(l);
        le = l;
        re = r;
        size = r - l + 1;
    }

    /*
     * 更新闭区间 [l,r]，此例调用时 l=r
     */
    void mod(int l, int r, const Tag& x) {
        opl = l;
        opr = r;
        mod(le, re, root, x);
    }

    /*
     * 查询闭区间 [l,r] 的维护信息
     */
    Data query(int l, int r) {
        opl = l;
        opr = r;
        return query(le, re, root);
    }

    void operator+=(const SegmentTree& other) {
        root = dfsMerge(le, re, root, other.root, other);
    }
};

class HLDTreeNode {
public:
    vector<int> son;
    int dfn, fa, w, link_head, d;
    int heavy, sub_dfn;
    int ans;
    SegmentTree st;
};

class HLDTree {
private:
    static const int MAX_N = N;
    int dfn = 1;
    HLDTreeNode hldNode[MAX_N];

    void dfs1(int cur, int fa, int d) {
        hldNode[cur].fa = fa;
        hldNode[cur].w = 1;
```

```
        hldNode[cur].d = d;
        hldNode[cur].st.build(1, N - 1);
        int heavy = -1, w = 0;
        for(int son : hldNode[cur].son) {
            if(son != fa) {
                dfs1(son, cur, d + 1);
                hldNode[cur].w += hldNode[son].w;
                if(hldNode[son].w > w) {
                    w = hldNode[son].w;
                    heavy = son;
                }
            }
        }
        if(heavy > 0) {
            hldNode[cur].heavy = heavy;
        }
    }

    void dfs2(int cur, int fa, int link_head) {
        inv_dfn[dfn] = cur;
        hldNode[cur].sub_dfn = dfn;
        hldNode[cur].dfn = dfn++;
        hldNode[cur].link_head = link_head;
        if(hldNode[cur].heavy > 0) {
            dfs2(hldNode[cur].heavy, cur, link_head);
            hldNode[cur].sub_dfn = hldNode[hldNode[cur].heavy].sub_dfn;
        }
        for(int son : hldNode[cur].son) {
            if(son != fa && son != hldNode[cur].heavy) {
                dfs2(son, cur, son);
                hldNode[cur].sub_dfn = hldNode[son].sub_dfn;
            }
        }
    }

    int find_lca(int x, int y) {
        while(hldNode[x].link_head != hldNode[y].link_head) { // 先找 LCA 和计算 st 之间的距离
            if(hldNode[hldNode[x].link_head].d < hldNode[hldNode[y].link_head].d) {
                swap(x, y);
            }
            x = hldNode[hldNode[x].link_head].fa;
        }
        return hldNode[x].d < hldNode[y].d ? x : y;
    }

public:
    int root = 1;
    int inv_dfn[MAX_N];

    /*
     * 添加无向边 (u,v)
     */
    void add_edge(int u, int v) {
        hldNode[u].son.push_back(v);
        hldNode[v].son.push_back(u);
    }

    /*
     * 以 tree_root 为根预处理树链信息
     */
    void build(int tree_root) {
```

```
        root = tree_root;
        dfs1(root, 0, 1);
        dfs2(root, 0, root);
    }

    /*
     * 在 x 到 y 的路径上加入颜色 z，使用树上差分
     */
    void path_add(int x, int y, int z) {

        int lca = find_lca(x, y);
        hldNode[x].st.mod(z, z, {1});
        hldNode[y].st.mod(z, z, {1});
        hldNode[lca].st.mod(z, z, {-1});
        if(hldNode[lca].fa) {
            hldNode[hldNode[lca].fa].st.mod(z, z, {-1});
        }
    }

    /*
     * 自底向上合并子树线段树，合并到父亲前保存当前点答案
     */
    void dfs3(int cur) {
        for(int son : hldNode[cur].son) {
            if(son != hldNode[cur].fa) {
                dfs3(son);
                hldNode[cur].st += hldNode[son].st;
            }
        }
        Data res = hldNode[cur].st.query(1, N - 1);
        hldNode[cur].ans = res.maxi == 0 ? 0 : res.idx;
    }

    int query(int cur) {
        return hldNode[cur].ans;
    }

};

HLDTree hldTree;
```

注意

节点池按 `n + 4m log V` 估算；链状树需注意递归深度。

3.7 可持久化线段树

时间复杂度 $O(\log n)$

```
class SegmentTree {
public:
    class Node {
    public:
        int ls = 0, rs = 0, cnt = 0;
    };
    vector<Node> node;
    vector<int> root;
    vector<int64> value;

    int mod(int u, int l, int r, int pos) {
        int v = int(node.size());
```

```
        node.push_back(node[u]);
        node[v].cnt++;
        if(l != r) {
            int mid = l + (r - l) / 2;
            if(pos <= mid) {
                node[v].ls = mod(node[u].ls, l, mid, pos);
            } else {
                node[v].rs = mod(node[u].rs, mid + 1, r, pos);
            }
        }
        return v;
    }
    /*
     * 将 a[1..n] 离散化并建立前缀版本
     */
    void build(const vector<int64>& a) {
        int n = int(a.size()) - 1;
        value.assign(a.begin() + 1, a.end());
        sort(value.begin(), value.end());
        value.erase(unique(value.begin(), value.end()), value.end());
        node.assign(1, Node());
        root.assign(n + 1, 0);
        int logn = 0;
        while((1LL << logn) < int(value.size())) {
            logn++;
        }
        node.reserve(1 + size_t(n) * (logn + 1));
        for(int i = 1; i <= n; i++) {
            int pos = lower_bound(value.begin(), value.end(), a[i]) - value.begin();
            root[i] = mod(root[i - 1], 0, int(value.size()) - 1, pos);
        }
    }
    /*
     * 查询闭区间 [l,r] 中第 k 小的原值，k 从 1 开始
     */
    int64 query(int l, int r, int k) const {
        int u = root[r], v = root[l - 1], le = 0, re = int(value.size()) - 1;
        while(le != re) {
            int mid = le + (re - le) / 2;
            int cnt = node[node[u].ls].cnt - node[node[v].ls].cnt;
            if(k <= cnt) {
                u = node[u].ls;
                v = node[v].ls;
                re = mid;
            } else {
                k -= cnt;
                u = node[u].rs;
                v = node[v].rs;
                le = mid + 1;
            }
        }
        return value[le];
    }
};
```

3.7.1 P3293 [SCOI2016] 美味

```
const int64 MAX_H = 18;
const int N = 200005;
const int M = 100005;
const int64 INVALID = 0;
```

```

class Tag {
public:
    int add;
    Tag operator+(const Tag& other) {
        return {add + other.add};
    }
};

class Data {
public:
    int64 sum;
    Data operator+(const Data& other) {
        return {sum + other.sum};
    }
};

inline Data merge(int64 l, int64 r, const Data& d, const Tag& tag) {
    return {d.sum + (r - l + 1) * tag.add};
}

class Node {
public:
    int64 ls, rs;
    Data data;
    Tag tag;
};

const Data DATA_EP = {0};
const Tag TAG_EP = {0};

Node seg_node[N * MAX_H];
int node_cnt = 1;

inline int getNode() {
    return node_cnt++;
}

class SegmentTree {
public:
    static const int64 MAX_N = N;
    int64 size, le, re, opl, opr;
    Node* node = seg_node;
    int root[N];

    void push_down(int64 l, int64 r, int64 cur) {
        Node& ls = node[node[cur].ls];
        Node& rs = node[node[cur].rs];
        int64 mid = (l + r) >> 1;
        ls.data = merge(l, mid, ls.data, node[cur].tag);
        ls.tag = ls.tag + node[cur].tag;
        rs.data = merge(mid + 1, r, rs.data, node[cur].tag);
        rs.tag = rs.tag + node[cur].tag;
        node[cur].tag = TAG_EP;
    }

    void push_up(int64 cur) {
        node[cur].data = node[node[cur].ls].data + node[node[cur].rs].data;
    }

    void mod(int64 l, int64 r, int64 cur, const int64 lst, const Tag& x) {
```

```
        if(opl ⩽ l && r ⩽ opr) {
            node[cur].data = merge(l, r, node[lst].data, x);
            node[cur].tag = node[lst].tag + x;
        } else {
            int64 mid = (l + r) >> 1;
            push_down(l, r, cur);
            if(max(opl, l) ⩽ min(opr, mid)) {
                node[cur].ls = getNode();
                mod(l, mid, node[cur].ls, node[lst].ls, x);
            } else {
                node[cur].ls = node[lst].ls;
            }
            if(max(opl, mid + 1) ⩽ min(opr, r)) {
                node[cur].rs = getNode();
                mod(mid + 1, r, node[cur].rs, node[lst].rs, x);
            } else {
                node[cur].rs = node[lst].rs;
            }
            push_up(cur);
        }
    }

    Data query(int64 l, int64 r, int64 cur, const int lst) {
        if(cur ≠ INVALID) {
            if(opl ⩽ l && r ⩽ opr) {
                return {node[cur].data.sum - node[lst].data.sum};
            } else {
                int64 mid = (l + r) >> 1;
                bool dual = max(opl, l) ⩽ min(opr, mid) && max(opl, mid + 1) ⩽ min(opr, r);

                Data res;
                if(dual) {
                    res = query(l, mid, node[cur].ls, node[lst].ls) +
                          query(mid + 1, r, node[cur].rs, node[lst].rs);
                } else if(max(opl, l) ⩽ min(opr, mid)) {
                    res = query(l, mid, node[cur].ls, node[lst].ls);
                } else {
                    res = query(mid + 1, r, node[cur].rs, node[lst].rs);
                }
                return merge(max(opl, l), min(opr, r), res, node[cur].tag);
            }
        } else {
            return DATA_EP;
        }
    }

    /*
     * 初始化值域闭区间 [l,r]
     */
    void build(int64 l, int64 r) {
        le = l;
        re = r;
        size = r - l + 1;
    }

    /*
     * 基于版本 ver 修改闭区间 [l,r]，写入版本 cur
     */
    void mod_from(int cur, int ver, int64 l, int64 r, const Tag& x) {
        opl = l;
        opr = r;
```

```
        if(!root[cur]) {
            root[cur] = getNode();
        }
        mod(le, re, root[cur], root[ver], x);
    }

    /*
     * 查询版本 cur 减去版本 ver 后的闭区间 [l,r] 信息
     */
    Data query_from(int cur, int ver, int64 l, int64 r) {
        opl = l;
        opr = r;
        return query(le, re, root[cur], root[ver]);
    }
};

SegmentTree sgt;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, a;
    cin >> n >> m;
    sgt.build(0, M - 1);
    for(int i = 1; i <= n; i++) {
        cin >> a;
        sgt.mod_from(i, i - 1, a, a, {1});
    }

    int b, x, vl, vr;
    int l, r, sum;
    for(int i = 1; i <= m; i++) {
        cin >> b >> x >> vl >> vr;
        sum = 0;
        for(int j = 1; j <= MAX_H; j++) {
            if((1 << (MAX_H - j)) & b) {
                int raw_l = sum - x;
                int raw_r = sum + (1 << (MAX_H - j)) - x - 1;
                Data res = DATA_EP;
                if(raw_l <= raw_r && raw_r >= 0 && raw_l <= M - 1) {
                    l = max(0, raw_l);
                    r = min(M - 1, raw_r);
                    res = sgt.query_from(vr, vl - 1, l, r);
                }
                if(res.sum > 0) {
                } else {
                    sum += 1 << (MAX_H - j);
                }
            } else {
                int raw_l = sum + (1 << (MAX_H - j)) - x;
                int raw_r = sum + (1 << (MAX_H - j + 1)) - x - 1;
                Data res = DATA_EP;
                if(raw_l <= raw_r && raw_r >= 0 && raw_l <= M - 1) {
                    l = max(0, raw_l);
                    r = min(M - 1, raw_r);
                    res = sgt.query_from(vr, vl - 1, l, r);
                }
                if(res.sum > 0) {
                    sum += 1 << (MAX_H - j);
                } else {
                }
```

```
            }
        }
        cout << (sum ^ b) << '\n';
    }
}
```

注意

版本编号从 0 开始；区间查询时先判断真实区间与值域是否有交，再钳位。

3.8 可持久化数组

时间复杂度 $O(\log n)$

3.8.1 P3919 【模板】可持久化线段树 1（可持久化数组）

```
template <class Data = int>
class PersistentArray {
public:
    class Node {
    public:
        int ls = 0, rs = 0;
        Data data{};
    };
    vector<Node> node;
    vector<int> version;
    int n = 0;

    /*
     * 由 a[1..n] 建立版本 0，capacity 仅用于预留节点空间
     */
    int build(int l, int r, const vector<Data>& a) {
        int u = int(node.size());
        node.emplace_back();
        if(l == r) {
            node[u].data = a[l];
        } else {
            int mid = l + (r - l) / 2;
            node[u].ls = build(l, mid, a);
            node[u].rs = build(mid + 1, r, a);
        }
        return u;
    }
    int modify(int u, int l, int r, int pos, const Data& x) {
        int v = int(node.size());
        node.push_back(node[u]);
        if(l == r) {
            node[v].data = x;
        } else {
            int mid = l + (r - l) / 2;
            if(pos <= mid) {
                node[v].ls = modify(node[u].ls, l, mid, pos, x);
            } else {
                node[v].rs = modify(node[u].rs, mid + 1, r, pos, x);
            }
        }
        return v;
    }
    /*
     * 由 a[1..n] 建立版本 0，capacity 仅用于预留节点空间
     */
    void build(const vector<Data>& a, size_t capacity = 0) {
```

```
        n = int(a.size()) - 1;
        node.clear();
        version.clear();
        node.reserve(capacity);
        version.push_back(build(1, n, a));
    }
    /*
     * 复制版本 ver，返回新版本号
     */
    int copy(int ver) {
        version.push_back(version[ver]);
        return int(version.size()) - 1;
    }
    /*
     * 基于版本 ver 修改 pos 处的值，返回新版本号
     */
    int modify(int ver, int pos, const Data& x) {
        int root = modify(version[ver], 1, n, pos, x);
        version.push_back(root);
        return int(version.size()) - 1;
    }
    /*
     * 查询版本 ver 中 pos 处的值，不修改历史版本
     */
    Data query(int ver, int pos) const {
        int u = version[ver], l = 1, r = n;
        while(l ≠ r) {
            int mid = l + (r - l) / 2;
            if(pos ⩽ mid) {
                u = node[u].ls;
                r = mid;
            } else {
                u = node[u].rs;
                l = mid + 1;
            }
        }
        return node[u].data;
    }
};
```

注意

只复制根指针，修改路径上的节点新建。

第 4 章

平衡树

4.1 Splay 树

时间复杂度 $O(\log n)$ 均摊

4.1.1 P3391 【模板】文艺平衡树

```
class Tag {
public:
    int64 add = 0;
    bool rev = false;
};

class Splay {
public:
    class Node {
    public:
        int son[2]{}, fa = 0, size = 0;
        int64 data = 0, sum = 0;
        Tag tag;
    };
    vector<Node> node;
    int root = 0, n = 0;

    void push_up(int u) {
        int l = node[u].son[0], r = node[u].son[1];
        node[u].size = node[l].size + node[r].size + 1;
        node[u].sum = node[l].sum + node[r].sum + node[u].data;
    }
    void apply(int u, const Tag& t) {
        if(!u) {
            return;
        }
        node[u].data += t.add;
        node[u].sum += t.add * node[u].size;
        node[u].tag.add += t.add;
        if(t.rev) {
            swap(node[u].son[0], node[u].son[1]);
            node[u].tag.rev ^= true;
        }
    }
    void push_down(int u) {
        apply(node[u].son[0], node[u].tag);
        apply(node[u].son[1], node[u].tag);
        node[u].tag = Tag();
    }
    bool dir(int u) const {
        return node[node[u].fa].son[1] == u;
    }
    void rot(int u) {
        int fa = node[u].fa, g = node[fa].fa, d = dir(u);
        int v = node[u].son[d ^ 1];
        if(g) {
            node[g].son[dir(fa)] = u;
        }
        node[u].fa = g;
```

```
        node[fa].son[d] = v;
        if(v) {
            node[v].fa = fa;
        }
        node[u].son[d ^ 1] = fa;
        node[fa].fa = u;
        push_up(fa);
        push_up(u);
    }
    void splay(int u, int goal = 0) {
        vector<int> st;
        for(int v = u; v != goal; v = node[v].fa) {
            st.push_back(v);
        }
        if(goal) {
            st.push_back(goal);
        }
        for(auto it = st.rbegin(); it != st.rend(); ++it) {
            push_down(*it);
        }
        while(node[u].fa != goal) {
            int fa = node[u].fa;
            if(node[fa].fa != goal) {
                rot(dir(u) == dir(fa) ? fa : u);
            }
            rot(u);
        }
        if(!goal) {
            root = u;
        }
    }
    /*
     * 由 a[1..n] 建立序列，两端补哨兵
     */
    int build(int l, int r, int fa, const vector<int64>& a) {
        if(l > r) {
            return 0;
        }
        int mid = (l + r) / 2, u = mid + 1;
        node[u].fa = fa;
        node[u].data = a[mid];
        node[u].son[0] = build(l, mid - 1, u, a);
        node[u].son[1] = build(mid + 1, r, u, a);
        push_up(u);
        return u;
    }
    int kth(int k) {
        int u = root;
        while(true) {
            push_down(u);
            int s = node[node[u].son[0]].size;
            if(k <= s) {
                u = node[u].son[0];
            } else if(k == s + 1) {
                return u;
            } else {
                k -= s + 1;
                u = node[u].son[1];
            }
        }
    }
```

```
    int split(int l, int r) {
        int x = kth(l);
        splay(x);
        int y = kth(r + 2);
        splay(y, x);
        return node[y].son[0];
    }
    /*
     * 由 a[1..n] 建立序列，两端补哨兵
     */
    void build(const vector<int64>& a) {
        n = int(a.size()) - 1;
        vector<int64> b(n + 2);
        for(int i = 1; i ⩽ n; i++) {
            b[i] = a[i];
        }
        node.assign(n + 3, Node());
        root = build(0, n + 1, 0, b);
    }
    /*
     * 对闭区间 [l,r] 施加加法和翻转标记 t
     */
    void mod(int l, int r, const Tag& t) {
        apply(split(l, r), t);
        push_up(node[root].son[1]);
        push_up(root);
    }
    /*
     * 翻转闭区间 [l,r]
     */
    void rev(int l, int r) {
        mod(l, r, {0, true});
    }
    /*
     * 查询闭区间 [l,r] 的和
     */
    int64 query(int l, int r) {
        return node[split(l, r)].sum;
    }
    /*
     * 按当前序列顺序返回所有值，不含哨兵
     */
    vector<int64> values() {
        vector<int64> res;
        vector<int> st;
        int u = root, idx = 0;
        while(u || !st.empty()) {
            while(u) {
                push_down(u);
                st.push_back(u);
                u = node[u].son[0];
            }
            u = st.back();
            st.pop_back();
            if(++idx > 1 && idx ⩽ n + 1) {
                res.push_back(node[u].data);
            }
            u = node[u].son[1];
        }
        return res;
    }
```

```
};
```

注意

P3391 文艺平衡树：两端加哨兵，`kth` 返回的是带哨兵的下标。

4.2 PBDS 平衡树

时间复杂度 $O(\log n)$

```
template <class Data = int>
class Splay {
public:
    class Node {
    public:
        int son[2]{}, fa = 0, cnt = 0, size = 0;
        Data data{};
    };
    vector<Node> node{Node()};
    int root = 0;

    bool dir(int u) const {
        return node[node[u].fa].son[1] == u;
    }
    void push_up(int u) {
        node[u].size = node[u].cnt + node[node[u].son[0]].size + node[node[u].son[1]].size;
    }
    void rot(int u) {
        int fa = node[u].fa, g = node[fa].fa, d = dir(u);
        int v = node[u].son[d ^ 1];
        if(g) {
            node[g].son[dir(fa)] = u;
        }
        node[u].fa = g;
        node[fa].son[d] = v;
        if(v) {
            node[v].fa = fa;
        }
        node[u].son[d ^ 1] = fa;
        node[fa].fa = u;
        push_up(fa);
        push_up(u);
    }
    void splay(int u) {
        while(node[u].fa) {
            int fa = node[u].fa;
            if(node[fa].fa) {
                rot(dir(u) == dir(fa) ? fa : u);
            }
            rot(u);
        }
        root = u;
    }
    int find(const Data& d) {
        int u = root, last = 0;
        while(u && !(node[u].data == d)) {
            last = u;
            u = node[u].son[node[u].data < d];
        }
        if(u) {
            splay(u);
```

```
        } else if(last) {
            splay(last);
        }
        return u;
    }
    /*
     * 清空平衡树
     */
    void clear() {
        node.assign(1, Node());
        root = 0;
    }
    int size() const {
        return node[root].size;
    }
    /*
     * 插入 d，重复值累计出现次数
     */
    void insert(const Data& d) {
        int u = root, fa = 0;
        while(u && !(node[u].data == d)) {
            fa = u;
            u = node[u].son[node[u].data < d];
        }
        if(!u) {
            u = int(node.size());
            node.emplace_back();
            node[u].data = d;
            node[u].fa = fa;
            if(fa) {
                node[fa].son[node[fa].data < d] = u;
            }
        }
        node[u].cnt++;
        push_up(u);
        splay(u);
    }
    /*
     * 删除一个 d，返回是否存在并删除
     */
    bool remove(const Data& d) {
        int u = find(d);
        if(!u) {
            return false;
        }
        if(--node[u].cnt) {
            push_up(u);
            return true;
        }
        int l = node[u].son[0], r = node[u].son[1];
        if(l) {
            node[l].fa = 0;
        }
        if(r) {
            node[r].fa = 0;
        }
        root = l;
        if(!l) {
            root = r;
        } else {
            int v = l;
```

```
            while(node[v].son[1]) {
                v = node[v].son[1];
            }
            splay(v);
            node[v].son[1] = r;
            if(r) {
                node[r].fa = v;
            }
            push_up(v);
        }
        return true;
    }
    /*
     * 返回 d 的排名，即小于 d 的元素个数加 1
     */
    int rank(const Data& d) {
        find(d);
        if(!root) {
            return 1;
        }
        return node[node[root].son[0]].size + (node[root].data < d ? node[root].cnt : 0) + 1;
    }
    /*
     * 将第 k 小的值写入 res，排名不存在时返回 false
     */
    bool kth(int k, Data& res) {
        if(k < 1 || k > size()) {
            return false;
        }
        int u = root;
        while(true) {
            int s = node[node[u].son[0]].size;
            if(k <= s) {
                u = node[u].son[0];
            } else if(k <= s + node[u].cnt) {
                break;
            } else {
                k -= s + node[u].cnt;
                u = node[u].son[1];
            }
        }
        res = node[u].data;
        splay(u);
        return true;
    }
    /*
     * 将严格小于 d 的最大值写入 res，不存在时返回 false
     */
    bool prev(const Data& d, Data& res) {
        return kth(rank(d) - 1, res);
    }
    /*
     * 将严格大于 d 的最小值写入 res，不存在时返回 false
     */
    bool next(const Data& d, Data& res) {
        int k = rank(d);
        if(root && node[root].data == d) {
            k += node[root].cnt;
        }
        return kth(k, res);
    }
```

```
};
```

4.2.1 PBDS 实现

```
/*
 * PBDS 实现：用 __gnu_pbds 的红黑树，通过 TreeUpdate 策略维护子树大小，
 * 从而支持 rank / kth，省去手写旋转。
 */
using namespace __gnu_pbds;

class Node {
public:
    int k;
    mutable int cnt;

    bool operator<(const Node& o) const {
        return k < o.k;
    }
};

class MetaData {
public:
    int size;
};

MetaData METADATA_EP = {0};

template <class cIt, class It, class Cmp, class _A>
class TreeUpdate {
public:
    typedef MetaData metadata_type;

#define nd(x) ((x).m_p_nd)
#define lson(x) (nd(x)->m_p_left)
#define rson(x) (nd(x)->m_p_right)
#define md(x) ((x) == node_end() ? METADATA_EP : (metadata_type)(nd(x)->m_metadata))
#define rmd(x) ((metadata_type&)(nd(x)->m_metadata))

    virtual cIt node_begin() const = 0;

    virtual cIt node_end() const = 0;

    void operator()(It u, cIt end) {
        It ls = lson(u);
        It rs = rson(u);
        metadata_type data = {0};
        if(ls != end) {
            data.size += md(ls).size;
        }
        if(rs != end) {

            data.size += md(rs).size;
        }
        data.size += (*u)->cnt;

        rmd(u) = data;
    }

    /*
     * 返回第 k 小的值
     */
    int kth(int k) {
```

```
        cIt u = node_begin();
        cIt end = node_end();

        while(u != end) {
            It ls = lson(u);
            It rs = rson(u);

            if(k <= md(ls).size) {
                u = ls;
            } else if(k <= md(ls).size + (*u)->cnt) {
                return (*u)->k;
            } else {
                k -= md(ls).size + (*u)->cnt;
                u = rs;
            }
        }
        return -1;
    }

    /*
     * 返回小于 k 的元素个数加一
     */
    int rank(int k) {
        cIt u = node_begin();
        cIt end = node_end();
        int rk = 1;

        while(u != end) {
            It ls = lson(u);
            It rs = rson(u);

            if(k <= (*u)->k) {
                u = ls;
            } else {
                rk += (*u)->cnt + md(ls).size;
                u = rs;
            }
        }
        return rk;
    }

#undef nd
#undef lson
#undef rson
#undef md
#undef rmd
};

class BT : public tree<Node, null_type, less<Node>, rb_tree_tag, TreeUpdate> {
private:
    using Base = tree<Node, null_type, less<Node>, rb_tree_tag, TreeUpdate>;
    typedef Base::iterator It;

    void update(It node) {
        update_to_top(node.m_p_nd, this);
    }

public:
    /*
     * 插入 k, 重复值累计出现次数
     */
```

```
    void insert(int k) {
        It it = lower_bound({k, 0});
        if(it != end() && it->k == k) {
            ++(it->cnt);
            update(it);
        } else {
            Base::insert({k, 1});
        }
    }

    /*
     * 删除一个 k
     */
    void erase(int k) {
        auto it = lower_bound(Node{k, 0});
        if(it != end() && it->cnt > 1) {
            --(it->cnt);
            update(it);
        } else {
            Base::erase(it);
        }
    }
};

BT bt;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, num;
    cin >> n >> m;

    for(int i = 1; i <= n; i++) {
        cin >> num;
        bt.insert(num);
    }

    int op, x, last = 0, ans = 0;
    while(m--) {
        cin >> op >> x;
        x ^= last;
        if(op == 1) {
            bt.insert(x);
        } else if(op == 2) {
            bt.erase(x);
        } else if(op == 3) {
            last = bt.rank(x);
            ans ^= last;
        } else if(op == 4) {
            last = bt.kth(x);
            ans ^= last;
        } else if(op == 5) {
            last = (--bt.lower_bound({x, 0}))->k;
            ans ^= last;
        } else if(op == 6) {
            last = (bt.upper_bound({x, 0}))->k;
            ans ^= last;
        }
    }
    cout << ans << '\n';
}
```

注意

PBDS 版本用 `TreeNode` 维护子树 size；`erase` 前必须确认元素存在。

4.3 笛卡尔树

时间复杂度 $O(n)$

```
const int N = 10000005;

class Node {
public:
    int x;
    int l, r;
};

Node node[N];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);

    int n;
    cin >> n;
    stack<int> st;
    long long ansl = 0, ansr = 0;
    for(int i = 1; i <= n; i++) {
        cin >> node[i].x;
        int lst = 0;
        while(!st.empty() && node[st.top()].x > node[i].x) {
            lst = st.top();
            ansl ^= lst * (node[lst].l + 1ll);
            ansr ^= lst * (node[lst].r + 1ll);
            st.pop();
        }
        if(!st.empty()) {
            node[st.top()].r = i;
        }
        node[i].l = lst;
        st.push(i);
    }
    while(!st.empty()) {
        ansl ^= st.top() * (node[st.top()].l + 1ll);
        ansr ^= st.top() * (node[st.top()].r + 1ll);
        st.pop();
    }
    cout << ansl << ' ' << ansr << endl;
}
```

注意

单调栈建树；下标从 1 开始。

第 5 章

分块

5.1 块状数组

时间复杂度 $O(\sqrt{n})$ 单次

```
const int N = 200005;
class BlockArray {
private:
    long long arr[N];
    long long idx[N];
    long long tag[500];
    long long n, S, bn;

    void bsort(long long b) {
        sort(idx + b * S,
             min(idx + (b + 1) * S, idx + n + 1),
             [this](long long A, long long B) {
                 return arr[A] < arr[B];
             });
    }

    long long bget(long long b, long long x) {
        return lower_bound(idx + b * S,
                           min(idx + (b + 1) * S, idx + n + 1),
                           x,
                           [this](long long A, long long B) {
                               return arr[A] < B;
                           }) -
               (idx + b * S);
    }

public:
    /*
     * 读入 size 个元素并初始化分块
     */
    void build(int size) {
        n = size;
        S = max(1ll, (long long)sqrt(n));
        bn = n / S + 1;
        for(long long i = 1; i ⩽ n; i++) {
            cin >> arr[i];
            idx[i] = i;
        }

        for(long long i = 0; i < bn; i++) {
            bsort(i);
        }
    }

    /*
     * 将闭区间 [l,r] 的元素加 x
     */
    void add(long long l, long long r, long long x) {
        long long lb = l / S;
        long long rb = r / S;
```

```
51
52          if(lb == rb) {
53              for(long long i = l; i <= r; i++) {
54                  arr[i] += x;
55              }
56              bsort(lb);
57          } else {
58              for(long long i = l; i < (lb + 1) * S; i++) {
59                  arr[i] += x;
60              }
61              bsort(lb);
62              for(long long i = lb + 1; i < rb; i++) {
63                  tag[i] += x;
64              }
65              for(long long i = rb * S; i <= r; i++) {
66                  arr[i] += x;
67              }
68              bsort(rb);
69          }
70      }
71
72      /*
73       * 统计闭区间 [l,r] 中小于 x 的元素个数
74       */
75      long long count(long long l, long long r, long long x) {
76          long long lb = l / S;
77          long long rb = r / S;
78          long long cnt = 0;
79
80          if(lb == rb) {
81              for(long long i = l; i <= r; i++) {
82                  cnt += arr[i] + tag[lb] < x;
83              }
84          } else {
85              for(long long i = l; i < (lb + 1) * S; i++) {
86                  cnt += arr[i] + tag[lb] < x;
87              }
88              for(long long i = lb + 1; i < rb; i++) {
89                  cnt += bget(i, x - tag[i]);
90              }
91              for(long long i = rb * S; i <= r; i++) {
92                  cnt += arr[i] + tag[rb] < x;
93              }
94          }
95          return cnt;
96      }
97
98  };
99
100 BlockArray ba;
101 int main() {
102     ios::sync_with_stdio(false);
103     cin.tie(nullptr);
104     long long n, q;
105     cin >> n;
106     q = n;
107     ba.build(n);
108     long long op;
109     long long l, r, x;
110     while(q--) {
111         cin >> op;
```

```
        cin >> l;
        cin >> r;
        cin >> x;

        if(op == 0) {
            ba.add(l, r, x);
        }
        if(op == 1) {
            cout << (ba.count(l, r, x * x)) << '\n';
        }
    }
}
```

```
const int N = 300005;
const int SQRTN = 600;
const long long INF = 1LL << 60;
class BlockArray {
private:
    int arr[N];
    long long eq[SQRTN];
    long long tag[SQRTN];
    int n, S, bn;

    void push_down(int b) {
        if(tag[b] != INF) {
            int r = min(n, (b + 1) * S - 1);
            for(int j = b * S; j <= r; j++) {
                arr[j] = tag[b];
            }
            tag[b] = INF;
        }
    }

    int bget(int l, int r, int x) {
        int lb = l / S, rb = r / S, ans = 0;
        if(lb == rb) {
            if(eq[lb] == x) {
                return r - l + 1;
            } else if(eq[lb] != INF) {
                if(r - l + 1 == S) {
                    eq[lb] = tag[lb] = x;
                } else {
                    push_down(lb);
                    eq[lb] = tag[lb] = INF;
                    for(int i = l; i <= r; i++) {
                        arr[i] = x;
                    }
                }
                return 0;
            } else {
                for(int i = l; i <= r; i++) {
                    ans += arr[i] == x;
                    arr[i] = x;
                }
                eq[lb] = r - l + 1 == S ? x : INF;
                tag[lb] = INF;
                return ans;
            }
        } else {
            return 0;
        }
```

```
    }

public:
    /*
     * 读入 size 个元素并初始化分块
     */
    void build(int size) {
        n = size;
        S = max(1, (int)sqrt(n));
        bn = n / S + 1;
        for(int i = 1; i ⩽ n; i++) {
            cin >> arr[i];
        }
        arr[0] = 0;
        for(int i = 0; i < bn; i++) {
            eq[i] = arr[i * S];
            tag[i] = INF;
        }
        eq[bn] = tag[bn] = INF;
        for(int i = 0; i ⩽ n; i++) {
            if(arr[i] ≠ eq[i / S]) {
                eq[i / S] = INF;
            }
        }
    }

    /*
     * 统计闭区间 [l,r] 中等于 x 的元素个数，并将该区间赋为 x
     */
    int get(int l, int r, int x) {
        int lb = l / S, rb = r / S, ans = 0;
        if(lb == rb) {
            return bget(l, r, x);
        } else {
            ans += bget(l, (lb + 1) * S - 1, x);
            for(int i = lb + 1; i < rb; i++) {
                ans += bget(i * S, (i + 1) * S - 1, x);
            }
            ans += bget(rb * S, r, x);
            return ans;
        }
    }
};

BlockArray ba;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n;
    q = n;
    ba.build(n);
    int op;
    int l, r, x;
    while(q--) {
        cin >> l;
        cin >> r;
        cin >> x;
        cout << (ba.get(l, r, x)) << '\n';
    }
}
```

```
const int N = 100005;
const int SQRTN = 320;
const int INF = 0x7FFFFFFF;

int cnt[N];

class BlockArray {
private:
    int inv[N];
    int arr[N];
    int pre[SQRTN][N];
    int mode[SQRTN][SQRTN];
    int n, S, bn;

public:
    /*
     * 读入 size 个元素并初始化分块
     */
    void build(int size) {
        n = size;
        S = max(1, (int)sqrt(n));
        bn = n / S + 1;
        for(int i = 1; i ⩽ n; i++) {
            cin >> arr[i];
            inv[i] = arr[i];
        }

        sort(inv + 1, inv + 1 + n);
        int len = unique(inv + 1, inv + 1 + n) - inv - 1;
        for(int i = 1; i ⩽ n; i++) {
            arr[i] = lower_bound(inv + 1, inv + len + 1, arr[i]) - inv;
        }

        for(int b = 0; b < bn; b++) {
            if(b) {
                for(int i = 1; i ⩽ len; i++) {
                    pre[b][i] = pre[b - 1][i];
                }
            }
            for(int i = b * S; i < min((b + 1) * S, n + 1); i++) {
                pre[b][arr[i]]++;
            }
        }

        for(int lb = 0; lb < bn; lb++) {
            int maxi = 0;
            for(int rb = lb; rb < bn; rb++) {
                if(rb > lb) {
                    mode[lb][rb] = mode[lb][rb - 1];
                }
                for(int i = rb * S; i < min((rb + 1) * S, n + 1); i++) {
                    int v = arr[i];
                    int ans = pre[rb][v];
                    if(lb) {
                        ans -= pre[lb - 1][v];
                    }
                    if(ans > maxi || ans == maxi && v < mode[lb][rb]) {
                        mode[lb][rb] = v;
                        maxi = ans;
                    }
                }
```

```
            }
        }
    }

    /*
     * 查询闭区间 [l,r] 的众数，出现次数相同取较小值
     */
    int get(int l, int r) {
        int lb = l / S, rb = r / S;
        int maxi = 0;
        int ans = 0;
        if(lb == rb) {
            for(int i = l; i <= r; i++) {
                cnt[arr[i]] = 0;
            }
            for(int i = l; i <= r; i++) {
                ++cnt[arr[i]];
                if(cnt[arr[i]] > maxi || cnt[arr[i]] == maxi && arr[i] < ans) {
                    maxi = cnt[arr[i]];
                    ans = arr[i];
                }
            }
        } else {
            for(int i = l; i <= (lb + 1) * S; i++) {
                cnt[arr[i]] = pre[rb - 1][arr[i]] - pre[lb][arr[i]];
            }
            for(int i = rb * S; i <= r; i++) {
                cnt[arr[i]] = pre[rb - 1][arr[i]] - pre[lb][arr[i]];
            }
            maxi = cnt[mode[lb + 1][rb - 1]] = pre[rb - 1][mode[lb + 1][rb - 1]] - pre[lb][mode[lb + 1][rb - 1]];
            ans = mode[lb + 1][rb - 1];
            for(int i = l; i < (lb + 1) * S; i++) {
                ++cnt[arr[i]];
                if(cnt[arr[i]] > maxi || cnt[arr[i]] == maxi && arr[i] < ans) {
                    maxi = cnt[arr[i]];
                    ans = arr[i];
                }
            }
            for(int i = rb * S; i <= r; i++) {
                ++cnt[arr[i]];
                if(cnt[arr[i]] > maxi || cnt[arr[i]] == maxi && arr[i] < ans) {
                    maxi = cnt[arr[i]];
                    ans = arr[i];
                }
            }
        }
        return inv[ans];
    }
};

BlockArray ba;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    cin >> n;
    q = n;
    ba.build(n);
    int l, r;
    while(q--) {
        cin >> l;
```

```
        cin >> r;
        cout << (ba.get(l, r)) << '\n';
    }
}
```

注意

块长 `S = max(1, sqrt(n))`；区间众数版本的 `pre` 按 $n \leq 1e5$ 开。

5.2 树分块

时间复杂度 $O(\sqrt{n})$

```
const int W = 64;
const int N = 40005;
const int LOGN = 17;
int cvt[N];

class Tree {
public:
    int S;
    class Node {
    public:
        int c, d;
        int p;
        int fa[LOGN];
        vector<int> oe;
    };

    class VNode {
    public:
        int fa;
        vector<int> oe;
        vector<bitset<N>> bm;
    };

    Node node[N];
    VNode vnode[N];

    /*
     * 添加无向边 (u,v)
     */
    void add(int u, int v) {
        node[u].oe.push_back(v);
        node[v].oe.push_back(u);
    }

    /*
     * 以 1 为根建立树分块和倍增信息
     */
    void build(int n) {
        S = sqrt(n) / W * W;
        dfs1(1);
        for(int i = 1; i < LOGN; i++) {
            for(int j = 1; j <= n; j++) {
                node[j].fa[i] = node[node[j].fa[i - 1]].fa[i - 1];
            }
        }
        dfs2(1);
    }

```

```
    /*
     * 返回 x、y 的最近公共祖先
     */
    int getLCA(int x, int y) {
        if(node[x].d < node[y].d) {
            swap(x, y);
        }
        for(int i = LOGN - 1; i ⩾ 0 && node[x].d > node[y].d; i--) {
            if(node[x].d ⩾ node[y].d + (1 << i)) {
                x = node[x].fa[i];
            }
        }
        if(x == y) {
            return x;
        } else {
            for(int i = LOGN - 1; i ⩾ 0; i--) {
                if(node[x].fa[i] != node[y].fa[i]) {
                    x = node[x].fa[i];
                    y = node[y].fa[i];
                }
            }
            return node[x].fa[0];
        }
    }

    void dfs1(int r) {
        node[r].d = 1;
        stack<int> st;
        st.push(r);
        while(!st.empty()) {
            int u = st.top(), p = node[u].p;
            st.pop();
            if(node[u].d % S == 1) {
                vnode[p].oe.push_back(u);
                vnode[u].fa = p;
                p = node[u].p = u;
            }
            for(int v : node[u].oe) {
                if(v != node[u].fa[0]) {
                    node[v].fa[0] = u;
                    node[v].d = node[u].d + 1;
                    node[v].p = p;
                    st.push(v);
                }
            }
        }
    }

    void dfs2(int r) {
        stack<int> st;
        st.push(r);
        while(!st.empty()) {
            int u = st.top(), fa = u;
            st.pop();
            vnode[u].bm.emplace_back();
            do {
                fa = node[fa].fa[0];
                vnode[u].bm[0][node[fa].c] = true;
            } while(fa != vnode[u].fa);
            for(auto& bm : vnode[vnode[u].fa].bm) {
                vnode[u].bm.emplace_back(vnode[u].bm[0] | bm);
```

```
            }
            for(int v : vnode[u].oe) {
                st.push(v);
            }
        }
    }

    /*
     * 查询 x 到 y 路径上的不同颜色数
     */
    int query(int x, int y) {
        int lca = getLCA(x, y);
        bitset<N> ans;
        for(int i = 1; i <= 2; i++, swap(x, y)) {
            if(node[lca].p != node[x].p) {
                int x0 = node[x].p;
                while(x != node[x].p && node[x].d > node[lca].d) {
                    ans[node[x].c] = true;
                    x = node[x].fa[0];
                }
                ans[node[x].c] = true;
                while((node[vnode[x].fa].d >= node[lca].d)) {
                    x = vnode[x].fa;
                }
                if(x0 != x) {
                    ans |= vnode[x0].bm[(node[x0].d - node[x].d) / S - 1];
                }
            }
            while(x != lca) {
                ans[node[x].c] = true;
                x = node[x].fa[0];
            }
        }
        ans[node[lca].c] = true;
        return ans.count();
    }
};

Tree tree;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, c;
    cin >> n;
    cin >> m;

    for(int i = 1; i <= n; i++) {
        cin >> c;
        cvt[i] = c;
        tree.node[i].c = c;
    }
    sort(cvt + 1, cvt + n + 1);
    int C = unique(cvt + 1, cvt + n + 1) - cvt - 1;
    for(int i = 1; i <= n; i++) {
        tree.node[i].c = lower_bound(cvt + 1, cvt + 1 + C, tree.node[i].c) - cvt;
    }

    int u, v;
    for(int i = 1; i < n; i++) {
        cin >> u;
```

```
171         cin >> v;
172         tree.add(u, v);
173     }
174 
175     tree.build(n);
176     int lastans = 0;
177     for(int i = 1; i ⩽ m; i++) {
178         cin >> u;
179         cin >> v;
180         u ^= lastans;
181         lastans = tree.query(u, v);
182         cout << (lastans) << '\n';
183     }
184 }
```

注意

按深度分块，块内维护到祖先的 bitset。

第 6 章

动态树

6.1 Link-Cut Tree

时间复杂度 $O(\log n)$ 均摊

Link-Cut Tree 用一组 Splay 辅助树维护森林。下面的通用实现封装了核心操作，题目只需给出 `Data`（聚合信息与 `operator+`）；需要额外维护虚子树等附加信息时，再重写 `on_access` / `on_link` 两个钩子。

```
const int N = 100005;

class Tag {
public:
    bool rev;
};

const Tag TAG_EP = {false};
const Tag TAG_REV = {true};

/*
 * 通用 LCT。Data 需提供 operator+ 和默认构造（单位元）；
 * self 保存节点自身的贡献，data = 左子 + self + 右子。
 * 需要维护虚子树等附加信息时，重写 on_access / on_link。
 */
template <class Data>
class LinkCutTree {
public:
    static const int MAXN = N;

    class Node {
    public:
        int son[2];
        int fa;
        Data data;
        Data self;
        Tag tag;

        void operator+=(const Tag& t) {
            if(t.rev) {
                swap(son[0], son[1]);
                tag.rev ^= t.rev;
            }
        }
    };

    Node node[MAXN];

    /*
     * 实链切换时的附加维护，默认无操作
     */
    virtual void on_access(int, int, int) {
    }

    /*
     * 加边后的附加维护，默认无操作
     */
    virtual void on_link(int, int) {
```

```
    }

#define ls(u) (node[u].son[0])
#define rs(u) (node[u].son[1])
#define fa(u) (node[u].fa)
#define data(u) (node[u].data)
#define tag(u) (node[u].tag)

    bool dir(const int u) {
        return rs(fa(u)) == u;
    }

    bool is_root(const int u) {
        return ls(fa(u)) != u && rs(fa(u)) != u;
    }

    void push_up(const int u) {
        data(u) = data(ls(u)) + node[u].self + data(rs(u));
    }

    void push_down(const int u) {
        if(ls(u)) {
            node[ls(u)] += tag(u);
        }
        if(rs(u)) {
            node[rs(u)] += tag(u);
        }
        tag(u) = TAG_EP;
    }

    void update(int u) {
        vector<int> st{u};
        for(int v = u; !is_root(v);) {
            v = fa(v);
            st.push_back(v);
        }
        for(auto it = st.rbegin(); it != st.rend(); ++it) {
            push_down(*it);
        }
    }

    void rot(int u) {
        int y = fa(u), z = fa(y), k = dir(u), v = node[u].son[!k];
        if(!is_root(y)) {
            node[z].son[dir(y)] = u;
        }
        node[y].son[k] = v;
        node[u].son[!k] = y;
        if(v) {
            fa(v) = y;
        }
        fa(y) = u;
        fa(u) = z;
        push_up(y);
        push_up(u);
    }

    void splay(int u) {
        update(u);
        for(int f = fa(u); !is_root(u); rot(u), f = fa(u)) {
            if(!is_root(f)) {
```

```
110                 rot(dir(u) == dir(f) ? f : u);
111             }
112         }
113     }
114
115     /*
116      * 打通原树根到 u 的实链
117      */
118     int access(int u) {
119         int v = 0;
120         for(; u; v = u, u = fa(u)) {
121             splay(u);
122             on_access(u, rs(u), v);
123             rs(u) = v;
124             push_up(u);
125         }
126         return v;
127     }
128
129     /*
130      * 将 u 设为所在原树的根
131      */
132     void make_root(int u) {
133         access(u);
134         splay(u);
135         node[u] += TAG_REV;
136     }
137
138     /*
139      * 返回 u 所在原树的根
140      */
141     int find_root(int u) {
142         access(u);
143         splay(u);
144         push_down(u);
145         while(ls(u)) {
146             u = ls(u);
147             push_down(u);
148         }
149         splay(u);
150         return u;
151     }
152
153     /*
154      * 将 x 到 y 的路径暴露到以 y 为根的辅助树，x、y 须连通
155      */
156     void split(int x, int y) {
157         make_root(x);
158         access(y);
159         splay(y);
160     }
161
162     /*
163      * 连接 x、y；只在两点不连通时加边
164      */
165     void link(int x, int y) {
166         make_root(x);
167         if(find_root(y) != x) {
168             splay(y);
169             fa(x) = y;
170             on_link(x, y);
```

```
            push_up(y);
        }
    }

    /*
     * 断开 x、y 之间的直接边
     */
    void cut(int x, int y) {
        make_root(x);
        if(find_root(y) == x && fa(y) == x && !ls(y)) {
            fa(y) = rs(x) = 0;
            push_up(x);
        }
    }

#undef ls
#undef rs
#undef fa
#undef data
#undef tag
};
```

6.1.1 P3690 【模板】动态树（LCT）

```
class Data {
public:
    int sum;

    Data operator+(const Data& other) const {
        return {sum ^ other.sum};
    }
};

int n, m;

LinkCutTree<Data> lct;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n;
    cin >> m;

    for(int i = 1; i <= n; i++) {
        cin >> lct.node[i].self.sum;
        lct.node[i].data.sum = lct.node[i].self.sum;
    }

    int op, x, y;
    while(m--) {
        cin >> op;
        cin >> x;
        cin >> y;
        if(op == 0) {
            lct.split(x, y);
            cout << (lct.node[y].data.sum) << '\n';
        } else if(op == 1) {
            lct.link(x, y);
        } else if(op == 2) {
            lct.cut(x, y);
        } else if(op == 3) {
```

```
            lct.make_root(x);
            lct.node[x].self.sum = y;
            lct.push_up(x);
        }
    }
}
```

注意

`Data::operator+` 需可交换（反转只交换儿子，不重算自身）；动态重心需重写 `on_access` / `on_link`。

第 7 章

其他结构

7.1 KD-Tree

时间复杂度 $O(\sqrt{n})$ 期望

```
const int64 INF = 0x3f3f3f3f3f3f3f3f;
const int N = 400005;
const int KD = 2;
int64 n;

class Point {
public:
    int64 coord[KD];
};
Point point[N];

class Node {
public:
    int l, r;
    int64 coord[KD];
    int64 min_coord[KD];
    int64 max_coord[KD];
};

bool cmp0(const Point& A, const Point& B) {
    return A.coord[0] < B.coord[0];
}

bool cmp1(const Point& A, const Point& B) {
    return A.coord[1] < B.coord[1];
}

class KDTree {
public:
    static const int MAXN = 400005;
    int kd = KD;
    int root = -1;
    int node_cnt = 0; // 0 for root;
    Node node[MAXN];

    int64 get_distance_squared(int tgt, int cur) {
        if(cur == -1) {
            return INF;
        }
        int64 gap, d2 = 0.;
        for(int i = 0; i < kd; i++) {
            gap = node[tgt].coord[i] - node[cur].coord[i];
            d2 += gap * gap;
        }
        return d2;
    }

    int64 potential_min_dis_squared(int tgt, int cur) {
        if(cur == -1) {
            return INF;
```

```
        }
        int64 gap, d2 = 0.;
        for(int i = 0; i < kd; i++) {
            if(node[tgt].coord[i] < node[cur].min_coord[i]) {
                gap = node[cur].min_coord[i] - node[tgt].coord[i];
            } else if(node[tgt].coord[i] > node[cur].max_coord[i]) {
                gap = node[tgt].coord[i] - node[cur].max_coord[i];
            } else {
                gap = 0;
            }
            d2 += gap * gap;
        }
        return d2;
    }

    void init_node(const Point& point, int cur) {
        for(int i = 0; i < kd; i++) {
            node[cur].coord[i] =
                node[cur].min_coord[i] =
                    node[cur].max_coord[i] = point.coord[i];
        }
        node[cur].l = -1;
        node[cur].r = -1;
    }

    void push_up(int cur) {
        for(int i = 0; i < kd; i++) {
            node[cur].min_coord[i] = node[cur].coord[i];
            node[cur].max_coord[i] = node[cur].coord[i];

            if(node[cur].l != -1) {
                node[cur].min_coord[i] = min(node[cur].min_coord[i], node[node[cur].l].min_coord[i]);
                node[cur].max_coord[i] = max(node[cur].max_coord[i], node[node[cur].l].max_coord[i]);
            }

            if(node[cur].r != -1) {
                node[cur].min_coord[i] = min(node[cur].min_coord[i], node[node[cur].r].min_coord[i]);
                node[cur].max_coord[i] = max(node[cur].max_coord[i], node[node[cur].r].max_coord[i]);
            }
        }
    }

    /*
     * 查询所有点对的最小距离平方
     */
    int64 batch_query() {
        int64 cur_ans = INF;
        int cur;
        for(int tgt = 0; tgt < node_cnt; tgt++) {
            stack<int> s;
            s.push(root);

            while(!s.empty()) {
                cur = s.top();
                s.pop();
                if(tgt == cur || potential_min_dis_squared(tgt, cur) > cur_ans) {
                    continue;
                } else {
                    cur_ans = min(cur_ans, get_distance_squared(tgt, cur));
                    if(node[cur].l != -1) {
                        s.push(node[cur].l);
```

```
                    }
                    if(node[cur].r != -1) {
                        s.push(node[cur].r);
                    }
                }
            }
        }
        return cur_ans;
    }

    /*
     * 按维度轮流划分 point[range_l..range_r], 返回根节点编号
     */
    int build(int range_l, int range_r, int d) {
        if(range_l == range_r) {
            init_node(point[range_l], node_cnt);
            return node_cnt++;
        } else if(range_l > range_r) {
            return -1;
        } else {
            int node_idx = node_cnt++;
            int mid = (range_r + range_l) >> 1;
            int d_idx = d % 2;

            if(d_idx == 0) {
                nth_element(point + range_l, point + mid, point + range_r, cmp0);
            } else {
                nth_element(point + range_l, point + mid, point + range_r, cmp1);
            }
            init_node(point[mid], node_idx);
            node[node_idx].l = build(range_l, mid - 1, d + 1);
            node[node_idx].r = build(mid + 1, range_r, d + 1);
            push_up(node_idx);
            return node_idx;
        }
    }
};

KDTree kdTree;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n;
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < KD; j++) {
            cin >> point[i].coord[j];
        }
    }

    kdTree.root = kdTree.build(0, int(n - 1), 0);

    cout << (kdTree.batch_query());
}
```

注意

重构阈值与维度相关；最近点对需要剪枝。

7.2 可持久化字典树

时间复杂度 $O(\log V)$

MAXH 为 1..64，num 和 x 为 MAXH 位无符号整数。查询排名不超过两个版本的计数差。

```
template <int MAXH = 32>
class PersistentTrie {
public:
    class Node {
    public:
        int son[2]{};
        int cnt = 0;
    };
    int ncnt;
    vector<Node> node;
    vector<int> root;

    /*
     * 初始化，版本 0 为空 Trie
     * n 为预计插入次数，用于预留空间
     */
    void init(int n = 0) {
        ncnt = 0;
        node.assign(1, Node());
        node.reserve(size_t(n) * (MAXH + 1) + 1);
        root.assign(1, 0);
    }
    int getNode(int lst) {
        node.push_back(node[lst]);
        node[++ncnt].cnt++;
        return ncnt;
    }
    /*
     * 在版本 ver 上插入 num，返回新版本号
     * 原版本保持不变
     */
    int push(int ver, uint64 num) {
        int u = getNode(root[ver]);
        root.push_back(u);
        for(int d = MAXH - 1; d >= 0; d--) {
            int c = (num >> d) & 1;
            int v = getNode(node[u].son[c]);
            node[u].son[c] = v;
            u = v;
        }
        return int(root.size()) - 1;
    }
    /*
     * 版本 vr 减去版本 vl 后的 Trie，与 x 异或后第 k 大的原数
     * 左开右闭，vl 为 vr 的祖先版本，k 从 1 开始
     */
    uint64 getKth(int vl, int vr, uint64 x, int k = 1) const {
        int l = root[vl], r = root[vr];
        uint64 res = 0;
        for(int d = MAXH - 1; d >= 0; d--) {
            int c = ((x >> d) & 1) ^ 1;
            int cnt = node[node[r].son[c]].cnt - node[node[l].son[c]].cnt;
            if(k > cnt) {
                k -= cnt;
                c ^= 1;
            }
            res |= uint64(c) << d;
            l = node[l].son[c];
            r = node[r].son[c];
```

```
        }
        return res;
    }
};
```

注意

版本区间查询用 `root[r] - root[l-1]`。

7.3 ST 表

时间复杂度 $O(n \log n)$ 建表，$O(1)$ 查询

```
template <int n>
class SparseTable {
public:
    static constexpr int logn = 32 - __builtin_clz(n);
    int64 st[logn][n + 1]{};
    /*
     * 由 st[0][1..n] 预处理区间最大值
     */
    void calc() {
        for(int j = 1; j < logn; j++) {
            for(int i = 1; i + (1 << j) - 1 ⩽ n; i++) {
                st[j][i] = max(st[j - 1][i], st[j - 1][i + (1 << (j - 1))]);
            }
        }
    }
    /*
     * 查询闭区间 [l,r] 的最大值
     */
    int64 query(int l, int r) const {
        int k = 31 - __builtin_clz(r - l + 1);
        return max(st[k][l], st[k][r - (1 << k) + 1]);
    }
};
```

注意

`logn = 32 - __builtin_clz(n)`；要求 $n \geq 1$。

第 II 编

图论

第 8 章

最短路

8.1 单源最短路

时间复杂度　Dijkstra $O(m \log m)$

8.1.1 P4779 【模板】单源最短路径（标准版）

```
#define N 100005
class Way {
public:
    int v, w;
};
class State {
public:
    int idx, d;
};
inline bool operator<(const State& A, const State& B) {
    return A.d > B.d;
}
vector<Way> way[N];
int d[N];
priority_queue<State> q;
bool visited[N];
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, r;
    memset(d, 0x3f, sizeof(d));
    cin >> n >> m >> r;
    int u, v, w;
    d[r] = 0;
    q.push({r, d[r]});
    while(m-- > 0) {
        cin >> u >> v >> w;
        way[u].push_back({v, w});
    }
    State now;
    while(!q.empty()) {
        now = q.top();
        q.pop();
        if(!visited[now.idx]) {
            visited[now.idx] = true;
            for(Way& e : way[now.idx]) {
                if(d[now.idx] + e.w < d[e.v]) {
                    d[e.v] = d[now.idx] + e.w;
                    q.push({e.v, d[e.v]});
                }
            }
        }
    }
    for(int i = 1; i <= n; i++) {
        cout << (d[i]) << ' ';
    }

}
```

8.1.2 P3953 [NOIP 2017 提高组] 逛公园

```
const int N = 100005;
const int K = 51;
const int INF = 0x3f3f3f3f;
class OriEdge {
public:
    int v, w;
};

class OriNode {
public:
    vector<OriEdge> oe;
    int dfn, low;
    int root;
    bool inStack;
};

class Edge {
public:
    int v, w;
};

class Node {
public:
    vector<Edge> oe;
    int dis;
    int ie;
    long long dp[K];
    bool onzl, visited;
};

OriNode old[N];
Node node[N];

int n, m, k, p;
int dfn = 1;
vector<int> st;

/*
 * 求 u 可达部分的强连通分量并记录分量编号
 */
void tarjan(int u) {
    old[u].low = old[u].dfn = dfn++;
    old[u].root = u;
    st.push_back(u);
    old[u].inStack = true;
    for(OriEdge& e : old[u].oe) {
        if(!e.w) {
            if(!old[e.v].dfn) {
                tarjan(e.v);
            }
            if(old[u].low > old[e.v].low && old[e.v].inStack) {
                old[u].low = old[e.v].low;
            }
        }
    }

    if(old[u].low == old[u].dfn) {
        if(st.back() != u) {
            node[u].onzl = true;
        }
```

```
61
62          while(st.back() != u) {
63              old[st.back()].root = u;
64              old[st.back()].inStack = false;
65              st.pop_back();
66          }
67          st.pop_back();
68          old[u].inStack = false;
69      }
70  }
71
72  class State {
73  public:
74      int u, dis;
75      bool operator<(const State& O) const {
76          return dis > O.dis;
77      }
78  };
79
80  int ptr[N];
81
82  int main() {
83      ios::sync_with_stdio(false);
84      cin.tie(nullptr);
85      int T;
86      cin >> T;
87      while(T--) {
88          cin >> n >> m >> k >> p;
89          dfn = 1;
90          st.clear();
91          for(int i = 1; i <= n; i++) {
92              old[i] = {{}, 0, 0, 0, false};
93              node[i] = {{}, INF, 0};
94              for(int j = 0; j <= k; j++) {
95                  node[i].dp[j] = 0;
96              }
97              node[i].onzl = false;
98              node[i].visited = false;
99          }
100         int u, v, w;
101         for(int i = 1; i <= m; i++) {
102             cin >> u >> v >> w;
103             old[u].oe.push_back({v, w});
104         }
105
106         for(int i = 1; i <= n; i++) {
107             if(!old[i].dfn) {
108                 tarjan(i);
109             }
110         }
111
112         long long ans;
113         {
114             for(u = 1; u <= n; u++) {
115                 for(OriEdge& e : old[u].oe) {
116                     int nu = old[u].root;
117                     int nv = old[e.v].root;
118                     if(nu != nv) {
119                         node[nu].oe.push_back({nv, e.w});
120                     }
121                 }
```

```
122             }
123
124             node[1].dis = 0;
125             priority_queue<State> q;
126             q.push({1, 0});
127             State cur;
128             while(!q.empty()) {
129                 cur = q.top();
130                 q.pop();
131                 if(node[cur.u].visited) {
132                     continue;
133                 } else {
134                     node[cur.u].visited = true;
135                 }
136                 for(Edge& e : node[cur.u].oe) {
137                     int dis = node[cur.u].dis + e.w;
138                     if(node[e.v].dis > dis) {
139                         node[e.v].dis = dis;
140                         node[e.v].ie = 1;
141                         q.push({e.v, dis});
142                     } else if(node[e.v].dis == dis) {
143                         node[e.v].ie++;
144                     }
145                 }
146             }
147             queue<int> topo;
148             for(int i = 1; i <= n; i++) {
149                 if(node[i].visited) {
150                     if(node[i].onzl) {
151                         node[i].dp[0] = -1;
152                     }
153                     if(!node[i].ie) {
154                         topo.push(i);
155                     }
156                 }
157             }
158
159             int tidx = 1;
160             while(!topo.empty()) {
161                 u = topo.front();
162                 topo.pop();
163                 ptr[tidx++] = u;
164                 for(Edge& e : node[u].oe) {
165                     if(node[u].dis + e.w - node[e.v].dis == 0) {
166                         node[e.v].ie--;
167                         if(!node[e.v].ie) {
168                             topo.push(e.v);
169                         }
170                     }
171                 }
172             }
173
174             if(!node[1].onzl) {
175                 node[1].dp[0] = 1;
176             }
177             for(int i = 0; i <= k; i++) {
178                 for(int j = 1; j < tidx; j++) {
179                     u = ptr[j];
180                     for(Edge& e : node[u].oe) {
181                         if(node[e.v].dp[i] != -1) {
182                             int g = node[u].dis + e.w - node[e.v].dis;
```

```
                            if(i + g <= k) {
                                if(node[u].dp[i] == -1) {
                                    node[e.v].dp[i + g] = -1;
                                } else {
                                    node[e.v].dp[i + g] += node[u].dp[i];
                                    node[e.v].dp[i + g] %= p;
                                }
                            }
                        }
                    }
                }
            }

            ans = 0;
            int root_n = old[n].root;
            for(int i = 0; i <= k; i++) {
                if(node[root_n].dp[i] == -1) {
                    ans = -1;
                    break;
                } else {
                    ans += node[root_n].dp[i];
                    ans %= p;
                }
            }
        }
        cout << (ans) << '\n';
    }
}
```

注意

Dijkstra 用 `visited` 判重，判不可达直接与 `INF` 比较；要求边权非负。

8.2 全源最短路

时间复杂度 Floyd $O(n^3)$；Johnson $O(nm \log n)$

稠密图（$n \lesssim 500$）用 Floyd，$O(n^3)$；稀疏图（m 与 n 同阶）用 Johnson，$O(nm \log n)$。两者都允许负边，都要求图中没有负环。读入前先把距离全部置 `INF`、`dis[i][i]` 置 0，重边取 min。

8.2.1 Floyd

```
const int N = 505;
const int64 INF = 0x3f3f3f3f3f3f3f3f;

int64 dis[N][N];

/*
 * n 个点的全源最短路，调用前 dis 已按上面的方式初始化
 */
void floyd(int n) {
    for(int k = 1; k <= n; k++) {
        for(int u = 1; u <= n; u++) {
            if(dis[u][k] >= INF) {
                continue;
            }
            for(int v = 1; v <= n; v++) {
                if(dis[k][v] < INF && dis[u][k] + dis[k][v] < dis[u][v]) {
                    dis[u][v] = dis[u][k] + dis[k][v];
                }
            }
        }
```

```
    }
}
```

8.2.2 Johnson

```
const int N = 3005;
const int INF = 0x3f3f3f3f;

class Edge {
public:
    int v;
    int w;
};

class Node {
public:
    vector<Edge> oe;
    int dis;   // johnson(s) 之后为 s 到本点的最短路
    int h;     // 势能，全图只求一次
};

Node node[N];
int n = 0;

/*
 * 所有点初值 0 同时入队，等价于虚拟源点向每个点连一条 0 权边
 * 调用一次求出势能 h；返回 false 表示图中有负环
 */
bool calc_potential() {
    vector<int> cnt(n + 1, 0);
    vector<bool> inq(n + 1, true);
    queue<int> q;
    for(int u = 1; u ≤ n; u++) {
        q.push(u);
    }
    while(!q.empty()) {
        int u = q.front();
        q.pop();
        inq[u] = false;
        for(auto& [v, w] : node[u].oe) {
            if(node[u].h + w < node[v].h) {
                node[v].h = node[u].h + w;
                cnt[v] = cnt[u] + 1;
                if(cnt[v] ⩾ n) {
                    return false;
                }
                if(!inq[v]) {
                    inq[v] = true;
                    q.push(v);
                }
            }
        }
    }
    return true;
}

/*
 * 负权单源最短路：先手动调用一次 calc_potential()，再对每个源点调用本函数
 * 之后 node[u].dis 为 s 到 u 的最短路
 */
void johnson(int s) {
```

```
    for(int u = 1; u ⩽ n; u++) {
        node[u].dis = INF;
    }

    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> pq;
    node[s].dis = 0;
    pq.push({0, s});
    while(!pq.empty()) {
        auto [dis, u] = pq.top();
        pq.pop();
        if(node[u].dis ≠ dis) {
            continue;
        }
        for(auto& [v, w] : node[u].oe) {
            int g = w + node[u].h - node[v].h;
            if(node[u].dis + g < node[v].dis) {
                node[v].dis = node[u].dis + g;
                pq.push({node[v].dis, v});
            }
        }
    }

    for(int u = 1; u ⩽ n; u++) {
        if(node[u].dis < INF) {
            node[u].dis += node[u].h - node[s].h;
        }
    }
}
```

注意

Floyd 的 `k` 必须在最外层；Johnson 先手动调用一次 `calc_potential()`（返回 false 表示有负环），再对每个源点调用 `johnson(s)`，之后读 `node[u].dis`。

8.3 差分约束

时间复杂度 $O(nm)$

8.3.1 P5960 【模板】差分约束

```
#define N 5005
class Edge {
public:
    int v, l;
};
int n, m;
int ie[N];
vector<Edge> oe[N];
int dist[N];
int io_cnt[N];
bool in_queue[N];
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m;
    int u, v, l;
    while(m-- > 0) {
        cin >> v >> u >> l;
        ie[v]++;
        oe[u].push_back({v, l});
    }

```

```
    for(int i = 1; i ⩽ n; i++) {
        ie[i]++;
        oe[0].push_back({i, 0});
    }

    memset(dist, 0x3f, sizeof(int) * (n + 1));
    dist[0] = 0;
    queue<int> q;
    q.push(0);
    int temp;
    bool negative_circle = false;
    in_queue[0] = true;
    while(!q.empty()) {
        temp = q.front();
        q.pop();
        io_cnt[temp]++;
        in_queue[temp] = false;
        if(io_cnt[temp] > n) {
            negative_circle = true;
            break;
        }
        for(Edge& e : oe[temp]) {

            if(dist[temp] + e.l < dist[e.v]) {
                dist[e.v] = dist[temp] + e.l;
                if(!in_queue[e.v]) {
                    in_queue[e.v] = true;
                    q.push(e.v);
                }
            }
        }
    }
    if(!negative_circle) {
        for(int i = 1; i ⩽ n; i++) {
            cout << (dist[i]) << ' ';
        }
    } else {
        cout << "NO";
    }
}
```

注意

最长路/最短路方向与不等式符号要统一；有负环则无解。

第 9 章

树上问题

9.1 LCA

时间复杂度 $O(\log n)$

```
/*
 * 返回 x、y 的最近公共祖先
 */
int getLCA(int x, int y) {
    if(node[x].d < node[y].d) {
        swap(x, y);
    }
    for(int i = LOGN - 1; i ⩾ 0 && node[x].d > node[y].d; i--) {
        if(node[x].d ⩾ node[y].d + (1 << i)) {
            x = node[x].fa[i];
        }
    }
    if(x == y) {
        return x;
    } else {
        for(int i = LOGN - 1; i ⩾ 0; i--) {
            if(node[x].fa[i] != node[y].fa[i]) {
                x = node[x].fa[i];
                y = node[y].fa[i];
            }
        }
        return node[x].fa[0];
    }
}

/*
 * 根据 fa[0] 预处理倍增祖先，深度在调用前填好
 */
void init() {
    for(int i = 1; i < LOGN; i++) {
        for(int j = 1; j ⩽ n; j++) {
            node[j].fa[i] = node[node[j].fa[i - 1]].fa[i - 1];
        }
    }
}
```

注意

倍增表 LOGN 要满足 2^LOGN > n。

9.2 直径

时间复杂度 $O(n)$

9.2.1 P3304 [SDOI2013] 直径

```
#define N 200005
class Edge {
public:
    int64 v, l, cnt;
};
```

```

class Node {
public:
    vector<Edge> oe;
    int64 depth;
};

Node node[N];
int64 n;

int64 max_d = 0;
int64 max_len = 0;
int64 s_idx = 0;
int64 le = -1, re = -1;
vector<int64> diameter;

void toOneSide(int64 idx, int64 fa, int64 d) {
    if(d > max_d) {
        s_idx = idx;
        max_d = d;
    }
    for(Edge& e : node[idx].oe) {
        if(e.v != fa) {
            toOneSide(e.v, idx, d + e.l);
        }
    }
}

void getDiameter(int64 idx, int64 fa, int64 d) {
    if(d > max_len && node[idx].oe.size() == 1) {
        max_len = d;
    }
    for(Edge& e : node[idx].oe) {
        if(e.v != fa) {
            getDiameter(e.v, idx, d + e.l);
        }
    }
}

bool getDiameterNode(int64 idx, int64 fa, int64 d) {
    diameter.push_back(idx);
    if(d == max_len) {
        return true;
    }
    for(Edge& e : node[idx].oe) {
        if(e.v != fa) {
            if(getDiameterNode(e.v, idx, d + e.l)) {
                return true;
            }
        }
    }
    diameter.pop_back();
    return false;
}

bool getMiddle(int64 idx, int64 fa, int64 d, int64 node_idx) {
    if(d * 2 == max_len) {
        le = re = idx;
        return true;
    }
    for(Edge& e : node[idx].oe) {
```

```
        if(e.v == diameter[node_idx + 1]) {
            if((d + e.l) * 2 > max_len) {
                le = idx;
                re = e.v;
                return true;
            }
            if(getMiddle(e.v, idx, d + e.l, node_idx + 1)) {
                return true;
            }
        }
    }
    return false;
}

int64 getSharedBranch(int64 idx, int64 fa, int64 d) {
    int64 cnt = 0, cnt_sum = 0;
    if(d == max_len) {
        return 1;
    } else {
        for(Edge& e : node[idx].oe) {
            if(e.v != fa) {
                cnt = getSharedBranch(e.v, idx, d + e.l);
                e.cnt = cnt;
                cnt_sum += cnt;
            }
        }
        return cnt_sum;
    }
}

int64 getDepth(int64 idx, int64 fa, int64 d) {
    int64 maximum = d;
    node[idx].depth = d;
    for(Edge& e : node[idx].oe) {
        if(e.v != fa) {
            maximum = max(maximum, getDepth(e.v, idx, d + e.l));
        }
    }
    return maximum;
}

int64 getCount(int64 idx, int64 fa, int64 d, int64 td) {
    if(d == td) {
        if(node[idx].oe.size() == 1 && d != 0) {
            return 1;
        } else {
            return 0;
        }
    }
    int64 cnt = 0, cnt_sum = 0;
    for(Edge& e : node[idx].oe) {
        if(e.v != fa) {
            cnt = getCount(e.v, idx, d + e.l, td);
            e.cnt = cnt;
            cnt_sum += cnt;
        }
    }
    return cnt_sum;
}

int64 getSharedNode(int64 idx, int64 fa, int64 tc) {
```

```
    int64 res = 0;
    for(Edge& e : node[idx].oe) {
        if(e.v != fa && e.cnt == tc) {
            res += getSharedNode(e.v, idx, tc) + 1;
        }
    }
    return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n;

    int64 u, v, l;
    for(int64 i = 1; i < n; i++) {
        cin >> u >> v >> l;
        node[u].oe.push_back({v, l, 0});
        node[v].oe.push_back({u, l, 0});
    }

    toOneSide(1, 0, 0);
    getDiameter(s_idx, 0, 0);
    getDiameterNode(s_idx, 0, 0);
    getMiddle(s_idx, 0, 0, 0);

    int64 l_max, r_max, l_cnt, r_cnt, l_share, r_share, middle;
    int64 e_cnt = 0;

    if(le == re) {
        middle = le;
        l_max = getDepth(le, re, 0);
        l_cnt = getCount(le, re, 0, l_max);

        int64 branch_cnt = node[middle].oe.size();
        for(Edge& e : node[middle].oe) {
            if(e.cnt != 0) {
                branch_cnt--;
                le = e.v;
            }
        }
        re = middle;

        if(branch_cnt != 2) {
            cout << (max_len) << '\n'
                 << (0ll) << '\n';
        } else {
            e_cnt = getSharedNode(le, 0, l_cnt);
        }
    }

    l_max = getDepth(le, re, 0);
    r_max = getDepth(re, le, 0);
    l_cnt = getCount(le, re, 0, l_max);
    r_cnt = getCount(re, le, 0, r_max);

    l_share = 0;
    r_share = 0;
    l_share = getSharedNode(le, re, l_cnt);
    r_share = getSharedNode(re, le, r_cnt);
    e_cnt = l_share + r_share + 1;
```

```
    cout << (max_len) << '\n'
         << (e_cnt) << '\n';
}
```

9.2.2 B4016 树的直径

```
const int N = 100005;

class Node {
public:
    vector<int> oe;
    int fa;
    int d;
};

class Graph {
public:
    Node node[N];
    int n = 0;
    int d_s = 0, d_t = 0, d_l = 0;
    vector<int> diameter;

    /*
     * 初始化 n 个点的空树
     */
    void init(int n_) {
        n = n_;
        for(int i = 0; i ⩽ n; i++) {
            node[i].oe.clear();
            node[i].fa = 0;
            node[i].d = 0;
        }
        d_s = d_t = d_l = 0;
        diameter.clear();
    }

    /*
     * 添加无向边
     */
    void add_edge(int u, int v) {
        node[u].oe.push_back(v);
        node[v].oe.push_back(u);
    }

    /*
     * 从 u 出发求各点距离与父亲，同时记录最远点
     */
    void dfs(int u, int fa, int d) {
        node[u].fa = fa;
        node[u].d = d;
        if(d > d_l) {
            d_l = d;
            d_s = u;
        }
        for(int v : node[u].oe) {
            if(v ≠ fa) {
                dfs(v, u, d + 1);
            }
        }
    }

```

```
    /*
     * 一次调用求直径：diameter 保存路径，d_l 保存长度
     */
    void get_diameter(int start = 1) {
        d_l = 0;
        d_s = start;
        dfs(start, 0, 1);
        int s = d_s;
        d_l = 0;
        d_s = s;
        dfs(s, 0, 1);
        int t = d_s;
        d_l = node[t].d-node[s].d;
        d_s = s;
        d_t = t;
        diameter.clear();
        for(int u = t; u; u = node[u].fa) {
            diameter.push_back(u);
            if(u == s) {
                break;
            }
        }
        reverse(diameter.begin(), diameter.end());
    }
};

Graph graph;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, u, v;
    cin >> n;
    graph.init(n);
    for(int i = 1; i < n; i++) {
        cin >> u >> v;
        graph.add_edge(u, v);
    }
    graph.get_diameter();
    cout << graph.d_l << '\n';
}
```

注意

两次 DFS 只适用于非负边权；`get_diameter` 把路径存入 `diameter`，长度存入 `d_l`。递归深度为树高，深链需开栈；带负权要用树形 DP。

9.3 偏心距

时间复杂度 $O(n)$

9.3.1 P1099 [NOIP 2007 提高组] 树网的核

```
const int N = 305;
int n, s;

class Edge {
public:
    int v;
    int l;
};
```

```

class Node {
public:
    int dis;
    bool on_diameter;
    vector<Edge> oe;
};

Node node[N];

vector<Edge> path;
vector<Edge> temp;
int max_d = 0, start;

void dfs1(int u, int fa, int d) {
    if(d > max_d) {
        start = u;
        max_d = d;
    }

    for(Edge& e : node[u].oe) {
        if(e.v != fa) {
            dfs1(e.v, u, d + e.l);
        }
    }
}

void dfs2(int u, int fa, int d) {
    if(d > max_d) {
        max_d = d;
        if(node[u].oe.size() <= 1) {
            path.assign(temp.begin(), temp.end());
        }
    }

    for(Edge& e : node[u].oe) {
        if(e.v != fa) {
            temp.push_back(e);
            dfs2(e.v, u, d + e.l);
            temp.pop_back();
        }
    }
}

int dfs3(int u, int fa) {
    for(Edge& e : node[u].oe) {
        if(e.v != fa) {
            node[u].dis = max(node[u].dis, dfs3(e.v, u) + e.l);
        }
    }
    return node[u].dis;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> s;
    int u, v, l;
    for(int i = 1; i < n; i++) {
        cin >> u >> v >> l;
        node[u].oe.push_back({v, l});
```

```
        node[v].oe.push_back({u, l});
    }

    max_d = -2147483648;
    dfs1(1, 0, 0);
    max_d = -2147483648;
    temp.push_back({start, 0});
    dfs2(start, 0, 0);

    int d = 0, cnt = 0;
    int middle = 0;
    int head, tail;
    for(int i = 0; i < path.size(); i++) {
        Edge& e = path[i];
        if((d + e.l) > max_d / 2) {
            if(max_d / 2 - d ⩾ d + e.l - max_d / 2) {
                middle = i;
            } else {
                middle = i - 1;
            }
            break;
        }
        d += e.l;
    }

    for(int i = 0; i < path.size(); i++) {
        Edge& e = path[i];
        node[e.v].on_diameter = true;
    }

    dfs3(path[middle].v, 0);

    int min_bias = 2147483647;
    int bias;
    bool is_legal;

    for(int i = 0; i ⩽ middle; i++) {
        d = 0;
        bias = 0;
        is_legal = false;
        if(i > 0) {
            bias = max(bias, node[path[i - 1].v].dis + path[i].l);
        }
        for(int j = i; j < path.size(); j++) {
            if(i ≠ j) {
                d += path[j].l;
            }
            if(d > s) {
                bias = max(bias, node[path[j].v].dis + path[j].l);
                break;
            }
            if(j == middle) {
                is_legal = true;
            }
            for(Edge& e : node[path[j].v].oe) {
                if(!node[e.v].on_diameter) {
                    bias = max(bias, node[e.v].dis + e.l);
                }
            }
        }
        if(is_legal) {
```

```
            min_bias = min(min_bias, bias);
        }
    }

    cout << (min_bias);
}
```

注意

单调队列维护直径上的最优区间。

9.4 树链剖分

时间复杂度 $O(\log^2 n)$

9.4.1 P3384 【模板】重链剖分/树链剖分

```
class HLD {
public:
    class Node {
    public:
        vector<int64> oe;
        int64 fa, d, w;
        int64 heavy, dfn, link_head;
    };

    vector<Node> node;
    int64 root = 1;
    int64 dfncnt = 0;

    /*
     * 初始化 n 个点的空树
     * 注意初始化半群结构，注意半群初始化需要 dfninv 转换
     */
    void init(int64 n) {
        node.assign(n + 1, Node());
        root = 1;
        dfncnt = 0;
    }

    /*
     * 添加无向边
     */
    void add_edge(int64 u, int64 v) {
        node[u].oe.push_back(v);
        node[v].oe.push_back(u);
    }

    void dfs1(int u, int fa) {
        node[u].fa = fa;
        node[u].d = node[fa].d + 1;
        node[u].w = 1;
        node[u].heavy = 0;
        int best = 0;
        for(int v : node[u].oe) {
            if(v != fa) {
                dfs1(v, u);
                node[u].w += node[v].w;
                if(node[v].w > best) {
                    best = node[v].w;
                    node[u].heavy = v;
                }
```

```
            }
        }
    }

    void dfs2(int u, int h) {
        node[u].dfn = ++dfncnt;
        node[u].link_head = h;
        if(node[u].heavy) {
            dfs2(node[u].heavy, h);
        }
        for(int v : node[u].oe) {
            if(v != node[u].fa && v != node[u].heavy) {
                dfs2(v, v);
            }
        }
    }

    /*
     * 以 u 为根建立树链剖分
     */
    void build(int64 u) {
        root = u;
        dfncnt = 0;
        dfs1(root, 0);
        dfs2(root, root);
    }

    /*
     * 求 u、v 的最近公共祖先
     */
    int64 lca(int64 u, int64 v) {
        while(node[u].link_head != node[v].link_head) {
            if(node[node[u].link_head].d < node[node[v].link_head].d) {
                swap(u, v);
            }
            u = node[node[u].link_head].fa;
        }
        return node[u].d < node[v].d ? u : v;
    }

    /*
     * 对 dfn 区间 [l, r] 施加修改 x; 在这里接入题目需要的半群
     */
    void dfnmod(int l, int r, int x) {
    }

    /*
     * 查询 dfn 区间 [l, r], x 为题目需要的查询参数
     */
    int64 dfnquery(int l, int r) {
        return 0;
    }

    /*
     * 将 u 到 v 路径上的点权加 x
     */
    void add(int u, int v, int x) {
        while(node[u].link_head != node[v].link_head) {
            if(node[node[u].link_head].d < node[node[v].link_head].d) {
                swap(u, v);
            }
```

```
            dfnmod(node[node[u].link_head].dfn, node[u].dfn, x);
            u = node[node[u].link_head].fa;
        }
        if(node[u].d > node[v].d) {
            swap(u, v);
        }
        dfnmod(node[u].dfn, node[v].dfn, x);
    }

    /*
     * 查询 u 到 v 路径上的信息
     */
    int query(int u, int v) {
        int res = 0;
        while(node[u].link_head != node[v].link_head) {
            if(node[node[u].link_head].d < node[node[v].link_head].d) {
                swap(u, v);
            }
            res += dfnquery(node[node[u].link_head].dfn, node[u].dfn);
            u = node[node[u].link_head].fa;
        }
        if(node[u].d > node[v].d) {
            swap(u, v);
        }
        res += dfnquery(node[u].dfn, node[v].dfn);
        return res;
    }
};
```

9.4.2 P4069 [SDOI2016] 游戏

```
const int64 INF = 0x3f3f3f3f3f3f3f3f;
const int64 INIT_VALUE = 123456789123456789;
const int64 N = 100005;

class Edge {
public:
    int64 v, l;
};

class HLDTreeNode {
public:
    vector<Edge> oe;                    // 边，含指向父节点的边和重边
    int64 dfn, fa, w, link_head, d, dis; // link_head 为链顶，fa 为父节点，dis 为该点到根的距离
    int64 segmentTreeIdx;               // 该点所在链对应的李超线段树的下标
    int64 heavy;                        // 重边下标
};

class HLDTree { // 树链剖分的树 Heavy-Light Decomposed Treee
public:
    static const int64 MAX_N = N;
    int64 root = 1;
    int64 dfn = 1;
    HLDTreeNode node[MAX_N]; // 所有的节点
    vector<LiChaoSegmentTree> segmentTree;

    void dfs1(int64 cur, int64 fa, int64 d, int64 dis) { // 第一次 dfs
        node[cur].fa = fa;
        node[cur].w = 1;
        node[cur].d = d;
        node[cur].dis = dis;
        int64 heavy = -1, w = 0;
```

```
        for(int64 i = 0; i < node[cur].oe.size(); i++) {
            Edge& e = node[cur].oe[i];
            if(e.v != fa) {

                dfs1(e.v, cur, d + 1, dis + e.l);
                node[cur].w += node[e.v].w;
                if(node[e.v].w > w) {
                    w = node[e.v].w;
                    heavy = i; // 找重链，记录重链在 oe 的下标
                }
            }
        }
        node[cur].heavy = heavy;
    }

    void dfs2(int64 cur, int64 fa, int64 link_head) {

        inv_dfn[dfn] = cur;
        node[cur].dfn = dfn++;
        node[cur].link_head = link_head;
        if(node[cur].heavy >= 0) {
            dfs2(node[cur].oe[node[cur].heavy].v, cur, link_head);
        } else {
            vector<int64> a(1);
            for(int i = node[link_head].dfn; i <= node[cur].dfn; i++) {
                a.push_back(node[inv_dfn[i]].dis);
            }
            node[link_head].segmentTreeIdx = segmentTree.size();
            segmentTree.emplace_back();
            segmentTree.back().build(a, INIT_VALUE);
        }
        node[cur].segmentTreeIdx = node[link_head].segmentTreeIdx;
        for(int i = 0; i < node[cur].oe.size(); i++) {
            Edge& e = node[cur].oe[i];
            if(e.v != fa && i != node[cur].heavy) {
                dfs2(e.v, cur, e.v);
            }
        }
    }

public:
    int64 inv_dfn[MAX_N];

    /*
     * 添加无向边
     */
    void add_edge(int64 u, int64 v, int64 l) {
        node[u].oe.push_back({v, l});
        node[v].oe.push_back({u, l});
    }

    /*
     * 以 u 为根建立树链剖分
     */
    void build(int64 u) {

        root = u;
        dfn = 1;
        segmentTree.clear();
        dfs1(root, 0, 1, 0);
        dfs2(root, 0, root);
```

```
    }

    /*
     * 返回 x、y 的最近公共祖先
     */
    int64 find_lca(int64 x, int64 y) {
        while(node[x].link_head != node[y].link_head) { // 先找 LCA 和计算 st 之间的距离
            if(node[node[x].link_head].d < node[node[y].link_head].d) {
                swap(x, y);
            }
            x = node[node[x].link_head].fa;
        }
        return node[x].d < node[y].d ? x : y;
    }

    /*
     * 在 s 到 t 路径的每个点 r 加入 a*dist(s,r)+b
     */
    void add_line(int64 s, int64 t, int64 a, int64 b) {
        int64 x = s, y = t;
        int64 lca = find_lca(s, t);

        while(node[node[x].link_head].d > node[lca].d) { // s 向 lca 计算
            segmentTree[node[x].segmentTreeIdx].add_line(1, node[x].dfn - node[node[x].link_head].dfn + 1, -a, b +
            ↪  node[s].dis * a);
            x = node[node[x].link_head].fa;
        }

        while(node[node[y].link_head].d > node[lca].d) { // t 向 lca 计算
            segmentTree[node[y].segmentTreeIdx].add_line(1, node[y].dfn - node[node[y].link_head].dfn + 1, a, b +
            ↪  (node[s].dis - node[lca].dis * 2) * a);
            y = node[node[y].link_head].fa;

            //考虑 dfn，更接近 lca 的 dfn 更小，且到 s 的距离更小，所以 k=a
        }

        // 最后 x 和 y 在同一条链上
        if(node[x].dfn ≤ node[y].dfn) { // 如果 x 是 y 的祖宗 或者它俩是一个点
            segmentTree[node[x].segmentTreeIdx].add_line(node[x].dfn - node[node[x].link_head].dfn + 1, node[y].dfn -
            ↪  node[node[x].link_head].dfn + 1, a, b + (node[s].dis - node[lca].dis * 2) * a);

            // x 的 dfn 更小，更靠近 x 的节点，到 s 的距离更小，所以 k=a
        } else { //
            segmentTree[node[y].segmentTreeIdx].add_line(node[y].dfn - node[node[y].link_head].dfn + 1, node[x].dfn -
            ↪  node[node[y].link_head].dfn + 1, -a, b + node[s].dis * a);
            // y 的 dfn 更小，更靠近 y 的节点，到 s 的距离更大，所以 k=-a
        }
    }

    /*
     * 查询 x 到 y 路径上的最小值
     */
    int64 get_minimum(int64 x, int64 y) {
        int64 ans = INF;
        while(node[x].link_head != node[y].link_head) {
            if(node[node[x].link_head].d < node[node[y].link_head].d) {
                swap(x, y); // 轮流上跳
            }
            ans = min(ans, segmentTree[node[x].segmentTreeIdx].range_query(1, node[x].dfn -
            ↪  node[node[x].link_head].dfn + 1));
            x = node[node[x].link_head].fa;
```

```
        }

        ans = min(ans,
                  segmentTree[node[x].segmentTreeIdx].range_query(min(node[x].dfn, node[y].dfn) -
                  ↪ node[node[x].link_head].dfn + 1, max(node[x].dfn, node[y].dfn) - node[node[x].link_head].dfn +
                  ↪ 1));
        return ans;
    }
};

HLDTree hldTree;

int64 n, m;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n;
    cin >> m;

    int64 u = 1, v, l;
    for(int64 i = 1; i < n; i++) {
        cin >> u;
        cin >> v;
        cin >> l;

        hldTree.add_edge(u, v, l);
    }
    hldTree.build(1);

    int64 op, s, t, a, b;
    while(m--) {
        cin >> op;
        cin >> s;
        cin >> t;
        if(op == 1) {
            cin >> a;
            cin >> b;

            hldTree.add_line(s, t, a, b);
        } else if(op == 2) {
            cout << (hldTree.get_minimum(s, t)) << '\n';
        }
    }
}
```

注意

`dfnmod` / `dfnquery` 是接入线段树或树状数组的位置，l、r 为 dfn 下标；子树区间为 [dfn[x], dfn[x] + w[x] - 1]。

1000 苹果树

OCS-4.9.45

Problem Description

给出一棵包含 n 个点的树，每个节点 i 都有一个权值 a_i。

有 m 次操作，每次操作都形如以下的两种：

- `1 x y`：查询 x 到 y 的路径上，最大的点权权值。
- `2 x z`：对于所有与 x 直接相连的点 i，令 $a_i \leftarrow a_i + z$。

不保证查询的 x, y 满足 $x \neq y$。

Input

每个测试点中包含多组测试数据。输入的第一行包含一个正整数 $T(1 \leq T \leq 110)$，表示数据组数。对于每组测试数据：

第一行两个正整数 $n, m(1 \leq n, m \leq 10^5)$。

第二行 n 个整数 $a_1, a_2, \cdots, a_n(1 \leq a_i \leq 10^4)$，表示初始每个节点的点权。

接下来 $n-1$ 行，每行两个正整数 $x, y(1 \leq x, y \leq n)$，表示有一条从 x 到 y 的无向边。

接下来 m 行，每行三个整数，第一个数 opt 表示操作类型：

- 若 $\text{opt} = 1$，则后面两个数 $x, y(1 \leq x, y \leq n)$，表示询问路径。
- 若 $\text{opt} = 2$，则后面两个数 $x, z(1 \leq x \leq n, 1 \leq z \leq 10^4)$，分别表示修改中心以及增加的值。

保证所有测试数据中 n 之和与 m 之和均不超过 4×10^5。

Output

对于每组测试数据：对于每一个 $\mathrm{opt} = 1$ 的操作，输出一行一个数表示答案。

OCS-4.9.45

Sample Input

```
1
5 10
3 7 9 1 6
2 1
3 1
4 2
5 4
2 1 2
2 5 3
1 1 4
1 3 1
2 4 3
2 2 9
2 1 5
1 4 2
2 3 4
1 4 4
```

Sample Output

```
9
11
17
13
```

9.5 点分治

时间复杂度 $O(n \log n)$

9.5.1 P4178 Tree

```
const int N = 40005;
const int K = 20005;

class Edge {
public:
    int v, w;
};

class Node {
public:
    vector<Edge> oe;
    int w;
    bool exist;
};

Node node[N];

int n;

void getW(int u, int fa) {
    node[u].w = 1;
    for(Edge& e : node[u].oe) {
        if(e.v != fa && node[e.v].exist) {
            getW(e.v, u);
            node[u].w += node[e.v].w;
        }
    }
}

int getCentroid(int u, int fa, int tot) {
    if(node[u].w * 2 < tot) {
        return 0;
    } else {
        for(Edge& e : node[u].oe) {
            if(e.v != fa && node[e.v].exist) {
                int ans = getCentroid(e.v, u, tot);
                if(ans > 0) {
                    return ans;
                }
            }
        }
        return u;
    }
}

BIT<> judge;
```

```
map<int, int> tmpmem;
int k;

void dfs(int u, int fa, int d) {
    if(d <= k) {
        if(tmpmem.count(d)) {
            tmpmem[d]++;
        } else {
            tmpmem[d] = 1;
```

```
        }
    }
    node[u].w = 1;
    for(Edge& e : node[u].oe) {
        if(e.v != fa && node[e.v].exist) {
            dfs(e.v, u, d + e.w);
            node[u].w += node[e.v].w;
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n;
    int u, v, w;
    long long ans = 0;

    for(int i = 1; i < n; i++) {
        node[i].exist = true;
        cin >> u >> v >> w;
        node[u].oe.push_back({v, w});
        node[v].oe.push_back({u, w});
    }
    node[n].exist = true;

    cin >> k;

    getW(1, 0);
    u = getCentroid(1, 0, node[1].w);
    judge.init(k + 1);
    queue<int> que;
    que.push(u);
    while(!que.empty()) {
        u = que.front();
        que.pop();
        node[u].exist = false;

        fill(judge.arr.begin(), judge.arr.end(), 0);
        judge.add(1, 1);

        for(Edge& e : node[u].oe) {
            if(node[e.v].exist) {
                tmpmem.clear();
                dfs(e.v, u, e.w);
                for(pair<int, int> p : tmpmem) {
                    ans += p.second * judge.prefix(k - p.first + 1);
                }
                for(pair<int, int> p : tmpmem) {
                    judge.add(p.first + 1, p.second);
                }
                que.push(getCentroid(e.v, u, node[e.v].w));
            }
        }
    }

    cout << (ans);
}
```

注意

每次找重心，容斥减去同一子树内的路径。

9.6 动态重心

时间复杂度 $O(\log n)$ 均摊

```
const int N = 100005;
int n, m;

class Data {
public:
    long long w, vw;
};

class Tag {
public:
    bool rev;
};

const Tag TAG_EP = {false};
const Tag TAG_REV = {true};

class Node {
public:
    int son[2];
    int fa;
    Data data;
    Tag tag;
    void operator+=(const Tag& t) {
        if(t.rev) {
            swap(son[0], son[1]);
            tag.rev ^= t.rev;
        }
    }
};

class LCT {
#define ls(u) (node[u].son[0])
#define rs(u) (node[u].son[1])
#define fa(u) (node[u].fa)
#define data(u) (node[u].data)
#define tag(u) (node[u].tag)
public:
    const static int MAXN = N;
    Node node[MAXN];

    bool dir(const int u) {
        return rs(fa(u)) == u;
    }

    bool is_root(const int u) {
        return ls(fa(u)) != u && rs(fa(u)) != u;
    }

    void push_up(const int u) {
        data(u).w = data(ls(u)).w + data(rs(u)).w + data(u).vw + 1;
    }

    void push_down(const int u) {
        if(ls(u)) {
            node[ls(u)] += tag(u);
        }
        if(rs(u)) {
```

```
58              node[rs(u)] += tag(u);
59          }
60          tag(u) = TAG_EP;
61      }
62
63      void update(int u) {
64          if(!is_root(u)) {
65              update(fa(u));
66          }
67          push_down(u);
68      }
69
70      void rot(int u) {
71          int y = fa(u), z = fa(y), k = dir(u), v = node[u].son[!k];
72          if(!is_root(y)) {
73              node[z].son[dir(y)] = u;
74          }
75          node[y].son[k] = v;
76          node[u].son[!k] = y;
77          fa(v) = y;
78          fa(y) = u;
79          fa(u) = z;
80          push_up(y);
81          push_up(u);
82      }
83
84      void splay(int u) {
85          update(u);
86          for(int f = fa(u); !is_root(u); rot(u), f = fa(u)) {
87              if(!is_root(f)) {
88                  rot(dir(u) == dir(f) ? f : u);
89              }
90          }
91      }
92
93      /*
94       * 打通原树根到 u 的实链
95       */
96      int access(int u) {
97          int v;
98          for(v = 0; u; v = u, u = fa(u)) {
99              splay(u);
100             data(u).vw += data(rs(u)).w - data(v).w;
101             rs(u) = v;
102             push_up(u);
103         }
104         return v;
105     }
106
107     /*
108      * 将 u 设为所在原树的根
109      */
110     void make_root(int u) {
111         access(u);
112         splay(u);
113         node[u] += TAG_REV;
114     }
115
116     /*
117      * 返回 u 所在原树的根
118      */
```

```
    int find_root(int u) {
        access(u);
        splay(u);
        push_down(u);
        while(ls(u)) {
            u = ls(u);
            push_down(u);
        }
        splay(u);
        return u;
    }

    /*
     * 将 x 到 y 的路径暴露到以 y 为根的辅助树，x、y 须连通
     */
    void split(int x, int y) {
        make_root(x);
        access(y);
        splay(y);
    }

    /*
     * 连接 x、y；只在两点不连通时加边
     */
    void link(int x, int y) {
        make_root(x);
        if(find_root(y) != x) {
            splay(y);
            fa(x) = y;
            data(y).vw += data(x).w;
            push_up(y);
        }
    }

    /*
     * 断开 x、y 之间的直接边
     */
    void cut(int x, int y) {
        make_root(x);
        if(find_root(y) == x && fa(y) == x && !ls(y)) {
            fa(y) = rs(x) = 0;
            push_up(x);
        }
    }

    /*
     * 沿维护的权值寻找重心候选
     */
    int find_w(int u) {
        long long lw = 0, fw = 0;
        long long tot = data(u).w;
        int lans = 0, rans = 0;
        while(u) {
            push_down(u);
            int ow = data(u).w - data(rs(u)).w;
            if((fw + ow) * 2 <= tot) {
                fw += ow;
                lans = u;
                lw = fw;
                u = rs(u);
            } else {
```

```
                rans = u;
                u = ls(u);
            }
        }

        if(lw == tot - lw) {
            return min(lans, rans);
        } else if(lw < tot - lw) {
            return rans;
        } else {
            return lans;
        }
    }
#undef ls
#undef rs
#undef fa
#undef data
#undef tag
};

LCT lct;
DSU dsu;
int capi[N];
char op[128];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m;
    dsu.init(n);
    int ans = 0;
    for(int i = 1; i <= n; i++) {
        lct.node[i].data.w = 1;
        capi[i] = i;
        ans ^= i;
    }
    int x, y;
    while(m--) {
        cin >> (op);
        if(op[0] == 'A') {
            cin >> x >> y;
            int cx = capi[dsu.find(x)];
            int cy = capi[dsu.find(y)];
            lct.link(x, y);
            lct.split(cx, cy);
            int cn = lct.find_w(cy);
            lct.splay(cn);
            ans ^= cx;
            ans ^= cy;
            ans ^= cn;
            dsu.join(x, y);
            capi[dsu.find(x)] = cn;
        } else if(op[0] == 'Q') {
            cin >> x;
            cout << (capi[dsu.find(x)]) << '\n';
        } else {
            cout << (ans) << '\n';
        }
    }
}
```

注意

LCT 维护虚子树大小；答案在路径上二分。

9.7 虚树

时间复杂度 $O(k \log k)$

9.7.1 P2495 [SDOI2011] 消耗战 /【模板】虚树

```
const int N = 250005;
const int LOGN = 19;
const int64 INF = 0x3f3f3f3f3f3f3f3f;
int n, m;
class Edge {
public:
    int v, w;
};

class Node {
public:
    vector<Edge> oe;
    int fa[LOGN], mini[LOGN];
    int d, dfn;
};

Node node[N];

class VNode {
public:
    vector<Edge> oe;
    bool critical;
    int c;
    int64 dp;
};

VNode vnode[N];

int dfncnt = 1;

void build(int u, int fa, int d) {
    node[u].d = d;
    node[u].fa[0] = fa;
    node[u].dfn = dfncnt++;
    for(Edge& e : node[u].oe) {
        if(e.v != fa) {
            node[e.v].mini[0] = e.w;
            build(e.v, u, d + 1);
        }
    }
}

int a[N * 2];
int h[N];

bool cmp(const int A, const int B) {
    return node[A].dfn < node[B].dfn;
}

void update(int u, int c) {
    if(vnode[u].c != c) {
```

```
        vnode[u].oe.clear();
        vnode[u].dp = INF;
        vnode[u].c = c;
        vnode[u].critical = false;
    }
}

/*
 * 在虚树上合并子树信息
 */
void dfs(int u) {
    int64 sum = 0;
    for(Edge& e : vnode[u].oe) {
        vnode[e.v].dp = e.w;
        dfs(e.v);
        sum += vnode[e.v].dp;
    }
    if(!vnode[u].critical) {
        vnode[u].dp = min(vnode[u].dp, sum);
    }
}

/*
 * 返回 u、v 的最近公共祖先，需先建好倍增表
 */
int getLCA(int u, int v) {
    if(node[u].d < node[v].d) {
        swap(u, v);
    }
    for(int j = LOGN - 1; j >= 0; j--) {
        if(node[u].d - (1 << j) >= node[v].d) {
            u = node[u].fa[j];
        }
    }
    if(u == v) {
        return u;
    }
    for(int j = LOGN - 1; j >= 0; j--) {
        if(node[u].fa[j] != node[v].fa[j]) {
            u = node[u].fa[j];
            v = node[v].fa[j];
        }
    }
    return node[u].fa[0];
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int u, v, w;
    cin >> n;
    for(int i = 1; i < n; i++) {
        cin >> u >> v >> w;
        node[u].oe.push_back({v, w});
        node[v].oe.push_back({u, w});
    }

    build(1, 0, 1);

    for(int i = 1; i < LOGN; i++) {
        for(int j = 1; j <= n; j++) {
```

```
            node[j].fa[i] = node[node[j].fa[i - 1]].fa[i - 1];
            node[j].mini[i] = min(node[j].mini[i - 1], node[node[j].fa[i - 1]].mini[i - 1]);
        }
    }

    cin >> m;
    for(int T = 1; T ⩽ m; T++) {
        int k;
        cin >> k;
        for(int i = 1; i ⩽ k; i++) {
            cin >> h[i];
        }
        sort(h + 1, h + 1 + k, cmp);

        int l = 0;
        a[++l] = 1;
        for(int i = 1; i < k; i++) {
            a[++l] = h[i];
            a[++l] = getLCA(h[i], h[i + 1]);
        }
        a[++l] = h[k];

        sort(a + 1, a + 1 + l, cmp);
        l = unique(a + 1, a + 1 + l) - (a + 1);

        for(int i = 1; i < l; i++) {
            w = 0x3f3f3f3f;
            int lca = getLCA(a[i], a[i + 1]);
            int x = a[i + 1];
            for(int j = LOGN - 1; j ⩾ 0; j--) {
                if(node[x].d - (1 << j) ⩾ node[lca].d) {
                    w = min(w, node[x].mini[j]);
                    x = node[x].fa[j];
                }
            }
            update(lca, T);
            update(a[i + 1], T);
            vnode[lca].oe.push_back({a[i + 1], w});
        }

        for(int i = 1; i ⩽ k; i++) {
            vnode[h[i]].critical = true;
        }

        vnode[1].dp = INF;
        dfs(1);

        cout << (vnode[1].dp) << '\n';
    }
    return 0;
}
```

注意

LOGN 按 n 取；建虚树前按 dfn 排序并加入相邻点 LCA。

9.8 基环树

时间复杂度 $O(n)$

9.8.1 P7752 [COCI 2013/2014 #2] PALETA

```
const int N = 1000005;
```

```
const long long MODER = 1000000007;
class Node {
public:
    int sub;
    int ptr;
    int dfn_or_ie;
    bool on_loop;
    bool visited;
};
Node node[N];
int n, k;
int dfn = 1;
int begin_dfn;
int loop_root_dfn;

long long int mul(int a, int b) {
    long long temp = a;
    temp *= (long long)b;
    temp %= MODER;
    return temp;
}

/*
 * 从 idx 寻找基环并记录环上节点
 */
inline void findLoop(int idx) {
    stack<int> s;

    while(!s.empty()) {
        node[idx].dfn_or_ie = dfn;
        dfn++;

        int v = node[idx].ptr;
        if(node[v].dfn_or_ie >= begin_dfn) {
            loop_root_dfn = node[v].dfn_or_ie;
        }
        if(!loop_root_dfn && !node[v].dfn_or_ie) {
            s.push(v);
        } else {
            s.pop();
        }
    }

    while(!s.empty()) {
        if(loop_root_dfn && node[idx].dfn_or_ie >= loop_root_dfn) {
            node[idx].on_loop = true;
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> k;
    int num;
    for(int i = 1; i <= n; i++) {
        cin >> num;
        node[i].ptr = num;
    }
    long long ans = 1;
    for(int i = 1; i <= n; i++) {
```

```
        if(!node[i].dfn_or_ie) {
            begin_dfn = dfn;
            loop_root_dfn = 0;
            findLoop(i);
        }
    }

    for(int i = 1; i ⩽ n; i++) {
        node[i].dfn_or_ie = 0;
        node[i].sub = 1;
    }

    for(int i = 1; i ⩽ n; i++) {
        node[node[i].ptr].dfn_or_ie++;
    }

    int cur;
    for(int i = 1; i ⩽ n; i++) {
        cur = i;
        while(!node[cur].dfn_or_ie && !node[cur].visited) {
            node[cur].visited = true;
            if(!node[cur].on_loop) {
                node[cur].sub = mul(node[cur].sub, k - 1);
            }
            node[node[cur].ptr].sub = mul(node[node[cur].ptr].sub, node[cur].sub);
            node[node[cur].ptr].dfn_or_ie--;
            cur = node[cur].ptr;
        }
    }

    for(int i = 1; i ⩽ n; i++) {
        if(!node[i].visited) {
            node[i].visited = true;
            int res = node[i].sub;
            int on_loop_cnt;
            int start = node[i].ptr;

            if(start == i) {
                on_loop_cnt = k;
            } else {
                int e_cnt = 1;
                on_loop_cnt = k - 1;
                for(int j = start; j =/= i; j = node[j].ptr) {
                    e_cnt++;
                    node[j].visited = true;
                    on_loop_cnt = mul(on_loop_cnt, (k - 1)) % MODER;
                    res = mul(res, node[j].sub) % MODER;
                }
                if(e_cnt % 2) {
                    on_loop_cnt = ((long long)on_loop_cnt + MODER - k + 1) % MODER;
                } else {
                    on_loop_cnt = ((long long)on_loop_cnt + k - 1) % MODER;
                }
            }
            ans = ans * res % MODER * on_loop_cnt % MODER;
        }
    }

    cout << (ans) << '\n';
}
```

注意

先找环，再对环上每棵树 DP，最后环上断边处理。

第 10 章

网络流

10.1 最大流

时间复杂度 ISAP $O(n^2m)$；HLPP $O(n^2\sqrt{m})$

调用前提：先 init(n)，节点编号在 1..n 内，容量非负，源汇不同。

```
class ISAP {
public:
    class Edge {
    public:
        int v;
        int64 c;
    };
    class Node {
    public:
        vector<int> oe;
        int d = 0, ce = 0, pre = 0;
    };
    vector<Edge> edge;
    vector<Node> node;
    vector<int> gap;
    int cnt = 0, src = 0, terminal = 0;

    /*
     * 初始化 n 个点的残量网络
     */
    void init(int n) {
        cnt = n;
        node.assign(n + 1, Node());
        gap.assign(n + 1, 0);
        edge.clear();
        src = terminal = 0;
    }

    /*
     * 添加 u 到 v、容量为 c 的边及其反向边
     */
    void add_edge(int u, int v, int64 c) {
        int id = int(edge.size());
        node[u].oe.push_back(id);
        node[v].oe.push_back(id + 1);
        edge.push_back({v, c});
        edge.push_back({u, 0});
    }

    /*
     * 第 i 条边（按 add_edge 的调用顺序，从 1 开始）跑了多少流
     */
    int64 flow(int i) {
        // 第 i 条边的正边编号是 2i-2、反向边是 2i-1，反向边容量就是流量
        return edge[2 * i - 1].c;
    }

    /*
     * 从汇点反向 BFS，求各点到汇点的最短距离并统计 gap
```

```
     */
    void get_level_graph() {
        for(auto& u : node) {
            u.d = cnt;
            u.ce = 0;
            u.pre = 0;
        }
        fill(gap.begin(), gap.end(), 0);
        queue<int> q;
        q.push(terminal);
        node[terminal].d = 0;
        gap[0] = 1;
        while(!q.empty()) {
            int u = q.front();
            q.pop();
            for(int id : node[u].oe) {
                int v = edge[id].v;
                if(edge[id ^ 1].c && node[v].d == cnt) {
                    node[v].d = node[u].d + 1;
                    gap[node[v].d]++;
                    q.push(v);
                }
            }
        }
    }

    /*
     * 沿 pre 从汇点回溯到源点，增广并返回本次流量
     */
    int64 augment() {
        int64 f = numeric_limits<int64>::max();
        for(int u = terminal; u != src; u = edge[node[u].pre ^ 1].v) {
            f = min(f, edge[node[u].pre].c);
        }
        for(int u = terminal; u != src; u = edge[node[u].pre ^ 1].v) {
            edge[node[u].pre].c -= f;
            edge[node[u].pre ^ 1].c += f;
        }
        return f;
    }

    /*
     * 求 s 到 t 的最大流，修改残量网络
     */
    int64 max_flow(int s, int t) {
        src = s;
        terminal = t;
        get_level_graph();
        int64 ans = 0;
        int u = src;
        while(node[src].d < cnt) {
            if(u == terminal) {
                ans += augment();
                u = src;
                continue;
            }
            bool found = false;
            for(int& i = node[u].ce; i < int(node[u].oe.size()); i++) {
                int id = node[u].oe[i];
                if(edge[id].c && node[u].d == node[edge[id].v].d + 1) {
                    node[edge[id].v].pre = id;
```

```
                    found = true;
                    u = edge[id].v;
                    break;
                }
            }
            if(found) {
                continue;
            }
            int min_d = cnt - 1;
            for(int id : node[u].oe) {
                if(edge[id].c && node[edge[id].v].d < min_d) {
                    min_d = node[edge[id].v].d;
                }
            }
            gap[node[u].d]--;
            if(gap[node[u].d] == 0) {
                break;
            }
            node[u].d = min_d + 1;
            gap[node[u].d]++;
            node[u].ce = 0;
            if(u != src) {
                u = edge[node[u].pre ^ 1].v;
            }
        }
        return ans;
    }
};
```

```
constexpr int N = 1200, M = 120000, INF = 0x3f3f3f3f;
int n, m, s, t;

struct qxx {
    int nex, t;
    long long v;
};

qxx e[M * 2 + 1];
int h[N + 1], cnt = 1;

void add_path(int f, int t, long long v) {
    e[++cnt] = qxx{h[f], t, v};
    h[f] = cnt;
}

/*
 * 添加容量边及对应反向边
 */
void add_flow(int f, int t, long long v) {
    add_path(f, t, v);
    add_path(t, f, 0);
}

int ht[N + 1];       // 高度;
long long ex[N + 1]; // 超额流;
int gap[N];          // gap 优化. gap[i] 为高度为 i 的节点的数量
stack<int> B[N];     // 桶 B[i] 中记录所有 ht[v]==i 的 v
int level = 0;       // 溢出节点的最高高度

int push(int u) {       // 尽可能通过能够推送的边推送超额流
    bool init = u == s; // 是否在初始化
    for(int i = h[u]; i; i = e[i].nex) {
```

```
        const int& v = e[i].t;
        const long long& w = e[i].v;
        // 初始化时不考虑高度差为 1
        if(!w || (init == false && ht[u] != ht[v] + 1) || ht[v] == INF) {
            continue;
        }
        long long k = init ? w : min(w, ex[u]);
        // 取到剩余容量和超额流的最小值，初始化时可以使源的溢出量为负数。
        if(v != s && v != t && !ex[v]) {
            B[ht[v]].push(v);
            level = max(level, ht[v]);
        }
        ex[u] -= k, ex[v] += k, e[i].v -= k, e[i ^ 1].v += k; // push
        if(!init && !ex[u]) {
            return 0; // 如果已经推送完就返回；初始化阶段源点可为负，不能提前返回
        }
    }
    return 1;
}

void relabel(int u) { // 重贴标签（高度）
    ht[u] = INF;
    for(int i = h[u]; i; i = e[i].nex) {
        if(e[i].v) {
            ht[u] = min(ht[u], ht[e[i].t]);
        }
    }
    if(++ht[u] < n) { // 只处理高度小于 n 的节点
        B[ht[u]].push(u);
        level = max(level, ht[u]);
        ++gap[ht[u]]; // 新的高度，更新 gap
    }
}

bool bfs_init() {
    memset(ht, 0x3f, sizeof(ht));
    queue<int> q;
    q.push(t);
    ht[t] = 0;
    while(q.size()) { // 反向 BFS，遇到没有访问过的结点就入队
        int u = q.front();
        q.pop();
        for(int i = h[u]; i; i = e[i].nex) {
            const int& v = e[i].t;
            if(e[i ^ 1].v && ht[v] > ht[u] + 1) {
                ht[v] = ht[u] + 1;
                q.push(v);
            }
        }
    }
    return ht[s] != INF; // 如果图不连通，返回 0
}

// 选出当前高度最大的节点之一，如果已经没有溢出节点返回 0
int select() {
    while(level > -1 && B[level].size() == 0) {
        level--;
    }
    return level == -1 ? 0 : B[level].top();
}

```

```
long long hlpp() { // 返回最大流
    if(!bfs_init()) {
        return 0; // 图不连通
    }
    memset(gap, 0, sizeof(gap));
    for(int i = 1; i ⩽ n; i++) {
        if(ht[i] ≠ INF) {
            gap[ht[i]]++; // 初始化 gap
        }
    }
    ht[s] = n;
    push(s); // 初始化预流
    int u;
    while((u = select())) {
        B[level].pop();
        if(push(u)) { // 仍然溢出
            if(!--gap[ht[u]]) {
                for(int i = 1; i ⩽ n; i++) {
                    if(i ≠ s && ht[i] > ht[u] && ht[i] < n + 1) {
                        ht[i] = n + 1; // 这里重贴成 n+1 的节点都不是溢出节点
                    }
                }
            }
            relabel(u);
        }
    }
    return ex[t];
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m >> s >> t;
    for(int i = 1, u, v, w; i ⩽ m; i++) {
        cin >> u >> v >> w;
        add_flow(u, v, w);
    }
    cout << (hlpp());
    return 0;
}
```

注意

ISAP 用 `d = cnt` 作哨兵；HLPP 初始化阶段源点超额可为负，不能提前返回。

10.2 费用流

时间复杂度 $O(Fm\log n)$

10.2.1 P3381 【模板】最小费用最大流

```
const int64 MAXN = 5005;
const int64 INF = 0x3f3f3f3f3f3f3f3f;

struct Edge {
    int64 v, cost, capacity, flow;
};

class Node {
public:
    vector<int64> level_oe;
    int64 cur_edge;
```

```
    int64 dis;
    int64 h;
    bool visited;
};

class State {
public:
    int64 u, cost;
};

bool operator<(const State& A, const State& B) {
    return A.cost > B.cost;
}

class PrimalDual {
public:
    int64 node_cnt;
    int64 edge_cnt;
    int64 src, terminal;
    int64 tot_flow;
    int64 tot_cost;
    vector<Edge> edges;
    Node node[MAXN];

    /*
     * 添加容量为 capacity、单位费用为 cost 的有向边
     */
    void add_edge(int64 u, int64 v, int64 capacity, int64 cost) {
        node[u].level_oe.push_back(edge_cnt++);
        node[v].level_oe.push_back(edge_cnt++);
        edges.push_back({v, cost, capacity, 0});
        edges.push_back({u, -cost, 0, 0});
    }

    void SPFA() {

        queue<int64> q;
        q.push(src);
        node[src].visited = true;

        int64 u;
        while(!q.empty()) {
            u = q.front();
            q.pop();
            node[u].visited = false;
            for(int64 i = 0; i < node[u].level_oe.size(); i++) {
                Edge& e = edges[node[u].level_oe[i]];
                if(e.capacity > e.flow && node[e.v].h > node[u].h + e.cost) {
                    node[e.v].h = node[u].h + e.cost;
                    if(!node[e.v].visited) {
                        q.push(e.v);
                        node[e.v].visited = true;
                    }
                }
            }
        }
    }

    bool dijkstra() {
        for(int i = 0; i ⩽ node_cnt; i++) {
            node[i].cur_edge = 0;
            node[i].dis = INF;
```

```
            node[i].visited = false;
        }

        priority_queue<State> q;
        q.push({src, 0});
        node[src].dis = 0;

        int64 u;
        int64 l;
        State tmp;
        while(!q.empty()) {
            tmp = q.top();
            u = tmp.u;
            q.pop();
            if(node[u].visited) {
                continue;
            } else {
                node[u].visited = true;
            }
            for(int64 i = 0; i < node[u].level_oe.size(); i++) {
                Edge& e = edges[node[u].level_oe[i]];
                l = e.cost + node[u].h - node[e.v].h;
                if(e.capacity > e.flow && node[e.v].dis > node[u].dis + l) {
                    node[e.v].dis = node[u].dis + l;
                    q.push({e.v, node[e.v].dis});
                }
            }
        }
        return node[terminal].dis != INF;
    }

    /*
     * 清空残量网络并设置点数
     */
    void init(int64 cnt) {
        node_cnt = cnt;
        edge_cnt = 0;
        for(int64 i = 0; i <= node_cnt; i++) {
            node[i] = Node();
        }
        edges.clear();
    }

    int64 augment(int64 v, int64 flow) {
        if(v == src) {
            return flow;
        }

        node[v].visited = true;
        int64 res;
        int64 ans = 0;
        int64 u, l;
        for(int64& i = node[v].cur_edge; i < node[v].level_oe.size(); i++) {
            Edge& come_e = edges[node[v].level_oe[i] ^ 1];
            Edge& back_e = edges[node[v].level_oe[i]];
            u = back_e.v;
            l = come_e.cost + node[u].h - node[v].h;

            if((!node[u].visited) && node[v].dis == node[u].dis + l && come_e.capacity > come_e.flow) {
                res = augment(u, min(come_e.capacity - come_e.flow, flow));
                ans += res;
```

```
                back_e.flow -= res;
                come_e.flow += res;
                flow -= res;
                if(!flow) {
                    break;
                }
            }
        }
        node[v].visited = false;
        return ans;
    }

    /*
     * 求 s 到 t 的最小费用最大流，结果保存在 tot_flow 和 tot_cost
     */
    void calc(int64 s, int64 t) {
        src = s;
        terminal = t;
        tot_cost = 0;
        tot_flow = 0;
        for(int i = 0; i ⩽ node_cnt; i++) {
            node[i].h = INF;
        }
        node[src].h = 0;
        SPFA(); // 第一轮依旧得 SPFA

        for(int i = 0; i ⩽ node_cnt; i++) {
            if(node[i].h == INF) {
                node[i].h = 0;
            }
        }
        int64 flow;
        while(dijkstra()) { // 后面用 Dji
            for(int i = 0; i ⩽ node_cnt; i++) {
                node[i].visited = false;
            }

            flow = augment(terminal, INF);

            for(int i = 0; i ⩽ node_cnt; i++) {
                if(node[i].dis ⩵ INF) {
                    node[i].h += node[i].dis;
                }
            }

            tot_flow += flow;
            tot_cost += flow * node[terminal].h;
        }
    }
};

PrimalDual primalDual;
```

10.2.2 P4843 清理雪道

```
/*
 * 本题边权为 1，路径费用只会是 0 或 -1。
 * PrimalDual 的其余成员直接继承，只覆盖 calc：最短路径费用非负时停止增广。
 */
class SnowFlow : public PrimalDual {
public:
```

```
    /* 其余方法继承自 PrimalDual, 不重复抄写 */
    void calc(int64 s, int64 t, int64 stop) {
        src = s;
        terminal = t;
        tot_cost = 0;
        tot_flow = 0;
        for(int i = 0; i ⩽ node_cnt; i++) {
            node[i].h = INF;
        }
        node[src].h = 0;
        SPFA();
        for(int i = 0; i ⩽ node_cnt; i++) {
            if(node[i].h == INF) {
                node[i].h = 0;
            }
        }
        int64 flow;
        while(dijkstra()) {
            if(node[terminal].h + node[terminal].dis ⩾ stop) {
                break;
            }
            for(int i = 0; i ⩽ node_cnt; i++) {
                node[i].visited = false;
            }
            flow = augment(terminal, INF);
            for(int i = 0; i ⩽ node_cnt; i++) {
                if(node[i].dis ≠ INF) {
                    node[i].h += node[i].dis;
                }
            }
            tot_flow += flow;
            tot_cost += flow * node[terminal].h;
        }
    }
};

SnowFlow snowFlow;

int64 ie[MAXN], oe[MAXN];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int64 n, m;

    cin >> n;
    snowFlow.init(n + 2);
    int64 v;
    for(int64 i = 1; i ⩽ n; i++) {
        cin >> m;
        oe[i] = m;
        for(int64 j = 0; j < m; j++) {
            cin >> v;
            ie[v]++;
            snowFlow.add_edge(i, v, 1, -1);
            snowFlow.add_edge(i, v, INF, 0);
        }
    }

    for(int64 i = 1; i ⩽ n; i++) {
        if(!ie[i]) {
```

```
            snowFlow.add_edge(n + 1, i, INF, 0);
        }
        if(!oe[i]) {
            snowFlow.add_edge(i, n + 2, INF, 0);
        }
    }

    snowFlow.calc(n + 1, n + 2, 0);
    cout << (snowFlow.tot_flow) << '\n';
}
```

注意

先 SPFA 求初始势能，之后 Dijkstra；负费用边需保证无负环。

10.3 上下界网络流

时间复杂度 ISAP 的 $O(n^2m)$

在 ISAP 上加一层：每条边记成流量限制 `[low, up]`，先按 `up - low` 建残量边，再把下界造成的盈亏记在 `d[]`，最后用超级源汇补齐。`init(n)` 会自动多开 n+1、n+2 两个点给超级源汇用。无源汇直接调 `feasible()`；有源汇用 `max_flow(s, t)` / `min_flow(s, t)`，两者内部会自己加一条 t 到 s 的无穷边，无解返回 -1。注意”无解”包含”不存在流量值非负的可行流”这一情况；`min_flow` 的答案是所有可行流里的最小值，如果它是负数，说明最优解是 t 往 s 送流，正常题目不会这么问。可行之后用 `flow(i)` 取第 i 条边（按 `add_edge` 的调用顺序，从 0 开始）的实际流量，直接输出就是一组可行解；自环、重边都按普通边处理。

```
class BoundedFlow : public ISAP {
public:
    vector<int64> d;    // 下界造成的盈亏
    vector<int64> lo;   // 每条边的下界，供 flow 还原实际流量
    int n = 0;
    int ss = 0;
    int tt = 0;

    /*
     * 多开两个点 n+1、n+2 作为超级源汇
     */
    void init(int n_) {
        n = n_;
        ISAP::init(n + 2);
        d.assign(n + 3, 0);
        lo.clear();
        ss = n + 1;
        tt = n + 2;
    }

    /*
     * 添加流量限制在 [low, up] 的边 u -> v
     */
    void add_edge(int u, int v, int64 low, int64 up) {
        lo.push_back(low);
        ISAP::add_edge(u, v, up - low);
        d[u] -= low;
        d[v] += low;
    }

    /*
     * 第 i 条边（按 add_edge 的调用顺序，从 1 开始）的实际流量
     * 要求所有边都在 feasible / max_flow / min_flow 之前加完
     */
    int64 flow(int i) {
```

```
        // 第 i 条边的正边编号是 2i-2、反向边是 2i-1，下界加上反向边容量就是流量
        return lo[i - 1] + edge[2 * i - 1].c;
    }

    /*
     * 无源汇可行流：把下界的盈亏接到超级源汇上跑一遍
     */
    bool feasible() {
        int64 need = 0;
        for(int u = 1; u <= n; u++) {
            if(d[u] > 0) {
                ISAP::add_edge(ss, u, d[u]);
                need += d[u];
            } else if(d[u] < 0) {
                ISAP::add_edge(u, tt, -d[u]);
            }
        }
        return ISAP::max_flow(ss, tt) == need;
    }

    /*
     * 有源汇上下界最大流，无解返回 -1
     */
    int64 max_flow(int s, int t) {
        int id = int(edge.size());
        ISAP::add_edge(t, s, INF);
        if(!feasible()) {
            return -1;
        }
        int64 base = edge[id ^ 1].c;
        edge[id].c = 0;
        edge[id ^ 1].c = 0;
        return base + ISAP::max_flow(s, t);
    }

    /*
     * 有源汇上下界最小流，无解返回 -1
     */
    int64 min_flow(int s, int t) {
        int id = int(edge.size());
        ISAP::add_edge(t, s, INF);
        if(!feasible()) {
            return -1;
        }
        int64 base = edge[id ^ 1].c;
        edge[id].c = 0;
        edge[id ^ 1].c = 0;
        return base - ISAP::max_flow(t, s);
    }
};
```

注意

`feasible()` 判无源汇可行流；有源汇用 `max_flow(s, t)` / `min_flow(s, t)`，无解返回 -1。`init(n)` 会多开两个点给超级源汇。

10.4 全局最小割（Stoer-Wagner）

时间复杂度 $O(n^3)$

无向图、边权非负，$O(n^3)$，适合 $n \le 600$ 的稠密图。`add_edge` 的重边会累加；图不连通时结果是 0；$n \le 1$

没有割，返回 INF。做法是每轮把” 最后被选中的点 t” 合并进” 前一个被选中的点 s”，该轮的割值就是 t 的权值和，所有轮取最小。

```
const int64 INF = 0x3f3f3f3f3f3f3f3f;

class StoerWagner {
public:
    vector<vector<int64>> g;
    vector<int64> w;
    vector<bool> vis;
    vector<bool> del;
    int n = 0;

    /*
     * 初始化 n 个点的空图
     */
    void init(int n_) {
        n = n_;
        g.assign(n + 1, vector<int64>(n + 1, 0));
        w.assign(n + 1, 0);
        vis.assign(n + 1, false);
        del.assign(n + 1, false);
    }

    /*
     * 添加无向边，重边累加
     */
    void add_edge(int u, int v, int64 c) {
        g[u][v] += c;
        g[v][u] += c;
    }

    /*
     * 全局最小割：每轮把最后选中的点 t 合并进前一个点 s
     */
    int64 min_cut() {
        int64 ans = INF;
        for(int i = 1; i < n; i++) {
            fill(w.begin(), w.end(), 0);
            fill(vis.begin(), vis.end(), false);
            int s = 0;
            int t = 0;
            for(int j = 1; j <= n - i + 1; j++) {
                t = 0;
                for(int u = 1; u <= n; u++) {
                    if(!del[u] && !vis[u] && (t == 0 || w[u] > w[t])) {
                        t = u;
                    }
                }
                if(j == n - i + 1) {
                    ans = min(ans, w[t]);
                    for(int u = 1; u <= n; u++) {
                        if(!del[u]) {
                            g[s][u] += g[t][u];
                            g[u][s] = g[s][u];
                        }
                    }
                    del[t] = true;
                } else {
                    s = t;
                    vis[t] = true;
```

```
                    for(int u = 1; u <= n; u++) {
                        if(!del[u] && !vis[u]) {
                            w[u] += g[t][u];
                        }
                    }
                }
            }
            if(ans == 0) {
                return 0;
            }
        }
        return ans;
    }
};
```

注意

无向图、边权非负；重边累加，图不连通返回 0，$n \le 1$ 没有割、返回 `INF`。

第 11 章

连通性

11.1 割点

时间复杂度 $O(n+m)$

Tarjan 边跑边标记：`node[u].isCut` 就是 u 是否为割点，不需要额外数组。根结点没有父结点，`fa == 0` 时不能用 `low` 判，改看儿子数是否 ≥ 2。图不保证连通，`build()` 对每个未访问点各开一棵 DFS 树。

11.1.1 P3388 【模板】割点（割顶）

```
const int N = 20005;

/*
 * 原图结点：邻接表与 Tarjan 需要的信息
 */
class Node {
public:
    vector<int> oe;
    int dfn = 0;
    int low = 0;
    bool isCut = false;   // 是否为割点
};

Node node[N];
int n = 0;
int dfncnt = 0;

/*
 * 从 u 出发跑 Tarjan 并标记割点，fa 为 DFS 树上的父结点，0 表示根
 */
void tarjan(int u, int fa) {
    node[u].dfn = node[u].low = ++dfncnt;
    int child = 0;
    for(int v : node[u].oe) {
        if(!node[v].dfn) {
            child++;
            tarjan(v, u);
            node[u].low = min(node[u].low, node[v].low);
            if(fa != 0 && node[v].low >= node[u].dfn) {
                node[u].isCut = true;
            }
        } else if(v != fa) {
            node[u].low = min(node[u].low, node[v].dfn);
        }
    }
    if(fa == 0 && child >= 2) {
        node[u].isCut = true;
    }
}

/*
 * 图不一定连通，每个未访问的点都要开一棵 DFS 树
 */
void build() {
    for(int u = 1; u <= n; u++) {
        if(!node[u].dfn) {
            tarjan(u, 0);
```

```
        }
    }
}
```

注意

node[v].low >= node[u].dfn 判割点，根结点靠 child >= 2 单独判定；图不连通时对每个未访问点各跑一次。

11.2 点双连通分量

时间复杂度 $O(n+m)$

点双用点栈：每遇到一个未访问的儿子 v，回溯后若 low[v] >= dfn[u]，就从栈里弹到 v，再把 u 补进同一块——所以割点会同时出现在它连着的每一块里。comp[k] 是一块包含的点，node[u].isCut 顺手标出割点。只有一条边的两点、以及孤立点，各自算一块。自环不影响双连通性，add_edge 里直接丢掉——不丢的话它既不是树边、又让孤立点判断失效，这个点一块都没有。

```
const int N = 20005;

/*
 * 无向边：两个方向各占一个编号
 */
class Edge {
public:
    int v;
    int id;   // 反向边的编号为 id ^ 1
};

/*
 * 原图结点：邻接表与 Tarjan 需要的信息
 */
class Node {
public:
    vector<Edge> oe;
    int dfn = 0;
    int low = 0;
    bool isCut = false;   // 是否为割点
};

Node node[N];
vector<int> st;
vector<vector<int>> comp;   // 每个点双连通分量包含的点
int n = 0;
int dfncnt = 0;
int ecnt = 0;
int root = 0;

void add_edge(int u, int v) {
    if(u == v) { // 去自环孤立点
        return;
    }
    node[u].oe.push_back({v, ecnt++});
    node[v].oe.push_back({u, ecnt++});
}

/*
 * 从 u 出发跑 Tarjan，iid 为进入 u 的那条边的编号，-1 表示根
 * 边回溯时弹栈，弹出的点连同 u 就是一块
 */
void tarjan(int u, int iid) {
    node[u].dfn = node[u].low = ++dfncnt;
```

```
    st.push_back(u);

    if(u == root && node[u].oe.empty()) {
        st.pop_back();
        comp.push_back({u});
        return;
    }

    int child = 0;
    for(Edge& e : node[u].oe) {
        if(e.id == iid) {
            continue;
        }
        if(!node[e.v].dfn) {
            child++;
            tarjan(e.v, e.id ^ 1);
            node[u].low = min(node[u].low, node[e.v].low);
            if(node[e.v].low >= node[u].dfn) {
                if(u != root || child > 1) {
                    node[u].isCut = true;
                }
                vector<int> c;
                while(true) {
                    int v = st.back();
                    st.pop_back();
                    c.push_back(v);
                    if(v == e.v) {
                        break;
                    }
                }
                c.push_back(u);
                comp.push_back(c);
            }
        } else {
            node[u].low = min(node[u].low, node[e.v].dfn);
        }
    }
}

/*
 * 图不一定连通，每个未访问的点都要开一棵 DFS 树
 */
void build() {
    for(int u = 1; u <= n; u++) {
        if(!node[u].dfn) {
            root = u;
            tarjan(u, -1);
        }

    }
}
```

注意

`low[v] >= dfn[u]` 时弹栈到 v，再把 u 补进同一块，割点因此属于多块；孤立点自成一块；自环在 `add_edge` 里直接丢掉，否则那个点一块都没有。

11.3 割边

时间复杂度 $O(n+m)$

无向图判桥必须按边的编号跳过父边：只按父结点 `v ≠ fa` 跳的话，两点之间并排的两条边会被误判成桥。`add_edge` 让一条无向边的两个方向占一对编号（`id` 与 `id ^ 1`），`node[v].isBridge` 表示 v 到父结点的那条树边是桥。图不保证连通，`build()` 对每个未访问点各开一棵 DFS 树。

```
const int N = 20005;

/*
 * 无向边：两个方向各占一个编号
 */
class Edge {
public:
    int v;
    int id;   // 反向边的编号为 id ^ 1
};

/*
 * 原图结点：邻接表与 Tarjan 需要的信息
 */
class Node {
public:
    vector<Edge> oe;
    int dfn = 0;
    int low = 0;
    bool isBridge = false;   // 到父结点的树边是桥
};

Node node[N];
int n = 0;
int dfncnt = 0;
int edgecnt = 0;

void add_edge(int u, int v) {
    node[u].oe.push_back({v, edgecnt++});
    node[v].oe.push_back({u, edgecnt++});
}

/*
 * 从 u 出发跑 Tarjan 并标记桥，inId 为进入 u 的那条边的编号，-1 表示根
 */
void tarjan(int u, int inId) {
    node[u].dfn = node[u].low = ++dfncnt;
    for(Edge& e : node[u].oe) {
        if(e.id == inId) {
            continue;
        }
        if(!node[e.v].dfn) {
            tarjan(e.v, e.id ^ 1);
            node[u].low = min(node[u].low, node[e.v].low);
            if(node[e.v].low > node[u].dfn) {
                node[e.v].isBridge = true;
            }
        } else {
            node[u].low = min(node[u].low, node[e.v].dfn);
        }
    }
}

/*
 * 图不一定连通，每个未访问的点都要开一棵 DFS 树
 */
void build() {
```

```
    for(int u = 1; u ⩽ n; u++) {
        if(!node[u].dfn) {
            tarjan(u, -1);
        }
    }
}
```

注意

`node[v].low > node[u].dfn` 判桥；必须按边编号跳过父边（`id ^ 1`），否则重边会被误判成桥。

11.4 边双连通分量

时间复杂度 $O(n+m)$

边双就是删掉所有桥之后剩下的连通块，这里用栈版 Tarjan 一次到底：在无向图上按 SCC 的写法弹栈，只是跳过父边的反向边。`node[u].color` 存所在边双的代表点，`node[u].color == u` 说明 u 是代表点，不需要另外的映射表。自环不影响双连通性，读入时丢掉。

```
const int N = 20005;

/*
 * 无向边：两个方向各占一个编号
 */
class Edge {
public:
    int v;
    int id;   // 反向边的编号为 id ^ 1
};

/*
 * 原图结点：邻接表与 Tarjan 需要的信息
 */
class Node {
public:
    vector<Edge> oe;
    int dfn = 0;
    int low = 0;
    int color = -1;   // 所在边双的代表点
    bool inStack = false;
};

Node node[N];
vector<int> st;
int n = 0;
int dfncnt = 0;
int ecnt = 0;

void add_edge(int u, int v) {
    node[u].oe.push_back({v, ecnt++});
    node[v].oe.push_back({u, ecnt++});
}

/*
 * 从 u 出发跑 Tarjan，iid 为进入 u 的那条边的编号，-1 表示根
 * 弹栈时把整块边双的 color 都记成代表点 u
 */
void tarjan2(int u, int iid) {
    node[u].dfn = node[u].low = ++dfncnt;
    st.push_back(u);
    node[u].inStack = true;
    for(Edge& e : node[u].oe) {
```

```
        if(e.id == iid) {
            continue;
        }
        if(!node[e.v].dfn) {
            tarjan2(e.v, e.id ^ 1);
            node[u].low = min(node[u].low, node[e.v].low);
        } else if(node[e.v].inStack) {
            node[u].low = min(node[u].low, node[e.v].dfn);
        }
    }

    if(node[u].low == node[u].dfn) {
        while(true) {
            int v = st.back();
            st.pop_back();
            node[v].inStack = false;
            node[v].color = u;
            if(v == u) {
                break;
            }
        }
    }
}

/*
 * 图不一定连通，每个未访问的点都要开一棵 DFS 树
 */
void build() {
    for(int u = 1; u <= n; u++) {
        if(!node[u].dfn) {
            tarjan2(u, -1);
        }
    }
}
```

注意

边双就是删掉所有桥后的连通块；本模板用栈版 Tarjan 一次到底，`node[u].color == u` 是代表点。

11.5 缩点

时间复杂度 $O(n+m)$

Tarjan 只做染色：`node[u].color` 存所在分量的代表点，`node[u].color == u` 说明 u 是代表点，不需要额外维护原结点到缩点后结点的映射。需要 DAG 时调用 `condense()`，`cnode[u].oe` 就是代表点 u 所在分量的出边表；分量上的其它信息自己往 `CNode` 里加字段。

```
const int N = 10005;

/*
 * 原图结点：邻接表与 Tarjan 需要的信息
 */
class Node {
public:
    vector<int> oe;
    int dfn = 0;
    int low = 0;
    int color = -1;   // 所在分量的代表点
    bool inStack = false;
};

```

```
/*
 * 缩点后的结点：以代表点为下标，存 DAG 上的出边
 */
class CNode {
public:
    vector<int> oe;
};

Node node[N];
CNode cnode[N];
vector<int> st;
int n = 0;
int dfncnt = 0;

/*
 * 从 u 出发跑 Tarjan，弹栈时把整块分量的 color 都记成代表点 u
 */
void tarjan(int u) {
    node[u].dfn = node[u].low = ++dfncnt;
    st.push_back(u);
    node[u].inStack = true;
    for(int v : node[u].oe) {
        if(!node[v].dfn) {
            tarjan(v);
            node[u].low = min(node[u].low, node[v].low);
        } else if(node[v].inStack) {
            node[u].low = min(node[u].low, node[v].dfn);
        }
    }

    if(node[u].low == node[u].dfn) {
        while(true) {
            int v = st.back();
            st.pop_back();
            node[v].inStack = false;
            node[v].color = u;
            if(v == u) {
                break;
            }
        }
    }
}

/*
 * 对 1 到 n 的每个点求强连通分量
 */
void build() {
    for(int u = 1; u <= n; u++) {
        if(!node[u].dfn) {
            tarjan(u);
        }
    }
}

/*
 * 缩点：跨分量的边收进代表点的出边表
 */
void condense() {
    for(int u = 1; u <= n; u++) {
        int cu = node[u].color;
        for(int v : node[u].oe) {
```

```
            int cv = node[v].color;
            if(cu != cv) {
                cnode[cu].oe.push_back(cv);
            }
        }
    }
}
```

注意

`node[u].color == u` 说明 u 是分量代表点；`color != u` 的点必然落在大小 ≥ 2 的分量里。`condense()` 只负责建 DAG，要不要用看题目。

11.6 2-SAT

时间复杂度 $O(n+m)$

三个求解接口，按需要选一个：

- `solve()`：Tarjan 弹栈顺序天然是逆拓扑序，弹栈编号小的分量在拓扑序里靠后，直接比 `color[id(x,1)] < color[id(x,0)]` 就知道该不该取真。不建 DAG，最快。
- `solveKahn()`：和 `solve()` 等价，但把缩点后的 DAG 建出来跑 Kahn 求拓扑序再比较。多一次 $O(n+m)$ 的建图和排队，解可能和 `solve()` 不同（都合法）。
- `solveLex()`：字典序最小的解，按变量顺序贪心，能取 0 就取 0，冲突再取 1，每次尝试沿蕴含边传播、失败回滚。$O(n(n+m))$，只适合小规模。

```
class TwoSAT {
public:
    int n = 0;
    int dfncnt = 0;
    int scccnt = 0;
    vector<vector<int>> oe;
    vector<int> dfn, low, color;
    vector<bool> inStack;
    vector<int> st;
    vector<int> state;

    /*
     * 初始化 n 个布尔变量，编号 1..n
     */
    void init(int n_) {
        n = n_;
        dfncnt = 0;
        scccnt = 0;
        oe.assign(2 * n + 1, {});
        dfn.assign(2 * n + 1, 0);
        low.assign(2 * n + 1, 0);
        color.assign(2 * n + 1, 0);
        inStack.assign(2 * n + 1, false);
        state.assign(n + 1, 0);
        st.clear();
    }

    /*
     * 变量 x 取 val 时对应的节点，val=0 表示假
     */
    int id(int x, int val) const {
        return x + val * n;
    }

    /*
```

```
     * 加入条件 (x=a) 或 (y=b)，a、b 取 0/1
     */
    void addOr(int x, int a, int y, int b) {
        oe[id(x, a ^ 1)].push_back(id(y, b));
        oe[id(y, b ^ 1)].push_back(id(x, a));
    }

    /*
     * 加入条件 x=a；a=0 即「非 x」。与条件连续调用即可
     */
    void addUnit(int x, int a) {
        oe[id(x, a ^ 1)].push_back(id(x, a));
    }

    /*
     * 加入条件 (x=a) -> (y=b)
     */
    void addImp(int x, int a, int y, int b) {
        addOr(x, a ^ 1, y, b);
    }

    /*
     * 从 u 出发求强连通分量，弹栈时按完成顺序编号
     * 递归深度为图上最长链，满约束下需要显式放大栈空间
     */
    void tarjan(int u) {
        dfn[u] = ++dfncnt;
        low[u] = dfn[u];
        st.push_back(u);
        inStack[u] = true;
        for(int v : oe[u]) {
            if(!dfn[v]) {
                tarjan(v);
            }
            if(inStack[v] && low[v] < low[u]) {
                low[u] = low[v];
            }
        }
        if(low[u] == dfn[u]) {
            while(true) {
                int v = st.back();
                st.pop_back();
                inStack[v] = false;
                color[v] = scccnt;
                if(v == u) {
                    break;
                }
            }
            scccnt++;
        }
    }

    /*
     * 求解；无解返回 false，有解时 state[1..n] 为各变量取值
     * 弹栈编号小 = 拓扑序靠后 = 该分量取真
     */
    bool solve() {
        for(int u = 1; u <= 2 * n; u++) {
            if(!dfn[u]) {
                tarjan(u);
            }
```

```
        }
        for(int x = 1; x <= n; x++) {
            if(color[id(x, 0)] == color[id(x, 1)]) {
                return false;
            }
            state[x] = color[id(x, 1)] < color[id(x, 0)];
        }
        return true;
    }

    /*
     * 同上，但建出缩点后的 DAG 跑 Kahn 求拓扑序再比较
     */
    bool solveKahn() {
        for(int u = 1; u <= 2 * n; u++) {
            if(!dfn[u]) {
                tarjan(u);
            }
        }
        vector<vector<int>> goe(scccnt);
        vector<int> indeg(scccnt, 0);
        for(int u = 1; u <= 2 * n; u++) {
            for(int v : oe[u]) {
                if(color[u] != color[v]) {
                    goe[color[u]].push_back(color[v]);
                    indeg[color[v]]++;
                }
            }
        }
        queue<int> q;
        for(int c = 0; c < scccnt; c++) {
            if(indeg[c] == 0) {
                q.push(c);
            }
        }
        vector<int> tpn(scccnt, 0);
        int cnt = 0;
        while(!q.empty()) {
            int c = q.front();
            q.pop();
            tpn[c] = ++cnt;
            for(int d : goe[c]) {
                if(--indeg[d] == 0) {
                    q.push(d);
                }
            }
        }
        for(int x = 1; x <= n; x++) {
            if(color[id(x, 0)] == color[id(x, 1)]) {
                return false;
            }
            state[x] = tpn[color[id(x, 1)]] > tpn[color[id(x, 0)]];
        }
        return true;
    }

    /*
     * 字面量 u 的否定
     */
    int neg(int u) const {
        return u > n ? u - n : u + n;
```

```
    }

    /*
     * 令 (x=a) 为真并沿蕴含边传播，失败则撤销本次所有赋值
     */
    bool assign(int x, int a, vector<int>& val) {
        vector<int> rec;
        if(val[id(x, a)] == 1) {
            return true;
        }
        auto push = [&](int u) -> bool {
            if(val[neg(u)] == 1) {
                return false;
            }
            if(val[u] == 1) {
                return true;
            }
            val[u] = 1;
            rec.push_back(u);
            return true;
        };
        if(!push(id(x, a))) {
            return false;
        }
        for(int i = 0; i < int(rec.size()); i++) {
            for(int v : oe[rec[i]]) {
                if(!push(v)) {
                    for(int w : rec) {
                        val[w] = -1;
                    }
                    return false;
                }
            }
        }
        return true;
    }

    /*
     * 字典序最小的解：按变量顺序，能取 0 就取 0，冲突再取 1
     */
    bool solveLex() {
        vector<int> val(2 * n + 1, -1);
        for(int x = 1; x <= n; x++) {
            if(!assign(x, 0, val) && !assign(x, 1, val)) {
                return false;
            }
        }
        for(int x = 1; x <= n; x++) {
            state[x] = val[id(x, 1)] == 1 ? 1 : 0;
        }
        return true;
    }
};
```

注意

Tarjan 递归深度可达 2n；满约束需开大栈或换非递归版。

第 12 章

图匹配

12.1 匈牙利算法

时间复杂度 $O(nm)$

12.1.1 P3386 【模板】二分图最大匹配

```
const int N = 505;

int n, m, e;

vector<int> edge[N];
int match[N];
int dfn[N];

/*
 * 从 u 寻找增广路，tag 用于标记本轮访问
 */
int dfs(int u, int tag) {
    if(dfn[u] == tag) {
        return 0;
    } else {
        dfn[u] = tag;
        for(int v : edge[u]) {
            if(match[v] == 0 || dfs(match[v], tag)) {
                match[v] = u;
                return 1;
            }
        }
        return 0;
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m >> e;

    int u, v;
    for(int i = 0; i < e; i++) {
        cin >> u >> v;
        edge[u].push_back(v);
    }
    int ans = 0;
    for(int i = 1; i <= n; i++) {
        ans += dfs(i, i);
    }

    cout << (ans) << '\n';
}
```

注意

每次增广前清空 `vis`。

12.2 带花树算法

时间复杂度 $O(n^3)$

```
const int N = 1005;
int m, n, cnt;
queue<int> q;

class Node {
public:
    vector<int> oe;
    int pre;
    int match;
    int check;
    int color;
    int fa;
    int on_flower;
};

Node node[N];

int find_root(int u) {
    bool flag = false;
    int last;
    while(node[u].fa != u) {
        if(flag) {
            node[last].fa = node[u].fa;
        }
        last = u;
        u = node[u].fa;
        flag = !flag;
    }
    return u;
}

void cat(int u, int v) {
    u = find_root(u);
    v = find_root(v);
    if(u != v) {
        node[u].fa = v;
        // This is a special parallel set whose root must be the flower root.
        // Merging by rank does not apply.
    }
}

int lca(int u, int v) {
    cnt++;
    u = find_root(u);
    v = find_root(v);
    while(node[u].on_flower != cnt) {
        node[u].on_flower = cnt;
        u = find_root(node[node[u].match].pre);
        if(v > 0) {
            swap(u, v);
        }
    }
    return u;
}

/*
 * 将奇环收缩到公共基点 r
```

```
 */
void shrink(int u, int v, int r) {
    while(find_root(u) != find_root(r)) {
        node[u].pre = v;
        v = node[u].match;
        if(node[v].color == 1) {
            node[v].color = 0;
            q.push(v);
        }
        if(find_root(u) == u) {
            cat(u, r);
        }
        if(find_root(v) == v) {
            cat(v, r);
        }
        // These ifs are essential. Bugs always occur here.
        u = node[v].pre;
    }
}

/*
 * 为当前匹配初始化增广搜索状态
 */
void init() {
    while(!q.empty()) {
        q.pop();
    }

    for(int i = 1; i <= n; i++) {
        node[i].fa = i;
        node[i].pre = 0;
        node[i].color = -1;
    }
}

/*
 * 反复寻找增广路，返回增加的匹配数
 */
int bfs() {
    int ans = 0;
    for(int i = 1; i <= n; i++) {
        if(!node[i].match) {
            init();
            q.push(i);
            node[i].color = 0;
            while(!q.empty() && !node[i].match) {
                int u = q.front();
                q.pop();
                for(int v : node[u].oe) {
                    if(node[v].color == 1 || find_root(u) == find_root(v)) {
                        continue;
                    } else if(node[v].color == 0) {
                        int root = lca(u, v);
                        shrink(u, v, root);
                        shrink(v, u, root);
                    } else {
                        node[v].color = 1;
                        node[v].pre = u;
                        if(!node[v].match) {
                            while(u > 0) {
                                int last_v = node[u].match;
```

```
                                int last_u = node[last_v].pre;
                                node[u].match = v;
                                node[v].match = u;
                                v = last_v;
                                u = last_u;
                            }
                            break;
                        } else {
                            node[node[v].match].color = 0;
                            q.push(node[v].match);
                        }
                    }
                }
            }
            if(node[i].match) {
                ans++;
            }
        }
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m;
    int u, v;
    for(int i = 0; i < m; i++) {
        cin >> u >> v;
        node[u].oe.push_back(v);
        node[v].oe.push_back(u);
    }

    cout << (bfs()) << '\n';
    for(int i = 1; i <= n; i++) {
        cout << (node[i].match) << ' ';
    }
    return 0;
}
```

注意

一般图最大匹配；缩花时注意 LCA 与并查集。

第 13 章

其他图论算法

13.1 折半搜索

时间复杂度 $O(2^{n/2})$

13.1.1 P2962 [USACO09NOV] Lights G

```
#define N 40
class Point {
public:
    vector<int> oe;
    bool state;
};

Point p[N + 1];

int n, m;
long long int a[N + 1];
map<long long, int> s;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m;
    int u, v;
    for(int i = 0; i < m; i++) {
        cin >> u >> v;
        p[u].oe.push_back(v);
        p[v].oe.push_back(u);
    }

    for(int i = 1; i ⩽ n; i++) {
        long long int t = 1 << (i - 1);
        for(int j : p[i].oe) {
            t ⊨ 1ll << (j - 1);
        }
        a[i - 1] = t;
    }

    int top = 1 << (n / 2);

    for(int i = 0; i < top; i++) {
        long long int code = 0;
        int d = 0;
        for(int j = 0; j < n / 2; j++) {
            if((i >> j) & 1) {
                code ^= a[j];
                d++;
            }
        }
        if(!s.count(code)) {
            s[code] = d;
        } else {
            s[code] = min(d, s[code]);
        }
    }
```

```
    int ans = 2147483647;
    top = 1 << (n - n / 2);
    for(int i = 0; i < top; i++) {
        long long int code = 0;
        int d = 0;
        for(int j = 0; j < n - n / 2; j++) {
            if((i >> j) & 1) {
                code ^= a[n / 2 + j];
                d++;
            }
        }
        if(s.count(((1ll << n) - 1) ^ code)) {
            ans = min(ans, d + s[((1ll << n) - 1) ^ code]);
        }
    }

    cout << (ans);
}
```

注意

两半分别枚举后排序双指针。

13.2 斯坦纳树

时间复杂度 $O(3^k n + 2^k m \log m)$

13.2.1 P4294 [WC2008] 游览计划

```
int dp[12][12][4096];
int w[12][12];
int zipper[12][12];
int128 arr[12][12][4096];
int n, m, k;
vector<pair<int, int>> key;

int direction[4][2] = {
    {0, 1},
    {1, 0},
    {0, -1},
    {-1, 0}};

class State {
public:
    int x, y, v;
};

bool operator<(const State& A, const State& B) {
    return A.v > B.v;
}

int128 bit = 1;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m;
    memset(dp, 0x3f, sizeof(dp));
    for(int i = 1; i <= n; i++) {
        for(int j = 1; j <= m; j++) {
            cin >> w[i][j];
```

```
            zipper[i][j] = (i - 1) * m + j - 1;
            if(!w[i][j]) {
                key.push_back({i, j});
                dp[i][j][1 << k] = 0;
                arr[i][j][1 << k] = bit << zipper[i][j];
                k++;
            }
        }
    }

    for(int s = 1; s < 1 << k; s++) {
        priority_queue<State> q;
        for(int i = 1; i <= n; i++) {
            for(int j = 1; j <= m; j++) {
                for(int sub = s & (s - 1); sub; sub = s & (sub - 1)) {
                    if(dp[i][j][s] > dp[i][j][sub] + dp[i][j][s ^ sub] - w[i][j]) {
                        dp[i][j][s] = dp[i][j][sub] + dp[i][j][s ^ sub] - w[i][j];
                        arr[i][j][s] = arr[i][j][sub] | arr[i][j][s ^ sub];
                    }
                }
                if(dp[i][j][s] != 0x3f3f3f3f) {
                    q.push({i, j, dp[i][j][s]});
                }
            }
        }

        while(!q.empty()) {
            State cur = q.top();
            q.pop();
            if(cur.v == dp[cur.x][cur.y][s]) {
                for(int facing = 0; facing < 4; facing++) {
                    int nxtx = cur.x + direction[facing][0];
                    int nxty = cur.y + direction[facing][1];
                    if(nxtx > 0 && nxtx <= n && nxty > 0 && nxty <= m) {
                        if(dp[nxtx][nxty][s] > dp[cur.x][cur.y][s] + w[nxtx][nxty]) {
                            dp[nxtx][nxty][s] = dp[cur.x][cur.y][s] + w[nxtx][nxty];
                            arr[nxtx][nxty][s] = arr[cur.x][cur.y][s] | (bit << zipper[nxtx][nxty]);
                            q.push({nxtx, nxty, dp[nxtx][nxty][s]});
                        }
                    }
                }
            }
        }
    }

    if(k > 0) {
        cout << (dp[key[0].first][key[0].second][(1 << k) - 1]) << '\n';
        for(int i = 1; i <= n; i++) {
            for(int j = 1; j <= m; j++) {
                if(!w[i][j]) {
                    cout << "x";
                } else {
                    int128 arrange = arr[key[0].first][key[0].second][(1 << k) - 1];
                    if(arrange & (bit << zipper[i][j])) {
                        cout << "o";
                    } else {
                        cout << "_";
                    }
                }
            }
            cout << '\n';
```

```
        }
    } else {
        cout << "0\n";
        for(int i = 1; i ⩽ n; i++) {
            for(int j = 1; j ⩽ m; j++) {
                if(!w[i][j]) {
                    cout << "x";
                } else {
                    cout << "_";
                }
            }
            cout << '\n';
        }
    }
}
```

注意

先子集 DP，再对每个状态跑最短路。

13.3 线段树优化建图

时间复杂度 $O(n + k \log n)$ 建图

把 `u` 连向整个区间、整个区间连向 `u` 这种 $O(n)$ 条边的建图压成 $O(\log n)$，用两棵共用 `1..n` 真实点当叶子的线段树：

- **出树**（父连子，叶子连到真实点）：`to(u, l, r)` 从 u 连到覆盖区间的几个出树结点，就能走到区间里的每个点。
- **入树**（子连父，真实点连到叶子）：`from(l, r, u)` 让区间里的点先爬到覆盖结点，再连到 u。
- `add(u, v)` 用来连普通的单条边。

两棵树是**第一次用到才建**的：只用 `to` 或只用 `from` 时总点数 `5n`，两个方向都用才 `9n`，什么都不用时就是 `n`。真实点固定占 `1..n`，树结点码在后面，偏移存在 `ob` / `ib` 里，所以**不要写死** `n + p`、`5 * n + p` 这类编号；`tot` 是当前实际用到的点数，跑最短路或 Tarjan 遍历 `1..tot` 即可。

`init` 只开 `n + 1` 个点，其余在 `ensure` 里第一次建树时才 `resize`；如果 `Node` 里还挂了别的字段（缩点的 `dfn` / `low` / `color` 之类），记得在同一处同步扩容。

`oe` 里存的是后继编号，不带边权。树边本身也会被算成一步，所以这份只保证**可达性**（缩点、2-SAT 这类）与题目图一致；要跑最短路得把树边设成 0 权，也就是把 `oe` 换成带权结构，只在 `build` 里填 0。

```
class Graph {
public:
    class Node {
    public:
        vector<int> oe;
    };
    vector<Node> node;
    int n = 0, tot = 0;
    int ob = 0, ib = 0;   // 出树、入树基址，0 表示还没建

    void init(int n_) {
        n = n_;
        tot = n;
        ob = ib = 0;
        node.assign(n + 1, Node());
    }

    void add(int u, int v) {
        node[u].oe.push_back(v);
    }
```

```
21
22      /*
23       * 第一次用到某个方向时才建树，dir 为 0 建出树、1 建入树，返回基址
24       * 有其他如缩点后的元素时，此处记得 resize
25       */
26      int ensure(int dir) {
27          int& base = dir ? ib : ob;
28          if(base) {
29              return base;
30          }
31          base = (ob || ib) ? 5 * n : n;
32          node.resize(base + 4 * n + 1);
33          tot = max(tot, base + 4 * n);
34          build(1, 1, n, base, dir);
35          return base;
36      }
37
38      /*
39       * 出树父连子、入树子连父，叶子连到真实点
40       */
41      void build(int p, int l, int r, int base, int dir) {
42          if(l == r) {
43              if(dir) {
44                  add(l, base + p);
45              } else {
46                  add(base + p, l);
47              }
48              return;
49          }
50          int mid = l + (r - l) / 2;
51          if(dir) {
52              add(base + 2 * p, base + p);
53              add(base + 2 * p + 1, base + p);
54          } else {
55              add(base + p, base + 2 * p);
56              add(base + p, base + 2 * p + 1);
57          }
58          build(2 * p, l, mid, base, dir);
59          build(2 * p + 1, mid + 1, r, base, dir);
60      }
61
62      /*
63       * dir 为 0 时 u 连向区间，为 1 时区间连向 u
64       */
65      void walk(int u, int l, int r, int p, int cl, int cr, int dir) {
66          if(l <= cl && cr <= r) {
67              if(dir) {
68                  add(ib + p, u);
69              } else {
70                  add(u, ob + p);
71              }
72              return;
73          }
74          int mid = cl + (cr - cl) / 2;
75          if(l <= mid) {
76              walk(u, l, r, 2 * p, cl, mid, dir);
77          }
78          if(r > mid) {
79              walk(u, l, r, 2 * p + 1, mid + 1, cr, dir);
80          }
81      }
```

```
82
83      /*
84       * u 连向 [l, r] 里的每个点
85       */
86      void to(int u, int l, int r) {
87          ensure(0);
88          walk(u, l, r, 1, 1, n, 0);
89      }
90
91      /*
92       * [l, r] 里的每个点连向 u
93       */
94      void from(int l, int r, int u) {
95          ensure(1);
96          walk(u, l, r, 1, 1, n, 1);
97      }
98  };
```

注意

两棵树按需建：只用一个方向时总点数 5n，两个方向都用才是 9n；树结点的偏移用 `ob` / `ib` 定位，跑最短路或 Tarjan 遍历 `1..tot`。

第 III 编

数学

第 14 章

数论

14.1 线性筛

时间复杂度 $O(n)$

```
class Sieve {
public:
    int pcnt = 0;
    vector<bool> is_not_prime;
    vector<int> prime, phi, mu;

    /*
     * 筛出 1..maxn 的素数、phi 和 mu，清空上次结果
     */
    void calc(int maxn) {
        pcnt = 0;
        prime.clear();
        is_not_prime.assign(maxn + 1, false);
        phi.assign(maxn + 1, 0);
        mu.assign(maxn + 1, 0);
        is_not_prime[0] = true;
        if(maxn ≥ 1) {
            is_not_prime[1] = true;
            phi[1] = mu[1] = 1;
        }
        for(int i = 2; i ≤ maxn; i++) {
            if(!is_not_prime[i]) {
                prime.push_back(i);
                pcnt++;
                phi[i] = i - 1;
                mu[i] = -1;
            }
            for(int p : prime) {
                if(p > maxn / i) {
                    break;
                }
                is_not_prime[p * i] = true;
                if(i % p == 0) {
                    phi[p * i] = phi[i] * p;
                    break;
                }
                phi[p * i] = phi[i] * (p - 1);
                mu[p * i] = -mu[i];
            }
        }
    }
};
```

注意

`maxn >= 0`；筛表大小按题目上限开。

14.2 杜教筛

时间复杂度 $O(n^{2/3})$

杜教筛

设现在要求积性函数 f 的前缀和， 设 $\sum_{i=1}^{n} f(i) = S(n)$。

再找一个积性函数 g ，则考虑它们的狄利克雷卷积的前缀和

$$\begin{aligned}
&\sum_{i=1}^{n}(f*g)(i) \\
&=\sum_{i=1}^{n}\sum_{d|i} f(d)g(\frac{i}{d}) \\
&=\sum_{d=1}^{n} g(d)\sum_{i=1}^{\lfloor \frac{n}{d} \rfloor} f(i) \\
&=\sum_{d=1}^{n} g(d)S(\lfloor \frac{n}{d} \rfloor)
\end{aligned}$$

其中得到第一行是根据狄利克雷卷积的定义。

得到第二行则是先枚举 d 提出 g 。

得到第三行则是把 $\sum_{i=1}^{\lfloor \frac{n}{d} \rfloor} f(i)$ 替换为 $S(\lfloor \frac{n}{d} \rfloor)$

接着考虑 $g(1)S(n)$ 等于什么。

可以发现，他就等于

$$\sum_{i=1}^{n} g(i)S(\lfloor \frac{n}{i} \rfloor) - \sum_{i=2}^{n} g(i)S(\lfloor \frac{n}{i} \rfloor)$$

（可以理解成从1开始的前缀和减去从2开始的前缀和就是第一项）

前面这个式子 $\sum_{i=1}^{n} g(i)S(\lfloor \frac{n}{i} \rfloor)$ ，根据刚才的推导，他就等于 $\sum_{i=1}^{n}(f*g)(i)$

所以得到杜教筛的核心式子：

$$g(1)S(n) = \sum_{i=1}^{n}(f*g)(i) - \sum_{i=2}^{n} g(i)S(\lfloor \frac{n}{i} \rfloor)$$

```
const int64 N = 1 << 31;

class DusSieve {
    // h = f*g
    // sf = sigma(f)
    // sg = sigma(g)
    // sh = sigma(h)
public:
    const static int64 MAXN = 1 << 21;
    int64 pcnt;

    bool is_not_prime[MAXN];
    int64 prime[MAXN];
    int64 s[MAXN];
    int64 mu[MAXN];
    map<int64, int64> sm;

    int64 f(int64 x) {
        return mu[x];
```

```
    }

    int64 g(int64 x) {
        return 1;
    }

    int64 sg(int64 x) {
        return x;
    }

    int64 sh(int64 x) {
        return 1;
    }

    /*
     * 用记忆化整除分块计算 f 在 1..x 的前缀和
     */
    int64 sf(int64 x) {
        int64 gs;
        if(x < MAXN) {
            gs = s[x];
        } else if(sm.count(x)) {
            gs = sm[x];
        } else {
            int64 ans = sh(x);
            for(int64 l = 2, r; l <= x; l = r + 1) {
                r = x / (x / l);
                ans = ans - sf(x / l) * (sg(r) - sg(l - 1));
            }
            gs = sm[x] = ans;
        }
        return gs / g(1);
    }

    /*
     * 线性筛预处理小范围的函数值与前缀和
     */
    void calc(int64 maxn) {}
};

DusSieve sieve;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int64 n;
    cin >> n;
    if(n == 1) {
        cout << (0ll) << '\n';
    } else {
        n--;
        sieve.calc(n);
        int64 ans = 0;
        for(int64 l = 1, r; l <= n; l = r + 1) {
            r = n / (n / l);
            ans += (n / l) * (n / l) * (sieve.sf(r) - sieve.sf(l - 1));
        }
        cout << (ans + 2) << '\n';
    }
}
```

注意

`calc()` 留空，使用时把线性筛的实现填进来；小范围查表，大范围记忆化整除分块。

延伸阅读

杜教筛+狄利克雷卷积小结

作者 zhiyangfan shotacon

发布时间 2022-01-19 22:15 分类 个人记录

杜教筛+狄利克雷卷积小结

前话——狄利克雷卷积

定义

对于两个数论函数 f,g，我们定义它们的狄利克雷卷积 $f*g$ 为：

$$(f*g)(n)=\sum_{d|n}f(d)g\left(\frac{n}{d}\right)$$

一些常见的数论函数：

- id，定义为 $\mathrm{id}(n)=n$，而 $\mathrm{id}_k(n)=n^k$。
- σ，定义为 $\sigma(n)=\sum_{d|n}d$，而 $\sigma_k(n)=\sum_{d|n}d^k$。
- ϵ，定义为 $\epsilon(n)=[n=1]$。
- φ，定义为 $\varphi(n)=\sum_{i=1}^{n}[\gcd(i,n)=1]$。
- μ，定义为，如果令 $n=\prod_{i=1}^{k}p_i^{c_i}$，其中 p_i 为质数，则

$$\mu(n)=\begin{cases}1 & n=1\\ (-1)^k & \prod_{i=1}^{k}c_i=1\\ 0 & \max\{c_i\}>1\end{cases}$$

- 1，定义为 $1(n)=1$。

积性函数

狄利克雷卷积的很多性质是围绕积性函数展开的，积性函数的定义是：

- 对于一个数论函数 f，如果 $f(1)=1$，且 $\forall\gcd(a,b)=1$，都有 $f(a)f(b)=f(ab)$，则称 f 为积性函数。特别地，如果 $\forall a,b$，都有 $f(a)f(b)=f(ab)$，则称 f 为完全积性函数。

可以发现，上文中的函数 $\mathrm{id},\epsilon,1$ 为完全积性函数，φ,μ,σ 是积性函数。

对于一个积性函数 f，它具有如下性质：

- $h(x)=f(x^p)$ 也是积性函数，如果注意到 $\gcd(a,b)=1$ 是 $\gcd(a^k,b^k)=1$ 的充分条件就能得到证明。
- $h(x)=f^p(x)$ 也是积性函数，可以根据乘积的幂等于幂的乘积证明。
- $h(x)=f(x)g(x)$ 也是积性函数，按照定义即可证明。
- $h(x)=(f*g)(x)$ 也是积性函数，依然可以直接套定义证明，只不过要稍微麻烦一点。

狄利克雷卷积与积性函数

可以通过狄利克雷卷积在积性函数之间建立起桥梁。

- $\mathrm{id}_k * 1 = \sigma_k$，其实就是：

$$(\mathrm{id}_k * 1)(n) = \sum_{d|n} d^k \cdot 1 = \sigma_k(n)$$

- $\varphi * 1 = \mathrm{id}$，证明如下：

  考虑令 $n = \prod_{i=1}^{k} p_i^{c_i}$，则现在的目的是要对 p^m，其中 p 是质数证明上式，剩下的可以通过积性函数的性质乘到一起。

  套定义有：

$$(\varphi * 1)(p^m) = \sum_{d|p^m} \varphi(d) = \sum_{d=1}^{m} \varphi(p^d) = \sum_{d=1}^{m} (p^d - p^{d-1}) = p^m = \mathrm{id}(p^m)$$

  得证。

- $\epsilon * f = f$，其中 f 是任意积性函数，其实就是：

$$(\epsilon * f)(n) = \sum_{d|n} \epsilon(d) f\left(\frac{n}{d}\right) = f(n)$$

  注意这其实反应了 ϵ 是狄利克雷卷积中的单位元。所以我们可以定义，如果 $f * g = \epsilon$，则称 f, g 在狄利克雷卷积中互逆。

- $1 * \mu = \epsilon$，证明如下：依然考虑刚刚证明 $\varphi * 1 = \mathrm{id}$ 的套路：

$$(1 * \mu)(p^m) = \sum_{d|p^m} \mu(d) = \sum_{d=0}^{m} \mu(p^d) = [m = 0] = \epsilon(p^m)$$

  注意倒数第二步成立是因为 $\mu(1) = 1, \mu(p) = -1, \mu(p^m) = 0 (m > 1)$。

  注意这反映了 $1, \mu$ 是互逆的，所以这引出了莫比乌斯反演的一种形式：

$$f * 1 = g \Leftrightarrow f = g * \mu$$

正文——杜教筛与积性函数前缀和

众所周知，一般来说，随着毒瘤出题人的不断进步，我们在题目中常常会遇到如下式子：

$$\sum_{i=1}^{n} f(i)$$

其中 f 是一个积性函数。我们当然可以用欧拉筛做到 $\mathcal{O}(n)$ 的求解，但当 $n \leq 10^{12}$ 的时候，线性的时间复杂度就不够用了。这时候，亚线性的筛法，杜教筛就该上场了，它能做到在 $\mathcal{O}(n^{\frac{2}{3}})$ 的时间复杂度求解。

推式子

我们令 $S(n) = \sum_{i=1}^{n} f(i)$，则杜教筛的主要思想就是找到一个 $S(n)$ 关于 $S\left(\left\lfloor \frac{n}{i} \right\rfloor\right)$ 的递推式。

注意到对于任意一个数论函数 g，它一定满足：

$$\sum_{i=1}^{n} (f * g)(i) = \sum_{i=1}^{n} g(i) S\left(\left\lfloor \frac{n}{i} \right\rfloor\right)$$

根据定义有，$\sum_{i=1}^{n}(f*g)(i)=\sum_{i=1}^{n}\sum_{d|i}f(d)g(\frac{i}{d})$，注意到 $f(d)g(\frac{i}{d})$ 就是对所有 $d\le n$ 的做贡献，所以考虑交换求和顺序，并改为枚举 $d,\frac{i}{d}$（分别对应下式的 i,j）：

$$\sum_{i=1}^{n}\sum_{j=1}^{\lfloor\frac{n}{i}\rfloor}g(i)f(j)=\sum_{i=1}^{n}g(i)\sum_{j=1}^{\lfloor\frac{n}{i}\rfloor}f(j)=\sum_{i=1}^{n}g(i)S\left(\left\lfloor\frac{n}{i}\right\rfloor\right)$$

得证。

所以我们就能得出杜教筛的主要式子：

$$g(1)S(n)=\sum_{i=1}^{n}(f*g)(i)-\sum_{i=2}^{n}g(i)S\left(\left\lfloor\frac{n}{i}\right\rfloor\right)$$

这个式子其实很简单，注意到根据上面证明的性质，右边第一项和第二项之差一个 $i=1$ 的情况，而这就等于左边。

而我们要做的主要任务就是寻找一个合适的 g，使 $f*g$ 性质足够好，能快速计算 $\sum_{i=1}^{n}(f*g)(i)$ 的值。

例题

> P4213 【模板】杜教筛（Sum）
>
> T 组数据，每组给出 n，分别求下面两个式子的值：
>
> $$\sum_{i=1}^{n}\varphi(i)\qquad\sum_{i=1}^{n}\mu(i)$$
>
> $(1\le T\le 10,1\le n\le 2^{31})$

板子题，我们分别来看这两个式子。

首先对于 φ，我们知道 $\varphi*1=\mathrm{id}$，而 $\sum_{i=1}^{n}\mathrm{id}(i)=\frac{1}{2}n(n+1)$ 非常好求。所以我们就选取 1 作为 g。

然后对于 μ，我们知道 $\mu*1=\epsilon$，而 $\sum_{i=1}^{n}\epsilon(i)=1$ 非常好求。所以我们就选取 1 作为 g。

分别选完之后套式子数论分块递归计算即可。直接算是 $\mathcal{O}(n^{\frac{4}{3}})$ 的，不过如果提前预处理出 $\le n^{\frac{2}{3}}$ 的答案，就能做到 $\mathcal{O}(n^{\frac{2}{3}})$ 了。

这里对于 φ 还有一种不需要单独找 g 的方法。考虑用 $\mathrm{id}*\mu=\varphi$ 的结论，原式即：

$$\sum_{i=1}^{n}\sum_{d|i}\frac{i}{d}\mu(d)=\sum_{d=1}^{n}\mu(d)\sum_{i=1}^{\lfloor\frac{n}{d}\rfloor}i=\sum_{d=1}^{n}\mu(d)\frac{(1+\lfloor\frac{n}{d}\rfloor)\lfloor\frac{n}{d}\rfloor}{2}$$

只要能快速求出 μ 的前缀和就可以数论分块做了。不过时间复杂度没有优化，依然是 $\mathcal{O}(n^{\frac{2}{3}})$。

```cpp
#include <cstdio>
#include <unordered_map>
const int N = 2e6 + 10; typedef long long ll;
int mu[N], sumMu[N], p[N], vis[N], tp; std::unordered_map<ll, ll> mp;
inline void getMu()
{
    mu[1] = 1;
    for (int i = 2; i < N; ++i)
    {
```

```
        if (!vis[i]) p[++tp] = i, mu[i] = -1;
        for (int j = 1; j <= tp && (ll)i * p[j] < N; ++j)
        {
            vis[i * p[j]] = 1;
            if (i % p[j] == 0) { mu[i * p[j]] = 0; break; }
            mu[i * p[j]] = -mu[i];
        }
    }
    for (int i = 1; i < N; ++i) sumMu[i] = sumMu[i - 1] + mu[i];
}
ll Smu(ll n)
{
    if (n < N) return sumMu[n]; if (mp[n]) return mp[n];
    ll ret = 1;
    for (ll l = 2, r; l <= n; l = r + 1)
    {
        r = n / (n / l);
        ret -= Smu(n / l) * (r - l + 1);
    }
    return mp[n] = ret;
}
ll Sphi(ll n)
{
    ll ret = 0;
    for (ll l = 1, r; l <= n; l = r + 1)
    {
        r = n / (n / l);
        ret += (Smu(r) - Smu(l - 1)) * (1 + n / l) * (n / l) / 2;
    }
    return ret;
}
int main()
{
    int T; scanf("%d", &T); getMu();
    while (T--)
    {
        ll n; scanf("%lld", &n);
        printf("%lld %lld\n", Sphi(n), Smu(n));
    }
    return 0;
}
```

> P3768 简单的数学题
>
> 求：
>
> $$\sum_{i=1}^{n}\sum_{j=1}^{n} ij\gcd(i,j)$$
>
> 答案对一个给定的质数 p 取模。($1 \le n \le 10^{10}, 5\times 10^8 \le p \le 1.1\times 10^9$)

需要一点莫比乌斯反演的板子题，常规莫反过程不再给出解释。

$$
\begin{aligned}
&\sum_{i=1}^{n}\sum_{j=1}^{n} ij\gcd(i,j)\\
=&\sum_{i=1}^{n}\sum_{j=1}^{n}\sum_{d=1}^{n} ij[\gcd(i,j)=d]\\
=&\sum_{d=1}^{n} d^3\sum_{i=1}^{\lfloor\frac{n}{d}\rfloor}\sum_{j=1}^{\lfloor\frac{n}{d}\rfloor} ij[\gcd(i,j)=1]\\
=&\sum_{d=1}^{n} d^3\sum_{i=1}^{\lfloor\frac{n}{d}\rfloor}\sum_{j=1}^{\lfloor\frac{n}{d}\rfloor} ij\sum_{p|i,p|j}\mu(p)\\
=&\sum_{d=1}^{n} d^3\sum_{p=1}^{\lfloor\frac{n}{d}\rfloor} p^2\mu(p)\sum_{i=1}^{\lfloor\frac{n}{dp}\rfloor}\sum_{j=1}^{\lfloor\frac{n}{dp}\rfloor} ij\\
=&\sum_{t=1}^{n} t^2\sum_{d|t}\frac{t}{d}\mu(d)\left(\frac{(1+\lfloor\frac{n}{t}\rfloor)\lfloor\frac{n}{t}\rfloor}{2}\right)^2\\
=&\sum_{t=1}^{n} t^2\varphi(t)\left(\frac{(1+\lfloor\frac{n}{t}\rfloor)\lfloor\frac{n}{t}\rfloor}{2}\right)^2
\end{aligned}
$$

推完了，现在任务就很明白了，前面的 $\mathrm{id}_2\varphi$ 的前缀和用杜教筛算出来，后面的那一坨用数论分块。而用杜教筛我们就要找 g。

单纯的 φ 我们可以卷上一个 1 处理，而现在变成 $\mathrm{id}_2\varphi$，我们就考虑卷上一个 id_2。来看看如果令 $g=\mathrm{id}_2$ 会有什么结果：

$$
\begin{aligned}
\sum_{i=1}^{n}(f*g)(i)&=\sum_{i=1}^{n}((\mathrm{id}_2\varphi)*\mathrm{id}_2)(i)\\
&=\sum_{i=1}^{n}\sum_{d|i}(\mathrm{id}_2\varphi)(d)\mathrm{id}_2\left(\frac{i}{d}\right)\\
&=\sum_{i=1}^{n}\sum_{d|i}d^2\varphi(d)\left(\frac{i}{d}\right)^2\\
&=\sum_{i=1}^{n}\sum_{d|i}i^2\varphi(d)\\
&=\sum_{i=1}^{n}i^2\sum_{d|i}\varphi(d)=\sum_{i=1}^{n}i^3=\left(\frac{1}{2}n(n+1)\right)^2
\end{aligned}
$$

其中最后一步用了 $1*\varphi=\mathrm{id}$。发现这个结果还不错，所以选用 $g=\mathrm{id}_2$ 套杜教筛的公式即可。时间复杂度 $\mathcal{O}(n^{\frac{2}{3}})$。

```
#include <cmath>
#include <cstdio>
#include <unordered_map>
const int N = 6e6 + 10; typedef long long ll;
int mod, inv6, inv4; ll n, pn; std::unordered_map<ll, ll> mp;
inline int ksm(int a, int b)
{
    int ret = 1;
    while (b)
    {
        if (b & 1) ret = (ll)ret * a % mod;
```

```
        a = (ll)a * a % mod; b >>= 1;
    }
    return ret;
}
int p[N], vis[N], phi[N], s[N], tp;
inline void getP()
{
    phi[1] = 1;
    for (int i = 2; i <= pn; ++i)
    {
        if (!vis[i]) p[++tp] = i, phi[i] = i - 1;
        for (int j = 1; j <= tp && (ll)p[j] * i <= pn; ++j)
        {
            vis[i * p[j]] = 1;
            if (i % p[j] == 0) { phi[i * p[j]] = (ll)phi[i] * p[j] % mod; break; }
            phi[i * p[j]] = (ll)phi[i] * phi[p[j]] % mod;
        }
    }
    for (int i = 1; i <= pn; ++i) s[i] = (s[i - 1] + (ll)i * i % mod * phi[i] % mod) % mod;
}
auto sum2 = [](ll x) { x %= mod; return x * (x + 1) % mod * (2 * x + 1) % mod * inv6 % mod; };
auto sum3 = [](ll x) { x %= mod; return x * x % mod * (x + 1) % mod * (x + 1) % mod * inv4 % mod;
};
int S(ll n)
{
    if (n <= pn) return s[n]; if (mp[n]) return mp[n];
    ll ret = sum3(n), pre = 1, cur;
    for (ll l = 2, r; l <= n; l = r + 1, pre = cur)
    {
        r = n / (n / l); cur = sum2(r);
        ret = (ret - S(n / l) * ((cur - pre + mod) % mod) % mod + mod) % mod;
    }
    return mp[n] = ret;
}
int work(ll n)
{
    int ret = 0, pre = 0, cur;
    for (ll l = 1, r; l <= n; l = r + 1, pre = cur)
    {
        r = n / (n / l); cur = S(r);
        ret = (ret + sum3(n / l) * ((cur - pre + mod) % mod) % mod) % mod;
    }
    return ret;
}
int main()
{
    scanf("%d%lld", &mod, &n); pn = pow(n, 2.0 / 3); getP();
    inv6 = ksm(6, mod - 2); inv4 = ksm(4, mod - 2); printf("%d\n", work(n)); return 0;
}
```

> P1587 [NOI2016] 循环之美
>
> 求出在 k 进制下有多少分数 $\frac{x}{y}$ 满足 $1 \leq x \leq n, 1 \leq y \leq n, x, y \in \mathbb{Z}$ 且对应的小数形式是纯循环的，一个小数是纯循环的当且仅当它能被表示成如下形式：
>
> $$a.\dot{c_1}c_2c_3\cdots c_{p-1}\dot{c_p}, a \in \mathbb{Z}, p \geq 1, c_i \in [0, k) \cap \mathbb{Z}$$
>
> 相同值的分数只算一次。($1 \leq n, m \leq 10^9, 2 \leq k \leq 2 \times 10^3$)

比较综合的一道题。

注意到一个分数 $\frac{x}{y}$ 在 k 进制下是纯循环的，当且仅当 $\gcd(y,k)=1$，而又因为相同值的分数只算一次，所以我们只统计最简分数就好，所以题目相当于在问：

$$\sum_{i=1}^{n}\sum_{j=1}^{m}[\gcd(i,j)=1][\gcd(j,k)=1]$$

注意到 $[\gcd(j,k)=1]$ 这个跟常规的莫反不太一样，先不动它，先整掉前面的 $[\gcd(i,j)=1]$。

$$\begin{aligned}&\sum_{i=1}^{n}\sum_{j=1}^{m}[\gcd(i,j)=1][\gcd(j,k)=1]\\=&\sum_{i=1}^{n}\sum_{j=1}^{m}\sum_{d|i,d|j}\mu(p)[\gcd(j,k)=1]\\=&\sum_{d=1}^{n}\mu(d)\sum_{i=1}^{\lfloor\frac{n}{d}\rfloor}\sum_{j=1}^{\lfloor\frac{m}{d}\rfloor}[\gcd(jd,k)=1]\\=&\sum_{d=1}^{n}\mu(d)\left\lfloor\frac{n}{d}\right\rfloor\sum_{j=1}^{\lfloor\frac{m}{d}\rfloor}[\gcd(jd,k)=1]\end{aligned}$$

到这儿，如果继续用莫反展开那个 $[\gcd(jd,k)=1]$，能获得一个这样的式子：

$$\sum_{d=1}^{n}\mu(d)\left\lfloor\frac{n}{d}\right\rfloor\sum_{p=1}^{k}[p|k]\mu(p)\left\lfloor\frac{m}{\operatorname{lcm}(d,p)}\right\rfloor$$

因为 lcm 的限制，这个式子没办法继续优化了，只能 $\mathcal{O}(nk)$ 暴力枚举，获得 **64pts**。

看来用莫反展开行不通，注意到这个式子难化开的瓶颈在于 $[\gcd(jd,k)=1]$ 这个玩意，所以考虑把它拆开。（注意到 $\gcd(jd,k)=1$ 和 $\gcd(j,k)=1$ 且 $\gcd(d,k)=1$ 是互为充要条件）

$$\sum_{d=1}^{n}\mu(d)\left\lfloor\frac{n}{d}\right\rfloor[\gcd(d,k)=1]\sum_{j=1}^{\lfloor\frac{m}{d}\rfloor}[\gcd(j,k)=1]$$

现在式子肉眼可见地被分为了两个部分，而我们的目标就是对这两个部分分别求前缀和。考虑设：

$$F(n)=\sum_{i=1}^{n}[\gcd(i,k)=1]\qquad S(n,k)=\sum_{d=1}^{n}\mu(d)[\gcd(d,k)=1]$$

现在就是要对这两个函数求出递归式。我们先考虑一眼看过去比较简单的 $F(n)$ 吧。观察到如果 $\gcd(i,k)=1$，则有 $\gcd(i+ak,k)=1,a\in\mathbb{N}$。这个可以通过辗转相除法的过程看出：

$$\gcd(i,k)=1\Leftrightarrow\gcd(k,i\bmod k)=1$$

所以先考虑 k 的剩余系中有多少与 k 互质的，再乘上 $\geq n$ 的数中能包含几个，最后算上剩下不完整的即可：

$$F(n)=\left\lfloor\frac{n}{k}\right\rfloor\varphi(k)+F(n\bmod k)$$

可以提前预处理出 $F(0\sim k)$ 的值，然后就能 $\mathcal{O}(1)$ 计算了，预处理的时间复杂度是 $\mathcal{O}(k\log k)$。

然后考虑 $S(n,k)$，化了半天也没什么思路，考虑先把 $[\gcd(d,k)=1]$ 用莫反展开：

$$\begin{aligned}S(n,k)&=\sum_{d=1}^{n}\mu(d)[\gcd(d,k)=1]\\&=\sum_{d=1}^{n}\mu(d)\sum_{p|d,p|k}\mu(p)\\&=\sum_{p=1}^{k}[p|k]\mu(p)\sum_{d=1}^{\lfloor\frac{n}{p}\rfloor}\mu(dp)\end{aligned}$$

这下难受了，$\mu(dp)$ 的求和是不连续的，不能用前缀和整出来。而如果想拆开，就必须满足 $\gcd(d,p)=1$，这也是无法保证的。等等，那如果 $\gcd(d,p)\neq 1$ 会怎么样？显然 dp 会出现重复的质因子，而这会导致 $\mu(dp)=0$！所以我们干脆直接套上一个 $\gcd(d,p)=1$ 的条件然后拆开就好，不会漏算东西。

$$\begin{aligned}&\sum_{p=1}^{k}[p|k]\mu(p)\sum_{d=1}^{\lfloor\frac{n}{p}\rfloor}\mu(dp)\\=&\sum_{p=1}^{k}[p|k]\mu^2(p)\sum_{d=1}^{\lfloor\frac{n}{p}\rfloor}\mu(d)[\gcd(d,p)=1]\\=&\sum_{p=1}^{k}[p|k]\mu^2(p)S\left(\left\lfloor\frac{n}{p}\right\rfloor,p\right)\end{aligned}$$

好了，这样强行拆开之后就得到了 $S(n,k)$ 的递归式了，而且总共只有 $\mathcal{O}(\sqrt{nk})$ 种状态，完全可以接受。但现在问题是，边界怎么办？显然有 $S(0,k)=0$，但是这还不够，$S(n,1)$ 我们也递归不下去。不过问题不大，因为当 $k=1$ 时，后面那个互质的条件就没了，即：

$$S(n,1)=\sum_{i=1}^{n}\mu(d)$$

n 很大，记忆化之后直接上杜教筛即可。

那这样这道题就结束了，时间复杂度 $\mathcal{O}(\sqrt{nk}+n^{\frac{2}{3}}+k\log k)$。用 `std::map` 记忆化的话会多一个 log 不过问题不大。

```
#include <map>
#include <cmath>
#include <cstdio>
#include <algorithm>
const int N = 2e6 + 10; typedef long long ll; std::map<std::pair<int, int>, ll> mp;
int p[N], vis[N], mu[N], phi[N], sum[N], f[N], tp, pn, k;
inline void getP()
{
    mu[1] = phi[1] = vis[1] = 1;
    for (int i = 2; i <= pn; ++i)
    {
        if (!vis[i]) p[++tp] = i, mu[i] = -1, phi[i] = i - 1;
        for (int j = 1; j <= tp && (ll)p[j] * i <= pn; ++j)
        {
            vis[i * p[j]] = 1;
            if (i % p[j] == 0) { phi[i * p[j]] = phi[i] * p[j]; mu[i * p[j]] = 0; break; }
            phi[i * p[j]] = phi[i] * phi[p[j]]; mu[i * p[j]] = -mu[i];
        }
    }
    for (int i = 1; i <= pn; ++i) sum[i] = sum[i - 1] + mu[i];
}
```

```cpp
inline int F(int n) { return n / k * phi[k] + f[n % k]; }
ll S(int n, int k)
{
    if ((k == 1 && n <= pn) || (!n)) return sum[n];
    auto now = std::make_pair(n, k);
    if (mp[now]) return mp[now];
    if (k == 1)
    {
        ll ret = 1;
        for (int l = 2, r; l <= n; l = r + 1)
            r = n / (n / l), ret -= S(n / l, k) * (r - l + 1);
        return mp[now] = ret;
    }
    ll ret = 0;
    for (int i = 1; i * i <= k; ++i)
    {
        if (k % i) continue;
        if (mu[i]) ret += S(n / i, i);
        if (i * i != k && mu[k / i]) ret += S(n / (k / i), k / i);
    }
    return mp[now] = ret;
}
int main()
{
    int n, m; scanf("%d%d%d", &n, &m, &k); pn = N - 1; getP();
    for (int i = 1; i <= k; ++i) f[i] = f[i - 1] + (std::__gcd(i, k) == 1);
    ll ans = 0, cur, pre = 0;
    for (int l = 1, r; l <= std::min(n, m); l = r + 1, pre = cur)
    {
        r = std::min(n / (n / l), m / (m / l));
        cur = S(r, k); ans += (cur - pre) * (n / l) * F(m / l);
    }
    printf("%lld\n", ans); return 0;
}
```

作者：zhiyangfan　创建时间：2022-01-19 22:15:40



14.3 迪利克雷前缀和

时间复杂度 $O(n \log \log n)$

14.3.1 P5495 【模板】Dirichlet 前缀和

```
const int N = 20000005;
unsigned int seed;
inline unsigned int getnext() {
    seed ^= seed << 13;
    seed ^= seed >> 17;
    seed ^= seed << 5;
    return seed;
}

int n;
bool is_not_prime[N];
unsigned int prime[N];
unsigned int cnt = 0;

class DirichletPrefixSum {
    const static int MAXN = N;

public:
    unsigned int h[MAXN];

    unsigned int g(unsigned int x) {
        return 1u;
    }

    void calc() {
        for(unsigned int i = 0; i < cnt; i++) {
            for(unsigned int j = n / prime[i]; j; j--) {
                for(long long t = prime[i]; j * t <= n; t *= prime[i]) {
                    h[j * t] += h[j] * g(t);
                }
            }
        }
    }
};

DirichletPrefixSum b;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> seed;
    for(int i = 1; i <= n; i++) {
        b.h[i] = getnext();
    }

    is_not_prime[0] = true;
    is_not_prime[1] = true;
    for(unsigned int i = 2; i <= n; i++) {
        if(!is_not_prime[i]) {
            prime[cnt] = i;
            cnt++;
        }
        for(unsigned int j = 0; j < cnt && prime[j] * i <= n; j++) {
            is_not_prime[prime[j] * i] = true;
            if(i % prime[j] == 0) {
```

```
                break;
            }
        }
    }

    b.calc();

    unsigned int ans = 0;
    for(unsigned int i = 1; i ⩽ n; i++) {
        ans ^= b.h[i];
    }

    cout << (ans);
}
```

注意

按质因子降序做后缀和；`h[j]` 在处理时仍是原数组。

14.4 迪利克雷卷积

时间复杂度 $O(n \log \log n)$

14.4.1 定义

欧拉函数（Euler's totient function），即 $\varphi(n)$，表示的是小于等于 n 和 n 互质的数的个数。

比如说 $\varphi(1) = 1$。

当 n 是质数的时候，显然有 $\varphi(n) = n - 1$。

14.4.2 性质

- 欧拉函数是 积性函数。
  即对任意满足 $\gcd(a, b) = 1$ 的整数 a, b，有 $\varphi(ab) = \varphi(a)\varphi(b)$。
  特别地，当 n 是奇数时 $\varphi(2n) = \varphi(n)$。
  证明参见 剩余系的复合。
- $n = \sum_{d|n} \varphi(d)$。
  利用 莫比乌斯反演 相关知识可以得出。
  也可以这样考虑：如果 $\gcd(k, n) = d$，那么 $\gcd(\frac{k}{d}, \frac{n}{d}) = 1, (k < n)$。
  如果我们设 $f(x)$ 表示 $\gcd(k, n) = x$ 的数的个数，那么 $n = \sum_{i=1}^{n} f(i)$。
  根据上面的证明，我们发现，$f(x) = \varphi(\frac{n}{x})$，从而 $n = \sum_{d|n} \varphi(\frac{n}{d})$。注意到约数 d 和 $\frac{n}{d}$ 具有对称性，所以上式化为 $n = \sum_{d|n} \varphi(d)$。
- 若 $n = p^k$，其中 p 是质数，那么 $\varphi(n) = p^k - p^{k-1}$。（根据定义可知）
- 由唯一分解定理，设 $n = \prod_{i=1}^{s} p_i^{k_i}$，其中 p_i 是质数，有 $\varphi(n) = n \times \prod_{i=1}^{s} \frac{p_i - 1}{p_i}$。
- 引理：设 p 为任意质数，那么 $\varphi(p^k) = p^{k-1} \times (p - 1)$。
  证明：显然对于从 1 到 p^k 的所有数中，除了 p^{k-1} 个 p 的倍数以外其它数都与 p^k 互素，故 $\varphi(p^k) = p^k - p^{k-1} = p^{k-1} \times (p - 1)$，证毕。
  接下来我们证明 $\varphi(n) = n \times \prod_{i=1}^{s} \frac{p_i - 1}{p_i}$。由唯一分解定理与 $\varphi(x)$ 函数的积性

$$\begin{aligned}\varphi(n) &= \prod_{i=1}^{s} \varphi(p_i^{k_i}) \\ &= \prod_{i=1}^{s} (p_i - 1) \times {p_i}^{k_i - 1} \\ &= \prod_{i=1}^{s} {p_i}^{k_i} \times (1 - \frac{1}{p_i}) \\ &= n \prod_{i=1}^{s} (1 - \frac{1}{p_i}) \qquad \square\end{aligned}$$

- 对任意不全为 0 的整数 m, n，$\varphi(mn)\varphi(\gcd(m,n)) = \varphi(m)\varphi(n)\gcd(m,n)$。
  可由上一条直接计算得出。

14.4.3 应用

欧拉函数常常用于化简一列最大公约数的和。国内有些文章称它为 **欧拉反演** [^1]。

在结论

$$n = \sum_{d|n} \varphi(d)$$

中代入 $n = \gcd(a, b)$，则有

$$\gcd(a,b) = \sum_{d|\gcd(a,b)} \varphi(d) = \sum_{d} [d|a][d|b]\varphi(d),$$

其中，$[\cdot]$ 称为 Iverson 括号，只有当命题 P 为真时 $[P]$ 取值为 1，否则取 0。对上式求和，就可以得到

$$\sum_{i=1}^{n} \gcd(i,n) = \sum_{d} \sum_{i=1}^{n} [d|i][d|n]\varphi(d) = \sum_{d} \left\lfloor \frac{n}{d} \right\rfloor [d|n]\varphi(d) = \sum_{d|n} \left\lfloor \frac{n}{d} \right\rfloor \varphi(d).$$

这里关键的观察是 $\sum_{i=1}^{n} [d|i] = \lfloor \frac{n}{d} \rfloor$，即在 1 和 n 之间能够被 d 整除的 i 的个数是 $\lfloor \frac{n}{d} \rfloor$。

利用这个式子，就可以遍历约数求和了。需要多组查询的时候，可以预处理欧拉函数的前缀和，利用数论分块查询。

14.4.4 欧拉定理

与欧拉函数紧密相关的一个定理就是欧拉定理。其描述如下：

若 $\gcd(a, m) = 1$，则 $a^{\varphi(m)} \equiv 1 \pmod{m}$。

扩展欧拉定理

当然也有扩展欧拉定理，用于处理一般的 a 和 m 的情形。

$$a^b \equiv \begin{cases} a^{b \bmod \varphi(m)}, & \gcd(a, m) = 1 \\ a^b, & \gcd(a, m) \neq 1, b < \varphi(m) \\ a^{b \bmod \varphi(m) + \varphi(m)}, & \gcd(a, m) \neq 1, b \geq \varphi(m) \end{cases} \pmod{m}$$

14.4.5 P2398 GCD SUM

```
const int64 N = 100005;
bool is_not_prime[N];
int64 prime[N / 10];
int64 euler[N];

int64 n;
int64 cnt = 0;
```

```
8
9  int main() {
10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12     cin >> n;
13     is_not_prime[0] = true;
14     is_not_prime[1] = true;
15     euler[0] = 0;
16     euler[1] = 1;
17
18     for(int64 i = 2; i ⩽ n; i++) {
19         if(!is_not_prime[i]) {
20             prime[cnt] = i;
21             euler[i] = i - 1;
22             cnt++;
23         }
24         for(int64 j = 0; j < cnt && prime[j] * i ⩽ n; j++) {
25             is_not_prime[prime[j] * i] = true;
26             if(i % prime[j] == 0) {
27                 euler[prime[j] * i] = euler[i] * prime[j];
28                 break;
29             } else {
30                 euler[prime[j] * i] = euler[i] * euler[prime[j]];
31             }
32         }
33     }
34
35     int64 ans = 0;
36     for(int64 i = 1; i ⩽ n; i++) {
37         ans += (n / i) * (n / i) * euler[i];
38     }
39
40     cout << (ans) << '\n';
41 }
```

注意

卷积与前缀和的枚举方向相反。

14.5 类欧几里得

时间复杂度 $O(\log n)$

```
1  const int64 MODER = 998244353;
2  const int64 i2 = (MODER + 1) / 2;
3  const int64 i6 = (MODER + 1) / 6;
4
5  class Data {
6  public:
7      int64 f, g, h;
8  };
9
10 /*
11  * 递归计算类欧几里得的三个和式，含端点 0 和 n
12  */
13 Data solve(int64 a, int64 b, int64 c, int64 n) {
14     int64 n2 = (n + 1) * n % MODER * i2 % MODER;
15     int64 n3 = (2 * n + 1) * (n + 1) % MODER * n % MODER * i6 % MODER;
16     Data res = {0, 0, 0};
17     if(a ⩾ c || b ⩾ c) {
18         Data tmp = solve(a % c, b % c, c, n);
```

```
        int64 qa = a / c, qb = b / c;
        res.f = (tmp.f + qa * n2 + qb * (n + 1)) % MODER;
        res.g = (tmp.g + qa * n3 + qb * n2) % MODER;
        res.h = (tmp.h + 2 * qb * tmp.f % MODER + 2 * qa * tmp.g % MODER +
                 qa * qa % MODER * n3 % MODER + qb * qb % MODER * (n + 1) % MODER +
                 2 * qa * qb % MODER * n2 % MODER) %
                MODER;
        return res;
    } else {
        int64 m = (a * n + b) / c;
        if(!m) {
            return res;
        }
        Data tmp = solve(c, c - b - 1, a, m - 1);
        res.f = (m * n - tmp.f + MODER) % MODER;
        res.g = (m * n2 + (MODER - tmp.f) * i2 + (MODER - tmp.h) * i2) % MODER;
        res.h = (n * m % MODER * m - tmp.f - tmp.g * 2 + 3 * MODER) % MODER;
        return res;
    }
}

/*
 * 求 i 从 0 到 n-1 时 floor((a*i+b)/m) 的和
 */
int64 floorSum(int64 n, int64 m, int64 a, int64 b) {
    int64 ans = 0;
    int64 flag = 1;
    while(m) {

        int64 ra = (a % m + m) % m;
        int64 rb = (b % m + m) % m;
        int64 s = n * (n - 1) / 2 * ((a - ra) / m) + n * ((b - rb) / m);
        int64 k = (ra * (n - 1) + rb) / m;
        ans += (s + n * k) * flag;
        flag = -flag;
        n = k;
        a = m;
        b = m + ra - rb - 1;
        m = ra;
    }
    return ans;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);
    int T;
    cin >> T;
    while(T--) {
        int n, a, b, c;
        cin >> n >> a >> b >> c;
        Data res = solve(a, b, c, n);
        cout << res.f << ' ' << res.h << ' ' << res.g << '\n';
    }
}
```

注意

注意本模板 g = Σ i • floor、h = Σ floor²，与常见记号相反。

14.6 乘法逆元

时间复杂度 $O(n)$ 预处理

```
const int N = 20000600;
int n, p;
int inv[N];
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> p;
    inv[1] = 1;
    for(int i = 2; i ⩽ n; i++) {
        inv[i] = 1ll * (p - p / i) * inv[p % i] % p;
    }

    for(int i = 1; i ⩽ n; i++) {
        cout << (inv[i]) << '\n';
    }
}
```

```
/*
 * 返回 gcd(a,b)，并求出满足 ax+by=gcd(a,b) 的 x、y
 */
long long exgcd(long long a, long long b, long long& x, long long& y) {
    if(!b) {
        x = 1;
        y = 0;
        return a;
    }
    long long g = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return g;
}
```

注意

线性递推要求 p 为素数且 n < p；exgcd 版本适用于任意互质模数。

14.7 大步小步算法（斐波那契同余）

时间复杂度 $O(\log n)$

14.7.1 题目大意

输入一个数 f，求最小的 x 使得 $F_x \equiv f \ (mod\ 10^{13})$，其中 $F = 0, 1, 1, 2, 3...$ 为斐波那契数列，下标从 0 开始。

```
const int N = 2;
const int LOGM = 14;
const __int128 MODER = 100000000000000;

class Matrix {
public:
    __int128 arr[2][2]{};
    Matrix operator*(const Matrix& o) const {
        Matrix res;
        for(int i = 0; i < 2; i++) {
            for(int k = 0; k < 2; k++) {
                for(int j = 0; j < 2; j++) {
                    res.arr[i][j] = (res.arr[i][j] + arr[i][k] * o.arr[k][j]) % MODER;
                }
            }
```

```
        }
        return res;
    }
    Matrix operator%(int64 mod) const {
        Matrix res = *this;
        for(auto& row : res.arr) {
            for(auto& x : row) {
                x %= mod;
            }
        }
        return res;
    }
    bool operator!=(const Matrix& o) const {
        for(int i = 0; i < 2; i++) {
            for(int j = 0; j < 2; j++) {
                if(arr[i][j] != o.arr[i][j]) {
                    return true;
                }
            }
        }
        return false;
    }
};

Matrix E = {
    1, 0,
    0, 1};

const Matrix A = {1, 1, 1, 0};
const Matrix st{1, 0, 0, 0};
vector<int64> ans[LOGM];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int64 f;
    cin >> f;
    f = (f % 10000000000000LL + 10000000000000LL) % 10000000000000LL;
    if(f == 0) {
        cout << 0 << endl;
        return 0;
    }
    if(f == 1) {
        cout << 1 << endl;
        return 0;
    }
    int64 moder = 10;
    int64 prod = 1;
    ans[0].push_back(0);
    for(int i = 1; i < LOGM; i++, moder *= 10) {
        int j = 0;
        bool flag = true;
        do {
            for(int64 y : ans[i - 1]) {
                Matrix Y = quickPow(A, prod * j + y, E) * st % (moder * 10);
                if(Y.arr[0][0] % (moder) == f % (moder)) {
                    ans[i].push_back(j * prod + y);
                    flag = false;
                }
            }
        } while(quickPow(A, 1 + prod * (++j), E) * st % moder != A * st);
```

```
        prod *= j;
        if(flag) {
            cout << -1 << endl;
            return 0;
        }
    }
    cout << ans[LOGM - 1][0] + 1 << endl;
    return 0;
}
```

注意

本文件实际是斐波那契同余的逐位提升，不是 BSGS。

第 15 章

代数

15.1 快速幂

时间复杂度 $O(\log k)$

15.1.1 模意义快速幂

`quickPow(a,k,moder)`；要求 k≥0、moder>0。int128 乘法支持正的 int64 模数。

```
/*
 * 求 a 的 k 次幂对 moder 取模
 */
long long quickPow(long long a, long long k, long long moder) {
    a = (a % moder + moder) % moder;
    long long res = 1 % moder;
    while(k) {
        if(k & 1) {
            res = res * a % moder;
        }
        a = a * a % moder;
        k >>= 1;
    }
    return res;
}
```

15.1.2 通用乘法快速幂

`quickPow(a,k,identity)`；适用于矩阵等满足结合律的乘法，单位元由调用者传入。若 T 是整数且需要普通整数乘方，注意与上面的模幂重载区分，可显式写 `quickPow<T>`。

```
template <class T>
/*
 * 求 a 的 k 次幂，res 传入乘法单位元
 */
T quickPow(T a, unsigned long long k, T res) {
    while(k) {
        if(k & 1) {
            res = res * a;
        }
        k >>= 1;
        if(k) {
            a = a * a;
        }
    }
    return res;
}
```

注意

模板按 int 模数设计；模数超过 3e9 时 `long long` 相乘会溢出。

15.2 矩阵（min-plus）

时间复杂度 $O(n^3 \log k)$

用于恰好 k 条边的最短路；INF 表示不可达，不能参与加法。要求所有有限中间结果严格位于 (-INF, INF)，正负权均可。普通矩阵乘法需另写乘加规则。

```cpp
// min-plus 矩阵：不是通常的矩阵乘法。T 为有符号整型。
template <typename T = long long, int N = 3>
class Matrix {
public:
    static constexpr T INF = numeric_limits<T>::max() / 4;
    T arr[N][N];
    /*
     * 将所有元素初始化为 x，默认表示不可达
     */
    explicit Matrix(T x = INF) {
        for(auto& row : arr) {
            fill(row, row + N, x);
        }
    }
    /*
     * min-plus 乘法，合并两段路径的最短距离
     */
    Matrix operator*(const Matrix& other) const {
        Matrix res;
        for(int i = 0; i < N; i++) {
            for(int k = 0; k < N; k++) {
                if(arr[i][k] != INF) {
                    for(int j = 0; j < N; j++) {
                        if(other.arr[k][j] != INF) {
                            res.arr[i][j] = min(res.arr[i][j], T(arr[i][k] + other.arr[k][j]));
                        }
                    }
                }
            }
        }
        return res;
    }
    /*
     * 构造 min-plus 乘法单位矩阵
     */
    static Matrix identity() {
        Matrix res;
        for(int i = 0; i < N; i++) {
            res.arr[i][i] = 0;
        }
        return res;
    }
};
```

注意

min-plus 矩阵的 `INF` 取 `max/4`，防止相加溢出。

15.3 线性基

时间复杂度 $O(n \log V)$

15.3.1 P4151 [WC2011] 最大 XOR 和路径

```cpp
const int N = 50005;

inline long long highBit(long long num) {
    if(!num) {
        return 0ll;
    } else {
        return num & (1ll << (64 - __builtin_clzll(num) - 1));
    }
```

```
}

inline int highPos(long long num) {
    if(!num) {
        return 0;
    } else {
        return 64 - __builtin_clzll(num);
    }
}

class HamelBasis {
public:
    int maxn;
    int cnt;
    vector<long long> vec;

    /*
     * 清空线性基，预留 _n 个向量
     */
    void init(int _n) {
        maxn = _n;
        cnt = 0;
        vec.clear();
        vec.reserve(_n);
    }

    /*
     * 插入 num，返回是否增加了独立向量
     */
    bool insert(long long num) {
        for(int i = 0; i < cnt; i++) {
            if(highBit(num) & highBit(vec[i])) {
                num ^= vec[i];
            } else if(highBit(num) > highBit(vec[i])) {
                swap(num, vec[i]);
            }
        }
        if(num) {
            cnt++;
            vec.push_back(num);
            return true;
        } else {
            return false;
        }
    }

    /*
     * 消去其他向量中的主元位，转为标准形
     */
    void standardize() {
        for(int i = cnt - 1; ~i; i--) {
            for(int j = i - 1; ~j; j--) {
                if(highBit(vec[i]) & vec[j]) {
                    vec[j] ^= vec[i];
                }
            }
        }
    }

    /*
     * 求 num 与线性基能表示的数异或后的最大值
```

```
     */
    long long getMax(long long num) {
        long long ans = num;
        for(int i = 0; i < cnt; i++) {
            long long b = highBit(vec[i]);
            if(highBit(vec[i]) ^ (ans & b)) {
                ans ^= vec[i];
            }
        }
        return ans;
    }
};

HamelBasis basis;

class Edge {
public:
    int v;
    long long w;
};

class Node {
public:
    vector<Edge> oe;
    int dfn;
    long long s;
};

Node node[N];

int n, m;
int dfn = 1;
long long ori;
void dfs(int u, int fa, long long s) {
    node[u].dfn = dfn++;
    node[u].s = s;
    if(u == n) {
        ori = s;
    }
    for(Edge& e : node[u].oe) {
        if(e.v != fa) {
            if(node[e.v].dfn) {
                basis.insert(s ^ e.w ^ node[e.v].s);
            } else {
                dfs(e.v, u, s ^ e.w);
            }
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    basis.init(64);
    cin >> n >> m;
    int u, v;
    long long w;
    for(int i = 1; i <= m; i++) {
        cin >> u >> v >> w;
        node[u].oe.push_back({v, w});
        node[v].oe.push_back({u, w});
```

```
    }

    dfs(1, 0, 0);
    basis.standardize();
    cout << (basis.getMax(ori)) << '\n';
}
```

15.3.2 图片题例：可回退线性基

Problem Description

请注意本题特殊的空间限制。

给出一个长度为 n 的非负整数序列 a，你可以在这些数中选取任意个数（可以是零个），但不能选取相邻的数，求选出来的数的异或和最大值。

Input

每个测试点中包含多组测试数据。输入的第一行包含一个正整数 $T(1 \leq T \leq 3)$，表示数据组数。对于每组测试数据：

第一行一个正整数 $n(1 \leq n \leq 50)$，表示序列长度。

第二行 n 个非负整数 $a_1, a_2, \cdots, a_n(0 \leq a_i \leq 10^{18})$，表示序列 a。

Output

对于每组测试数据：输出一行一个整数，表示答案。

```
const int N = 55;
int n;

inline long long highBit(long long num) {
    if(!num) {
        return 0ll;
    } else {
        return num & (1ll << (64 - __builtin_clzll(num) - 1));
    }
}

class HamelBasis {
public:
    int maxn;
    int cnt;
    vector<long long> vec;

    /*
     * 清空线性基，预留 _n 个向量
     */
    void init(int _n) {
```

```
        maxn = _n;
        cnt = 0;
        vec.clear();
        vec.resize(_n);
    }

    /*
     * 插入 num，返回插入位置；相关向量返回-1
     */
    int insert(long long num) {
        for(int i = maxn - 1; ~i && num; i--) {
            if(num & (1ll << i)) {
                if(!vec[i]) {
                    vec[i] = num;
                    return i;
                } else {
                    num ^= vec[i];
                }
            }
        }
        return -1;
    }
    /*
     * 撤销 idx 处的向量插入
     */
    void pop(int idx) {
        vec[idx] = 0;
    }

    /*
     * 求 num 与线性基能表示的数异或后的最大值
     */
    long long getMax(long long num = 0) {
        long long ans = num;
        for(int i = maxn - 1; ~i; i--) {
            if(!(ans & (1ll << i)) && vec[i]) {
                ans ^= vec[i];
            }
        }
        return ans;
    }
};

HamelBasis basis;

long long a[N];
long long ans;

void dfs(int k) {
    if(k >= n) {
        ans = max(ans, basis.getMax());
        return;
    }
    int idx = basis.insert(a[k]);
    dfs(k + 2);
    dfs(k + 3);
    if(idx >= 0) {
        basis.pop(idx);
    }
}

```

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int T;
    cin >> T;
    while(T--) {
        cin >> n;
        for(int i = 0; i < n; i++) {
            cin >> a[i];
        }
        ans = 0;
        basis.init(60);
        dfs(0);
        dfs(1);
        cout << (ans) << '\n';
    }
}
```

注意

插入按最高位消元；查询最大值从高位贪心。

15.4 线性方程组

时间复杂度 $O(n^3)$

15.4.1 P2455 [SDOI2006] 线性方程组

```
class LinearSystem {
public:
    static const int N = 55;
    static constexpr double EPS = 1e-9;

    int n = 0;
    double a[N][N + 1];

    /*
     * 初始化 n 元方程组，系数矩阵清零
     */
    void init(int n_) {
        n = n_;
        for(int i = 0; i < n; i++) {
            for(int j = 0; j ⩽ n; j++) {
                a[i][j] = 0;
            }
        }
    }

    /*
     * 高斯-约当消元。返回 -1 无解、0 无穷多解、1 唯一解；
     * 唯一解时 a[i][n] 即 x_{i+1}
     */
    int solve() {
        int rank = 0;
        for(int col = 0; col < n && rank < n; col++) {
            int pivot = -1;
            for(int r = rank; r < n; r++) {
                if(abs(a[r][col]) > EPS) {
                    pivot = r;
                    break;
                }
```

```
            }
            if(pivot == -1) {
                continue;
            }
            for(int j = 0; j <= n; j++) {
                swap(a[rank][j], a[pivot][j]);
            }
            double k = a[rank][col];
            for(int j = col; j <= n; j++) {
                a[rank][j] /= k;
            }
            for(int r = 0; r < n; r++) {
                if(r == rank || abs(a[r][col]) <= EPS) {
                    continue;
                }
                double f = a[r][col];
                for(int j = col; j <= n; j++) {
                    a[r][j] -= f * a[rank][j];
                }
            }
            rank++;
        }

        for(int i = rank; i < n; i++) {
            if(abs(a[i][n]) > EPS) {
                return -1;
            }
        }
        return rank < n ? 0 : 1;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    LinearSystem ls;
    ls.init(n);
    for(int i = 0; i < n; i++) {
        for(int j = 0; j <= n; j++) {
            cin >> ls.a[i][j];
        }
    }

    int res = ls.solve();
    if(res == -1) {
        cout << -1 << '\n';
        return 0;
    }
    if(res == 0) {
        cout << 0 << '\n';
        return 0;
    }
    for(int i = 0; i < n; i++) {
        double x = abs(ls.a[i][n]) < LinearSystem::EPS ? 0.0 : ls.a[i][n];
        cout << "x" << (i + 1) << "=" << fixed << setprecision(2) << x << '\n';
    }
    return 0;
}
```

注意

返回 -1 无解、0 无穷多解、1 唯一解；唯一解在 `a[i][n]`。

15.5 拉格朗日插值

时间复杂度 $O(n^2)$ 预处理

调用前提：先 init，x/y 长度一致；moder 为质数，x 模意义互异。

```cpp
// moder 为质数，x 在模 moder 下两两不同；O(n^2 log moder)。
class Interpolation {
public:
    int64 moder = 998244353;
    int64 norm(int128 a) const {
        return (a % moder + moder) % moder;
    }

    vector<int64> x, y;
    /*
     * 分配 n+1 个插值点，调用处填充 x、y
     */
    void init(int n) {
        x.assign(n + 1, 0);
        y.assign(n + 1, 0);
    }
    /*
     * 根据插值点求 t 处的函数值，x 在模 moder 下须两两不同
     */
    int64 operator()(int64 t) const {
        int64 ans = 0;
        for(int i = 0; i < int(x.size()); i++) {
            int64 p = norm(y[i]);
            for(int j = 0; j < int(x.size()); j++) {
                if(i != j) {
                    int64 div = norm(int128(x[i]) - x[j]);

                    p = norm(int128(p) * norm(int128(t) - x[j]));
                    p = norm(int128(p) * quickPow(div, moder - 2, moder));
                }
            }
            ans = norm(int128(ans) + p);
        }
        return ans;
    }
};
```

注意

分母用费马小定理求逆；需要模数为素数。

第 16 章

组合

16.1 容斥原理

时间复杂度 $O(4s+2^4n)$

16.1.1 P1450 [HAOI2008] 硬币购物

```
const long long N = 100005;
long long c[5];
long long n;
long long d[5];
long long s;
long long dp[N];
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> c[0] >> c[1] >> c[2] >> c[3] >> n;
    for(long long T = 1; T ⩽ n; T++) {
        cin >> d[0] >> d[1] >> d[2] >> d[3] >> s;

        for(long long i = 0; i ⩽ s; i++) {
            dp[i] = 0;
        }

        dp[0] = 1;
        for(long long i = 0; i < 4; i++) {
            for(long long j = c[i]; j ⩽ s; j++) {
                dp[j] += dp[j - c[i]];
            }
        }

        long long ans = 0;
        for(long long i = 0; i < 1 << 4; i++) {
            long long k = s;
            for(long long j = 0; j < 4; j++) {
                if((i >> j) & 1) {
                    k -= c[j] * (d[j] + 1);
                }
            }
            if(k ⩾ 0) {
                if(__builtin_popcountll(i) & 1) {
                    ans -= dp[k];
                } else {
                    ans += dp[k];
                }
            }
        }
        cout << (ans) << '\n';
    }
}
```

注意

每组询问重跑背包会超时；先读入所有询问取 maxS 预处理。

16.2 约瑟夫环问题

时间复杂度 $O(n)$ 或 $O(k \log n)$

16.2.1 问题描述

n 个人标号 $0, 1, \cdots, n-1$。逆时针站一圈，从 0 号开始，每一次从当前的人逆时针数 k 个，然后让这个人出局。问最后剩下的人是谁。

16.2.2 过程

朴素算法

最朴素的算法莫过于直接枚举。用一个环形链表枚举删除的过程，重复 $n-1$ 次得到答案。复杂度 $\Theta(n^2)$。

简单优化

寻找下一个人的过程可以用线段树优化。具体地，开一个 $0, 1, \cdots, n-1$ 的线段树，然后记录区间内剩下的人的个数。寻找当前的人的位置以及之后的第 k 个人可以在线段树上二分做。

线性算法

设 $J_{n,k}$ 表示规模分别为 n, k 的约瑟夫问题的答案。我们有如下递归式

$$J_{n,k} = (J_{n-1,k} + k) \bmod n$$

这个也很好推。你从 0 开始数 k 个，让第 $k-1$ 个人出局后剩下 $n-1$ 个人，你计算出在 $n-1$ 个人中选的答案后，再加一个相对位移 k 得到真正的答案。这个算法的复杂度显然是 $\Theta(n)$ 的。

```
/*
 * 返回每次数到 k 出局后的幸存者编号，从 0 开始
 */
int josephus(int n, int k) {
    int res = 0;
    for(int i = 1; i ⩽ n; ++i) {
        res = (res + k) % i;
    }
    return res;
}
```

对数算法

对于 k 较小 n 较大的情况，本题还有一种复杂度为 $\Theta(k \log n)$ 的算法。

考虑到我们每次走 k 个删一个，那么在一圈以内我们可以删掉 $\lfloor \frac{n}{k} \rfloor$ 个，然后剩下了 $n - \lfloor \frac{n}{k} \rfloor$ 个人。这时我们在第 $\lfloor \frac{n}{k} \rfloor \cdot k$ 个人的位置上。而你发现它等于 $n - n \bmod k$。于是我们继续递归处理，算完后还原它的相对位置。还原相对位置的依据是：每次做一次删除都会把数到的第 k 个人删除，他们的编号被之后的人逐个继承，也即用 $n - \lfloor \frac{n}{k} \rfloor$ 人环算时每 k 个人即有 1 个人的位置失算，因此在得数小于 0 时，用还没有被删去 k 倍数编号的 n 人环的的 n 求模，在得数大于等于 0 时，即可以直接乘 $\frac{k}{k-1}$，于是得到如下的算法：

```
/*
 * 返回每次数到 k 出局后的幸存者编号，从 0 开始
 */
int josephus(int n, int k) {
    if(n == 1) {
        return 0;
    }
    if(k == 1) {
        return n - 1;
    }
```

```
    if(k > n) {
        return (josephus(n - 1, k) + k) % n; // 线性算法
    }
    int res = josephus(n - n / k, k);
    res -= n % k;
    if(res < 0) {
        res += n; // mod n
    } else {
        res += res / (k - 1); // 还原位置
    }
    return res;
}
```

可以证明这个算法的复杂度是 $\Theta(k\log n)$ 的。我们设这个过程的递归次数是 x，那么每一次问题规模会大致变成 $n\left(1-\frac{1}{k}\right)$，于是得到

$$n\left(1-\frac{1}{k}\right)^x=1$$

解这个方程得到

$$x=-\frac{\ln n}{\ln\left(1-\frac{1}{k}\right)}$$

注意

两个版本二选一；注意 $k\geq 1$、$n\geq 1$。

16.3 康托展开

时间复杂度 $O(n\log n)$

16.3.1 P5367 【模板】康托展开

```
class Cantor {
    static constexpr int64 MODER = 998244353;

public:
    // a[1..n] 是 1..n 的排列，返回从 1 开始的排名。
    int64 rank(const vector<int>& a) const {
        int n = int(a.size()) - 1;
        vector<int> arr(n + 1);
        vector<int64> fac(n + 1, 1);
        for(int i = 1; i <= n; i++) {
            arr[i] = i & -i;
            fac[i] = fac[i - 1] * i % MODER;
        }
        int64 ans = 1;
        for(int i = 1; i <= n; i++) {
            int cnt = 0;
            for(int j = a[i] - 1; j; j -= j & -j) {
                cnt += arr[j];
            }
            ans = (ans + cnt * fac[n - i]) % MODER;
            for(int j = a[i]; j <= n; j += j & -j) {
                arr[j]--;
            }
        }
        return ans;
    }
};
```

注意

配合树状数组求排名；阶乘取模。

第 17 章

计算几何

17.1 扫描线

时间复杂度 $O(n \log n)$

17.1.1 P5490 【模板】扫描线 & 矩形面积并

```
const int N = 100005;
const int M = 1000000007;
int n;

class Data {
public:
    int cnt;

    Data operator+(const Data& other) const {
        return {cnt + other.cnt};
    }
};

class Tag {
public:
    int add;

    Tag operator+(const Tag& other) const {
        return {add + other.add};
    }

    /*
     * 标记持久化：add 是累积覆盖次数，> 0 时整段都被覆盖
     */
    Data apply(const Data& d, int l, int r) const {
        if(add > 0) {
            return {r - l + 1};
        }
        return d;
    }
};

class Line {
public:
    int x1, x2;
    int y;
    bool upper;

    bool operator<(const Line& other) const {
        return y < other.y;
    }
};

Line line[N * 2];

/*
 * 注意：这里用的是标记持久化线段树，标记不下传，不是普通的懒惰标记线段树。
 * Tag::apply 接收的是累积后的标记；原理见 数据结构/线段树/标记持久化线段树.md。
 */
```

```
SegmentTree<Data, Tag> sgt;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n;
    sgt.init(0, M, 32 * n);
    int x1, x2, y1, y2;

    for(int i = 0; i < n; i++) {
        cin >> x1 >> y1 >> x2 >> y2;
        line[i * 2] = {x1, x2 - 1, y1, false};
        line[i * 2 + 1] = {x1, x2 - 1, y2, true};
    }

    sort(line, line + n * 2);

    long long w = 0;
    long long ly = 0;
    long long s = 0;
    for(int i = 0; i < n * 2; i++) {
        s += w * (line[i].y - ly);
        if(line[i].upper) {
            sgt.mod(line[i].x1, line[i].x2, {-1});
        } else {
            sgt.mod(line[i].x1, line[i].x2, {1});
        }
        w = sgt.query(0, M).cnt;
        ly = line[i].y;
    }
    cout << (s) << '\n';
}
```

注意

依赖标记持久化线段树：覆盖次数 > 0 返回区间全长，否则合并儿子。

第 IV 编

字符串

第 18 章

字符串

18.1 哈希

时间复杂度 $O(1)$ 单次查询

使用 unsigned long long 的模 2^64 自然溢出；反转等价类保存排序后的正反哈希二元组，避免把两个哈希相乘导致的额外信息丢失。哈希仍有碰撞可能，不提供确定性判等保证。

18.1.1 P3498 [POI 2010] KOR-Beads

```
#define N 200005
#define BASE 200019
class DeHash {
public:
    uint64 asc, rev;
};
inline bool operator<(const DeHash& A, const DeHash& B) {
    if(A.asc != B.asc) {
        return A.asc < B.asc;
    } else {
        return A.rev < B.rev;
    }
}
inline bool operator==(const DeHash& A, const DeHash& B) {
    return A.asc == B.asc && A.rev == B.rev || A.asc == B.rev && A.rev == B.asc;
}
DeHash de_hash[N];
set<pair<uint64, uint64>> s;
uint64 base[N];
uint64 n;
uint64 bead[N];
/*
 * 返回闭区间 [l,r] 子串的正反向哈希，反转后的字符串视为同类
 */
inline DeHash getSegHash(uint64 l, uint64 r) {
    return {
        de_hash[r].asc - de_hash[l - 1].asc * base[r - l + 1],
        de_hash[l].rev - de_hash[r + 1].rev * base[r - l + 1]};
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n;
    for(uint64 i = 1; i <= n; i++) {
        cin >> bead[i];
    }
    base[0] = 1;
    for(uint64 i = 1; i <= n; i++) {
        base[i] = base[i - 1] * BASE;
        de_hash[i].asc = de_hash[i - 1].asc * BASE + bead[i];
        de_hash[n - i + 1].rev = de_hash[n - i + 2].rev * BASE + bead[n - i + 1];
    }
    vector<uint64> ans_k;
    uint64 max_seg = 0, size;
    DeHash tmp;
    for(uint64 i = 1; i <= n; i++) {
```

```
        s.clear();

        for(uint64 j = 1; j + i - 1 <= n; j += i) {
            tmp = getSegHash(j, j + i - 1);
            if(!s.count(minmax(tmp.asc, tmp.rev))) {
                s.insert(minmax(tmp.asc, tmp.rev));
            } else {
            }
        }
        size = s.size();
        if(size > max_seg) {
            max_seg = size;
            ans_k.clear();
            ans_k.push_back(i);
        } else if(size == max_seg) {
            ans_k.push_back(i);
        }
    }
    cout << (max_seg) << ' ' << (ans_k.size()) << '\n';
    for(uint64 e : ans_k) {
        cout << (e) << ' ';
    }
}
```

注意

双哈希降低碰撞；模数与底数取大质数。

18.2 KMP

时间复杂度 $O(n+m)$

18.2.1 P3375 【模板】KMP

```
class KMP {
public:
    int n;
    string pat;
    vector<int> nxt;
    /*
     * 由模式串 pat 预处理失配数组
     */
    explicit KMP(const string& pat) : n(int(pat.size())), pat(pat) {
        nxt.resize(n + 1);
        nxt[0] = -1;
        for(int i = 0, j = -1; i < n;) {
            if(j == -1 || pat[i] == pat[j]) {
                ++i;
                ++j;
                nxt[i] = j;
            } else {
                j = nxt[j];
            }
        }
    }

    /*
     * 返回模式串在 s 中出现的所有起点，下标从 0 开始
     */
    vector<int> search(const string& s) const {
        vector<int> ans;
        int m = s.size();
```

```
        if(n == 0) {
            for(int i = 0; i <= m; i++) {
                ans.push_back(i);
            }
            return ans;
        }
        for(int i = 0, j = 0; i < m;) {
            if(j == -1 || s[i] == pat[j]) {
                ++i;
                ++j;
                if(j == n) {
                    ans.push_back(i - j);
                    j = nxt[j];
                }
            } else {
                j = nxt[j];
            }
        }
        return ans;
    }
};
```

18.2.2 P9770 [HUSTFC 2023] 逆 KMP

```
const int maxn = 200005;
int ans[maxn], n, fa[maxn][18];
int find(int x, int k) {
    return fa[x][k] == x ? x : fa[x][k] = find(fa[x][k], k);
}
void merge(int x, int y, int k) {
    x = find(x, k);
    y = find(y, k);
    if(x < y) {
        swap(x, y);
    }
    if(x != y) {
        fa[x][k] = y;
    }
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n;
    const int maxk = __lg(n);
    for(int i = 1; i <= n; ++i) {
        for(int k = 0; k <= maxk; ++k) {
            fa[i][k] = i;
        }
    }
    for(int i = 1, l1, r1, l2, r2; i <= n; ++i) {
        int x;
        cin >> x;
        if(x) {
            l1 = 1;
            r1 = x;
            l2 = i - x + 1;
            r2 = i;
            for(int k = maxk; ~k; --k) {
                if(l1 + (1 << k) - 1 <= r1) {
                    merge(l2, l1, k);
                    l1 += 1 << k;
```

```
                    l2 += 1 << k;
                }
            }
        }
    }
    for(int k = maxk; k; --k) {
        for(int i = 1; i + (1 << k) - 1 <= n; ++i) {
            int pos = find(i, k);
            merge(i, pos, k - 1);
            merge(i + (1 << (k - 1)), pos + (1 << (k - 1)), k - 1);
        }
    }
    int cnt = 0;
    for(int i = 1; i <= n; ++i) {
        if(find(i, 0) == i) {
            ans[i] = ++cnt;
        } else {
            ans[i] = ans[find(i, 0)];
        }
    }
    for(int i = 1; i <= n; ++i) {
        cout << (ans[i]) << ' ';
    }
    return 0;
}
```

注意

nxt[0] = -1；匹配成功后 j = nxt[j] 继续找重叠匹配。

18.3 AC 自动机

时间复杂度 $O(\sum |s|)$ 建树

18.3.1 P5357 【模板】AC 自动机

```
class ACAutomaton {
public:
    class Node {
    public:
        int next[26]{}, fail = 0;
    };
    vector<Node> node{Node()};
    vector<int> order;
    bool built = false;
    /*
     * 清空自动机
     */
    void clear() {
        node.assign(1, Node());
        order.clear();
        built = false;
    }
    /*
     * 插入模式串 s，返回终止节点编号
     */
    int insert(const string& s) {
        int u = 0;
        for(char c : s) {
            int x = c - 'a';

```

```
            if(!node[u].next[x]) {
                int v = int(node.size());
                node.emplace_back();
                node[u].next[x] = v;
            }
            u = node[u].next[x];
        }
        return u;
    }
    /*
     * 插入全部模式串后建立 fail 指针和转移
     */
    void build() {
        if(built) {
            return;
        }
        for(int v : node[0].next) {
            if(v) {
                order.push_back(v);
            }
        }

        for(int i = 0; i < int(order.size()); i++) {
            int u = order[i];
            for(int c = 0; c < 26; c++) {
                int v = node[u].next[c];
                if(v) {
                    node[v].fail = node[node[u].fail].next[c];
                    order.push_back(v);
                } else {
                    node[u].next[c] = node[node[u].fail].next[c];
                }
            }
        }
        built = true;
    }
    /*
     * 统计文本 s 中各节点对应模式串的出现次数，重复查询不累积
     */
    vector<long long> query(const string& s) const {
        vector<long long> ans(node.size());
        int u = 0;
        for(char c : s) {
            u = node[u].next[c - 'a'];
            ans[u]++;
        }
        for(auto it = order.rbegin(); it != order.rend(); ++it) {
            ans[node[*it].fail] += ans[*it];
        }
        return ans;
    }
};
```

注意

先全部插入再 `build()`；fail 树上逆 BFS 累加。

18.4 后缀自动机

时间复杂度 $O(n)$

18.4.1 题意翻译

输入两个字符串，输出它们的最长公共子串长度，若不存在公共子串则输出 0。

```
class State {
public:
    int len{0}, link{0};
    map<char, int> next;
};

class SuffixAutomaton {
    vector<State> state;
    int size{0}, last{0};

public:
    explicit SuffixAutomaton(int n = 0) {
        init(n);
    }

    /*
     * 清空自动机，n 为预计字符串长度
     */
    void init(int n = 0) {
        state.assign(1, State());
        state.reserve(max(1, 2 * n));
        state[0].link = -1;
        size = 1;
        last = 0;
    }

    /*
     * 在当前字符串末尾加入字符 c
     */
    void extend(char c) {
        int cur = size++;
        state.emplace_back();
        state[cur].len = state[last].len + 1;
        int p = last;
        while(p != -1 && !state[p].next.count(c)) {
            state[p].next[c] = cur;
            p = state[p].link;
        }
        if(p == -1) {
            state[cur].link = 0;
        } else {
            int q = state[p].next[c];
            if(state[p].len + 1 == state[q].len) {
                state[cur].link = q;
            } else {
                int clone = size++;
                state.emplace_back();
                state[clone].len = state[p].len + 1;
                state[clone].next = state[q].next;
                state[clone].link = state[q].link;
                while(p != -1 && state[p].next[c] == q) {
                    state[p].next[c] = clone;
                    p = state[p].link;
                }
                state[q].link = state[cur].link = clone;
            }
        }
        last = cur;
```

```
    }

    /*
     * 返回两个字符串的一个最长公共子串，调用时重建自动机
     */
    string lcs(const string& string1, const string& string2) {
        init(int(string1.size()));
        for(char c : string1) {
            extend(c);
        }
        int v = 0, l = 0, best = 0, bestpos = -1;
        for(int i = 0; i < string2.size(); i++) {
            while(v && !state[v].next.count(string2[i])) {
                v = state[v].link;
                l = state[v].len;
            }
            if(state[v].next.count(string2[i])) {
                v = state[v].next[string2[i]];
                l++;
            }
            if(l > best) {
                best = l;
                bestpos = i;
            }
        }
        return string2.substr(bestpos - best + 1, best);
    }

};
```

题目输入输出示例（与正文组装后编译，正文不重复驱动代码）。

注意

clone 节点完整复制转移与 link，再重定向。

第 V 编

动态规划

第 19 章

动态规划

19.1 线性动规

时间复杂度 $O(n \log n)$（LIS）

19.1.1 最长上升子序列

```
#define N 1000005
int n;
int num[N];
int d[N];
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n;
    int l = 0;
    for(int i = 0; i < n; i++) {
        cin >> num[i];
    }

    l = 1;
    d[0] = num[0];
    int idx;
    int ans = 1;
    for(int i = 1; i < n; i++) {
        if(num[i] > d[l - 1]) {
            d[l] = num[i];
            l++;
        } else {
            idx = lower_bound(d, d + l, num[i]) - d;
            d[idx] = num[i];
        }
        ans = max(ans, l);
    }
    cout << (ans) << '\n';
}
```

19.1.2 P1439 两个排列的最长公共子序列

```
const int N = 100005;
int inv[N];
int a[N];
int dp[N];
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, num;
    cin >> n;
    for(int i = 1; i ⩽ n; i++) {
        cin >> num;
        inv[num] = i;
        dp[i] = 0x3f3f3f3f;
    }

    for(int i = 1; i ⩽ n; i++) {
        cin >> num;
        num = inv[num];
```

```
        a[i] = num;
    }

    for(int i = 1; i ⩽ n; i++) {
        int j = lower_bound(dp, dp + i, a[i]) - dp;
        j--;
        if(dp[j] < a[i] && a[i] < dp[j + 1]) {
            dp[j + 1] = a[i];
        }
    }

    int ans = 0;
    for(int i = n; i > 0; i--) {
        if(dp[i] =≠ 0x3f3f3f3f) {
            ans = i;
            break;
        }
    }

    cout << (ans);
}
```

19.1.3 图片题例：概率 DP

此题输入格式见原图，无独立文本样例。

1009 线段染色

从左到右（从 1 到 n ）进行染色，记已尝试染色整点 $1\sim i$ ，且 $1\sim i$ 内的线段已全被染色，且当前最后一个被染色点为 j 的概率为 $dp_{i,j}$ 。

尝试从 $i-1$ 转移至 i :

1. 当点 i 被染色时，所有状态有同样的 p_i 的概率被改变（最后一个染色点被后移至 i ），则使全局的所有值乘以 $1-p_i$ ，再将损失的值加到位置 i :

$$dp_{i,j}=\begin{cases}dp_{i-1,j}\cdot(1-p_i),j<i\\\sum\limits_{k=0}^{j-1}dp_{i-1,k},j=i\\0,j>i\end{cases}$$

1. 这之后关注右端点为 i 的线段，取其中最大的左端点为 mxl_i （没有线段则记其为 0 ），此时最后一个被染色点为 $j<mxl_i$ 则确定不符合"染色所有线段"的条件，使所有 $dp_{i,j<mxl_i}$ 变为 0 。

最后要求的 $Ans=\sum\limits_{j=0}^{n}dp_{n,j}$ 。

易线性求得每个 mxl_i 和 $mxl'_i=\min\limits_{j=0}^{i}mxl_j$；但用区乘线段树以 $\mathcal{O}(n\log n)$ 转移 dp 可能过慢。

优化:

维护"全局的所有值之和"为 sum ；

1. 将上文的第1步改为:
   - 若 $p_i\neq 1(a_i\neq 10)$ ，全局的值不变，仅赋位置 i 的值为 $sum\cdot\frac{p_i}{1-p_i}$ （最后再使答案乘回 $1-p_i$ ）；
   - 否则赋位置 i 的值为 sum，并视 mxl_i 为 i ；
2. 维护"存在有效值"的区间 $[mxl'_i,i]$ ；区间的左右端点都只会向右移动，移动时容易维护区间内的 "sum" ；

最后要求的 $Ans=sum\cdot\prod\limits_{i=1\sim n,p_i\neq 1}(1-p_i)$ 。

依赖：扩展欧几里得 • 代码块 1。

```
const int MODER = 1000000007;
const int N = 200005;
const int INF = 0x3f3f3f3f;

long long inv[11];
long long dp[N];
int p[N];
int maxl[N];

int n, m;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    for(int i = 1; i ⩽ 10; i++) {
        int64 y;
        exgcd(i, MODER, inv[i], y);
        inv[i] = (inv[i] % MODER + MODER) % MODER;
    }

    int T;
    cin >> T;
    while(T--) {
        cin >> n;
        cin >> m;
        long long pie = 1;
        for(int i = 1; i ⩽ n; i++) {
            cin >> p[i];
            maxl[i] = 0;
            dp[i] = 0;
            if(p[i] ≠ 10) {
                pie = pie * (10 - p[i]) % MODER * inv[10] % MODER;
            }
        }

        dp[0] = 1;

        int l, r;
        for(int i = 0; i < m; i++) {
            cin >> l;
            cin >> r;
            maxl[r] = max(maxl[r], l);
        }

        long long sum = 1;
        l = 0;

        for(int i = 1; i ⩽ n; i++) {
            if(p[i] ≠ 10) {
                dp[i] = sum * p[i] % MODER * inv[10 - p[i]] % MODER;
                sum = (sum + dp[i]) % MODER;

                if(maxl[i] > l) {
                    for(; l < maxl[i]; l++) {
                        sum -= dp[l];
                        dp[l] = 0;
                    }
                    sum = (sum % MODER + MODER) % MODER;
                }
            } else {
                dp[i] = sum;
```

```
                for(; l < i; l++) {
                    dp[l] = 0;
                }
            }
        }
        sum = sum * pie % MODER;

        cout << (sum) << '\n';
    }
}
```

注意

LIS 用 `lower_bound` 求严格上升；`n = 0` 需单独处理。

1003 奸商

OCS-4.9.45

Problem Description

你是一个奸商，你甚至将金钱看得比你的性命还重要，所以你被巨人抓了起来。

巨人给了你一个字符串，字符串中只包含字母表中的前 17 个字母，即 $a \sim q$，并将会对所有长度 $\geq len$ 的子串进行检查，如果有任意一个子串不是优秀的，他就会处决你。

对于一个长度为 n 的字符串 s，如果至少存在一个位置 i $(1 \leq i \leq n)$ 满足 $s[i] = s[n+1-i]$，就称此字符串是优秀的。

现在，你可以花费一定代价使巨人对某些字母产生幻视，例如，如果巨人对字母 d 产生幻视，那么在检查过程中，巨人可能会将字符 $'d'$ 识别成某个 $ascii$ 码 $\geq 'd'$ 且 $ascii$ 码 $\leq 'q'$ 的字符，即 $'d'$，$'e'$，$'f'$，$\ldots$，$'q'$。请注意：

- 巨人会对字符串中所有的字符 $'d'$ 产生幻视；
- 巨人在检查不同子串时，可能会对同一个字符 $'d'$ 有不同的幻视。

例如，对于字符串 $acde$，假设巨人仅对字母 a 产生幻视：

- 检查不优秀的子串 $acde$ 时，巨人可能识别成优秀的字符串 $ecde$；
- 检查不优秀的子串 ac 时，巨人可能识别成优秀的字符串 cc。

前文提到，你爱财如命，所以你希望在有生还希望的前提下，花费最小的代价，请你打印这个代价。

Input

第一行包含一个整数 T，表示测试数据的组数，$1 \leq T \leq 100$。

对于每组测试数据：

第一行包含一个整数 n，表示字符串的长度；

第二行给出一个长度为 n 的字符串；

第 w_i $(1 \le w_i \le 1e6)$，分别表示使巨人对字母 a ~ q 产生的恍惚变的代价；

第四行包含一个整数 len $(len \le n)$，巨人会对所有长度 $\ge len$ 的子串进行检查。

数据保证：$\sum n \le 3000$。

OCS-4.9.45

Output

请你打印在有生还希望的前提下，花费的最小代价。

Sample Input

```
1
5
encle
1 1 8 3 4 4 8 4 2 4 6 8 9 8 2 2 8
2
```

Sample Output

```
12
```

19.2 区间 DP

时间复杂度 $O(n^3)$

19.2.1 P4170 [CQOI2007] 涂色

19.2.2 思路

$$f_{l,r} = min_{i=l}^{r-1}(f_{l,i} + f_{i+1,r})$$

```
const int N = 55;
int dp[N][N];
char color[N];
int n;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> (color);
    memset(dp, 0x3f, sizeof(dp));
    for(; color[n]; n++) {
        dp[n][n] = 1;
    }

    for(int len = 2; len <= n; len++) {
        for(int l = 0; l < n; l++) {
            int r = l + len - 1;

            if(r >= n) {
                break;
            }

            if(color[l] == color[r]) {
                dp[l][r] = min(dp[l + 1][r], dp[l][r - 1]);
            }

            for(int i = l; i < r; i++) {
                dp[l][r] = min(dp[l][r], dp[l][i] + dp[i + 1][r]);
            }
        }
    }

    cout << (dp[0][n - 1]);
}
```

19.2.3 P1880 [NOI1995] 石子合并

```
#define N 128
int dp[N * 2][N * 2];
int dp2[N * 2][N * 2];
int prefix[N * 2];
int num[N * 2];
int n;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n;
    for(int i = 1; i <= n; i++) {
        cin >> num[i];
        prefix[i] = prefix[i - 1] + num[i];
    }

    for(int i = n + 1; i <= n * 2; i++) {
```

```
        num[i] = num[i - n];
        prefix[i] = prefix[i - 1] + num[i];
    }

    for(int l = 2; l ⩽ n; l++) { // [i: i+l]
        for(int i = 1; i ⩽ 2 * n - l + 1; i++) {
            int maximum = 0, minimum = 2147483647;
            for(int j = 1; j < l; j++) {
                maximum = max(maximum, dp[i][i + j - 1] + dp[i + j][i + l - 1]);
                minimum = min(minimum, dp2[i][i + j - 1] + dp2[i + j][i + l - 1]);
            }
            dp[i][i + l - 1] = maximum + prefix[i + l - 1] - prefix[i - 1];
            dp2[i][i + l - 1] = minimum + prefix[i + l - 1] - prefix[i - 1];
        }
    }

    int minimum = 2147483647;
    int maximum = 0;

    for(int i = 1; i ⩽ n; i++) {
        minimum = min(minimum, dp2[i][i + n - 1]);
        maximum = max(maximum, dp[i][i + n - 1]);
    }

    cout << (minimum) << '\n'
         << (maximum) << '\n';
}
```

19.2.4 P6934 [ICPC 2017 WF] Posterize

```
const int64 N = 257;
int64 f[N][N];
int64 g[N][N];
int64 pix[N];
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int64 d, k;
    memset(f, 0x3f, sizeof(f));
    memset(g, 0x3f, sizeof(g));
    cin >> d >> k;
    for(int64 i = 1; i ⩽ d; i++) {
        int64 r, p;
        cin >> r >> p;
        pix[r + 1] = p;
    }

    int64 sum;
    for(int64 i = 1; i < N; i++) {
        for(int64 j = i; j < N; j++) {
            for(int64 t = i; t ⩽ j; t++) {
                sum = 0;
                for(int64 c = i; c ⩽ j; c++) {
                    sum += (c - t) * (c - t) * pix[c];
                }
                f[i][j] = min(f[i][j], sum);
            }
        }
    }

    for(int64 i = 0; i < N; i++) {
        g[i][0] = 0;
```

```
    }

    for(int64 i = 1; i ⩽ k; i++) {
        for(int64 r = 1; r < N; r++) {
            for(int64 l = 1; l ⩽ r; l++) {
                g[i][r] = min(g[i][r], g[i - 1][l - 1] + f[l][r]);
            }
        }
    }
    cout << (g[k][N - 1]);
}
```

注意

按区间长度递增枚举；环形问题拆成 2n。

延伸阅读

1009 线段染色

Problem Description

给出一包含整点 $1 \sim n$ 的数轴和其上的 m 条线段，第 i 条线段的左右端点分别为 l_i, r_i 。

现对数轴上的每个整点都进行一次染色，整点 i 被染色的成功率为 p_i ，不同整点被染色的成功率互相独立。

称一线段被染色，**当且仅当**该线段上的至少一点被染色；求所有线段都被染色的概率。

Input

第一行含一个正整数 t $(1 \leq t \leq 5 \times 10^4)$ ，表示数据组数；接下来对于每组数据：

第一行包含 2 个正整数 n, m $(1 \leq n \leq 2 \times 10^5, 0 \leq m \leq 2 \times 10^5)$ ，表示数轴的长度和线段的个数；

第二行为序列 a ，包含 n 个整数，第 i 个数 a_i $(0 \leq a_i \leq 10)$ 代表整点 i 被染色的成功率为 $p_i = \frac{a_i}{10}$ ；

接下来的 m 行，每行包括 2 个正整数，第 i 行的两数依次代表第 i 个线段的左右端点位置 l_i, r_i $(1 \leq l_i \leq r_i \leq n)$ 。

保证 $\sum n, \sum m \leq 2.5 \times 10^6$ 。

Output

对于每组数据，输出一个非负整数独占一行，表示所有线段都被染色的概率对 $10^9 + 7$ 取模后的结果。

Sample Input

```
6
4 2
5 5 5 5
2 3
3 4
4 2
5 5 5 5
2 2
3 4
1 0
3
4 1
0 2 3 4
2 4
4 2
5 0 5 5
1 3
2 4
4 3
1 1 1 0
2 4
3 4
4 4
```

Sample Output

```
625000005
375000003
1
48000001
625000005
0
```

Hint

样例中各答案的真实值依次为 $\frac{5}{8}, \frac{3}{8}, 1, \frac{83}{125}, \frac{5}{8}, 0$ 。

19.3 树上 DP

时间复杂度 $O(nm)$

19.3.1 P1642 规划

```
const int N = 128;
const double ETA = 1e-4;
const double INF = 1e9;

class Node {
public:
    int w, c, s;
    vector<int> oe;
    double dp[N];
};

Node node[N];
int n, m;
double c;

void predfs(int cur, int fa) {
    node[cur].s = 1;
    for(int v : node[cur].oe) {
        if(v != fa) {
            predfs(v, cur);
            node[cur].s += node[v].s;
        }
    }
}

void dfs(int cur, int fa) {
    for(int v : node[cur].oe) {
        if(v != fa) {
            dfs(v, cur);
        }
    }

    node[cur].dp[1] = node[cur].w - node[cur].c * c;
    for(int i = 2; i <= node[cur].s; i++) {
        node[cur].dp[i] = -INF;
    }

    for(int v : node[cur].oe) {
        if(v != fa) {
            for(int i = node[cur].s; i >= 2; i--) {
                for(int j = 1; j <= min(i - 1, node[v].s); j++) {
                    node[cur].dp[i] = max(node[cur].dp[i], node[cur].dp[i - j] + node[v].dp[j]);
                }
            }
        }
    }
}

bool check() {
    dfs(1, 0);

    for(int i = 1; i <= n; i++) {
        if(node[i].s >= n - m && node[i].dp[n - m] > -ETA) {
            return true;
        }
```

```
    }
    return false;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m;
    for(int i = 1; i ⩽ n; i++) {
        cin >> node[i].w;
    }

    for(int i = 1; i ⩽ n; i++) {
        cin >> node[i].c;
    }

    int u, v;
    for(int i = 1; i < n; i++) {
        cin >> u >> v;
        node[u].oe.push_back(v);
        node[v].oe.push_back(u);
    }

    predfs(1, 0);

    double ans = 0.;
    double l = 0, r = 10005.0, mid;

    while(abs(r - l) > ETA) {
        c = mid = (l + r) / 2;
        if(check()) {
            ans = l = mid;
        } else {
            r = mid;
        }
    }

    cout << fixed << setprecision(1) << (ans);
}
```

19.3.2 P3478 [POI 2008] STA-Station

数据规模与约定 对于全部的测试点，保证 $1 \leq n \leq 10^6$，$1 \leq u, v \leq n$，给出的是一棵树。

```
#define N 1000005
int64 n;
class Node {
public:
    vector<int64> son;
    int64 d, w;
    int64 f;
};
Node node[N];

void getWeight(int64 idx, int64 fa) {
    node[idx].w = 1;
    node[idx].f = 0;
    if(fa > 0) {
        node[idx].d = node[fa].d + 1;
    } else {
        node[idx].d = 0;
    }
```

```

    for(int64 son : node[idx].son) {
        if(son != fa) {
            getWeight(son, idx);
            node[idx].w += node[son].w;
        }
    }
}

void getSumDep(int64 idx, int64 fa) {
    for(int64 son : node[idx].son) {
        if(son != fa) {
            node[son].f = node[idx].f - node[son].w + (n - node[son].w);
            getSumDep(son, idx);
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n;
    int64 u, v;
    for(int64 i = 1; i < n; i++) {
        cin >> u >> v;
        node[u].son.push_back(v);
        node[v].son.push_back(u);
    }

    getWeight(1, -1);

    getSumDep(1, -1);

    int64 ans_idx;
    int64 max_d_sum = -2147483648;

    for(int64 i = 1; i <= n; i++) {
        if(max_d_sum < node[i].f) {
            max_d_sum = node[i].f;
            ans_idx = i;
        }
    }

    cout << (ans_idx) << '\n';
}
```

19.3.3 P2014 [CTSC1997] 选课

```
#define N 305
int n, m;

class Node {
public:
    vector<int> son;
    vector<int> f;
    int score;
};

Node node[305];

int dfs(int root) {
```

```
    int s;
    int l = node[root].son.size();
    node[root].f.resize(m + 1, 0);

    if(root == 0) {
        s = 0; // This node itself doesn't count.
        for(int i = 0; i < l; i++) {
            int son = node[root].son[i];
            s += dfs(son);
            for(int j = min(s, m); j >= 0; j--) {
                for(int k = 0; k <= j; k++) {
                    node[root].f[j] = max(node[root].f[j], node[root].f[j - k] + node[son].f[k]);
                }
            }
        }
    } else {
        s = 1; // This node itself.
        node[root].f[1] = node[root].score;
        for(int i = 0; i < l; i++) {
            int son = node[root].son[i];
            s += dfs(son);
            for(int j = min(s, m); j > 0; j--) {
                for(int k = 0; k < j; k++) {
                    node[root].f[j] = max(node[root].f[j], node[root].f[j - k] + node[son].f[k]);
                }
            }
        }
    }
    return s;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m;

    int fa, score;
    for(int i = 1; i <= n; i++) {
        cin >> fa >> score;
        node[fa].son.push_back(i);
        node[i].score = score;
    }

    node[0].score = 0;

    dfs(0);

    cout << (node[0].f[node[0].f.size() - 1]) << '\n';
}
```

注意

树上背包合并时枚举上界取 `min(i-1, s[v])`；递归深度可达 1e6。

19.4 背包

时间复杂度 $O(nV)$

19.4.1 引入

在具体讲何为「背包 dp」前，先来看如下的例题：

题意概要：有 n 个物品和一个容量为 W 的背包，每个物品有重量 w_i 和价值 v_i 两种属性，要求选若干物品放入背包使背包中物品的总价值最大且背包中物品的总重量不超过背包的容量。

在上述例题中，由于每个物体只有两种可能的状态（取与不取），对应二进制中的 0 和 1，这类问题便被称为「0-1 背包问题」。

19.4.2 0-1 背包

解释

例题中已知条件有第 i 个物品的重量 w_i，价值 v_i，以及背包的总容量 W。

设 DP 状态 $f_{i,j}$ 为在只能放前 i 个物品的情况下，容量为 j 的背包所能达到的最大总价值。

考虑转移。假设当前已经处理好了前 $i-1$ 个物品的所有状态，那么对于第 i 个物品，当其不放入背包时，背包的剩余容量不变，背包中物品的总价值也不变，故这种情况的最大价值为 $f_{i-1,j}$；当其放入背包时，背包的剩余容量会减小 w_i，背包中物品的总价值会增大 v_i，故这种情况的最大价值为 $f_{i-1,j-w_i}+v_i$。

由此可以得出状态转移方程：

$$f_{i,j}=\max(f_{i-1,j},f_{i-1,j-w_i}+v_i)$$

这里如果直接采用二维数组对状态进行记录，会出现 MLE。可以考虑改用滚动数组的形式来优化。

由于对 f_i 有影响的只有 f_{i-1}，可以去掉第一维，直接用 f_i 来表示处理到当前物品时背包容量为 i 的最大价值，得出以下方程：

$$f_j=\max\left(f_j,f_{j-w_i}+v_i\right)$$

务必牢记并理解这个转移方程，因为大部分背包问题的转移方程都是在此基础上推导出来的。

实现

```
for(int i = 1; i ⩽ n; i++) {
    for(int l = W; l ⩾ w[i]; l--) {
        f[l] = max(f[l], f[l - w[i]] + v[i]);
    }
}
```

19.4.3 完全背包

解释

完全背包模型与 0-1 背包类似，与 0-1 背包的区别仅在于一个物品可以选取无限次，而非仅能选取一次。

我们可以借鉴 0-1 背包的思路，进行状态定义：设 $f_{i,j}$ 为只能选前 i 个物品时，容量为 j 的背包可以达到的最大价值。

需要注意的是，虽然定义与 0-1 背包类似，但是其状态转移方程与 0-1 背包并不相同。

过程

可以考虑一个朴素的做法：对于第 i 件物品，枚举其选了多少个来转移。这样做的时间复杂度是 $O(n^3)$ 的。

状态转移方程如下：

$$f_{i,j}=\max_{k=0}^{+\infty}(f_{i-1,j-k\times w_i}+v_i\times k)$$

考虑做一个简单的优化。可以发现，对于 $f_{i,j}$，只要通过 $f_{i,j-w_i}$ 转移就可以了。因此状态转移方程为：

$$f_{i,j}=\max(f_{i-1,j},f_{i,j-w_i}+v_i)$$

理由是当我们这样转移时，$f_{i,j-w_i}$ 已经由 $f_{i,j-2\times w_i}$ 更新过，那么 $f_{i,j-w_i}$ 就是充分考虑了第 i 件物品所选次数后得到的最优结果。换言之，我们通过局部最优子结构的性质重复使用了之前的枚举过程，优化了枚举的复杂度。

与 0-1 背包相同，我们可以将第一维去掉来优化空间复杂度。如果理解了 0-1 背包的优化方式，就不难明白压缩后的循环是正向的（也就是上文中提到的错误优化）。

19.4.4 多重背包

多重背包也是 0-1 背包的一个变式。与 0-1 背包的区别在于每种物品有 k_i 个，而非一个。

一个很朴素的想法就是：把「每种物品选 k_i 次」等价转换为「有 k_i 个相同的物品，每个物品选一次」。这样就转换成了一个 0-1 背包模型，套用上文所述的方法就可已解决。状态转移方程如下：

$$f_{i,j} = \max_{k=0}^{k_i}(f_{i-1,j-k\times w_i} + v_i \times k)$$

时间复杂度 $O(W\sum_{i=1}^{n} k_i)$。

```
for(int i = 1; i ⩽ n; i++) {
    for(int weight = W; weight ⩾ w[i]; weight--) {
        // 多遍历一层物品数量
        for(int k = 1; k * w[i] ⩽ weight && k ⩽ cnt[i]; k++) {
            dp[weight] = max(dp[weight], dp[weight - k * w[i]] + k * v[i]);
        }
    }
}
```

二进制分组优化

考虑优化。我们仍考虑把多重背包转化成 0-1 背包模型来求解。

解释

显然，复杂度中的 $O(nW)$ 部分无法再优化了，我们只能从 $O(\sum k_i)$ 处入手。为了表述方便，我们用 $A_{i,j}$ 代表第 i 种物品拆分出的第 j 个物品。

在朴素的做法中，$\forall j \le k_i$，$A_{i,j}$ 均表示相同物品。那么我们效率低的原因主要在于我们进行了大量重复性的工作。举例来说，我们考虑了「同时选 $A_{i,1}, A_{i,2}$」与「同时选 $A_{i,2}, A_{i,3}$」这两个完全等效的情况。这样的重复性工作我们进行了许多次。那么优化拆分方式就成为了解决问题的突破口。

过程

我们可以通过「二进制分组」的方式使拆分方式更加优美。

具体地说就是令 $A_{i,j}\ (j \in [0, \lfloor \log_2(k_i+1) \rfloor - 1])$ 分别表示由 2^j 个单个物品「捆绑」而成的大物品。特殊地，若 k_i+1 不是 2 的整数次幂，则需要在最后添加一个由 $k_i - 2^{\lfloor \log_2(k_i+1) \rfloor - 1}$ 个单个物品「捆绑」而成的大物品用于补足。

举几个例子：

- $6 = 1 + 2 + 3$
- $8 = 1 + 2 + 4 + 1$
- $18 = 1 + 2 + 4 + 8 + 3$
- $31 = 1 + 2 + 4 + 8 + 16$

显然，通过上述拆分方式，可以表示任意 $\le k_i$ 个物品的等效选择方式。将每种物品按照上述方式拆分后，使用 0-1 背包的方法解决即可。

时间复杂度 $O(W\sum_{i=1}^{n} \log_2 k_i)$

实现

```
index = 0;
for(int i = 1; i ⩽ m; i++) {
    int c = 1, p, h, k;
    cin >> p >> h >> k;
```

```
    while(k > c) {
        k -= c;
        list[++index].w = c * p;
        list[index].v = c * h;
        c *= 2;
    }
    list[++index].w = p * k;
    list[index].v = h * k;
}
```

19.4.5 单调队列优化多重背包

???+ note "问题描述" 你有 n 个物品，每个物品重量为 w_i，价值为 v_i，数量为 k_i。你有一个承重上限为 W 的背包，现在要求你在不超过重量上限的情况下选取价值和尽可能大的物品放入背包。求最大价值。

不了解背包 DP 的请先阅读 背包 DP。设 $f_{i,j}$ 表示前 i 个物品装入承重为 j 的背包的最大价值，朴素的转移方程为

$$f_{i,j}=\max_{k=0}^{k_i}(f_{i-1,j-k\times w_i}+v_i\times k)$$

时间复杂度 $O(W\sum k_i)$。

考虑优化 f_i 的转移。为方便表述，设 $g_{x,y}=f_{i,x\times w_i+y},g'_{x,y}=f_{i-1,x\times w_i+y}$，其中 $0\le y<w_i$，则转移方程可以表示为：

$$g_{x,y}=\max_{k=0}^{k_i}(g'_{x-k,y}+v_i\times k)$$

设 $G_{x,y}=g'_{x,y}-v_i\times x$。则方程可以表示为：

$$g_{x,y}=\max_{k=0}^{k_i}(G_{x-k,y})+v_i\times x$$

这样就转化为一个经典的单调队列优化形式了。$G_{x,y}$ 可以 $O(1)$ 计算，因此对于固定的 y，我们可以在 $O\left(\left\lfloor\frac{W}{w_i}\right\rfloor\right)$ 的时间内计算出 $g_{x,y}$。因此求出所有 $g_{x,y}$ 的复杂度为 $O\left(\left\lfloor\frac{W}{w_i}\right\rfloor\right)\times O(w_i)=O(W)$。这样转移的总复杂度就降为 $O(nW)$。

在实现的时候，我们需要先枚举 y，这样才能保证枚举 x 的时候利用单调队列进行优化，而单调队列中存储的是 $x-k$，并不存储 k，这样使用的时候需要通过 `f[last][q.front() * w[i] + y] - q.front() * v[i]` 获取对应的 $G_{x-k,y}$，不难发现 $x-k\in[x-k_i,x]$，因此在枚举 x 的时候，我们需要删除队列中不在这个范围内的元素。

```
constexpr int MAXV = 4e4 + 10;
constexpr int MAXN = 1e2 + 10;
int n, W, last = 0, now = 1;
int v[MAXN], w[MAXN], k[MAXN];
int f[2][MAXV];
deque<int> q;

int main() {
    ios::sync_with_stdio(false);
    cin >> n >> W;
    for(int i = 1; i <= n; i++) {
        cin >> v[i] >> w[i] >> k[i];
    }
    for(int i = 1; i <= n; i++) {
        for(int y = 0; y < w[i]; y++) {
            // 清空队列
            deque<int>().swap(q);
            for(int x = 0; x * w[i] + y <= W; x++) {
                // 弹出不在范围的元素
                while(!q.empty() && q.front() < x - k[i]) {
```

```
                    q.pop_front();
                }
                // 保证队列单调
                while(!q.empty() && f[last][q.back() * w[i] + y] - q.back() * v[i] <
                                    f[last][x * w[i] + y] - x * v[i]) {
                    q.pop_back();
                }
                q.push_back(x);
                f[now][x * w[i] + y] =
                    f[last][q.front() * w[i] + y] - q.front() * v[i] + x * v[i];
            }
        }
        swap(last, now);
    }
    cout << f[last][W] << endl;
    return 0;
}
```

19.4.6 混合背包

混合背包就是将前面三种的背包问题混合起来，有的只能取一次，有的能取无限次，有的只能取 k 次。

这种题目看起来很吓人，可是只要领悟了前面几种背包的中心思想，并将其合并在一起就可以了。下面给出伪代码：

19.4.7 二维费用背包

有 n 个任务需要完成，完成第 i 个任务需要花费 t_i 分钟，产生 c_i 元的开支。

现在有 T 分钟时间，W 元钱来处理这些任务，求最多能完成多少任务。

这道题是很明显的 0-1 背包问题，可是不同的是选一个物品会消耗两种价值（经费、时间），只需在状态中增加一维存放第二种价值即可。

这时候就要注意，再开一维存放物品编号就不合适了，因为容易 MLE。

实现

```
for(int k = 1; k ⩽ n; k++) {
    for(int i = m; i ⩾ mi; i--) {     // 对经费进行一层枚举
        for(int j = t; j ⩾ ti; j--) { // 对时间进行一层枚举
            dp[i][j] = max(dp[i][j], dp[i - mi][j - ti] + 1);
        }
    }
}
```

19.4.8 分组背包

有 n 件物品和一个大小为 m 的背包，第 i 个物品的价值为 w_i，体积为 v_i。同时，每个物品属于一个组，同组内最多只能选择一个物品。求背包能装载物品的最大总价值。

这种题怎么想呢？其实是从「在所有物品中选择一件」变成了「从当前组中选择一件」，于是就对每一组进行一次 0-1 背包就可以了。

再说一说如何进行存储。我们可以将 $t_{k,i}$ 表示第 k 组的第 i 件物品的编号是多少，再用 cnt_k 表示第 k 组物品有多少个。

实现

```
for(int k = 1; k ⩽ ts; k++) {             // 循环每一组
    for(int i = m; i ⩾ 0; i--) {          // 循环背包容量
        for(int j = 1; j ⩽ cnt[k]; j++) { // 循环该组的每一个物品
            if(i ⩾ w[t[k][j]]) {          // 背包容量充足
                dp[i] = max(dp[i],
                            dp[i - w[t[k][j]]] + c[t[k][j]]); // 像 0-1 背包一样状态转移
```

```
            }
        }
    }
}
```

这里要注意：**一定不能搞错循环顺序**，这样才能保证正确性。

19.4.9 有依赖的背包

金明有 n 元钱，想要买 m 个物品，第 i 件物品的价格为 v_i，重要度为 p_i。有些物品是从属于某个主件物品的附件，要买这个物品，必须购买它的主件。

目标是让所有购买的物品的 $v_i \times p_i$ 之和最大。

考虑分类讨论。对于一个主件和它的若干附件，有以下几种可能：只买主件，买主件 + 某些附件。因为这几种可能性只能选一种，所以可以将这看成分组背包。

如果是多叉树的集合，则要先算子节点的集合，最后算父节点的集合。

19.4.10 泛化物品的背包

这种背包，没有固定的费用和价值，它的价值是随着分配给它的费用而定。在背包容量为 V 的背包问题中，当分配给它的费用为 v_i 时，能得到的价值就是 $h(v_i)$。这时，将固定的价值换成函数的引用即可。

19.4.11 杂项

小优化

根据贪心原理，当费用相同时，只需保留价值最高的；当价值一定时，只需保留费用最低的；当有两件物品 i, j 且 i 的价值大于 j 的价值并且 i 的费用小于 j 的费用时，只需保留 i。

背包问题变种

输出方案 输出方案其实就是记录下来背包中的某一个状态是怎么推出来的。我们可以用 $g_{i,v}$ 表示第 i 件物品占用空间为 v 的时候是否选择了此物品。然后在转移时记录是选用了哪一种策略（选或不选）。输出时的伪代码：

```
int v = V; // 记录当前的存储空间

// 因为最后一件物品存储的是最终状态，所以从最后一件物品进行循环
for(从最后一件循环至第一件) {
    if(g[i][v]) {
        选了第 i 项物品;
        v -= 第 i 项物品的重量;
    } else {
        未选第 i 项物品;
    }
}
```

求方案数 对于给定的一个背包容量、物品费用、其他关系等的问题，求装到一定容量的方案总数。

这种问题就是把求最大值换成求和即可。

例如 0-1 背包问题的转移方程就变成了：

$$dp_i = \sum(dp_i, dp_{i-c_i})$$

初始条件：$dp_0 = 1$

因为当容量为 0 时也有一个方案，即什么都不装。

求最优方案总数 要求最优方案总数，我们要对 0-1 背包里的 dp 数组的定义稍作修改，DP 状态 $f_{i,j}$ 为在只能放前 i 个物品的情况下，容量为 j 的背包「正好装满」所能达到的最大总价值。

这样修改之后，每一种 DP 状态都可以用一个 $g_{i,j}$ 来表示方案数。

$f_{i,j}$ 表示只考虑前 i 个物品时背包体积「正好」是 j 时的最大价值。

$g_{i,j}$ 表示只考虑前 i 个物品时背包体积「正好」是 j 时的方案数。

转移方程：

如果 $f_{i,j} = f_{i-1,j}$ 且 $f_{i,j} \neq f_{i-1,j-v} + w$ 说明我们此时不选择把物品放入背包更优，方案数由 $g_{i-1,j}$ 转移过来，

如果 $f_{i,j} \neq f_{i-1,j}$ 且 $f_{i,j} = f_{i-1,j-v} + w$ 说明我们此时选择把物品放入背包更优，方案数由 $g_{i-1,j-v}$ 转移过来，

如果 $f_{i,j} = f_{i-1,j}$ 且 $f_{i,j} = f_{i-1,j-v} + w$ 说明放入或不放入都能取得最优解，方案数由 $g_{i-1,j}$ 和 $g_{i-1,j-v}$ 转移过来。

初始条件：

```
memset(f, -0x3f, sizeof(f)); // 最大值 DP 的不可达哨兵，避免未装满状态参与转移
f[0] = 0;
g[0] = 1; // 什么都不装是一种方案
```

因为背包体积最大值有可能装不满，所以最优解不一定是 f_m。

最后我们通过找到最优解的价值，把 g_j 数组里取到最优解的所有方案数相加即可。

```
for(int i = 0; i < N; i++) {
    for(int j = V; j >= v[i]; j--) {
        int tmp = max(dp[j], dp[j - v[i]] + w[i]);
        int c = 0;
        if(tmp == dp[j]) {
            c += cnt[j]; // 如果从 dp[j] 转移
        }
        if(tmp == dp[j - v[i]] + w[i]) {
            c += cnt[j - v[i]]; // 如果从 dp[j-v[i]] 转移
        }
        dp[j] = tmp;
        cnt[j] = c;
    }
}
int ans = 0; // 寻找最优解
for(int i = 0; i <= V; i++) {
    ans = max(ans, dp[i]);
}
int res = 0;
for(int i = 0; i <= V; i++) {
    if(dp[i] == ans) {
        res += cnt[i]; // 求和最优解方案数
    }
}
```

背包的第 k 优解 普通的 0-1 背包是要求最优解，在普通的背包 DP 方法上稍作改动，增加一维用于记录当前状态下的前 k 优解，即可得到求 0-1 背包第 k 优解的算法。具体来讲：$dp_{i,j,k}$ 记录了前 i 个物品中，选择的物品总体积为 j 时，能够得到的第 k 大的价值和。这个状态可以理解为将普通 0-1 背包只用记录一个数据的 $dp_{i,j}$ 扩展为记录一个有序的优解序列。转移时，普通背包最优解的求法是 $dp_{i,j} = \max(dp_{i-1,j}, dp_{i-1,j-v_i} + w_i)$，现在我们则是要合并 $dp_{i-1,j}$，$dp_{i-1,j-v_i} + w_i$ 这两个大小为 k 的递减序列，并保留合并后前 k 大的价值记在 $dp_{i,j}$ 里，这一步利用双指针法，复杂度是 $O(k)$ 的，整体时间复杂度为 $O(nmk)$。空间上，此方法与普通背包一样可以压缩掉第一维，复杂度是 $O(mk)$ 的。

```
memset(dp, 0, sizeof(dp));
int i, j, p, x, y, z;
cin >> n >> m >> K;
for(i = 0; i < n; i++) {
    cin >> w[i];
}
for(i = 0; i < n; i++) {
```

```
    cin >> c[i];
}
for(i = 0; i < n; i++) {
    for(j = m; j >= c[i]; j--) {
        for(p = 1; p <= K; p++) {
            a[p] = dp[j - c[i]][p] + w[i];
            b[p] = dp[j][p];
        }
        a[p] = b[p] = -1;
        x = y = z = 1;
        while(z <= K && (a[x] != -1 || b[y] != -1)) {
            if(a[x] > b[y]) {
                dp[j][z] = a[x++];
            } else {
                dp[j][z] = b[y++];
            }
            if(dp[j][z] != dp[j][z - 1]) {
                z++;
            }
        }
    }
}
cout << (dp[m][K]) << '\n';
```

19.4.12 P1417 烹调方案

```
#define N 64
int64 t, n;

class Item {
public:
    int64 a, b, c;
};

bool operator<(const Item& A, const Item& B) {
    return A.c * B.b < A.b * B.c;
}

Item item[N];

int64 dp[100005];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> t >> n;
    for(int64 i = 0; i < n; i++) {
        cin >> item[i].a;
    }

    for(int64 i = 0; i < n; i++) {
        cin >> item[i].b;
    }

    for(int64 i = 0; i < n; i++) {
        cin >> item[i].c;
    }

    sort(item, item + n);

    for(int64 i = 0; i < n; i++) {
        for(int64 j = t; j >= 0; j--) {
```

```
            if(j - item[i].c ⩾ 0) {
                dp[j] = max(dp[j - item[i].c] + item[i].a - item[i].b * j, dp[j]);
            }
        }
    }

    int64 maximum = 0;

    for(int64 i = 0; i ⩽ t; i++) {
        maximum = max(maximum, dp[i]);
    }
    cout << (maximum);
}
```

注意

0/1 背包容量倒序；「恰好装满」的最大值 DP 用负哨兵（`memset(f, -0x3f, ...)`），不要用正无穷。

19.5 SOSDP

时间复杂度 $O(n \cdot 2^{\omega(x)})$

1003

首先，奇数长度的字符串一定优秀，若所有长度为 $n(n \geq 2)$ 且 n 是偶数的字符串优秀，则所有长度为 $m(m \geq n)$ 且 m 是偶数的字符串优秀。所以我们只需要检查一个 len 即可，若 len 是奇数，则加 1 变成偶数。

以 17 位二进制表示一种方案，初始所有方案都合法，若某位为 1，则表示此方案不对该字母幻视。

对所有长度为 len 的串进行判断，若其不优秀，我们就会得到一个非法方案，比如对于 $abcd$，非法方案就是同时不选 a, b。

对于一个 17 位二进制，如果它有子集是不合法的，则它也必定是不合法的。因为如果不选 a, b 不合法，不选 a, b, c 当然也不合法。

通过 $SOSdp$ 处理出所有合法情况，取最小即可。

时间复杂度：$\mathcal{O}(T * 17 * 2^{17})$

```
const int N = 100005;
int n, x;

inline int gcd(int a, int b) {
    if(b == 0) {
        return a;
    } else {
        return gcd(b, a % b);
    }
}

inline long long lcm(int a, int b) {
    return 1LL * a / gcd(a, b) * b;
}

class Node {
public:
    vector<int> oe;
    int b;
    int f[1 << 8];
};

Node node[N];

```

```
int p[8];
int k[8];
int pcnt;
long long ans;

void dfs(int cur, int fa) {
    if(node[cur].b != -1) {
        node[cur].f[node[cur].b] = 1;
        if(node[cur].b == (1 << pcnt) - 1) {
            ans++;
        }

        for(int v : node[cur].oe) {
            if(v != fa) {
                int s[1 << pcnt];
                for(int i = 0; i < 1 << pcnt; i++) {
                    s[i] = node[cur].f[i];
                }

                for(int j = 0; j < pcnt; j++) {
                    for(int i = 0; i < 1 << pcnt; i++) {
                        if(!(i & (1 << j))) {
                            s[i] += s[i ^ (1 << j)];
                        }
                    }
                }

                dfs(v, cur);
                if(node[v].b != -1) {
                    for(int i = 0; i < 1 << pcnt; i++) {
                        ans += 1ll * s[((1 << pcnt) - 1) ^ (i | node[cur].b)] * node[v].f[i];
                        node[cur].f[i | node[cur].b] += node[v].f[i];
                    }
                }
            }
        }
    } else {
        for(int v : node[cur].oe) {
            if(v != fa) {
                dfs(v, cur);
            }
        }
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int T;
    cin >> T;
    while(T--) {
        cin >> n >> x;
        pcnt = 0;
        ans = 0;
        for(int i = 0; i < 8; i++) {
            k[i] = p[i] = 0;
        }
        int a = x;
        for(int i = 2; 1LL * i * i <= a; i++) {
            if(a % i == 0) {
                p[pcnt] = i;
```

```
                while(a % i == 0) {
                    a /= i;
                    k[pcnt]++;
                }
                pcnt++;
            }
        }

        if(a > 1) {
            p[pcnt] = a;
            k[pcnt] = 1;
            pcnt++;
        }

        for(int i = 1; i <= n; i++) {
            node[i].oe.clear();
            node[i].b = 0;
            for(int j = 0; j < 1 << pcnt; j++) {
                node[i].f[j] = 0;
            }
        }

        int u, v;
        for(int i = 1; i < n; i++) {
            cin >> u >> v;
            node[u].oe.push_back(v);
            node[v].oe.push_back(u);
        }

        for(int i = 1; i <= n; i++) {
            cin >> a;
            if(lcm(a, x) != x) {
                node[i].b = -1;
            } else {
                for(int j = 0; j < pcnt; j++) {
                    int cnt = 0;
                    while(a % p[j] == 0) {
                        a /= p[j];
                        cnt++;
                    }
                    if(cnt == k[j]) {
                        node[i].b |= 1 << j;
                    }
                }
            }
        }

        dfs(1, 0);
        cout << (ans) << '\n';
    }
}
```

1003

首先，奇数长度的字符串一定优秀，若所有长度为 $n(n \geq 2)$ 且 n 是偶数的字符串优秀，则所有长度为 $m(m \geq n)$ 且 m 是偶数的字符串优秀。所以我们只需要检查一个 len 即可，若 len 是奇数，则加 1 变成偶数。

以 17 位二进制表示一种方案，初始所有方案都合法，若某位为 1，则表示此方案不对该字母幻视。

对所有长度为 len 的串进行判断，若其不优秀，我们就会得到一个非法方案，比如对于 $abcd$，非法方案就是同时不选 a, b。

对于一个 17 位二进制，如果它有子集是不合法的，则它也必定是不合法的。因为如果不选 a, b 不合法，不选 a, b, c 当然也不合法。

通过 $SOSdp$ 处理出所有合法情况，取最小即可。

时间复杂度：$\mathcal{O}(T * 17 * 2^{17})$

```
const int N = 3005;
const int K = 17;
int n;
int l;

char s[N];
int w[K];
bool legal[1 << K];
int cost(int state) {
    int tot = 0;
    for(int i = 0; i < K; i++) {
        if(!(state & (1 << i))) {
            tot += w[i];
        }
    }
    return tot;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int T;
    cin >> T;
    while(T--) {
        cin >> n;
        cin >> (s);
        for(int i = 0; i < K; i++) {
            cin >> w[i];
        }
        cin >> l;

        if(l & 1) {
            l++;
        }

        if(l > n) {
            cout << (0) << '\n';
        } else {
            memset(legal, 1, sizeof(legal));
            for(int i = 0; i + l <= n; i++) {
                int state = 0;
                for(int j = 0; j < l / 2; j++) {
                    if(s[i + j] == s[i + l - 1 - j]) {
                        state = 0;
                        break;
                    } else {
                        char mini = min(s[i + j], s[i + l - 1 - j]);
```

```
                        state |= 1 << (mini - 'a');
                    }
                }
                if(state) {
                    legal[state] = false;
                }
            }

            for(int j = 0; j < K; j++) {
                for(int i = 0; i < 1 << K; i++) {
                    if(i & (1 << j)) {
                        legal[i] &= legal[i ^ (1 << j)];
                    }
                }
            }

            int ans = 0x3f3f3f3f;
            for(int i = 0; i < 1 << K; i++) {
                if(legal[i]) {
                    ans = min(ans, cost(i));
                }
            }

            cout << (ans) << '\n';
        }
    }
}
```

注意

$x \leq 1e7$ 最多 8 个不同质因子；`lcm` 用 `1LL * a / gcd * b`，分解到 `i*i <= a`。

1007 树上LCM

OCS-4.9.45

Problem Description

给你一棵由 n 个节点的树和一个数 x，其中每个节点都有一个值。有多少条简单路径的值的 lcm 为 x？

一条简单路径的 lcm 的定义为路径上所有节点的值的 lcm。

Input

第一行输入一个整数 T ($1 \leq T \leq 200$)，表示测试的总数。

对于每个测试样例， 第一行输入两个数 n ($1 \leq n \leq 10^5$)， x ($2 \leq x \leq 10^7$)，表示节点的个数和目标值 x。

接下来 $n-1$ 行，每行两个数 u 和 v，表示节点 u 和 v 之间存在一条边。

接下来一行 n 个数 $a_1, a_2, \cdots, a_n$ ($1 \leq a_i \leq 10^9$)，每个节点的值。

保证样例中 n 的总和不超过 3×10^5。

Output

对于每个样例，输出一个数，满足条件的路径的数量。

Sample Input

```
2
3 2
1 2
2 3
2 2 2
```

```
6 6
1 2
1 3
2 4
2 5
3 6
6 1 4 2 3 5
```

OCS-4.9.45

Sample Output

```
6
5
```

Hint

对于第一个样例，任何路径都满足条件。因此答案为 $3 \times 2 = 6$。

对于第二个样例，满足条件的路径为 `[1, 1]`、`[1, 2]`、`[1, 4]`、`[1, 5]`、`[4, 5]`。因此答案为 5。

19.6 DP 优化

时间复杂度 $O(n \log n)$ / $O(nm)$

19.6.1 P3216 [HNOI2011] 数学作业

```
const uint64 N = 3;
uint64 n, m;

class Matrix {
public:
    uint64 arr[N][N];

    Matrix operator*(const Matrix& o) {
        Matrix res = {0};

        for(uint64 i = 0; i < N; i++) {
            for(uint64 j = 0; j < N; j++) {
                for(uint64 k = 0; k < N; k++) {
                    res.arr[i][j] += arr[i][k] * o.arr[k][j];
                }
                res.arr[i][j] %= m;
            }
        }

        return res;
    }
};
Matrix A, B;
Matrix E = {
    1, 0, 0,
    0, 1, 0,
    0, 0, 1};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m;

    A.arr[0][0] = 0;
    A.arr[1][0] = 1;
    A.arr[2][0] = 1;
    B.arr[0][1] = 1;
    B.arr[1][1] = 1;
    B.arr[1][2] = 1;
    B.arr[2][2] = 1;

    for(uint64 i = 1; i <= n; i *= 10) {
        B.arr[0][0] = i * 10 % m;
        A = quickPow(B, min(n + 1, i * 10) - i, E) * A;
    }

    cout << (A.arr[0][0]) << '\n';
}
```

19.6.2 P2513 [HAOI2009] 逆序对数列

```
const int MODER = 10000;
const int N = 1024;
int dp[N][N];

int main() {
```

```
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, k;
    cin >> n >> k;

    dp[0][0] = 1;
    for(int i = 0; i < n; i++) {

        for(int j = 1; j ⩽ k; j++) {
            dp[i][j] += dp[i][j - 1];
            dp[i][j] %= MODER;
        }

        for(int j = 0; j ⩽ k; j++) {
            int l = j - min(i, j) - 1;
            if(l < 0) {
                dp[i + 1][j] = dp[i][j];
            } else {
                dp[i + 1][j] = (dp[i][j] - dp[i][l] + MODER) % MODER;
            }
        }
    }
    cout << (dp[n][k] % MODER) << '\n';
}
```

注意

矩阵快速幂按位数分组；斜率优化注意单调性前提。

第 VI 编

杂项与工具

第 20 章

其他算法

20.1 莫队

时间复杂度 $O((n+m)\sqrt{n})$

20.1.1 P1494 [国家集训队] 小 Z 的袜子

```
const int N = 50005;
int S = 250;

long long gcd(long long A, long long B) {
    return B == 0 ? A : gcd(B, A % B);
}

class Query {
public:
    int l, r;
    long long ans;
    bool operator<(const Query& o) const {
        if(l / S != o.l / S) {
            return l / S < o.l / S;
        } else {
            return (l / S) & 1 ? r < o.r : r > o.r;
            ;
        }
    }
};

Query q[N];
int ptr[N];

bool cmp(const int A, const int B) {
    return q[A] < q[B];
}

int a[N];
int cnt[N];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n;
    cin >> m;
    S = max(1, int(n / sqrt(max(1, m))));
    for(int i = 1; i <= n; i++) {
        cin >> a[i];
    }
    for(int i = 1; i <= m; i++) {
        cin >> q[i].l;
        cin >> q[i].r;
        ptr[i] = i;
    }
    sort(ptr + 1, ptr + 1 + m, cmp);

    int l = 1, r = 1;
```

```
    long long ans = 0;
    cnt[a[1]]++;
    for(int i = 1; i <= m; i++) {
        Query& query = q[ptr[i]];
        while(l > query.l) {
            ans += cnt[a[--l]]++;
        }
        while(r < query.r) {
            ans += cnt[a[++r]]++;
        }
        while(l < query.l) {
            ans -= --cnt[a[l++]];
        }
        while(r > query.r) {
            ans -= --cnt[a[r--]];
        }
        query.ans = ans;
    }

    for(int i = 1; i <= m; i++) {
        if(q[i].ans == 0) {
            cout << "0/1\n";
        } else {
            long long tot = (q[i].r - q[i].l + 1ll) * (q[i].r - q[i].l) / 2;
            long long g = gcd(q[i].ans, tot);
            cout << (q[i].ans / g) << "/" << (tot / g) << '\n';
        }
    }
}
```

20.1.2 P1903 [国家集训队] 数颜色 / 维护队列 /【模板】带修莫队

```
const int N = 140005;
const int C = 1000005;
int S = 400;

class Query {
public:
    int l, r, t;
    long long ans;
    bool operator<(const Query& o) const {
        if(l / S != o.l / S) {
            return l / S < o.l / S;
        } else if(r / S != o.r / S) {
            return r / S < o.r / S;
        } else {
            return (r / S) & 1 ? t < o.t : t > o.t;
            ;
        }
    }
};

Query q[N];
int qcnt;
int ptr[N];
int mod[N][3];

bool cmp(const int A, const int B) {
    return q[A] < q[B];
}

int a[N];
```

```
int cnt[C];

int main() {
    int n, m;
    cin >> n >> m;
    S = pow(n, 0.66);
    for(int i = 1; i ⩽ n; i++) {
        cin >> a[i];
    }
    int t = 0;
    int l = 1, r = 1;
    long long ans = 0;
    char op;
    for(int i = 1; i ⩽ m; i++) {
        cin >> op >> l >> r;
        if(op == 'Q') {
            q[++qcnt] = {l, r, t};
            ptr[qcnt] = qcnt;
        } else {
            mod[t][0] = l;
            mod[t][1] = a[l];
            mod[t][2] = r;
            a[l] = r;
            ++t;
        }
    }
    sort(ptr + 1, ptr + 1 + qcnt, cmp);

    l = r = 1;
    cnt[a[1]]++;
    ans = 1;

    for(int i = 1; i ⩽ qcnt; i++) {
        Query& query = q[ptr[i]];
        while(l > query.l) {
            --l;
            if(cnt[a[l]] == 0) {
                ans++;
            }
            cnt[a[l]]++;
        }
        while(r < query.r) {
            ++r;
            if(cnt[a[r]] == 0) {
                ans++;
            }
            cnt[a[r]]++;
        }
        while(l < query.l) {
            cnt[a[l]]--;
            if(cnt[a[l]] == 0) {
                ans--;
            }
            l++;
        }
        while(r > query.r) {
            cnt[a[r]]--;
            if(cnt[a[r]] == 0) {
                ans--;
            }
            r--;
```

```
        }

        while(t < query.t) {
            if(mod[t][0] ⩾ l && mod[t][0] ⩽ r) {
                cnt[mod[t][1]]--;
                if(cnt[mod[t][1]] == 0) {
                    ans--;
                }
                if(cnt[mod[t][2]] == 0) {
                    ans++;
                }
                cnt[mod[t][2]]++;
            }
            a[mod[t][0]] = mod[t][2];
            ++t;
        }
        while(t > query.t) {
            --t;
            if(mod[t][0] ⩾ l && mod[t][0] ⩽ r) {
                cnt[mod[t][2]]--;
                if(cnt[mod[t][2]] == 0) {
                    ans--;
                }
                if(cnt[mod[t][1]] == 0) {
                    ans++;
                }
                cnt[mod[t][1]]++;
            }
            a[mod[t][0]] = mod[t][1];
        }
        query.ans = ans;
    }

    for(int i = 1; i ⩽ qcnt; i++) {
        cout << (q[i].ans) << '\n';
    }
}
```

注意

块长取 `max(1, n / sqrt(m))`，防止 m 很大时除零；`ptr` 要初始化到 m。

20.2 环计数

时间复杂度 $O(m\sqrt{m})$

20.2.1 P1989 无向图三元环计数

```
const int N = 100005;

class Node {
public:
    vector<int> oe;
    int degree;
    bool visited;
};

Node node[N];

int n, m;

int main() {
```

```
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m;
    set<int> nidx;
    vector<pair<int, int>> edge;
    for(int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        edge.push_back({u, v});
        node[u].degree++;
        node[v].degree++;
        nidx.insert(u);
        nidx.insert(v);
    }

    for(pair<int, int> e : edge) {
        if(node[e.first].degree > node[e.second].degree || (node[e.first].degree == node[e.second].degree && e.first
        ↪  > e.second)) {
            node[e.first].oe.push_back(e.second);
        } else {
            node[e.second].oe.push_back(e.first);
        }
    }

    int ans = 0;

    for(int u : nidx) {
        for(int v : node[u].oe) {
            node[v].visited = true;
        }
        for(int v : node[u].oe) {
            for(int w : node[v].oe) {
                if(node[w].visited) {
                    ans++;
                }
            }
        }
        for(int v : node[u].oe) {
            node[v].visited = false;
        }
    }

    cout << (ans) << '\n';
}
```

20.2.2 题意精炼

给定一个无向图，有 n 个节点（城镇）和 m 条边（铁路连接）。每条边的权重为 a。此外，对于图中任意两个节点 u 和 v，如果它们没有直接边相连，但存在一个公共邻居 w（即存在边 (u,w) 和 (w,v)），则添加一条边 (u,v)，权重为 b。

现在，给定源节点 k，要求计算从 k 到每个节点的最短路径成本。注意，从 k 到自身的成本为 0。

```
const int N = 100005;

int n, m, k, a, b;

class Node {
public:
    vector<int> sub;
    vector<int> leap;
    int dis;
```

```
    int even_dis;
    bool visited;
    bool flag;
};

Node node[N];

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m >> k >> a >> b;
    for(int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        node[u].sub.push_back(v);
        node[v].sub.push_back(u);
    }

    for(int i = 0; i ⩽ n; i++) {
        node[i].leap = vector<int>(node[i].sub);
        node[i].dis = 0x3f3f3f3f;
        node[i].even_dis = 0x3f3f3f3f;
    }

    node[k].dis = 0;
    node[k].visited = true;
    queue<int> q;
    q.push(k);
    int u;
    while(!q.empty()) {
        u = q.front();
        q.pop();

        for(int v : node[u].sub) {
            if(!node[v].visited) {
                node[v].dis = node[u].dis + 1;
                node[v].visited = true;

                q.push(v);
            }
        }
    }

    if(b < 2 * a) {
        node[k].even_dis = 0;
        q.push(k);
        node[k].visited = false;
        while(!q.empty()) {
            u = q.front();
            q.pop();
            for(int v : node[u].leap) {
                node[v].flag = true;
            }
            for(int v : node[u].sub) {
                vector<int> remained;
                remained.reserve(node[v].leap.size());

                for(int w : node[v].leap) {
                    if(node[w].visited && !node[w].flag) {
                        node[w].even_dis = node[u].even_dis + 2;
                        node[w].visited = false;
                        q.push(w);
```

```
                    } else {
                        remained.push_back(w);
                    }
                }
                node[v].leap = remained;
            }
            for(int v : node[u].leap) {
                node[v].flag = false;
            }
        }

        for(int i = 1; i ⩽ n; i++) {
            cout << (min((node[i].dis % 2) * a + node[i].dis / 2 * b, node[i].even_dis != 0x3f3f3f3f ?
            ↪  node[i].even_dis / 2 * b : 0x3f3f3f3f)) << '\n';
        }
    } else {
        for(int i = 1; i ⩽ n; i++) {
            cout << (node[i].dis * a) << '\n';
        }
    }
}
```

注意

按度数定向；不可达判 `=/= INF`，不要用 `<= n`。

20.3 分数规划

时间复杂度 $O(n \log n)$ 每次判定

分数规划用来求一个分式的极值。

形象一点就是，给出 a_i 和 b_i，求一组 $w_i \in \{0,1\}$，最小化或最大化

$$\frac{\sum_{i=1}^{n} a_i \times w_i}{\sum_{i=1}^{n} b_i \times w_i}$$

另外一种描述：每种物品有两个权值 a 和 b，选出若干个物品使得 $\frac{\sum a}{\sum b}$ 最小/最大。

一般分数规划问题还会有一些奇怪的限制，比如『分母至少为 W』。

20.3.1 求解

二分法

分数规划问题的通用方法是二分。

假设我们要求最大值。二分一个答案 mid，然后推式子（为了方便少写了上下界）：

$$\begin{aligned} &\frac{\sum a_i \times w_i}{\sum b_i \times w_i} > mid \\ \Longrightarrow &\sum a_i \times w_i - mid \times \sum b_i \cdot w_i > 0 \\ \Longrightarrow &\sum w_i \times (a_i - mid \times b_i) > 0 \end{aligned}$$

那么只要求出不等号左边的式子的最大值就行了。如果最大值比 0 要大，说明 mid 是可行的，否则不可行。

20.3.2 P4377 [USACO18OPEN] Talent Show G

```
const double eta = 1e-8;
int n, W;
double p[256];
int w[256];
```

```
double dp[1024];
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> W;
    for(int i = 1; i ⩽ n; i++) {
        cin >> w[i] >> p[i];
    }

    double l = 0, r = 1e7, mid = 0.0, sum_w, ans = 0.0;
    while(r - l > eta) {
        mid = (l + r) / 2;
        for(int k = 0; k ⩽ W; k++) {
            dp[k] = -1e32;
        }
        dp[0] = 0;

        bool flag = false;

        for(int i = 1; i ⩽ n; i++) {
            for(int j = W + w[i]; j ⩾ w[i]; j--) {
                int k = min(j, W);
                dp[k] = max(dp[k], dp[j - w[i]] + p[i] - mid * w[i]);
                if(dp[W] > eta) {
                    flag = true;
                    break;
                }
            }
            if(flag) {
                break;
            }
        }
        if(flag) {
            ans = mid;
            l = mid;
        } else {
            r = mid;
        }
    }

    cout << (int((ans + eta * 10) * 1000));
}
```

注意

二分答案后转成判定性问题。

20.4 梯度下降

时间复杂度 迭代次数 $\times O(n)$

20.4.1 P1337 [JSOI2004] 平衡点 / 吊打 XXX

```
const double ETA = 1e-4;
const int N = 1024;
double lr = 1e-3;
const double alpha = 0.9;
class Vec {
public:
    double x, y;
    Vec operator+(const Vec& o) const {
        return {x + o.x, y + o.y};
```

```
    }

    Vec operator*(double a) const {
        return {x * a, y * a};
    }

    double len() const {
        return sqrt(x * x + y * y);
    }
};

int n;
int X[N], Y[N], W[N];

Vec getForce(double x, double y, int b) {
    double dx = X[b] - x;
    double dy = Y[b] - y;
    double l = sqrt(dx * dx + dy * dy);
    if(l > ETA) {
        return {W[b] * dx / l, W[b] * dy / l};
    } else {
        return {0, 0};
    }
}

Vec totForce(const Vec& p) {
    Vec f = {0, 0};
    for(int i = 1; i ⩽ n; i++) {
        f = f + getForce(p.x, p.y, i);
    }
    return f;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n;

    double sum = 0, sx = 0, sy = 0;
    for(int i = 1; i ⩽ n; i++) {
        cin >> X[i] >> Y[i] >> W[i];
        sum += W[i];
        sx += X[i] * W[i];
        sy += Y[i] * W[i];
    }

    int cnt = 0;
    Vec p = {sx / sum, sy / sum};
    Vec f = {0, 0};
    while(++cnt) {
        f = f * alpha + totForce(p) * (1 - alpha); //动量梯度下降
        if(f.len() < ETA) {
            break;
        } else {
            p = p + f * lr;
        }
    }

    cout << fixed << setprecision(3) << (p.x) << ' ' << fixed << setprecision(3) << (p.y) << '\n';
}
```

注意

步长与迭代上限要写死，否则可能在合法数据上不收敛。

20.5 高精度

时间复杂度 $O(n^2)$

```
struct BigInt {
    int sign;
    vector<int> v;

    BigInt() : sign(1) {
    }
    BigInt(const string& s) {
        *this = s;
    }
    BigInt(int v) {
        char buf[21];
        sprintf(buf, "%d", v);
        *this = buf;
    }
    /*
     * 转换内部进制表示
     */
    void zip(int unzip) {
        if(unzip == 0) {
            for(int i = 0; i < (int)v.size(); i++) {
                v[i] = get_pos(i * 4) + get_pos(i * 4 + 1) * 10 + get_pos(i * 4 + 2) * 100 + get_pos(i * 4 + 3) *
                ↪ 1000;
            }
        } else {
            for(int i = (v.resize(v.size() * 4), (int)v.size() - 1), a; i >= 0; i--) {
                a = (i % 4 >= 2) ? v[i / 4] / 100 : v[i / 4] % 100;
                v[i] = (i & 1) ? a / 10 : a % 10;
            }
        }
        setsign(1, 1);
    }
    int get_pos(unsigned pos) const {
        return pos >= v.size() ? 0 : v[pos];
    }
    BigInt& setsign(int newsign, int rev) {
        for(int i = (int)v.size() - 1; i > 0 && v[i] == 0; i--) {
            v.erase(v.begin() + i);
        }
        sign = (v.size() == 0 || (v.size() == 1 && v[0] == 0)) ? 1 : (rev ? newsign * sign : newsign);
        return *this;
    }
    /*
     * 转换为十进制字符串，保留符号
     */
    string to_str() const {
        BigInt b = *this;
        string s;

        for(int i = (b.zip(1), 0); i < (int)b.v.size(); ++i) {
            s += char(*(b.v.rbegin() + i) + '0');
        }
        return (sign < 0 ? "-" : "") + (s.empty() ? string("0") : s);
    }
    bool absless(const BigInt& b) const {
```

```
        if(v.size() != b.v.size()) {
            return v.size() < b.v.size();
        }
        for(int i = (int)v.size() - 1; i >= 0; i--) {
            if(v[i] != b.v[i]) {
                return v[i] < b.v[i];
            }
        }
        return false;
    }
    BigInt operator-() const {
        BigInt c = *this;
        c.sign = (v.size() > 1 || v[0]) ? -c.sign : 1;
        return c;
    }
    BigInt& operator=(const string& s) {
        if(s[0] == '-') {
            *this = s.substr(1);
        } else {
            for(int i = (v.clear(), 0); i < (int)s.size(); ++i) {
                v.push_back(*(s.rbegin() + i) - '0');
            }
            zip(0);
        }
        return setsign(s[0] == '-' ? -1 : 1, sign = 1);
    }
    bool operator<(const BigInt& b) const {
        return sign != b.sign ? sign < b.sign : (sign == 1 ? absless(b) : b.absless(*this));
    }
    bool operator==(const BigInt& b) const {
        return v == b.v && sign == b.sign;
    }
    BigInt& operator+=(const BigInt& b) {
        if(sign != b.sign) {
            return *this = (*this) - -b;
        }
        v.resize(max(v.size(), b.v.size()) + 1);
        for(int i = 0, carry = 0; i < (int)b.v.size() || carry; i++) {
            carry += v[i] + b.get_pos(i);
            v[i] = carry % 10000;
            carry /= 10000;
        }
        return setsign(sign, 0);
    }
    BigInt operator+(const BigInt& b) const {
        BigInt c = *this;
        return c += b;
    }
    /*
     * 将 b 乘 mul 后累加到当前数
     */
    void add_mul(const BigInt& b, int mul) {
        v.resize(max(v.size(), b.v.size()) + 2);
        for(int i = 0, carry = 0; i < (int)b.v.size() || carry; i++) {
            carry += v[i] + b.get_pos(i) * mul;
            v[i] = carry % 10000;
            carry /= 10000;
        }
    }
    BigInt operator-(const BigInt& b) const {
        if(b.v.empty() || b.v.size() == 1 && b.v[0] == 0) {
```

```
            return *this;
        }
        if(sign != b.sign) {
            return (*this) + -b;
        }
        if(absless(b)) {
            return -(b - *this);
        }
        BigInt c;
        for(int i = 0, borrow = 0; i < (int)v.size(); i++) {
            borrow += v[i] - b.get_pos(i);
            c.v.push_back(borrow);
            c.v.back() -= 10000 * (borrow >= 31);
        }
        return c.setsign(sign, 0);
    }
    BigInt operator*(const BigInt& b) const {
        if(b < *this) {
            return b * *this;
        }
        BigInt c, d = b;
        for(int i = 0; i < (int)v.size(); i++, d.v.insert(d.v.begin(), 0)) {
            c.add_mul(d, v[i]);
        }
        return c.setsign(sign * b.sign, 0);
    }
    BigInt operator/(const BigInt& b) const {
        BigInt c, d;
        BigInt e = b;
        e.sign = 1;

        d.v.resize(v.size());
        double db = 1.0 / (b.v.back() + (b.get_pos((unsigned)b.v.size() - 2) / 1e4) +
                           (b.get_pos((unsigned)b.v.size() - 3) + 1) / 1e8);
        for(int i = (int)v.size() - 1; i >= 0; i--) {
            c.v.insert(c.v.begin(), v[i]);
            int m = (int)((c.get_pos((int)e.v.size()) * 10000 + c.get_pos((int)e.v.size() - 1)) * db);
            c = c - e * m;
            c.setsign(c.sign, 0);
            d.v[i] += m;
            while(!(c < e)) {
                c = c - e;
                d.v[i] += 1;
            }
        }
        return d.setsign(sign * b.sign, 0);
    }
    BigInt operator%(const BigInt& b) const {
        return *this - *this / b * b;
    }
    bool operator>(const BigInt& b) const {
        return b < *this;
    }
    bool operator<=(const BigInt& b) const {
        return !(b < *this);
    }
    bool operator>=(const BigInt& b) const {
        return !(*this < b);
    }
    bool operator!=(const BigInt& b) const {
        return !(*this == b);
```

```
    }
};
```

注意

压位宽度与进位处理要一致。

20.6 加工生产调度（Johnson 规则）

时间复杂度 $O(n \log n)$

20.6.1 P1248 加工生产调度

```
struct Request {
    int a, b, idx;
};
typedef struct Request Request;
Request request[1005];
Request q[1005];
bool cmp(const Request& a, const Request& b) {
    return min(a.a, a.b) < min(b.a, b.b);
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    for(int i = 0; i < n; i++) {
        cin >> request[i].a;
    }

    for(int i = 0; i < n; i++) {
        cin >> request[i].b;
        request[i].idx = i + 1;
    }

    sort(request, request + n, cmp);
    int h = 0, e = n - 1;
    for(int i = 0; i < n; i++) {
        if(request[i].a < request[i].b) {
            q[h] = request[i];
            h++;
        } else {
            q[e] = request[i];
            e--;
        }
    }
    int ta = 0, tb = 0;
    for(int i = 0; i < n; i++) {
        ta += q[i].a;
        tb = max(tb + q[i].b, ta + q[i].b);
    }
    cout << (tb) << '\n';

    for(int i = 0; i < n; i++) {
        cout << (q[i].idx) << ' ';
    }
}
```

注意

两台机器的作业排序：a < b 的用头指针从前往后填，否则用尾指针从后往前填。这是排序规则，与全源最短路的 Johnson 无关。

第 21 章

工具

21.1 STL

时间复杂度 —

序列式容器

- **向量** (vector) 后端可高效增加元素的顺序表。
- **数组** (array)C++11，定长的顺序表，C 风格数组的简单包装。
- **双端队列** (deque) 双端都可高效增加元素的顺序表。
- **列表** (list) 可以沿双向遍历的链表。
- **单向列表** (forward_list) 只能沿一个方向遍历的链表。

关联式容器

- **集合** (set) 用以有序地存储 **互异**元素的容器。其实现是由节点组成的红黑树，每个节点都包含着一个元素，节点之间以某种比较元素大小的谓词进行排列。
- **多重集合** (multiset) 用以有序地存储元素的容器。允许存在相等的元素。
- **映射** (map) 由 {键，值} 对组成的集合，以某种比较键大小关系的谓词进行排列。
- **多重映射** (multimap) 由 {键，值} 对组成的多重集合，亦即允许键有相等情况的映射。

无序（关联式）容器

- **无序（多重）集合** (unordered_set/unordered_multiset)C++11，与 set/multiset 的区别在于元素无序，只关心「元素是否存在」，使用哈希实现。
- **无序（多重）映射** (unordered_map/unordered_multimap)C++11，与 map/multimap 的区别在于键 (key) 无序，只关心 “键与值的对应关系”，使用哈希实现。

容器适配器

容器适配器其实并不是容器。它们不具有容器的某些特点（如：有迭代器、有 clear() 函数……）。

「适配器是使一种事物的行为类似于另外一种事物行为的一种机制」，适配器对容器进行包装，使其表现出另外一种行为。

- **栈** (stack) 后进先出 (LIFO) 的容器，默认是对双端队列（deque）的包装。
- **队列** (queue) 先进先出 (FIFO) 的容器，默认是对双端队列（deque）的包装。
- **优先队列** (priority_queue) 元素的次序是由作用于所存储的值对上的某种谓词决定的的一种队列，默认是对向量（vector）的包装。

21.1.1 共同点

共有函数

=：有赋值运算符以及复制构造函数。
begin()：返回指向开头元素的迭代器。
end()：返回指向末尾的下一个元素的迭代器。end() 不指向某个元素，但它是末尾元素的后继。
size()：返回容器内的元素个数。
max_size()：返回容器 **理论上**能存储的最大元素个数。依容器类型和所存储变量的类型而变。

`empty()`：返回容器是否为空。

`swap()`：交换两个容器。

`clear()`：清空容器。

`==`/`!=`/`<`/`>`/`<=`/`>=`：按 **字典序**比较两个容器的大小。（比较元素大小时 `map` 的每个元素相当于 `set<pair<key, value>>`，无序容器不支持 `<`/`>`/`<=`/`>=`。）

在 STL 中，迭代器（Iterator）用来访问和检查 STL 容器中元素的对象，它的行为模式和指针类似，但是它封装了一些有效性检查，并且提供了统一的访问格式。类似的概念在其他很多高级语言中都存在，如 Python 的 `__iter__` 函数，C# 的 `IEnumerator`。

21.1.2 基础使用

迭代器听起来比较晦涩，其实迭代器本身可以看作一个数据指针。迭代器主要支持两个运算符：自增 (`++`) 和解引用（单目 `*` 运算符），其中自增用来移动迭代器，解引用可以获取或修改它指向的元素。

指向某个 STL 容器 `container` 中元素的迭代器的类型一般为 `container::iterator`。

迭代器可以用来遍历容器，例如，下面两个 for 循环的效果是一样的：

```
vector<int> data(10);

for (int i = 0; i < data.size(); i++)
  cout << data[i] << endl;  // 使用下标访问元素

for (vector<int>::iterator iter = data.begin(); iter != data.end(); iter++)
  cout << *iter << endl;  // 使用迭代器访问元素
// 在 C++11 后可以使用 auto iter = data.begin() 来简化上述代码
```

NOI 系列比赛（包括 CSP J/S）在评测时将使用 **C++14**，这个版本已经支持了 `auto` 关键字。

21.1.3 分类

在 STL 的定义中，迭代器根据其支持的操作依次分为以下几类：

- InputIterator（输入迭代器）：只要求支持拷贝、自增和解引访问。
- OutputIterator（输出迭代器）：只要求支持拷贝、自增和解引赋值。
- ForwardIterator（向前迭代器）：同时满足 InputIterator 和 OutputIterator 的要求。
- BidirectionalIterator（双向迭代器）：在 ForwardIterator 的基础上支持自减（即反向访问）。
- RandomAccessIterator（随机访问迭代器）：在 BidirectionalIterator 的基础上支持加减运算和比较运算（即随机访问）。
- ContiguousIterator（连续迭代器）：在 RandomAccessIterator 的基础上要求对可解引用的迭代器 `a + n` 满足 `*(a + n)` 与 `*(std::address_of(*a) + n)` 等价（即连续存储，其中 `a` 为连续迭代器、`n` 为整型值）。ContiguousIterator 于 C++17 中正式引入。

「输入」指的是「可以从迭代器中获取输入」，而「输出」指的是「可以输出到迭代器」。

「输入」和「输出」的施动者是程序的其它部分，而不是迭代器自身。

其实这个「分类」并不互斥——一个「类别」是可以包含另一个「类别」的。例如，在要求使用向前迭代器的地方，同样可以使用双向迭代器。

指针满足随机访问迭代器的所有要求，可以当作随机访问迭代器使用。

21.1.4 相关函数

很多 STL 函数 都使用迭代器作为参数。

可以使用 `std::advance(it, n)` 将迭代器 `it` 向后移动 `n` 步；若 `n` 为负数，则对应向前移动，此时迭代器必须满足双向迭代器，否则行为未定义。

在 C++11 以后可以使用 `std::next(it)` 获得向前迭代器 `it` 的后继（此时迭代器 `it` 不变），`std::next(it, n)` 获得向前迭代器 `it` 的第 `n` 个后继。

在 C++11 以后可以使用 `std::prev(it)` 获得双向迭代器 `it` 的前驱（此时迭代器 `it` 不变），`std::prev(it, n)` 获得双向迭代器 `it` 的第 `n` 个前驱。

STL 容器 一般支持从一端或两端开始的访问，以及对 const 修饰符 的支持。例如容器的 `begin()` 函数可以获得指向容器第一个元素的迭代器，`rbegin()` 函数可以获得指向容器最后一个元素的反向迭代器，`cbegin()` 函数可以获得指向容器第一个元素的 const 迭代器，`end()` 函数可以获得指向容器尾端（「尾端」并不是最后一个元素，可以看作是最后一个元素的后继；「尾端」的前驱是容器里的最后一个元素，其本身不指向任何一个元素）的迭代器。

21.1.5 vector

`std::vector` 是 STL 提供的 **内存连续的、可变长度**的数组（亦称列表）数据结构。能够提供线性复杂度的插入和删除，以及常数复杂度的随机访问。

为什么要使用 vector

作为 OIer，对程序效率的追求远比对工程级别的稳定性要高得多，而 `vector` 由于其对内存的动态处理，时间效率在部分情况下低于静态数组，并且在 OJ 服务器不一定开全优化的情况下更加糟糕。所以在正常存储数据的时候，通常不选择 `vector`。下面给出几个 `vector` 优秀的特性，在需要用到这些特性的情况下，`vector` 能给我们带来很大的帮助。

vector 可以动态分配内存 很多时候我们不能提前开好那么大的空间（eg：预处理 1~n 中所有数的约数）。尽管我们能知道数据总量在空间允许的级别，但是单份数据还可能非常大，这种时候我们就需要 `vector` 来把内存占用量控制在合适的范围内。`vector` 还支持动态扩容，在内存非常紧张的时候这个特性就能派上用场了。

vector 重写了比较运算符及赋值运算符 `vector` 重载了六个比较运算符，以字典序实现，这使得我们可以方便的判断两个容器是否相等（复杂度与容器大小成线性关系）。例如可以利用 `vector<char>` 实现字符串比较（当然，还是用 `std::string` 会更快更方便）。另外 `vector` 也重载了赋值运算符，使得数组拷贝更加方便。

vector 便利的初始化 由于 `vector` 重载了 `=` 运算符，所以我们可以方便的初始化。此外从 C++11 起 `vector` 还支持 列表初始化，例如 `vector<int> data {1, 2, 3};`。

vector 的使用方法

构造函数 用例参见如下代码（假设你已经 `using` 了 `std` 命名空间相关类型）：

```
// 1. 创建空 vector；常数复杂度
vector<int> v0;
// 1+. 这句代码可以使得向 vector 中插入前 3 个元素时，保证常数时间复杂度
v0.reserve(3);
// 2. 创建一个初始空间为 3 的 vector，其元素的默认值是 0；线性复杂度
vector<int> v1(3);
// 3. 创建一个初始空间为 3 的 vector，其元素的默认值是 2；线性复杂度
vector<int> v2(3, 2);
// 4. 创建一个初始空间为 3 的 vector，其元素的默认值是 1，
// 并且使用 v2 的空间配置器；线性复杂度
vector<int> v3(3, 1, v2.get_allocator());
// 5. 创建一个 v2 的拷贝 vector v4，其内容元素和 v2 一样；线性复杂度
vector<int> v4(v2);
// 6. 创建一个 v4 的拷贝 vector v5，其内容是 {v4[1], v4[2]}；线性复杂度
vector<int> v5(v4.begin() + 1, v4.begin() + 3);
// 7. 移动 v2 到新创建的 vector v6，不发生拷贝；常数复杂度；需要 C++11
vector<int> v6(std::move(v2));  // 或者 v6 = std::move(v2);
```

```
// 以下是测试代码，有兴趣的同学可以自己编译运行一下本代码。
cout << "v1 = ";
copy(v1.begin(), v1.end(), ostream_iterator<int>(cout, " "));
cout << endl;
cout << "v2 = ";
copy(v2.begin(), v2.end(), ostream_iterator<int>(cout, " "));
```

```
cout << endl;
cout << "v3 = ";
copy(v3.begin(), v3.end(), ostream_iterator<int>(cout, " "));
cout << endl;
cout << "v4 = ";
copy(v4.begin(), v4.end(), ostream_iterator<int>(cout, " "));
cout << endl;
cout << "v5 = ";
copy(v5.begin(), v5.end(), ostream_iterator<int>(cout, " "));
cout << endl;
cout << "v6 = ";
copy(v6.begin(), v6.end(), ostream_iterator<int>(cout, " "));
cout << endl;
```

可以利用上述的方法构造一个 `vector`，足够我们使用了。

元素访问 `vector` 提供了如下几种方法进行元素访问

1. `at()`
   `v.at(pos)` 返回容器中下标为 `pos` 的引用。如果数组越界抛出 `std::out_of_range` 类型的异常。
2. `operator[]`
   `v[pos]` 返回容器中下标为 `pos` 的引用。不执行越界检查。
3. `front()`
   `v.front()` 返回首元素的引用。
4. `back()`
   `v.back()` 返回末尾元素的引用。
5. `data()`
   `v.data()` 返回指向数组第一个元素的指针。

迭代器 vector 提供了如下几种 迭代器

1. `begin()/cbegin()`
   返回指向首元素的迭代器，其中 `*begin = front`。
2. `end()/cend()`
   返回指向数组尾端占位符的迭代器，注意是没有元素的。
3. `rbegin()/crbegin()`
   返回指向逆向数组的首元素的逆向迭代器，可以理解为正向容器的末元素。
4. `rend()/crend()`
   返回指向逆向数组末元素后一位置的迭代器，对应容器首的前一个位置，没有元素。

以上列出的迭代器中，含有字符 `c` 的为只读迭代器，你不能通过只读迭代器去修改 `vector` 中的元素的值。如果一个 `vector` 本身就是只读的，那么它的一般迭代器和只读迭代器完全等价。只读迭代器自 C++11 开始支持。

长度和容量 `vector` 有以下几个与容器长度和容量相关的函数。注意，`vector` 的长度（size）指有效元素数量，而容量（capacity）指其实际分配的内存长度，相关细节请参见后文的实现细节介绍。

与长度相关：

- `empty()` 返回一个 `bool` 值，即 `v.begin() == v.end()`，`true` 为空，`false` 为非空。
- `size()` 返回容器长度（元素数量），即 `std::distance(v.begin(), v.end())`。
- `resize()` 改变 `vector` 的长度，多退少补。补充元素可以由参数指定。
- `max_size()` 返回容器的最大可能长度。

与容量相关：

- `reserve()` 使得 `vector` 预留一定的内存空间，避免不必要的内存拷贝。

- `capacity()` 返回容器的容量，即不发生拷贝的情况下容器的长度上限。
- `shrink_to_fit()` 使得 `vector` 的容量与长度一致，多退但不会少。

元素增删及修改

- `clear()` 清除所有元素
- `insert()` 支持在某个迭代器位置插入元素、可以插入多个。**复杂度与 `pos` 距离末尾长度成线性而非常数的**
- `erase()` 删除某个迭代器或者区间的元素，返回最后被删除的迭代器。复杂度与 `insert` 一致。
- `push_back()` 在末尾插入一个元素，均摊复杂度为 **常数**，最坏为线性复杂度。
- `pop_back()` 删除末尾元素，常数复杂度。
- `swap()` 与另一个容器进行交换，此操作是 **常数复杂度**而非线性的。

21.1.6 `array`(C++11)

`std::array` 是 STL 提供的 **内存连续的、固定长度**的数组数据结构。其本质是对原生数组的直接封装。

为什么要用 `array`

`array` 实际上是 STL 对数组的封装。它相比 `vector` 牺牲了动态扩容的特性，但是换来了与原生数组几乎一致的性能（在开满优化的前提下）。因此如果能使用 C++11 特性的情况下，能够使用原生数组的地方几乎都可以直接把定长数组都换成 `array`，而动态分配的数组可以替换为 `vector`。

成员函数

隐式定义的成员函数

函数	作用
`operator=`	以来自另一 `array` 的每个元素重写 `array` 的对应元素

元素访问

函数	作用
`at`	访问指定的元素，同时进行越界检查
`operator[]`	访问指定的元素，不进行越界检查
`front`	访问第一个元素
`back`	访问最后一个元素
`data`	返回指向内存中数组第一个元素的指针

`at` 若遇 `pos >= size()` 的情况会抛出 `std::out_of_range`。

容量

函数	作用
`empty`	检查容器是否为空
`size`	返回容纳的元素数
`max_size`	返回可容纳的最大元素数

由于每个 `array` 都是固定大小容器，`size()` 返回的值等于 `max_size()` 返回的值。

操作

函数	作用
`fill`	以指定值填充容器
`swap`	交换内容

注意，交换两个 array 是 $\Theta(\text{size})$ 的，而非与常规 STL 容器一样为 $O(1)$。

非成员函数

函数	作用
`operator==` 等	按照字典序比较 array 中的值
`std::get`	访问 array 的一个元素
`std::swap`	特化的 `std::swap` 算法

下面是一个 array 的使用示例：

```
// 1. 创建空 array，长度为 3; 常数复杂度
std::array<int, 3> v0;
// 2. 用指定常数创建 array; 常数复杂度
std::array<int, 3> v1{1, 2, 3};

v0.fill(1);  // 填充数组

// 访问数组
for (int i = 0; i != v0.size(); ++i) cout << v0[i] << " ";
```

21.1.7 deque

`std::deque` 是 STL 提供的 双端队列 数据结构。能够提供线性复杂度的插入和删除，以及常数复杂度的随机访问。

构造函数 参见如下代码（假设你已经 `using` 了 `std` 命名空间相关类型）：

```
// 1. 定义一个 int 类型的空双端队列 v0
deque<int> v0;
// 2. 定义一个 int 类型的双端队列 v1，并设置初始大小为 10; 线性复杂度
deque<int> v1(10);
// 3. 定义一个 int 类型的双端队列 v2，并初始化为 10 个 1; 线性复杂度
deque<int> v2(10, 1);
// 4. 复制已有的双端队列 v1; 线性复杂度
deque<int> v3(v1);
// 5. 创建一个 v2 的拷贝 deque v4，其内容是 v4[0] 至 v4[2]; 线性复杂度
deque<int> v4(v2.begin(), v2.begin() + 3);
// 6. 移动 v2 到新创建的 deque v5，不发生拷贝; 常数复杂度; 需要 C++11
deque<int> v5(std::move(v2));
```

元素访问 与 `vector` 一致，但无法访问底层内存。其高效的元素访问速度可参考实现细节部分。

- `at()` 返回容器中指定位置元素的引用，执行越界检查，**常数复杂度**。
- `operator[]` 返回容器中指定位置元素的引用。不执行越界检查，**常数复杂度**。
- `front()` 返回首元素的引用。
- `back()` 返回末尾元素的引用。

元素增删及修改 与 `vector` 一致，并额外有向队列头部增加元素的函数。

- `clear()` 清除所有元素
- `insert()` 支持在某个迭代器位置插入元素、可以插入多个。**复杂度与 pos 与两端距离较小者成线性**。

- `erase()` 删除某个迭代器或者区间的元素，返回最后被删除的迭代器。复杂度与 `insert` 一致。
- `push_front()` 在头部插入一个元素，**常数复杂度**。
- `pop_front()` 删除头部元素，**常数复杂度**。
- `push_back()` 在末尾插入一个元素，**常数复杂度**。
- `pop_back()` 删除末尾元素，**常数复杂度**。
- `swap()` 与另一个容器进行交换，此操作是 **常数复杂度**而非线性的。

21.1.8 list

`std::list` 是 STL 提供的 双向链表 数据结构。能够提供线性复杂度的随机访问，以及常数复杂度的插入和删除。

list 的使用方法

`list` 的使用方法与 `deque` 基本相同，但是增删操作和访问的复杂度不同。详细内容 请参见 C++ 文档。`list` 的迭代器、长度、元素增删及修改相关的函数与 `deque` 相同，因此不作详细介绍。

元素访问 由于 `list` 的实现是链表，因此它不提供随机访问的接口。若需要访问中间元素，则需要使用迭代器。

- `front()` 返回首元素的引用。
- `back()` 返回末尾元素的引用。

操作 `list` 类型还提供了一些针对其特性实现的 STL 算法函数。由于这些算法需要 随机访问迭代器，因此 `list` 提供了特别的实现以便于使用。这些算法有 `splice()`、`remove()`、`sort()`、`unique()`、`merge()` 等。

forward_list 的使用方法

`forward_list` 的使用方法与 `list` 几乎一致，但是迭代器只有单向的，因此其具体用法不作详细介绍。详细内容 请参见 C++ 文档

21.1.9 set

`set` 是关联容器，含有键值类型对象的已排序集，搜索、移除和插入拥有对数复杂度。`set` 内部通常采用 红黑树 实现。平衡二叉树 的特性使得 `set` 非常适合处理需要同时兼顾查找、插入与删除的情况。

和数学中的集合相似，`set` 中不会出现值相同的元素。如果需要有相同元素的集合，需要使用 `multiset`。`multiset` 的使用方法与 `set` 的使用方法基本相同。

插入与删除操作

- `insert(x)` 当容器中没有等价元素的时候，将元素 x 插入到 `set` 中。
- `erase(x)` 删除值为 x 的 **所有**元素，返回删除元素的个数。
- `erase(pos)` 删除迭代器为 pos 的元素，要求迭代器必须合法。
- `erase(first,last)` 删除迭代器在 $[first, last)$ 范围内的所有元素。
- `clear()` 清空 `set`。

insert 函数的返回值类型为 `pair<iterator, bool>`，其中 iterator 是一个指向所插入元素（或者是指向等于所插入值的原本就在容器中的元素）的迭代器，而 bool 则代表元素是否插入成功，由于 `set` 中的元素具有唯一性质，所以如果在 `set` 中已有等值元素，则插入会失败，返回 false，否则插入成功，返回 true；`map` 中的 insert 也是如此。

迭代器

`set` 提供了以下几种迭代器：

1. `begin()/cbegin()`
   返回指向首元素的迭代器，其中 `*begin = front`。

2. `end()/cend()`
   返回指向数组尾端占位符的迭代器，注意是没有元素的。
3. `rbegin()/crbegin()`
   返回指向逆向数组的首元素的逆向迭代器，可以理解为正向容器的末元素。
4. `rend()/crend()`
   返回指向逆向数组末元素后一位置的迭代器，对应容器首的前一个位置，没有元素。

以上列出的迭代器中，含有字符 `c` 的为只读迭代器，你不能通过只读迭代器去修改 `set` 中的元素的值。如果一个 `set` 本身就是只读的，那么它的一般迭代器和只读迭代器完全等价。只读迭代器自 C++11 开始支持。

查找操作

- `count(x)` 返回 `set` 内键为 x 的元素数量。
- `find(x)` 在 `set` 内存在键为 x 的元素时会返回该元素的迭代器，否则返回 `end()`。
- `lower_bound(x)` 返回指向首个不小于给定键的元素的迭代器。如果不存在这样的元素，返回 `end()`。
- `upper_bound(x)` 返回指向首个大于给定键的元素的迭代器。如果不存在这样的元素，返回 `end()`。
- `empty()` 返回容器是否为空。
- `size()` 返回容器内元素个数。

`set` 自带的 `lower_bound` 和 `upper_bound` 的时间复杂度为 $O(\log n)$。

但使用 `algorithm` 库中的 `lower_bound` 和 `upper_bound` 函数对 `set` 中的元素进行查询，时间复杂度为 $O(n)$。

`set` 没有提供自带的 `nth_element`。使用 `algorithm` 库中的 `nth_element` 查找第 k 大的元素时间复杂度为 $O(n)$。

如果需要实现平衡二叉树所具备的 $O(\log n)$ 查找第 k 大元素的功能，需要自己手写平衡二叉树或权值线段树，或者选择使用 pb_ds 库中的平衡二叉树。

使用样例

`set` 在贪心中的使用 在贪心算法中经常会需要出现类似 **找出并删除最小的大于等于某个值的元素**。这种操作能轻松地通过 `set` 来完成。

```
// 现存可用的元素
set<int> available;
// 需要大于等于的值
int x;

// 查找最小的大于等于 x 的元素
set<int>::iterator it = available.lower_bound(x);
if (it == available.end()) {
  // 不存在这样的元素，则进行相应操作……
} else {
  // 找到了这样的元素，将其从现存可用元素中移除
  available.erase(it);
  // 进行相应操作……
}
```

21.1.10 `map`

`map` 是有序键值对容器，它的元素的键是唯一的。搜索、移除和插入操作拥有对数复杂度。`map` 通常实现为 红黑树。

设想如下场景：现在需要存储一些键值对，例如存储学生姓名对应的分数：`Tom 0`，`Bob 100`，`Alan 100`。但是由于数组下标只能为非负整数，所以无法用姓名作为下标来存储，这个时候最简单的办法就是使用 STL 中的 `map`。

`map` 重载了 `operator[]`，可以用任意定义了 `operator <` 的类型作为下标（在 `map` 中叫做 `key`，也就是索引）：

```
map<Key, T> yourMap;
```

其中，`Key` 是键的类型，`T` 是值的类型，下面是使用 `map` 的实例：

```
map<string, int> mp;
```

map 中不会存在键相同的元素，multimap 中允许多个元素拥有同一键。multimap 的使用方法与 map 的使用方法基本相同。

插入与删除操作

- 可以直接通过下标访问来进行查询或插入操作。例如 mp["Alan"]=100。
- 通过向 map 中插入一个类型为 pair<Key, T> 的值可以达到插入元素的目的,例如 mp.insert(pair<string,int>("Alan",1
- erase(key) 函数会删除键为 key 的 所有元素。返回值为删除元素的数量。
- erase(pos): 删除迭代器为 pos 的元素，要求迭代器必须合法。
- erase(first,last): 删除迭代器在 $[first, last)$ 范围内的所有元素。
- clear() 函数会清空整个容器。

在利用下标访问 map 中的某个元素时，如果 map 中不存在相应键的元素，会自动在 map 中插入一个新元素，并将其值设置为默认值（对于整数，值为零；对于有默认构造函数的类型，会调用默认构造函数进行初始化）。

当下标访问操作过于频繁时，容器中会出现大量无意义元素，影响 map 的效率。因此一般情况下推荐使用 find() 函数来寻找特定键的元素。

查询操作

- count(x): 返回容器内键为 x 的元素数量。复杂度为 $O(\log(size) + ans)$（关于容器大小对数复杂度，加上匹配个数）。
- find(x): 若容器内存在键为 x 的元素，会返回该元素的迭代器；否则返回 end()。
- lower_bound(x): 返回指向首个不小于给定键的元素的迭代器。
- upper_bound(x): 返回指向首个大于给定键的元素的迭代器。若容器内所有元素均小于或等于给定键，返回 end()。
- empty(): 返回容器是否为空。
- size(): 返回容器内元素个数。

使用样例

map 用于存储复杂状态 在搜索中，我们有时需要存储一些较为复杂的状态（如坐标，无法离散化的数值，字符串等）以及与之有关的答案（如到达此状态的最小步数）。map 可以用来实现此功能。其中的键是状态，而值是与之相关的答案。下面的示例展示了如何使用 map 存储以 string 表示的状态。

```
// 存储状态与对应的答案
map<string, int> record;

// 新搜索到的状态与对应答案
string status;
int ans;
// 查找对应的状态是否出现过
map<string, int>::iterator it = record.find(status);
if (it == record.end()) {
  // 尚未搜索过该状态，将其加入状态记录中
  record[status] = ans;
  // 进行相应操作……
} else {
  // 已经搜索过该状态，进行相应操作……
}
```

21.1.11 遍历容器

可以利用迭代器来遍历关联式容器的所有元素。

```
set<int> s;
using si = set<int>::iterator;
for (si it = s.begin(); it != s.end(); it++) cout << *it << endl;
```

需要注意的是，对 map 的迭代器解引用后，得到的是类型为 pair<Key, T> 的键值对。

在 C++11 中，使用范围 for 循环会让代码简洁很多：

```
set<int> s;
for (auto x : s) cout << x << endl;
```

对于任意关联式容器，使用迭代器遍历容器的时间复杂度均为 $O(n)$。

21.1.12 自定义比较方式

set 在默认情况下的比较函数为 <（如果是非内置类型需要 重载 < 运算符）。然而在某些特殊情况下，我们希望能自定义 set 内部的比较方式。

这时候可以通过传入自定义比较器来解决问题。

具体来说，我们需要定义一个类，并在这个类中 重载 () 运算符。

例如，我们想要维护一个存储整数，且较大值靠前的 set，可以这样实现：

```
struct cmp {
  bool operator()(int a, int b) const { return a > b; }
};

set<int, cmp> s;
```

对于其他关联式容器，可以用类似的方式实现自定义比较，这里不再赘述。

21.1.13 概述

自 C++11 标准起，四种基于 哈希 实现的无序关联式容器正式纳入了 C++ 的标准模板库中，分别是：unordered_set，unordered_multiset，unordered_map，unordered_multimap。

在 C++11 之前，无序关联式容器属于 C++ 的 TR1 扩展。所以，如果编译器不支持 C++11，在使用时需要在头文件的名称中加入 tr1/ 前缀，并且使用 std::tr1 命名空间。如 #include <unordered_map> 需要改成 #include <tr1/unordered_map>；std::unordered_map 需要改为 std::tr1::unordered_map（如果使用 using namespace std;，则为 tr1::unordered_map）。

它们与相应的关联式容器在功能，函数等方面有诸多共同点，而最大的不同点则体现在普通的关联式容器一般采用红黑树实现，内部元素按特定顺序进行排序；而这几种无序关联式容器则采用哈希方式存储元素，内部元素不以任何特定顺序进行排序，所以访问无序关联式容器中的元素时，访问顺序也没有任何保证。

采用哈希存储的特点使得无序关联式容器 **在平均情况下**大多数操作（包括查找，插入，删除）都能在常数时间复杂度内完成，相较于关联式容器与容器大小成对数的时间复杂度更加优秀。

在最坏情况下，对无序关联式容器进行插入、删除、查找等操作的时间复杂度会 **与容器大小成线性关系！**这一情况往往在容器内出现大量哈希冲突时产生。

同时，由于无序关联式容器的操作时通常存在较大的常数，其效率有时并不比普通的关联式容器好太多。

因此应谨慎使用无序关联式容器，尽量避免滥用（例如懒得离散化，直接将 unordered_map<int, int> 当作空间无限的普通数组使用）。

21.1.14 制造哈希冲突

上文中提到了，在最坏情况下，对无序关联式容器进行一些操作的时间复杂度会与容器大小成线性关系。

在哈希函数确定的情况下，可以构造出数据使得容器内产生大量哈希冲突，导致复杂度达到上界。

在标准库实现里，每个元素的散列值是将值对一个质数取模得到的，更具体地说，是 这个列表 中的质数（g++ 6 及以前版本的编译器，这个质数一般是 126271，g++ 7 及之后版本的编译器，这个质数一般是 107897）。

因此可以通过向容器中插入这些模数的倍数来达到制造大量哈希冲突的目的。

21.1.15 自定义哈希函数

使用自定义哈希函数可以有效避免构造数据产生的大量哈希冲突。

要想使用自定义哈希函数，需要定义一个结构体，并在结构体中重载 `()` 运算符，像这样：

```
struct my_hash {
  size_t operator()(int x) const { return x; }
};
```

当然，为了确保哈希函数不会被迅速破解（例如 Codeforces 中对使用无序关联式容器的提交进行 hack），可以试着在哈希函数中加入一些随机化函数（如时间）来增加破解的难度。

例如，在 这篇博客 中给出了如下哈希函数：

```
struct my_hash {
  static uint64_t splitmix64(uint64_t x) {
    x += 0x9e3779b97f4a7c15;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
    x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
    return x ^ (x >> 31);
  }

  size_t operator()(uint64_t x) const {
    static const uint64_t FIXED_RANDOM =
        chrono::steady_clock::now().time_since_epoch().count();
    return splitmix64(x + FIXED_RANDOM);
  }

  // 针对 std::pair<int, int> 作为主键类型的哈希函数
  size_t operator()(pair<uint64_t, uint64_t> x) const {
    static const uint64_t FIXED_RANDOM =
        chrono::steady_clock::now().time_since_epoch().count();
    return splitmix64(x.first + FIXED_RANDOM) ^
           (splitmix64(x.second + FIXED_RANDOM) >> 1);
  }
};
```

写完自定义的哈希函数后,就可以通过 `unordered_map<int, int, my_hash> my_map;` 或者 `unordered_map<pair<int, int>, int, my_hash> my_pair_map;` 的定义方式将自定义的哈希函数传入容器了。

21.1.16 栈

STL 栈(`std::stack`) 是一种后进先出 (Last In, First Out) 的容器适配器，仅支持查询或删除最后一个加入的元素（栈顶元素），不支持随机访问，且为了保证数据的严格有序性，不支持迭代器。

头文件

```
#include <stack>
```

定义

```
std::stack<TypeName> s;  // 使用默认底层容器 deque，数据类型为 TypeName
std::stack<TypeName, Container> s;  // 使用 Container 作为底层容器
std::stack<TypeName> s2(s1);        // 将 s1 复制一份用于构造 s2
```

成员函数

以下所有函数均为常数复杂度

- `top()` 访问栈顶元素（如果栈为空，此处会出错）
- `push(x)` 向栈中插入元素 x

- `pop()` 删除栈顶元素
- `size()` 查询容器中的元素数量
- `empty()` 询问容器是否为空

简单示例

```cpp
std::stack<int> s1;
s1.push(2);
s1.push(1);
std::stack<int> s2(s1);
s1.pop();
std::cout << s1.size() << " " << s2.size() << std::endl;  // 1 2
std::cout << s1.top() << " " << s2.top() << std::endl;    // 2 1
s1.pop();
std::cout << s1.empty() << " " << s2.empty() << std::endl;  // 1 0
```

21.1.17 队列

STL 队列(`std::queue`) 是一种先进先出 (First In, First Out) 的容器适配器，仅支持查询或删除第一个加入的元素（队首元素），不支持随机访问，且为了保证数据的严格有序性，不支持迭代器。

头文件

```cpp
#include <queue>
```

定义

```cpp
std::queue<TypeName> q;  // 使用默认底层容器 deque，数据类型为 TypeName
std::queue<TypeName, Container> q;  // 使用 Container 作为底层容器

std::queue<TypeName> q2(q1);  // 将 s1 复制一份用于构造 q2
```

成员函数

以下所有函数均为常数复杂度

- `front()` 访问队首元素（如果队列为空，此处会出错）
- `push(x)` 向队列中插入元素 x
- `pop()` 删除队首元素
- `size()` 查询容器中的元素数量
- `empty()` 询问容器是否为空

简单示例

```cpp
std::queue<int> q1;
q1.push(2);
q1.push(1);
std::queue<int> q2(q1);
q1.pop();
std::cout << q1.size() << " " << q2.size() << std::endl;    // 1 2
std::cout << q1.front() << " " << q2.front() << std::endl;  // 1 2
q1.pop();
std::cout << q1.empty() << " " << q2.empty() << std::endl;  // 1 0
```

21.1.18 优先队列

优先队列 `std::priority_queue` 是一种 堆，一般为 二叉堆。

头文件

```
#include <queue>
```

定义

```
std::priority_queue<TypeName> q;             // 数据类型为 TypeName
std::priority_queue<TypeName, Container> q;  // 使用 Container 作为底层容器
std::priority_queue<TypeName, Container, Compare> q;
// 使用 Container 作为底层容器，使用 Compare 作为比较类型

// 默认使用底层容器 vector
// 比较类型 less<TypeName>（此时为它的 top() 返回为最大值）
// 若希望 top() 返回最小值，可令比较类型为 greater<TypeName>
// 注意：不可跳过 Container 直接传入 Compare

// 从 C++11 开始，如果使用 lambda 函数自定义 Compare
// 则需要将其作为构造函数的参数代入，如：
auto cmp = [](const std::pair<int, int> &l, const std::pair<int, int> &r) {
  return l.second < r.second;
};
std::priority_queue<std::pair<int, int>, std::vector<std::pair<int, int>>,
                    decltype(cmp)>
    pq(cmp);
```

成员函数

以下所有函数均为常数复杂度

- `top()` 访问堆顶元素（此时优先队列不能为空）
- `empty()` 询问容器是否为空
- `size()` 查询容器中的元素数量

以下所有函数均为对数复杂度

- `push(x)` 插入元素，并对底层容器排序
- `pop()` 删除堆顶元素（此时优先队列不能为空）

简单示例

```
std::priority_queue<int> q1;
std::priority_queue<int, std::vector<int>> q2;
// C++11 后空格可省略
std::priority_queue<int, std::deque<int>, std::greater<int>> q3;
// q3 为小根堆
for (int i = 1; i ⩽ 5; i++) q1.push(i);
// q1 中元素 :  [1, 2, 3, 4, 5]
std::cout << q1.top() << std::endl;
// 输出结果 : 5
q1.pop();
// 堆中元素 : [1, 2, 3, 4]
std::cout << q1.size() << std::endl;
// 输出结果 : 4
for (int i = 1; i ⩽ 5; i++) q3.push(i);
// q3 中元素 :  [1, 2, 3, 4, 5]
std::cout << q3.top() << std::endl;
// 输出结果 : 1
```

STL 提供了大约 100 个实现算法的模版函数，基本都包含在 `<algorithm>` 之中，还有一部分包含在 `<numeric>` 和 `<functional>`。完备的函数列表请 参见参考手册，排序相关的可以参考 排序内容的对应页面。

- `find`：顺序查找。`find(v.begin(), v.end(), value)`，其中 `value` 为需要查找的值。

- `reverse`：翻转数组、字符串。`reverse(v.begin(), v.end())` 或 `reverse(a + begin, a + end)`。
- `unique`：去除容器中相邻的重复元素。`unique(ForwardIterator first, ForwardIterator last)`，返回值为指向 **去重后**容器结尾的迭代器，原容器大小不变。与 `sort` 结合使用可以实现完整容器去重。
- `random_shuffle`：随机地打乱数组。`random_shuffle(v.begin(), v.end())` 或 `random_shuffle(v + begin, v + end)`。

在 C++11 以及更新的标准中，您可以使用 `shuffle` 函数代替原来的 `random_shuffle`。使用方法为 `shuffle(v.begin(), v.end(), rng)`（最后一个参数传入的是使用的随机数生成器，一般情况使用以真随机数生成器 `random_device` 播种的梅森旋转伪随机数生成器 `mt19937`）。

```
// #include <random>
std::mt19937 rng(std::random_device{}());
std::shuffle(v.begin(), v.end(), rng);
```

- `sort`：排序。`sort(v.begin(), v.end(), cmp)` 或 `sort(a + begin, a + end, cmp)`，其中 `end` 是排序的数组最后一个元素的后一位，`cmp` 为自定义的比较函数。
- `stable_sort`：稳定排序，用法同 `sort()`。
- `nth_element`：按指定范围进行分类，即找出序列中第 n 大的元素，使其左边均为小于它的数，右边均为大于它的数。`nth_element(v.begin(), v.begin() + n, v.end(), cmp)` 或 `nth_element(a + begin, a + begin + n, a + end, cmp)`。
- `binary_search`：二分查找。`binary_search(v.begin(), v.end(), value)`，其中 `value` 为需要查找的值。
- `merge`：将两个（已排序的）序列 **有序合并**到第三个序列的 **插入迭代器**上。`merge(v1.begin(), v1.end(), v2.begin(), v2.end() ,back_inserter(v3))`。
- `inplace_merge`：将两个（已按小于运算符排序的）：`[first,middle)`, `[middle,last)` 范围 **原地合并为一个有序序列**。`inplace_merge(v.begin(), v.begin() + middle, v.end())`。
- `lower_bound`：在一个有序序列中进行二分查找，返回指向第一个 **大于等于** x 的元素的位置的迭代器。如果不存在这样的元素，则返回尾迭代器。`lower_bound(v.begin(),v.end(),x)`。
- `upper_bound`：在一个有序序列中进行二分查找，返回指向第一个 **大于** x 的元素的位置的迭代器。如果不存在这样的元素，则返回尾迭代器。`upper_bound(v.begin(),v.end(),x)`。

在一般的数组里，这两个函数的时间复杂度均为 $O(\log n)$，但在 `set` 等关联式容器中，直接调用 `lower_bound(s.begin(),` 的时间复杂度是 $O(n)$ 的。

`set` 等关联式容器中已经封装了 `lower_bound` 等函数（像 `s.lower_bound(val)` 这样），这样调用的时间复杂度是 $O(\log n)$ 的。

- `next_permutation`：将当前排列更改为 **全排列中的下一个排列**。如果当前排列已经是 **全排列中的最后一个排列**（元素完全从大到小排列），函数返回 `false` 并将排列更改为 **全排列中的第一个排列**（元素完全从小到大排列）；否则，函数返回 `true`。`next_permutation(v.begin(), v.end())` 或 `next_permutation(v + begin, v + end)`。
- `prev_permutation`：将当前排列更改为 **全排列中的上一个排列**。用法同 `next_permutation`。
- `partial_sum`:求前缀和。设源容器为 x，目标容器为 y，则令 $y[i] = x[0]+x[1]+\cdots+x[i]$。`partial_sum(src.begin(), src.end(), back_inserter(dst))`。

使用样例

- 使用 `next_permutation` 生成 1 到 9 的全排列。例题：Luogu P1706 全排列问题

```
int N = 9, a[] = {1, 2, 3, 4, 5, 6, 7, 8, 9};
do {
    for (int i = 0; i < N; i++) cout << a[i] << " ";
    cout << endl;
} while (next_permutation(a, a + N));
```

- 使用 `lower_bound` 与 `upper_bound` 查找有序数组 a 中小于 x，等于 x，大于 x 元素的分界线。

```
int N = 10, a[] = {1, 1, 2, 4, 5, 5, 7, 7, 9, 9}, x = 5;
int i = lower_bound(a, a + N, x) - a, j = upper_bound(a, a + N, x) - a;
// a[0] ~ a[i - 1] 为小于 x 的元素， a[i] ~ a[j - 1] 为等于 x 的元素，
// a[j] ~ a[N - 1] 为大于 x 的元素
cout << i << " " << j << endl;
```

- 使用 `partial_sum` 求解 src 中元素的前缀和，并存储于 dst 中。

```
vector<int> src = {1, 2, 3, 4, 5}, dst;
// 求解 src 中元素的前缀和，dst[i] = src[0] + ... + src[i]
// back_inserter 函数作用在 dst 容器上，提供一个迭代器
partial_sum(src.begin(), src.end(), back_inserter(dst));
for (unsigned int i = 0; i < dst.size(); i++) cout << dst[i] << " ";
```

- 使用 `lower_bound` 查找有序数组 a 中最接近 x 的元素。例题：UVa10487 Closest Sums

```
int N = 10, a[] = {1, 1, 2, 4, 5, 5, 8, 8, 9, 9}, x = 6;
// lower_bound 将返回 a 中第一个大于等于 x 的元素的地址，计算出的 i 为其下标
int i = lower_bound(a, a + N, x) - a;
// 在以下两种情况下，a[i] (a 中第一个大于等于 x 的元素) 即为答案：
// 1. a 中最小的元素都大于等于 x；
// 2. a 中存在大于等于 x 的元素，且第一个大于等于 x 的元素 (a[i])
// 相比于第一个小于 x 的元素 (a[i - 1]) 更接近 x；
// 否则，a[i - 1] (a 中第一个小于 x 的元素) 即为答案
if (i == 0 || (i < N && a[i] - x < x - a[i - 1]))
    cout << a[i];
else
    cout << a[i - 1];
```

- 使用 `sort` 与 `unique` 查找数组 a 中 **第 k 小的值**（注意：重复出现的值仅算一次，因此本题不是求解第 k 小的元素）。例题：Luogu P1138 第 k 小整数

```
int N = 10, a[] = {1, 3, 3, 7, 2, 5, 1, 2, 4, 6}, k = 3;
sort(a, a + N);
// unique 将返回去重之后数组最后一个元素之后的地址，计算出的 cnt 为去重后数组的长度
int cnt = unique(a, a + N) - a;
cout << a[k - 1];
```

21.1.19 介绍

`std::bitset` 是标准库中的一个存储 `0/1` 的大小不可变容器。严格来讲，它并不属于 STL。

由此看来，`bitset` 并不属于 STL，而是一种标准库中的 “Special Container”。事实上，它作为一种容器，也并不满足 STL 容器的要求。说它是适配器，它也并不依赖于其它 STL 容器作为底层实现。

由于内存地址是按字节即 `byte` 寻址，而非比特 `bit`，一个 `bool` 类型的变量，虽然只能表示 `0/1`, 但是也占了 1 byte 的内存。

`bitset` 就是通过固定的优化，使得一个字节的八个比特能分别储存 8 位的 `0/1`。

对于一个 4 字节的 `int` 变量，在只存 `0/1` 的意义下，`bitset` 占用空间只是其 $\frac{1}{32}$，计算一些信息时，所需时间也是其 $\frac{1}{32}$。

在某些情况下通过 `bitset` 可以优化程序的运行效率。至于其优化的是复杂度还是常数，要看计算复杂度的角度。一般 `bitset` 的复杂度有以下几种记法：（设原复杂度为 $O(n)$）

1. $O(n)$，这种记法认为 `bitset` 完全没有优化复杂度。
2. $O(\frac{n}{32})$，这种记法不太严谨（复杂度中不应出现常数），但体现了 `bitset` 能将所需时间优化至 $\frac{1}{32}$。
3. $O(\frac{n}{w})$，其中 $w = 32$（计算机的位数），这种记法较为普遍接受。
4. $O(\frac{n}{\log w})$，其中 w 为计算机一个整型变量的大小。

另外，`vector` 的一个特化 `vector<bool>` 的储存方式同 `bitset` 一样，区别在于其支持动态开空间，`bitset` 则和我们一般的静态数组一样，是在编译时就开好了的。然而，`bitset` 有一些好用的库函数，不仅方便，而且有时可以实现 SIMD 进而减小常数。另外，`vector<bool>` 的部分表现和 `vector` 不一致（如对 `std::vector<bool> vec` 来说，`&vec[0] + i` 不等于 `&vec[i]`）。因此，一般不使用 `vector<bool>`。

头文件

```
#include <bitset>
```

指定大小

```
std::bitset<1000> bs;  // a bitset with 1000 bits
```

构造函数

- `bitset()`: 每一位都是 `false`。
- `bitset(unsigned long val)`: 设为 `val` 的二进制形式。
- `bitset(const string& str)`: 设为 01 串 `str`。

运算符

- `operator []`: 访问其特定的一位。
- `operator ==`/`operator !=`: 比较两个 `bitset` 内容是否完全一样。
- `operator &`/`operator &=`/`operator |`/`operator |=`/`operator ^`/`operator ^=`/`operator ~`: 进行按位与/或/异或/取反操作。
  注意：**`bitset` 只能与 `bitset` 进行位运算**，若要和整型进行位运算，要先将整型转换为 `bitset`。
- `operator <<`/`operator >>`/`operator <<=`/`operator >>=`: 进行二进制左移/右移。

此外，`bitset` 还提供了 C++ 流式 IO 的支持，这意味着你可以通过 `cin`/`cout` 进行输入输出。

成员函数

- `count()`: 返回 `true` 的数量。
- `size()`: 返回 `bitset` 的大小。
- `test(pos)`: 它和 `vector` 中的 `at()` 的作用是一样的，和 `[]` 运算符的区别就是越界检查。
- `any()`: 若存在某一位是 `true` 则返回 `true`，否则返回 `false`。
- `none()`: 若所有位都是 `false` 则返回 `true`，否则返回 `false`。
- `all()`: 若所有位都是 `true` 则返回 `true`，否则返回 `false`。
- 1. `set()`: 将整个 `bitset` 设置成 `true`。
  2. `set(pos, val = true)`: 将某一位设置成 `true`/`false`。
- 1. `reset()`: 将整个 `bitset` 设置成 `false`。
  2. `reset(pos)`: 将某一位设置成 `false`。相当于 `set(pos, false)`。
- 1. `flip()`: 翻转每一位。（$0 \leftrightarrow 1$，相当于异或一个全是 1 的 `bitset`）
  2. `flip(pos)`: 翻转某一位。
- `to_string()`: 返回转换成的字符串表达。
- `to_ulong()`: 返回转换成的 `unsigned long` 表达（`long` 在 NT 及 32 位 POSIX 系统下与 `int` 一样，在 64 位 POSIX 下与 `long long` 一样）。
- `to_ullong()`:（**C++11** 起）返回转换成的 `unsigned long long` 表达。

另外，libstdc++ 中有一些较为实用的内部成员函数 [^bitset1]：

- `_Find_first()`: 返回 `bitset` 第一个 `true` 的下标，若没有 `true` 则返回 `bitset` 的大小。
- `_Find_next(pos)`: 返回 `pos` 后面（下标严格大于 `pos` 的位置）第一个 `true` 的下标，若 `pos` 后面没有 `true` 则返回 `bitset` 的大小。

21.1.20 应用

「LibreOJ β Round #2」贪心只能过样例

这题可以用 dp 做，转移方程很简单：

$f(i,j)$ 表示前 i 个数的平方和能否为 j，那么 $f(i,j)=\bigvee_{k=a}^{b} f(i-1,j-k^2)$（或起来）。

但如果直接做的话是 $O(n^5)$ 的，（看起来）过不了。

发现可以用 `bitset` 优化，左移再或起来就好了：

```
#include <bitset>
#include <cstdio>
#include <iostream>

using namespace std;

constexpr int N = 101;

int n, a[N], b[N];
bitset<N * N * N> f[N];

int main() {
    int i, j;

    cin >> n;

    for (i = 1; i ⩽ n; ++i) cin >> a[i] >> b[i];

    f[0][0] = 1;

    for (i = 1; i ⩽ n; ++i) {
    for (j = a[i]; j ⩽ b[i]; ++j) {
        f[i] ⊨ (f[i - 1] << (j * j));
    }
    }

    cout << f[n].count();

    return 0;
}
```

由于 libstdc++ 的实现为压 `__CHAR_BIT__ * sizeof(unsigned long)` 位的 [^bitset2]，在一些平台中其为 32。所以，可以手写 `bitset`（只需要支持左移后或起来这一种操作）压 64 位（`__CHAR_BIT__ * sizeof(unsigned long long)`）来进一步优化：

```
#include <cstdio>

using namespace std;

constexpr int N = 101;
constexpr int W = 64;

struct Bitset {
    unsigned long long a[N * N * N >> 6];

    void shiftor(const Bitset &y, int p, int l, int r) {
    int t = p - p / W * W;
    int tt = (t == 0 ? 0 : W - t);
    int to = (r + p) / W;
    int qaq = (p + W - 1) / W;

    for (int i = (l + p) / W; i ⩽ to; ++i) {
        if (i - qaq ⩾ 0) a[i] ⊨ y.a[i - qaq] >> tt;

        a[i] ⊨ ((y.a[i - qaq + 1] & ((1ull << tt) - 1)) << t);
    }
```

```
    }
} f[N];

int main() {
    int n, a, b, l = 0, r = 0, ans = 0;

    scanf("%d", &n);

    f[0].a[0] = 1;

    for (int i = 1; i ⩽ n; ++i) {
    scanf("%d%d", &a, &b);

    for (int j = a; j ⩽ b; ++j) f[i].shiftor(f[i - 1], j * j, l, r);

    l += a * a;
    r += b * b;
    }

    for (int i = l / W; i ⩽ r / W; ++i)
    ans += __builtin_popcount(f[n].a[i] & 0xffffffffu) +
            __builtin_popcount(f[n].a[i] >> 32);

    printf("%d", ans);


    return 0;
}
```

21.1.21 string 是什么

std::string 是在标准库 <string>（注意不是 C 语言中的 <string.h> 库）中提供的一个类，本质上是 std::basic_string<char> 的别称。

21.1.22 为什么要使用 string

在 C 语言中，提供了字符串的操作，但只能通过字符数组的方式来实现字符串。而 string 则是一个简单的类，使用简单，在 OI 竞赛中被广泛使用。并且相较于其他 STL 容器，string 的常数可以算是非常优秀的，基本与字符数组不相上下。

string 可以动态分配空间

和许多 STL 容器相同，string 能动态分配空间，这使得我们可以直接使用 std::cin 来输入，但其速度则同样较慢。这一点也同样让我们不必为内存而烦恼。

string 重载了加法运算符和比较运算符

string 的加法运算符可以直接拼接两个字符串或一个字符串和一个字符。和 std::vector 类似，string 重载了比较运算符，同样是按字典序比较的，所以我们可以直接调用 std::sort 对若干字符串进行排序。

声明

```
std::string s;
```

转 char 数组

在 C 语言里，也有很多字符串的函数，但是它们的参数都是 char 指针类型的，为了方便使用，string 有两个成员函数能够将自己转换为 char 指针——data()/c_str()（它们几乎是一样的，但最好使用 c_str()，因为 c_str() 保证末尾有空字符，而 data() 则不保证），如：

```
printf("%s", s);          // 编译错误
printf("%s", s.data());   // 编译通过，但是是 undefined behavior
printf("%s", s.c_str());  // 一定能够正确输出
```

获取长度

很多函数都可以返回 string 的长度：

```
printf("s 的长度为 %zu", s.size());
printf("s 的长度为 %zu", s.length());
printf("s 的长度为 %zu", strlen(s.c_str()));
```

`strlen()` 的复杂度一定是与字符串长度线性相关的。

`size()` 和 `length()` 的复杂度在 C++98 中没有指定，在 C++11 中被指定为常数复杂度。但在常见的编译器上，即便是 C++98，这两个函数的复杂度也是常数。

这三个函数（以及下面将要提到的 `find` 函数）的返回值类型都是 `size_t`（`unsigned long`）。因此，这些返回值不支持直接与负数比较或运算，建议在需要时进行强制转换。

寻找某字符（串）第一次出现的位置

`find(str,pos)` 函数可以用来查找字符串中一个字符/字符串在 `pos`（含）之后第一次出现的位置（若不传参给 `pos` 则默认为 `0`）。如果没有出现，则返回 `string::npos`（被定义为 `-1`，但类型仍为 `size_t`/`unsigned long`）。

示例：

```
string s = "OI Wiki", t = "OI", u = "i";
int pos = 5;
printf(" 字符 I 在 s 的 %lu 位置第一次出现\n", s.find('I'));
printf(" 字符 a 在 s 的 %lu 位置第一次出现\n", s.find('a'));
printf(" 字符 a 在 s 的 %d 位置第一次出现\n", s.find('a'));
printf(" 字符串 t 在 s 的 %lu 位置第一次出现\n", s.find(t));
printf(" 在 s 中自 pos 位置起字符串 u 第一次出现在 %lu 位置", s.find(u, pos));
```

输出：

截取子串

`substr(pos, len)` 函数的参数返回从 `pos` 位置开始截取最多 `len` 个字符组成的字符串（如果从 `pos` 开始的后缀长度不足 `len` 则截取这个后缀）。

示例：

```
string s = "OI Wiki", t = "OI";
printf(" 从字符串 s 的第四位开始的最多三个字符构成的子串是 %s\n",
       s.substr(3, 3).c_str());
printf(" 从字符串 t 的第二位开始的最多三个字符构成的子串是 %s",
       t.substr(1, 3).c_str());
```

输出：

插入/删除字符（串）

`insert(index,count,ch)` 和 `insert(index,str)` 是比较常见的插入函数。它们分别表示在 `index` 处连续插入 `count` 次字符串 `ch` 和插入字符串 `str`。

`erase(index,count)` 函数将字符串 `index` 位置开始（含）的 `count` 个字符删除（若不传参给 `count` 则表示删去 `index` 位置及以后的所有字符）。

示例：

```
string s = "OI Wiki", t = " Wiki";
char u = '!';
s.erase(2);
printf(" 从字符串 s 的第三位开始删去所有字符后得到的字符串是 %s\n", s.c_str());
s.insert(2, t);
printf(" 在字符串 s 的第三位处插入字符串 t 后得到的字符串是 %s\n", s.c_str());
s.insert(7, 3, u);
printf(" 在字符串 s 的第八位处连续插入 3 次字符串 u 后得到的字符串是 %s",
       s.c_str());
```

输出：

替换字符（串）

`replace(pos,count,str)` 和 `replace(first,last,str)` 是比较常见的替换函数。它们分别表示将从 `pos` 位置开始 `count` 个字符的子串替换为 `str` 以及将以 `first` 开始（含）、`last` 结束（不含）的子串替换为 `str`，其中 `first` 和 `last` 均为迭代器。

示例：

```
string s = "OI Wiki";
s.replace(2, 5, "");
printf(" 将字符串 s 的第 3~7 位替换为空串后得到的字符串是 %s\n", s.c_str());
s.replace(s.begin(), s.begin() + 2, "NOI");
printf(" 将字符串 s 的前两位替换为 NOI 后得到的字符串是 %s", s.c_str());
```

输出：

`std::pair` 是标准库中定义的一个类模板。用于将两个变量关联在一起，组成一个「对」，而且两个变量的数据类型可以是不同的。

类模板（class template）本身不是一个类，而是可以根据 **不同数据类型**产生 **不同类**的「模板」。

在使用时，编译器会根据传入的数据类型产生对应的类，再创建对应实例。

模板属于 C++ 较为高级的语言特性，在信息学竞赛中几乎不会出现。如果对此感兴趣，可以进一步阅读《C++ Primer》以学习更深层次的 C++ 知识。

通过灵活使用 `pair`，可以轻松应对 **需要将关联数据捆绑存储、处理**的场景。

与自定义的 `struct` 相比，`pair` 不需要额外定义结构与重载运算符，因此使用起来更加简便。

然而，自定义 `struct` 的变量命名往往更加清晰（`pair` 只能使用 `first` 与 `second` 访问包含的两个变量）。同时，如果需要将两个以上的变量进行关联，自定义 `struct` 会更加合适。

21.1.23 使用

初始化

可以在定义时直接完成 `pair` 的初始化。

```
pair<int, double> p0(1, 2.0);
```

也可以使用先定义，后赋值的方法完成 `pair` 的初始化。

```
pair<int, double> p1;
p1.first = 1;
p1.second = 2.0;
```

还可以使用 `std::make_pair` 函数。该函数接受两个变量，并返回由这两个变量组成的 `pair`。

```
pair<int, double> p2 = make_pair(1, 2.0);
```

一种常用的方法是使用宏定义 `#define mp make_pair`，将有些冗长的 `make_pair` 化简为 `mp`。

在 C++11 以及之后的版本中，`make_pair` 可以配合 `auto` 使用，以避免显式声明数据类型。

```
auto p3 = make_pair(1, 2.0);
```

访问

通过成员函数 first 与 second，可以访问 pair 中包含的两个变量。

```
int i = p0.first;
double d = p0.second;
```

也可以对其进行修改。

```
p1.first++;
```

比较

pair 已经预先定义了所有的比较运算符，包括 <、>、<=、>=、==、≠。当然，这需要组成 pair 的两个变量所属的数据类型定义了 == 和/或 < 运算符。

其中，<、>、<=、>= 四个运算符会先比较两个 pair 中的第一个变量，在第一个变量相等的情况下再比较第二个变量。

```
if (p2 ⩾ p3) {
  cout << "do something here" << endl;
}
```

由于 pair 定义了 STL 中常用的 < 与 ==，使得其能够很好的与其他 STL 函数或数据结构配合。比如，pair 可以作为 priority_queue 的数据类型。

```
priority_queue<pair<int, double>> q;
```

赋值与交换

可以将 pair 的值赋给另一个类型一致的 pair。

```
p0 = p1;
```

也可以使用 swap 函数交换 pair 的值。

```
swap(p0, p1);
p2.swap(p3);
```

21.1.24 应用举例

离散化

pair 可以轻松实现离散化。

我们可以创建一个 pair 数组，将原始数据的值作为每个 pair 第一个变量，将原始数据的位置作为第二个变量。在排序后，将原始数据值的排名（该值排序后所在的位置）赋给该值原本所在的位置即可。

```
// a 为原始数据
pair<int, int> a[MAXN];
// ai 为离散化后的数据
int ai[MAXN];
for (int i = 0; i < n; i++) {
  // first 为原始数据的值，second 为原始数据的位置
  scanf("%d", &a[i].first);
  a[i].second = i;
}
// 排序
sort(a, a + n);
for (int i = 0; i < n; i++) {
  // 将该值的排名赋给该值原本所在的位置
  ai[a[i].second] = i;
}
```

Dijkstra

如前所述，pair 可以作为 priority_queue 的数据类型。

那么，在 Dijkstra 算法的堆优化中，可以使用 pair 与 priority_queue 维护节点，将节点当前到起点的距离作为第一个变量，将节点编号作为第二个变量。

```
priority_queue<pair<int, int>, std::vector<pair<int, int>>,
               std::greater<pair<int, int>>>
    q;
... while (!q.empty()) {
  // dis 为入堆时节点到起点的距离，i 为节点编号
  int dis = q.top().first, i = q.top().second;
  q.pop();
  ...
}
```

pair 与 map

map 的是 C++ 中存储键值对的数据结构。很多情况下，map 中存储的键值对通过 pair 向外暴露。

```
map<int, double> m;
m.insert(make_pair(1, 2.0));
```

关于 map 更多的内容，请见 关联式容器 与 无序关联式容器 中相关部分。

21.1.25 pb_ds

pb_ds 库全称 Policy-Based Data Structures。

pb_ds 库封装了很多数据结构，比如哈希（Hash）表，平衡二叉树，字典树（Trie 树），堆（优先队列）等。

就像 vector、set、map 一样，其组件均符合 STL 的相关接口规范。部分（如优先队列）包含 STL 内对应组件的所有功能，但比 STL 功能更多。

pb_ds 只在使用 libstdc++ 为标准库的编译器下可以用。

可以使用 begin() 和 end() 来获取 iterator 从而遍历

可以 increase_key,decrease_key 以及删除单个元素

由于 pb_ds 库的主要内容在以下划线开头的 __gnu_pbds 命名空间中，在 NOI 系列活动中的合规性一直没有确定。2021 年 9 月 1 日，根据 《关于 NOI 系列活动中编程语言使用限制的补充说明》，允许使用以下划线开头的库函数或宏（但具有明确禁止操作的库函数和宏除外），在 NOI 系列活动中使用 pb_ds 库的合规性有了文件上的依据。

21.1.26 __gnu_pbds::priority_queue

```
#include <ext/pb_ds/priority_queue.hpp>
using namespace __gnu_pbds;
__gnu_pbds::priority_queue<T, Compare, Tag, Allocator>
```

21.1.27 模板形参

- T: 储存的元素类型
- Compare: 提供严格的弱序比较类型
- Tag: 是 __gnu_pbds 提供的不同的五种堆，Tag 参数默认是 pairing_heap_tag 五种分别是：
  - pairing_heap_tag：配对堆官方文档认为在非原生元素（如自定义结构体/std::string/pair）中，配对堆表现最好
  - binary_heap_tag：二叉堆官方文档认为在原生元素中二叉堆表现最好，不过笔者测试的表现并没有那么好
  - binomial_heap_tag：二项堆二项堆在合并操作的表现要优于二叉堆，但是其取堆顶元素操作的复杂度比二叉堆高
  - rc_binomial_heap_tag：冗余计数二项堆

  - `thin_heap_tag`：除了合并的复杂度都和 Fibonacci 堆一样的一个 tag

- `Allocator`：空间配置器，由于 OI 中很少出现，故这里不做讲解

由于本篇文章只是提供给学习算法竞赛的同学们，故对于后四个 tag 只会简单的介绍复杂度，第一个会介绍成员函数和使用方法。

至少对于 OIer 来讲，除了配对堆的其他四个 tag 都是鸡肋，要么没用，要么常数大到不如 `std` 的，且有可能造成 MLE，故这里只推荐用默认的配对堆。同样，配对堆也优于 `algorithm` 库中的 `make_heap()`。

21.1.28 构造方式

要注明命名空间因为和 `std` 的类名称重复。

```
// __gnu_pbds::priority_queue<int>;
// __gnu_pbds::priority_queue<int, greater<int>>;
// __gnu_pbds::priority_queue<int, greater<int>, pairing_heap_tag>;
__gnu_pbds::priority_queue<int>::point_iterator id;  // 点类型迭代器
// 在 modify 和 push 的时候都会返回一个 point_iterator，下文会详细的讲使用方法
id = q.push(1);
```

21.1.29 成员函数

- `push()`: 向堆中压入一个元素，返回该元素位置的迭代器。
- `pop()`: 将堆顶元素弹出。
- `top()`: 返回堆顶元素。
- `size()` 返回元素个数。
- `empty()` 返回是否非空。
- `modify(point_iterator, const key)`: 把迭代器位置的 `key` 修改为传入的 `key`，并对底层储存结构进行排序。
- `erase(point_iterator)`: 把迭代器位置的键值从堆中擦除。
- `join(__gnu_pbds::priority_queue &other)`: 把 `other` 合并到 `*this` 并把 `other` 清空。

使用的 tag 决定了每个操作的时间复杂度：

	push	pop	modify	erase	Join
`pairing_heap_tag`	$O(1)$	最坏 $\Theta(n)$ 均摊 $\Theta(\log(n))$	最坏 $\Theta(n)$ 均摊 $\Theta(\log(n))$	最坏 $\Theta(n)$ 均摊 $\Theta(\log(n))$	$O(1)$
`binary_heap_tag`	最坏 $\Theta(n)$ 均摊 $\Theta(\log(n))$	最坏 $\Theta(n)$ 均摊 $\Theta(\log(n))$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
`binomial_heap_tag`	最坏 $\Theta(\log(n))$ 均摊 $O(1)$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$
`rc_binomial_heap_tag`	$O(1)$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$	$\Theta(\log(n))$
`thin_heap_tag`	$O(1)$	最坏 $\Theta(n)$ 均摊 $\Theta(\log(n))$	最坏 $\Theta(\log(n))$ 均摊 $O(1)$	最坏 $\Theta(n)$ 均摊 $\Theta(\log(n))$	$\Theta(n)$

21.1.30 示例

```
#include <algorithm>
#include <cstdio>
#include <ext/pb_ds/priority_queue.hpp>
#include <iostream>
using namespace __gnu_pbds;
// 由于面向 OIer，本文以常用堆 : pairing_heap_tag 作为范例
// 为了更好的阅读体验，定义宏如下 :
using pair_heap = __gnu_pbds::priority_queue<int>;
pair_heap q1;  // 大根堆，配对堆
pair_heap q2;
pair_heap::point_iterator id;  // 一个迭代器

int main() {
```

```
  id = q1.push(1);
  // 堆中元素 : [1];
  for (int i = 2; i <= 5; i++) q1.push(i);
  // 堆中元素 : [1, 2, 3, 4, 5];
  std::cout << q1.top() << std::endl;
  // 输出结果 : 5;
  q1.pop();
  // 堆中元素 : [1, 2, 3, 4];
  id = q1.push(10);
  // 堆中元素 : [1, 2, 3, 4, 10];
  q1.modify(id, 1);
  // 堆中元素 : [1, 1, 2, 3, 4];
  std::cout << q1.top() << std::endl;
  // 输出结果 : 4;
  q1.pop();
  // 堆中元素 : [1, 1, 2, 3];
  id = q1.push(7);
  // 堆中元素 : [1, 1, 2, 3, 7];
  q1.erase(id);
  // 堆中元素 : [1, 1, 2, 3];
  q2.push(1), q2.push(3), q2.push(5);
  // q1 中元素 : [1, 1, 2, 3], q2 中元素 : [1, 3, 5];
  q2.join(q1);
  // q1 中元元素, q2 中元素 : [1, 1, 1, 2, 3, 3, 5];
}
```

21.1.31 ___gnu_pbds 迭代器的失效保证（invalidation_guarantee）

在上述示例以及一些实践中（如使用本章的 pb-ds 堆来编写单源最短路等算法），常常需要保存并使用堆的迭代器（如 `__gnu_pbds::priority_queue<int>::point_iterator` 等）。

可是例如对于 `__gnu_pbds::priority_queue` 中不同的 Tag 参数，其底层实现并不相同，迭代器的失效条件也不一样，根据 ___gnu_pbds 库的设计，以下三种由上至下派生的情况：

1. 基本失效保证（basic_invalidation_guarantee）：即不修改容器时，点类型迭代器（point_iterator）、指针和引用（key/value）**保持**有效。
2. 点失效保证（point_invalidation_guarantee）：即 **修改**容器后，点类型迭代器（point_iterator）、指针和引用（key/value）只要对应在容器中没被删除 **保持**有效。
3. 范围失效保证（range_invalidation_guarantee）：即 **修改**容器后，除（2）的特性以外，任何范围类型的迭代器（包括 `begin()` 和 `end()` 的返回值）是正确的，具有范围失效保证的 Tag 有 `rb_tree_tag` 和适用于 `__gnu_pbds::tree` 的 `splay_tree_tag`，以及适用于 `__gnu_pbds::trie` 的 `pat_trie_tag`。

从运行下述代码中看出，除了 `binary_heap_tag` 为 `basic_invalidation_guarantee` 在修改后迭代器会失效，其余的均为 `point_invalidation_guarantee` 可以实现修改后点类型迭代器 (point_iterator) 不失效的需求。

```
#include <iostream>
using namespace std;
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/priority_queue.hpp>
using namespace __gnu_pbds;
#include <cxxabi.h>

template <typename T>
void print_invalidation_guarantee() {
  using gute = typename __gnu_pbds::container_traits<T>::invalidation_guarantee;
  cout << abi::__cxa_demangle(typeid(gute).name(), 0, 0, 0) << endl;
}

int main() {
  using pairing =
      __gnu_pbds::priority_queue<int, greater<int>, pairing_heap_tag>;
```

```
  using binary = __gnu_pbds::priority_queue<int, greater<int>, binary_heap_tag>;
  using binomial =
      __gnu_pbds::priority_queue<int, greater<int>, binomial_heap_tag>;
  using rc_binomial =
      __gnu_pbds::priority_queue<int, greater<int>, rc_binomial_heap_tag>;
  using thin = __gnu_pbds::priority_queue<int, greater<int>, thin_heap_tag>;
  print_invalidation_guarantee<pairing>();
  print_invalidation_guarantee<binary>();
  print_invalidation_guarantee<binomial>();
  print_invalidation_guarantee<rc_binomial>();
  print_invalidation_guarantee<thin>();
  return 0;
}
```

21.1.32 __gnu_pbds::tree

附：官方文档地址

```
#include <ext/pb_ds/assoc_container.hpp>  // 因为 tree 定义在这里 所以需要包含这个头文件
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
__gnu_pbds::tree<Key, Mapped, Cmp_Fn = std::less<Key>, Tag = rb_tree_tag,
                 Node_Update = null_tree_node_update,
                 Allocator = std::allocator<char>>
```

21.1.33 模板形参

- Key: 储存的元素类型，如果想要存储多个相同的 Key 元素，则需要使用类似于 std::pair 和 struct 的方法，并配合使用 lower_bound 和 upper_bound 成员函数进行查找
- Mapped: 映射规则（Mapped-Policy）类型，如果要指示关联容器是 **集合**，类似于存储元素在 std::set 中，此处填入 null_type，低版本 g++ 此处为 null_mapped_type；如果要指示关联容器是 **带值的集合**，类似于存储元素在 std::map 中，此处填入类似于 std::map<Key, Value> 的 Value 类型
- Cmp_Fn: 关键字比较函子，例如 std::less<Key>
- Tag: 选择使用何种底层数据结构类型，默认是 rb_tree_tag。__gnu_pbds 提供不同的三种平衡树，分别是：
  - rb_tree_tag：红黑树，一般使用这个，后两者的性能一般不如红黑树
  - splay_tree_tag：splay 树
  - ov_tree_tag：有序向量树，只是一个由 vector 实现的有序结构，类似于排序的 vector 来实现平衡树，性能取决于数据想不想卡你
- Node_Update:用于更新节点的策略,默认使用 null_node_update,若要使用 order_of_key 和 find_by_order 方法，需要使用 tree_order_statistics_node_update
- Allocator：空间分配器类型

21.1.34 构造方式

```
__gnu_pbds::tree<std::pair<int, int>, __gnu_pbds::null_type,
                 std::less<std::pair<int, int>>, __gnu_pbds::rb_tree_tag,
                 __gnu_pbds::tree_order_statistics_node_update>
    trr;
```

21.1.35 成员函数

- insert(x)：向树中插入一个元素 x，返回 std::pair<point_iterator, bool>，其中第一个元素代表插入位置的迭代器，第二个元素代表是否插入成功。
- erase(x)：从树中删除一个元素/迭代器 x。如果 x 是迭代器，则返回指向 x 下一个的迭代器（如果 x 是 end() 则返回 end()）；如果 x 是 Key，则返回是否删除成功（如果不存在则删除失败）。
- order_of_key(x)：返回严格小于 x 的元素个数（以 Cmp_Fn 作为比较逻辑），即从 0 开始的排名。
- find_by_order(x)：返回 Cmp_Fn 比较的排名所对应元素的迭代器。
- lower_bound(x)：返回第一个不小于 x 的元素所对应的迭代器（以 Cmp_Fn 作为比较逻辑）。

- `upper_bound(x)`：返回第一个严格大于 `x` 的元素所对应的迭代器（以 `Cmp_Fn` 作为比较逻辑）。
- `join(x)`：将 `x` 树并入当前树，`x` 树被清空（必须确保两树的 **比较函数**和 **元素类型**相同）。
- `split(x,b)`：以 `Cmp_Fn` 比较，小于等于 `x` 的属于当前树，其余的属于 `b` 树。
- `empty()`：返回是否为空。
- `size()`：返回大小。

`join(x)` 函数需要保证并入树的键的值域与被并入树的键的值域 **不相交**（也就是说并入树内所有值必须全部大于/小于当前树内的所有值），否则会抛出 `join_error` 异常。

如果要合并两棵值域有交集的树，需要将一棵树的元素一一插入到另一棵树中。

21.1.36 示例

```
// Common Header Simple over C++11
#include <iostream>
using namespace std;
using ll = long long;
using ull = unsigned long long;
using ld = long double;
using pii = pair<int, int>;
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
__gnu_pbds::tree<pair<int, int>, __gnu_pbds::null_type, less<pair<int, int>>,
                 __gnu_pbds::rb_tree_tag,
                 __gnu_pbds::tree_order_statistics_node_update>
    trr;

int main() {
  int cnt = 0;
  trr.insert(make_pair(1, cnt++));
  trr.insert(make_pair(5, cnt++));
  trr.insert(make_pair(4, cnt++));
  trr.insert(make_pair(3, cnt++));
  trr.insert(make_pair(2, cnt++));
  // 树上元素 {(1,0), (2,4), (3,3), (4,2), (5,1)}

  auto it = trr.lower_bound(make_pair(2, 0));
  trr.erase(it);
  // 树上元素 {(1,0), (3,3), (4,2), (5,1)}

  // 输出排名 0 1 2 3 中的排名 1 的元素的 first
  auto it2 = trr.find_by_order(1);
  cout << (*it2).first << endl;  // 输出：3

  // 输出其排名
  int pos = trr.order_of_key(*it2);
  cout << pos << endl;  // 输出：1

  // 按照 it2 分裂 trr
  decltype(trr) newtr;
  trr.split(*it2, newtr);
  for (auto i = newtr.begin(); i != newtr.end(); ++i) {
    cout << (*i).first << ' ';  // 输出：4 5
  }
  cout << endl;

  // 将 newtr 树并入 trr 树，newtr 树被清空。
  trr.join(newtr);
  for (auto i = trr.begin(); i != trr.end(); ++i) {
    cout << (*i).first << ' ';  // 输出：1 3 4 5
  }
```

```
  cout << endl;
  cout << newtr.size() << endl;  // 输出: 0

  return 0;
}
```

注意

`pb_ds` 与 `rope` 需要 GCC 扩展头文件；比赛环境注意可用性。

延伸阅读

search...

如梦山河千帆舞，夕朝日耀照吾影。不为凡躯囿我情，青春本因江湖游！

XichenOC + 关注

园龄：8个月 粉丝：7 关注：0

收藏 闪存 小组 博问

算法学习笔记 rope

2025-01-21 21:02 阅读: 475 评论: 1 推荐: 1

神级STL结构-rope大法（学习笔记）

简介：

他是一个 STL 中自带的一种数据结构，是 **pb_ds（Policy-Based Data Structures）库** 的一个分支，由于他的底层是可持久化平衡树——红黑树，或块状链表。所以他的操作的复杂度几乎约等于 $O(logn)$ 或 $\sqrt{n}$ 。它可支持操作较多，完全可以理解成加强版的 $vector$。

最为抽象的是，它可以几乎以 $O(1)$ 的复杂度复制一份，且空间消耗不算太大，这导致他可以代替实现繁琐的可持久化平衡树、可持久化线段树（主席树）、可持久化并查集等等。

其可以以很小的代码量和调试难度骗得较高得分，且 $noip$ 可以使用。

使用方法：

准备：

对与 $rope$ 的使用必须要要调用其库和名名空间名。（因为它不包含在万能头中，且不是 std 命名）

```
#include<ext/rope>
using namespace __gnu_cxx;
```

定义：

其定义方式与其他 STL 的定义方式相同，如 `rope<int>a` 。若要定义一个字符 $rope$ 可以 `rope<char>c` 或 `crope c` ，其类似与一个字符串。

操作：

字符串rope（crope）

首先，我们定义了一个 `crope a;`

- `a.push_back(x)` ：在 `a` 的末尾添加字符 `c` ；
- `a.insert(p,x)` ：在 `a` 的下标 `p` 后面添加 `x` ；
- 或 `a.insert(p,s,n)` ：将字符串 `s` 的前 `n` 位插入 `a` 的下标 `p` 处；
- `a.erase(p,x)` ：从 `a` 的下标 `p` 开始删除 `x` 个元素；
- `a.replace(p,s)` ：从 `a` 的下标 `p` 开始换成字符串 `s` ，若 `a` 的位数不够则补足；
- `a.copy(p,n,s)` ：从 `a` 的下标 `p` 开始的 `n` 个字符换成字符串 `s` ，若位数不够则补足；

- a.substr(p,x) ：从 a 的下标 p 开始截取 x 个元素；
- a[x] 或 a.at(x) 访问下标为 x 的元素；
- a.append(s,p,n) ：把字符串 s 中从下标 p 开始的 n 个字符连接到 a 的结尾，如没有参数 n 则把字符串 s 中下标 p 后的所有字符连接到 a 的结尾，如参数 p 也没有则把整个字符串 s 连接到 a 的结尾；

以上， s 均为字符串（ string 或 char[] ）类型， c 为字符（ char ）类型， n ， p ， x 均为整型。

数组rope（rope）

和字符串 rope 的操作差不多，只是把上文中的字符串参数换成了数组。

数组 rope 可以用 a.append(x) 来在末尾插入一个数 x ，当然也可以用类似于上文的方法把一个数组连接到末尾，这里不再赘述。

数组 rope 也支持 substr 操作，用于截取其中的一段数。

其他常用操作：

与其他普通的 STL 一样支持：

- size() ：返回大小；
- empty() ：返回是否为空；
- begin() ：返回首指针；
- end() ：返回末尾指针后一位；
- clear() ：清空该 $rope$；

可持久操作：

可持久操作主要分为两种，朴素版和指针版，由于复杂度差不多，且指针版可能不太好理解 ~~可能是我菜~~，这里主要将朴素版

我们定义两个 $rope$ 一个为 now 和 $past$ 分别表示当前正在维护的，和之前的版本。对于每一次操作结束后都可以将

`past[++cnt]=now` （没错直接等于就 $O(1)$ 复制了）其中 cnt 为版本数。这样需要调用时就直接找到历史版本查询即可。

例题：

P1383 高级打字机

P6166 [IOI2012] scrivener

两道题几乎一模一样的。

我们直接插入，储存版本，输出就可以，用 $rope$ 十分简单，与 $vector$ 无二。

```cpp
#include<bits/stdc++.h>
#include<ext/rope>
using namespace std;
using namespace __gnu_cxx;
const int N=1e6+10;
rope<char>now,past[N];
int main(){
    int n;
    scanf("%d",&n);
    int cnt;
    while(n--){
        char op;
        cin>>op;
        if(op=='T'){
            char k;
            cin>>k;
            now.push_back(k);
            past[++cnt]=now;
        }
        else if(op=='U'){
            int k;
            scanf("%d",&k);
            now=past[cnt-k];
            past[++cnt]=now;
        }
        else if(op=='Q'){
            int k;
            scanf("%d",&k);
            printf("%c\n",now[k]);
        }
```

```
    }
}
```

优缺点：

优点：

写起来简单，学起来简单，调起来简单

相比各种可持久化数据结构，$rope$ 可谓简短而明了，而且写起来不费劲也不费时，因此对于学不懂/没有来得及学/没有时间写可持久化数据结构的人来说，$rope$ 简直是天降之物，救赎苍生；

缺点：

- 运行较慢：

> 对于它单次操作复杂度为 $O(\sqrt{n})$，在特定情况下较快，但对于根据题目变换而对应的专门的数据结构，其效率是远远不如的。

- 空间较大：

> 由于是完全复制，他的空间远远不如专门改良过后的可持久化数据结构。所以要小心使用

- 可以维护的东西较少

> 对于有些特定的操作就难以维护,不如手写的可自定义化强

练习：

1.自带版本控制功能的IDE Version Controlled IDE **题解**
2.P3850 [TJOI2007] 书架 **题解**

上一篇 P1486 [NOI2004] 郁闷的出纳员
下一篇 P3850 [TJOI2007] 书架

本文作者：XichenOC
本文链接：
https://www.cnblogs.com/XichenOC/p/18684418
版权声明：本作品采用知识共享署名-非商业性使用-禁止演绎 2.5 中国大陆许可协议进行许可。

关注我 收藏该文 1 0

Posted @ 2025-01-21 21:02 XichenOC 阅读(475) 评论(1) 收藏 举报

博客园 © 2004-2025

21.2 数据生成器与对拍

时间复杂度 —

```
const string CHECKER = "checker";
const string TGT = "U224067";

/*
 * 生成闭区间 [a,b] 内的随机整数
 */
int64 randint(int64 a, int64 b) {
    int64 g = b - a + 1;
    int64 rand64 = 0;
    for(int i = 0; i < 4; i++) {
        rand64 <<= 16;
        rand64 ^= rand();
    }
    return a + rand64 % g;
}

/*
 * 生成 T 组规模为 N 的测试数据
 */
string generate_data(int64 T, int64 N) {
    string output;
    int64 n = randint(1, N);
    int64 m = randint(1, N);
    output += to_string(n) + " " + to_string(m) + "\n";

    for(int64 j = 0; j < n; j++) {
        int64 a = randint(1, m);
        int64 b = randint(1, N);
        output += to_string(a) + " " + to_string(b) + "\n";
    }

    return output;
}

/*
 * 比较两个输出文件的内容
 */
bool compare_files(const string& file1, const string& file2) {
    ifstream f1(file1);
    ifstream f2(file2);
    if(!f1.is_open() || !f2.is_open()) {
        cerr << " 无法打开输出文件进行比较" << endl;
        return false;
    }
    string line1, line2;
    int64 lineNum = 0;
    while(getline(f1, line1) && getline(f2, line2)) {
        lineNum++;
        if(line1 != line2) {
            cout << " 第 " << lineNum << " 行不一致:" << endl;
            cout << "Ans: " << line1 << endl;
            cout << "Res: " << line2 << endl;
            f1.close();
            f2.close();
            return false;
        }
    }
```

```
    if(getline(f1, line1) || getline(f2, line2)) {
        cout << " 文件长度不一致" << endl;
        f1.close();
        f2.close();
        return false;
    }
    f1.close();
    f2.close();
    return true;
}

int main() {
    srand(time(0)); // 初始化随机种子

    int64 T = 1;
    int64 N = 2000;

    while(true) {
        string data = generate_data(T, N);
        ofstream f_in("in.in");
        f_in << data;
        f_in.close();

        string cmd1 = CHECKER + " < in.in > outc.out";
        system(cmd1.c_str());

        auto start = chrono::high_resolution_clock::now();
        string cmd2 = TGT + " < in.in > outtgt.out";
        system(cmd2.c_str());
        auto end = chrono::high_resolution_clock::now();
        chrono::duration<double> elapsed = end - start;

        if(compare_files("outc.out", "outtgt.out")) {
            cout << "Accepted! \n 运行时间: " << elapsed.count() << " 秒" << endl;
        } else {

            break;
        }
    }
    return 0;
}
```

注意

随机种子可复现；每轮对拍前清空输出文件，并检查 `system()` 返回值。

第 A 章

Dev C++ 的使用

A.1 快捷键

表 A.1: Dev C++ 快捷键一览

快捷键	功能	快捷键	功能
Ctrl+N	新建源代码	Ctrl+O	打开文件
Ctrl+F9	编译程序	Ctrl+F10	运行
F9	编译并运行	F8	调试程序
Ctrl+W	查看变量	Ctrl+Alt+F2	终止程序
Ctrl+.	注释	Ctrl+M	取消注释
Ctrl+Shift+A	格式化代码	Ctrl+Space	代码提示

A.2 调试

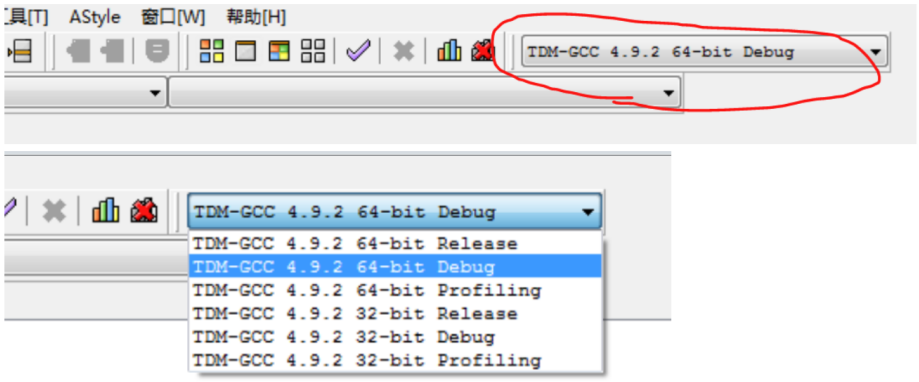


图 A.1: 打开调试功能

先打开调试功能。在行号处点击，这一行会变红，断点就设置在这里；再点击调试按钮，页面会出现黑色的运行框，同时红色断点行变蓝。

输入数据有两种方法：

1. 先单步到 `scanf` 之后，再到运行框输入数据并回车。
2. 先在运行框输入（输入时不会回显），回车后回到代码框单步，输入内容才会显示。

A.3 C++ 标准设置

1. 在 Dev 的【工具】里找到【编译选项】。
2. 【编译时加入以下命令】原本是空白且未勾选；勾选后填入 `-std=c++17`（旧版本可填 `-std=c++11`）。

A.4 优化开关

O1 主要优化分支、常量与表达式；O2 在此基础上做更多寄存器级与指令级优化，编译更慢；O3 在 O2 上继续增加内联与循环优化。

```
1 #pragma GCC optimize(2)
```



图 A.2: 调整编译标准

```
#pragma GCC optimize(3,"Ofast","inline")
```